

OFFICIAL REFERENCE MANUAL

ModelScript Engineering Toolchain

ModelScript

The Polyglot Cyber-Physical Modeling, Simulation & Formal Verification Environment

Architecture, Languages & Algorithmic Foundations

Edition & Target
2026 Comprehensive Edition
WebAssembly (WASM) & Node.js ≥ 24

Open Source License
GNU AGPL v3.0
github.com/modelscript/modelscript

Documentation Portal
docs.modelscript.org
Compiler & Engine Core

Table of Contents

Part I: Getting Started & User Guide

1. Introduction
2. Installation & Requirements
3. Getting Started
4. Platform Administration

Part II: Compiler & Pipeline Architecture

5. Compiler Pipeline Overview
6. WASM GLR Native Parser
7. Salsa Query Engine
8. Linear DAE Arena
9. Triple Graph Grammars
10. Nelson-Oppen Coordinator
11. Simulation Solvers
12. SIL & Real-Time HIL Runtime

Part III: Polyglot Languages & Formats

13. Language Support Overview
14. ModelScript (.msx) Native Language
15. Modelica Physical Modeling
16. SysML v2 & KerML
17. PSCAD Power Systems
18. STEP CAD (ISO 10303)
19. OWL2 Ontologies
20. CSV Telemetry
21. CFD (SU2 & OpenFOAM)
22. FEA Structural Analysis
23. OpenSCAD CSG Geometry
24. SSP System Packaging

Part IV: Comprehensive Algorithms Catalog

25. Algorithms Overview & Map
26. Pantelides Index Reduction
27. Tarjan BLT Partitioning
28. Cellier-Elmqvist Tearing
29. Acausal Connector Balancing
30. Automatic Differentiation
31. Numerical ODE/DAE Solvers
32. Optimization Algorithms
33. Nelson-Oppen & SMT Solvers
34. CHC & IC3 Verification
35. Abstract Interpretation Domains
36. Graph Rewriting & E-Graphs
37. Linear Memory Structures
38. Scientific Computing & Surrogates
39. SDP, BVP & Geometric CSG
40. Diagram Layout & Routing

Part V: Command-Line Interface (msx)

41. CLI Overview & Flags
42. Command Reference

Part VI: Reference & Diagnostics

43. Compiler Linter Rules
44. Standards & Export Formats
45. MCP Server & IDE Workbench

PART I

Getting Started & User Guide

CHAPTER 1

Introduction

Welcome to **ModelScript**, the open-source **computable digital thread** platform and polyglot compiler designed for modern multi-domain engineering.

What is the Computable Digital Thread?

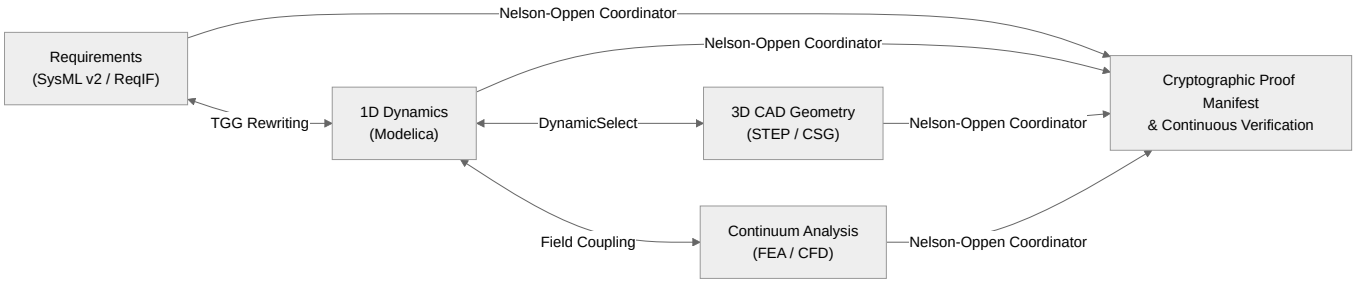
Modern complex systems — from electric vehicles and aerospace systems to industrial robotics — require seamless coordination across disparate engineering domains:

- **System Architecture & Requirements** (SysML v2, KerML, ReqIF)
- **1D Multi-Domain Physical Simulation** (Modelica 3.x, SSP)
- **3D CAD Geometry & Envelopes** (STEP ISO 10303, OpenSCAD CSG)
- **Continuum Field Physics** (FEA structural stress, CFD aerodynamics)
- **Ontological Taxonomies & Rules** (OWL2 Functional Syntax)

Traditionally, these disciplines operate in disconnected silos. Enterprise "digital threads" have historically been **passive and descriptive** — composed of static document links, slow PLM databases, and manual spreadsheets that fall out of sync the moment a parameter changes.

ModelScript makes the digital thread *computable*.

Instead of passive hyperlinks, ModelScript treats engineering models as **executable, code-first specifications** evaluated continuously by native WebAssembly compilers, graph rewriting engines, and formal solvers:



Core Pillars of ModelScript

1. Code-First Engineering Models

Every artifact lives as clean, text-based code versioned in Git. Say goodbye to proprietary binary files and monolithic GUI databases. Diff, review, merge, and test engineering models using standard modern developer workflows and CI/CD pipelines.

2. High-Performance WebAssembly CST & DAE Arenas

Powered by ahead-of-time generated WebAssembly GLR incremental parsers and a zero-allocation Struct-of-Arrays (SoA) linear memory architecture (`DAEBuilder`). Hierarchical models flatten into Differential Algebraic Equations with zero garbage collection pause, enabling microsecond cycle times inside IDEs and web browsers.

3. Bidirectional Triple Graph Grammars (TGG)

Keep architecture and physics in permanent alignment. Declarative TGG rules compile into AOT WebAssembly dispatch tables that automatically propagate architectural changes (SysML v2) forward into physical simulation models (Modelica), while back-annotating numerical simulation sizing results back into architectural specifications.

4. Nelson-Oppen Multi-Theory Formal Verification

Cross-domain verification is solved through an extensible Nelson-Oppen theory coordinator. Exchange equality facts and conflict clauses across six signature-disjoint theory oracles — Ontology (OWL2), Interval Constraints, Abstract Interpretation, Continuous Reachability (Zonotopes), Dynamic DAE Simulation, and 3D Spatial CAD clearance.

5. Multi-Language Language Server Protocol (LSP)

A unified background worker indexing mixed-language workspaces simultaneously. Enjoy instant go-to-definition, hover documentation, semantic completion, and real-time compiler diagnostics across Modelica, SysML v2, STEP, and OWL2 in VS Code or browser environments.

Ready to Get Started?

Proceed to the Installation guide to set up the `msx` CLI and VS Code extension, or follow Getting Started to build your first connected model.

CHAPTER 2

Installation

ModelScript can be installed as a VS Code extension for end users, or set up locally for developers looking to contribute.

For Users

The easiest way to use ModelScript is via the VS Code Marketplace.

1. Open VS Code.
2. Go to the Extensions tab (`Ctrl+Shift+X`).
3. Search for **ModelScript**.
4. Click **Install**.

The extension bundles the Language Server, the 3D Viewer, and all necessary polyglot tools out-of-the-box.

For Developers

To set up ModelScript for local development, you will need **Node.js ≥ 24** (matching `.nvmrc`) and `npm`. Docker is optional and only required if running backend database/API services.

1. Clone the Repository

```
git clone https://github.com/modelscript/modelscript.git
cd modelscript
```

2. Install Dependencies

We use `npm` workspaces to manage our monorepo packages.

```
npm install
```

3. Build the Monorepo

ModelScript compiles its native WebAssembly GLR parsers, AssemblyScript flattening kernels, and TypeScript packages via Nx:

```
npm run build:libs    # Build core compiler packages and language runtimes
npm run build         # Build all monorepo targets
```

4. Run the Dev Server

You can launch the IDE and the local web environment via:

```
npm run dev
```

This will spawn the compiler backend, the LSP workers, and open the web UI in your browser.

CHAPTER 3

Getting Started

This guide walks you through building your first **computable digital thread** in ModelScript — linking 1D dynamic physics in Modelica with 3D STEP CAD geometry and simulating the combined system.

---

1. Creating a Workspace

1. Open an empty project folder in VS Code.
2. Run the command `ModelScript: Initialize Workspace` from the command palette (`Ctrl+Shift+P`).
3. This creates a standard `.modelscript` workspace index configuration.

---

2. Writing Your First Physics Model

Create a new file called `BouncingBall.mo` :

```
model BouncingBall
  parameter Real e = 0.7 "Coefficient of restitution";
  parameter Real g = 9.81 "Gravity acceleration";
  Real h(start=1.0) "Height of ball";
  Real v(start=0.0) "Velocity of ball";
equation
  der(h) = v;
  der(v) = -g;
  when h <= 0 and v < 0 then
    reinit(v, -e * pre(v));
  end when;
end BouncingBall;
```

The native WebAssembly GLR language server instantly activates, providing sub-millisecond diagnostics, type checking, and hover info.

---

3. Weaving the Digital Thread: Binding 3D CAD

In a computable digital thread, physical states directly drive geometric positions. If you have a STEP file (e.g., `ball.step`), you can bind the 3D geometry directly to the dynamic variable `h` using a `DynamicSelect` CAD annotation:

```
model BouncingBall
  parameter Real e = 0.7 "Coefficient of restitution";
  parameter Real g = 9.81 "Gravity acceleration";
  Real h(start=1.0) "Height of ball" annotation(
    CAD(
      shape="ball.step",
      transform=DynamicSelect(
        translate(0, 0, 0),
        translate(0, h, 0)
      )
    )
  );
  Real v(start=0.0) "Velocity of ball";
equation
  der(h) = v;
  der(v) = -g;
  when h <= 0 and v < 0 then
    reinit(v, -e * pre(v));
  end when;
end BouncingBall;
```

Open the `BouncingBall.mo` file and click the **Open 3D Viewer** icon in the editor toolbar to see the 3D STEP mesh animate in synchrony with the continuous physics!

---

4. Simulating via the CLI

Execute the model using the unified `msx` command-line tool:

```
# Flatten to linear DAE arena and simulate
msx simulate BouncingBall BouncingBall.mo --stop-time 3.0 --step-size 0.01

# Render the interactive SVG schematic diagram
msx render BouncingBall BouncingBall.mo > schematic.svg
```

---

5. Next Steps

- Explore Polyglot Languages to integrate SysML v2 architectural requirements and OWL2 ontologies into the digital thread.
- Learn about the Triple Graph Grammar (TGG) engine for automated bidirectional model synchronization.
- Read about the Nelson-Oppen Coordinator for cross-domain formal verification.

CHAPTER 4

Platform Administration & Enterprise Hub Guide

The **ModelScript Enterprise Hub** (`@modelscript/api` and `@modelscript/web`) provides a self-hostable or cloud-managed control plane for polyglot package management, Git Smart HTTP hosting, OMG SysML v2 / Semantic Web APIs, Slurm HPC simulation offload, operational digital twins, and cryptographically verifiable regulatory compliance.

---

1. Administrative Console & Role-Based Access Control (RBAC)

Access to the Platform Administration Console (`/admin`) and `/api/v1/admin/*` endpoints is strictly enforced via JWT authentication and role-based middleware (`requireAdmin`):

- **`admin`** : Full platform superuser privileges (user moderation, runtime feature flag mutation, system maintenance windows, compute wallet minting, and audit log verification).
- **`moderator`** : Read/moderation access to public registry packages, flagged comments, and community discussions.
- **`user` / Organization Roles (`owner` , `admin` , `member` , `billing` , `viewer`)**: Scoped access within `/api/v1/organizations/:org` namespaces.

User & Account Governance (`/api/v1/admin/users`)

Platform administrators can search, filter, and manage user accounts via the Admin Console or REST API:

- **Role Assignment (`PATCH /api/v1/admin/users/:id/role`)**: Promote or demote accounts between `user` , `moderator` , and `admin` . Self-demotion by the active administrator is blocked to prevent accidental lockout.
- **Account Suspension (`POST /api/v1/admin/users/:id/suspend` & `/unsuspend`)**: Immediately revokes active JWT refresh tokens, blocks package publishing, and halts running cloud HPC jobs while preserving historical audit trails.
- **Platform Telemetry (`GET /api/v1/admin/stats`)**: Real-time metrics covering total registered users, published package versions, active Co-Simulation sessions, cloud HPC queue depth, and object storage consumption.

---

2. Runtime Feature Flags & Progressive Rollouts

ModelScript includes an in-memory and database-backed **Feature Flag Engine** (`apps/api/src/services/feature-flag-service.ts`) that allows operators to toggle experimental compilers, cloud solvers, or UI capabilities at runtime without redeploying containers.

Flag Evaluation Precedence

1. **Environment Variable Override**: Setting `FEATURE_FLAG_<UPPER_SNAKE_KEY>=true|false` (for example, `FEATURE_FLAG_WEBGPU_CLOUD_SOLVER=true`) overrides database state for deterministic staging/CI configuration.
2. **Role Gate (`allowedRoles`)**: If a flag specifies `allowedRoles` (e.g., `["admin", "moderator"]`), only authenticated users holding one of those roles receive `true` .
3. **Deterministic Percentage Rollout (`rolloutPercentage`)**: When `0 < rolloutPercentage < 100` , the service computes a deterministic hash over `(flagKey, userId)` so a given user receives a consistent cohort experience across sessions.

Administrative Flag Endpoints

Method & Endpoint	Access	Description
`GET /api/v1/flags`	Public / User	Evaluate all feature flags for the current caller's `userId` and `role` .
`GET /api/v1/admin/flags`	`admin`	List all registered feature flags with full rollout metadata, role gates, and environment override states.
`PATCH /api/v1/admin/flags/:key`	`admin`	Dynamically update `enabled` (`boolean`), `rolloutPercentage` (`0` – `100`), `allowedRoles` (`string[]`), or `description` , and record an immutable admin audit event.

---

3. Enterprise Authentication, SSO, 2FA & Bot Accounts

The authentication subsystem (`/api/v1/auth` , `/api/v1/auth/sso` , `/api/v1/auth/2fa` , `/api/v1/bots`) supports defense-in-depth enterprise identity requirements:

- **OpenID Connect (OIDC) & SAML Enterprise SSO (`/api/v1/auth/sso`)**: Connect Okta, Azure AD / Entra ID, Keycloak, or Google Workspace identity providers with automatic Just-in-Time (JIT) organization provisioning and group-to-role mapping.
- **TOTP Two-Factor Authentication (`/api/v1/auth/2fa`)**: RFC 6238 Time-Based One-Time Password enrollment (QR code provisioning, backup recovery codes, and mandatory 2FA enforcement policies per organization).
- **Machine & CI/CD Bot Accounts (`/api/v1/bots`)**: Provision scoped, non-interactive service accounts bearing `ms_bot_*` API tokens with granular permissions (`package:read` , `package:publish` , `cloud:dispatch` , `mcp:invoke` , `dhf:export`) and automatic token rotation schedules.
- **Signed Outbound Webhooks (`/api/v1/webhooks`)**: Subscribe external CI/CD pipelines to workspace events (`package.published` , `simulation.completed` , `verification.failed` , `twin.anomaly_detected`) signed with HMAC-SHA256 (`X-ModelScript-Signature-256`).

---

4. Native Git Smart HTTP Server & Content-Addressable Registry

The Hub integrates both a **Git Smart HTTP Backend** and a **Signed Content-Addressable Package Registry**:

Native Git Smart HTTP Server (`/git/:namespace/:project` & `/api/v1/git-server`)

Engineers can clone, fetch, branch, and push polyglot model repositories using standard `git` clients over HTTPS:

```
git clone https://hub.example.com/git/aerospace/flight-control.git
```

- Supports `git-upload-pack` (`GET /info/refs?service=git-upload-pack`) and `git-receive-pack` (`POST /git-receive-pack`) authenticated via Basic Auth (`username` + PAT or `ms_bot_*` token).
- Pre-receive hooks optionally enforce zero-error `msx lint` checks and Digital Thread correspondence verification before accepting pushes to protected branches.

Cryptographic Package Signing & Multi-Lens Manifests (`/api/v1/packages`)

Every package published via `msx publish` is stored in a SHA-256 Content-Addressable Storage (CAS) blob store and signed with the publisher's **Ed25519** private key:

- **Integrity & Provenance**: `msx install --verify-signatures` validates both the SHA-256 tarball digest against `modelscript.lock` and the Ed25519 signature against the registry's public keyring.
- **Multi-Lens Manifest Projection (`GET /api/v1/packages/:name/:version?lens=<lens>`)**: The registry dynamically projects any `modelscript.json` package manifest into three industry interoperability schemas on the fly:
  - `?lens=npm` : Standard Node/NPM-compatible package metadata for JavaScript/TypeScript tooling.
  - `?lens=aas` : **IDTA Asset Administration Shell (AAS v3.0)** JSON submodel descriptor containing `TechnicalData` and `SimulationModels` (FMU) references.
  - `?lens=okh` : **DIN SPEC 3105 / OSHWA Open Know-How (OKH v1.0)** hardware manifest exposing CAD/ECAD bill-of-materials, manufacturing files, and SPDX license metadata.

---

5. Standards-Compliant Systems & Semantic Web APIs

The Hub natively implements open MBSE and Semantic Web specifications so external enterprise tools can query models without proprietary adapters:

API Surface	Base Path	Standard / Capabilities
OMG SysML v2 REST API	`/api/v1/sysml2` (and root `/` alias)	Implements the **OMG Systems Modeling API and Services v2.0** specification: `/projects`, `/projects/{id}/commits`, `/projects/{id}/branches`, `/projects/{id}/tags`, `/projects/{id}/commits/{cid}/elements`, and `/relationships`.
GraphQL Library API	`/api/v1/libraries/graphql`	Strongly typed GraphQL schema for querying packages, Modelica classes, ports, equations, physical units, and dependency graphs in a single round-trip.
W3C RDF Serialization	`/api/v1/libraries/rdf`	Exports package and workspace ontologies as `text/turtle`, `application/ld+json`, or `application/n-triples`.
W3C SPARQL 1.1 Endpoint	`/api/v1/libraries/sparql`	Executes SPARQL 1.1 `SELECT`, `CONSTRUCT`, and `ASK` queries with optional OWL 2 RL entailment reasoning over indexed engineering libraries.
OSLC 3.0 & TRS 3.0	`msx oslc serve` / `/api/v1/threads`	Exposes OASIS OSLC Requirements (`oslc_rm`) and Architecture (`oslc_am`) catalogs with Tracked Resource Set (TRS 3.0) incremental change logs for IBM DOORS Next and Polarion synchronization.

---

6. Cloud Slurm HPC Offload, CAE Sweeps & Compute Credit Billing

When engineers run `msx simulate --cloud` or `msx cloud dispatch`, the Hub orchestrates containerized batch execution across Slurm HPC clusters or Kubernetes GPU pools:

- **Cloud Simulation & CAE Endpoints (`/api/v1/cloud` & `/api/v1/cae`)**:
  - Dispatches single-run stiff DAE simulations, multi-thousand-run Monte Carlo ensembles, SU2 CFD runs, and CalculiX FEA jobs to isolated worker nodes.
  - Streams live progress events and solver logs via WebSockets (`/ws`) and persists output trajectories (`CSV`, `Parquet`, `VTK`, `FRD`) to S3-compatible object storage.
- **Compute Credit Wallet & Stripe Billing (`/api/v1/billing`)**:
  - Each user or organization maintains an ACID transactional **Compute Credit Wallet**.
  - Job submission places a pre-authorization hold (`reserveCredits`) based on the requested hardware profile (`standard`, `high-cpu`, `gpu-a100`, `slurm-mpi`); upon completion or cancellation (`msx cloud cancel`), exact wall-clock consumption is settled and unused credits are refunded.
  - Supports self-service credit top-ups via Stripe Checkout webhooks (`/api/v1/billing/webhook`) or manual enterprise quota allocation by platform admins.

---

7. Digital Thread Hash-Chained Audit Ledger & Operational Digital Twins

For safety-critical industries (aerospace DO-178C, automotive ISO 26262, medical FDA 21 CFR Part 11 / 820, and nuclear IEC 61513), the Hub maintains a tamper-evident digital thread and live twin telemetry pipeline:

Cryptographic Hash-Chained Audit Ledger (`/api/v1/threads/audit-log`)

Every mutation to a requirement, model equation, TGG alignment link, verification proof, or administrative setting is appended to a PostgreSQL audit ledger where each record k stores:

$$H_k = \text{SHA256}(H_{k-1} \parallel \text{timestamp}_k \parallel \text{actorId}_k \parallel \text{action}_k \parallel \text{payloadHash}_k)$$

Operators and external auditors can verify the unbroken integrity of the entire historical chain at any time via the CLI or REST API:

```
# Verify cryptographic hash-chain integrity across the PostgreSQL audit ledger
msx db verify

# Export and verify a signed Design History File (DHF) compliance archive
msx dhf export ./workspace -o release_v1.dhf.md
msx dhf audit release_v1.dhf.md
```

Operational Digital Twins (`/api/v1/twins` & `/api/v1/instances`)

- **Asset Instance Registry (`/api/v1/instances`)**: Tracks serialized physical assets in the field (e.g., tail numbers, inverter serial IDs, or battery packs) bound to specific immutable commits of their ModelScript digital twin.
- **Live Telemetry & Drift Detection (`/api/v1/twins`)**: Ingests time-series sensor telemetry via REST/WebSockets, executes an online Extended Kalman Filter (EKF) / moving-horizon observer against the compiled WASM DAE arena, and triggers automated alerts when physical parameter drift or residual anomalies exceed safety thresholds.

---

8. Hosted MCP SSE Gateway (`/api/v1/mcp`)

Enterprise teams can connect cloud AI agents and internal copilot orchestrators directly to the Hub's **Hosted Model Context Protocol (MCP) Gateway**:

- **`GET /api/v1/mcp/sse`** : Authenticates via Bearer token (`JWT` or `ms_bot_*`) and opens a server-sent events transport session.
- **`POST /api/v1/mcp/messages`**: Dispatches any of the 29 MCP engineering tools inside sandboxed worker isolates with automatic compute credit metering and full audit-ledger logging of every AI tool invocation.

PART II

Compiler & Pipeline Architecture

CHAPTER 5

Architecture Overview

ModelScript is built on a **data-oriented, WebAssembly-first compiler architecture**. Rather than allocating deep hierarchies of garbage-collected JavaScript AST wrapper objects, every stage of the compiler—from GLR parsing to DAE flattening, sparse linear algebra, and formal verification—operates on contiguous **Struct-of-Arrays (SoA)** arenas in WebAssembly linear memory.

End-to-End Compilation Pipeline



Core Architectural Pillars

1. Zero-Wrapper Linear Memory CST & DAE Arena

Equations, expressions, and statements are lowered directly from 32-bit linear-memory CST node pointers (`u32`) into `DAEBuilder` integer IDs (`VarId` , `ExprId` , `EqId` , `StmtId`). This eliminates garbage-collection pauses even when flattening multi-thousand-equation Modelica Standard Library (MSL) models.

2. Demand-Driven Incremental Compilation (Salsa)

Inspired by the Rust compiler (`rust-analyzer`), `QueryEngine` implements the **Salsa** incremental recomputation algorithm (`fetch → deep_verify → execute → backdate`). When a user edits a parameter in the IDE, only queries whose transitive input hashes actually changed are recomputed.

3. Polyglot Synchronization via Triple Graph Grammars (TGG)

Cross-language traceability is enforced algebraically via **Double-Pushout (DPO)** graph rewriting (`packages/dsl/src/tgg.ts`) with **Critical Pair Analysis (CPA)** confluence verification (`packages/dsl/src/cpa.ts`). Changes in SysML v2 requirements, Modelica parameters, STEP CAD assemblies, or ModelScript `@unmanaged` structs propagate bidirectionally without losing comments or formatting.

4. Multi-Domain Theory Coordinator ($\mathrm{CDCL}(\mathrm{T})$)

Thirteen specialized **Domain Theory Oracles**—spanning OWL2 Description Logic (`OntologyTheoryOracle`), SysML v2 intervals (`ConstraintTheoryOracle`), Zones/Octagons (`AbstractDomainOracle`), Barrier Certificates & Taylor Flowpipes (`ContinuousSafetyOracle`), DAE trajectories (`DynamicSimulationOracle`), 3D B-Rep clearance (`SpatialPhysicsOracle`), FEA/CFD contracts (`CaeContractOracle`), DAE structural matching (`CausalizationOracle`), SI dimensional analysis (`DimensionalOracle`), Assume-Guarantee refinement (`EntailmentOracle`), Kirchhoff/stream conservation (`FlowAlgebraOracle`), AP242 GD&T stack-up (`ToleranceStackOracle`), and Conformal UQ (`UqOracle`)—exchange variable bounds and Nelson-Oppen equalities until a global fixpoint or minimal unsatisfiable core (`UnsatCore`) is reached.

Monorepo Package Matrix

Package	Path	Primary Responsibility
`@modelscript/dsl`	`packages/dsl`	Grammar combinators, Ahead-of-Time WASM GLR parser compiler (`buildParser`), TGG/DPO engine, Critical Pair Analysis, E-graphs
`@modelscript/runtime`	`packages/runtime`	Linear-memory `DAEBuilder` , `ArenaDAEPrinter` , Salsa `QueryEngine` , `WorkspaceIndex` , BLT/Pantelides solver, 13 Formal Theory Oracles
`@modelscript/simulate`	`packages/simulate`	Numerical integrators (SUNDIALS CVODE/IDA, Radau IIA, DOPRI5, Dommel EMT), WebGPU batched solver, FNO surrogates, Monte Carlo
`@modelscript/cad`	`packages/cad`	OpenCascade (OCCT) B-Rep kernel, CSG evaluator, STEP AP214/AP242 serializer, 3D tetrahedral FEA solver, Gerber ECAD parser
`@modelscript/diagram`	`packages/diagram`	Polyglot schematic layout engine, orthogonal Manhattan connection router, SVG/Canvas renderer, dark-mode color inversion
`@modelscript/exchange`	`packages/exchange`	FMI 2.0/3.0 FMU import/export, SSP 1.0/2.0 container toolkit, Co-Simulation Master Orchestrator, WebSerial/AudioWorklet HIL
`@modelscript/lsp`	`packages/lsp`	Multi-language LSP server, semantic tokens, hover, go-to-definition, code actions, background worker pool
`@modelscript/mcp`	`packages/mcp`	Unified Model Context Protocol server exposing 25+ polyglot compilation, simulation, CAD, and verification tools to AI agents
`@modelscript/ide-client`	`packages/ide`	Reusable WebAssembly IDE client components, Monaco/CodeMirror language bindings, and dockable engineering viewports
`@modelscript/examples`	`packages/examples`	Curated multi-domain reference workspaces (CubeSat, EV Battery Pack, IEEE 39-Bus Grid, Robotic Arm, Aero-Structural Wing)
`modelscript` (`msx`)	`apps/cli`	Unified command-line interface (`flatten` , `simulate` , `verify` , `align` , `polyglot` , `fmu` , `csg` , `surrogate` , `mc` , `hil`)
`@modelscript/api`	`apps/api`	REST, GraphQL, SPARQL, and WebSocket simulation backend server
`@modelscript/web`	`apps/web`	Browser-native WebAssembly Polyglot Playground and visual modeling environment

CHAPTER 6

WebAssembly GLR Incremental Parser

ModelScript avoids external runtime parser dependencies. Instead, every supported language in `languages/` is parsed using a custom, ahead-of-time (AOT) compiled **Generalized LR (GLR)** parser executing directly in WebAssembly (`parser.wasm`).

The parser compiler is implemented in `@modelscript/dsl` under `packages/dsl/src/codegen/`.

Why GLR in WebAssembly?

1. **Handling Complex Engineering Grammars**: Engineering modeling languages like Modelica and SysML v2 possess ambiguities that cannot be parsed by strict deterministic LR(1) or LALR parsers. The GLR algorithm handles non-deterministic grammar paths by branching parse stacks and pruning invalid branches dynamically.
2. **Zero-Allocation Memory Footprint**: Rather than allocating JavaScript AST object wrappers for every identifier and operator, tokens and CST nodes are laid out in a flat WebAssembly linear memory arena.
3. **Web-Native Performance**: The compiled `parser.wasm` executes at near-native speeds in both Node.js server environments and client-side browser contexts (such as the IDE and the Web Playground).

Parser Initialization

In TypeScript or AssemblyScript code, parsers are loaded and registered via `createWasmParser`:

```
import { createWasmParser } from "@modelscript/modelica/parser";
import { Context } from "./src/context.js";

// Load the compiled WebAssembly parser binary
const modelicaWasmPath = require.resolve("@modelscript/modelica/parser.wasm");
const { parser } = await createWasmParser(modelicaWasmPath);

// Register file extension mapping in the compilation Context
Context.registerParser(".mo", parser);
```

Linear CST Memory Layout

When a source file is parsed, the parser allocates a continuous buffer of node headers in linear memory. Each CST node is referenced by a 32-bit integer pointer (`u32` node offset) rather than an object reference:

```
┌──────────────────────────────────────────────────┐
│                  Linear CST Node Arena           │
├─────────────┬────────────┬──────────────┬────────┤
│ Field       │ Offset     │ Type         │ Description     │
├─────────────┼────────────┼──────────────┼─────────────────┤
│ typeId      │ 0          │ uint16       │ Node type code  │
│ flags       │ 2          │ uint16       │ Semantic flags  │
│ startOffset │ 4          │ uint32       │ Start byte pos  │
│ endOffset   │ 8          │ uint32       │ End byte pos    │
│ firstChild  │ 12         │ uint32       │ Offset to child │
│ nextSibling │ 16         │ uint32       │ Offset to sib   │
└─────────────┴────────────┴──────────────┴─────────────────┘
```

Direct Field Accessors (`Cst`)

High-performance accessors read typed node properties directly from the linear buffer:

```
import { Cst } from "@modelscript/modelica/language";

// Accessing child fields without constructing intermediate wrappers
const comp = Cst.ComponentClause.componentDeclarationList(node);
for (const decl of comp) {
  const nameNode = Cst.ComponentDeclaration.declaration(decl);
  const identifier = db.ast.text(nameNode);
}
```

Incremental Re-Parsing

When editing code in the Language Server Protocol (LSP) or Web IDE, the GLR parser performs incremental tree re-use:

- Text edits pass line and character delta offsets.
- Unaffected subtrees in linear memory remain unchanged and are spliced into the new parse tree.
- Syntax diagnostics are generated instantly during parsing and stored in the diagnostics arena.

CHAPTER 7

Salsa Incremental Query Engine

ModelScript's semantic analysis and symbol resolution pipeline is driven by an incremental query computation engine inspired by the open-source Salsa incremental query framework.

The engine lives in `@modelscript/runtime` and provides memoized, demand-driven evaluation across multi-file engineering workspaces.

The Query Life Cycle

Every query executed in ModelScript follows a 4-phase lifecycle to ensure maximum cache re-use and correct invalidation:



The 4 Stages

1. **`fetch`**: Checks if the query result exists in the memoization table.
2. **`deep_verify`**: If the workspace revision has advanced, transitively checks whether any input queries or files that contributed to this result have actually changed.
3. **`execute`**: Re-runs the query definition if inputs have genuinely changed, tracking any newly accessed queries.
4. **`backdate`**: If re-executing the query produces an output value identical to the previous result, the query's revision is backdated to prevent cascading invalidation down the dependency tree.

Key Compiler Queries

The query engine handles semantic resolution, type checking, and specialization:

Query	Scope	Description
`instantiate`	Class	Resolves all direct components, inherited elements, and nested declarations of a class.
`classInstance`	Component	Specializes a generic class definition with specific modifier expressions.
`resolveSimpleName`	Scope	Performs hierarchical lexical scoping and imports resolution.
`variability`	Component	Computes effective variability: `constant`, `parameter`, `discrete`, or `continuous`.
`causality`	Component	Computes signal causality: `input`, `output`, or `internal`.
`effectiveModification`	Hierarchy	Resolves and merges class modification clauses across multi-level inheritance graphs.

Salsa & Linear CST Synergy

AST wrapper nodes are **never** stored inside the Salsa cache. Storing heavy AST objects would duplicate memory and defeat garbage collection guarantees.

Instead:

- The Salsa cache stores semantic integer handles and primitive metadata (e.g. `SymbolId`, resolved `TypeInfo`, variability enums).
- Concrete CST nodes are retrieved on-demand from linear memory using `db.cstNode(symbolId)` only when lowering or evaluating equations.

CHAPTER 8

Linear Memory DAE Arena & Lowering

The Differential Algebraic Equations (DAE) representation in ModelScript is implemented via `DAEBuilder` in `@modelscript/runtime` (`packages/runtime/src/dae/wasm_dae.ts`).

Unlike traditional compilers that construct linked expression trees using heap-allocated objects, `DAEBuilder` operates strictly on **flat, contiguous typed arrays** (Struct-of-Arrays) in WebAssembly linear memory.

Arena Struct-of-Arrays (SoA) Layout

The arena maintains three primary contiguous arrays:

```
┌──────────────────────────────────────────────────────────┐
│                    DAEBuilder Linear Memory              │
├──────────────┬──────────────┬────────────────────────────┤
│ Buffer       │ Element Size │ Contents                   │
├──────────────┼──────────────┼────────────────────────────┤
│ varData      │ 32 bytes     │ Name, type, variability,   │
│              │              │ causality, start value, flags│
├──────────────┼──────────────┼────────────────────────────┤
│ exprData     │ 16 bytes     │ OpKind, data1, leftExprId, │
│              │              │ rightExprId                │
├──────────────┼──────────────┼────────────────────────────┤
│ eqData       │ 24 bytes     │ EqKind, lhsExprId,         │
│              │              │ rhsExprId, flags           │
└──────────────┴──────────────┴────────────────────────────┘
```

Every variable, expression, and equation is assigned a persistent 32-bit integer handle:

- `VarId` : Handle to a variable in `varData`
- `ExprId` : Handle to an expression node in `exprData`
- `EqId` : Handle to an equation in `eqData`

Lowering Pipeline

Flattening a high-level Modelica model into linear DAE form proceeds in two coordinated layers:

Layer 1: Components & Hierarchy

Reads Salsa queries (`instantiate` , `classInstance` , `variability`) to allocate variables and parameters in `DAEBuilder` :

- Scalar variables (`Real` , `Integer` , `Boolean` , `String`)
- Array dimensions and indices
- Start values, nominals, and min/max boundaries

Layer 2: Equations & Statements

Walks the linear CST of the class body (`db.cstNode(classId)`):

- Lowers binary, unary, and call expressions directly into `ExprId` nodes in linear memory.
- Emits equality equations (`lhs = rhs`), initial equations, and algorithm statements.

Expression Lowering Rules

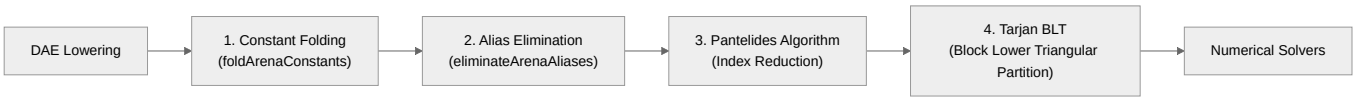
To match OpenModelica's canonical output, the flattener applies specific normalization rules:

1. **Real Coercion in Binary Operations**: If one operand of an arithmetic operator is `Real` and the other is `Integer` , the integer operand is automatically cast to Real using `castToRealExpr()` .

2. **Negation Distribution**: Expressions of the form `-(a * b)` are rewritten canonically to `(-a) * b`.
3. **Binding Literal Kind Matching**: When a binding expression is lowered, `addIntLiteral()` is used for `VarType.Integer` and `addRealLiteral()` for `VarType.Real` (e.g. `parameter Real R = 100` becomes `100.0`).

Optimization & Transformation Passes

Once lowered to the arena, equations pass through four high-performance transformations:



1. **Constant Folding (`foldArenaConstants`)**: Recursively simplifies compile-time constant arithmetic expressions directly in the arena.
2. **Alias Elimination (`eliminateArenaAliases`)**: Detects trivial equality equations ($x = y$) generated by connectors and replaces all references with a single canonical variable handle.
3. **Pantelides Index Reduction**: Differentiates high-index algebraic constraints to reduce the DAE index to 1 or 0 for stable numerical integration.
4. **Tarjan BLT Partitioning**: Constructs the bipartite incidence graph between variables and equations, computes maximum matching, and finds Strongly Connected Components (SCC) using Tarjan's algorithm.

CHAPTER 9

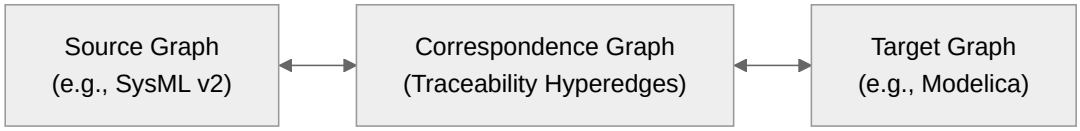
Polyglot Triple Graph Grammars (TGG)

ModelScript provides declarative, ahead-of-time (AOT) compiled **Triple Graph Grammars (TGG)** and Double-Pushout (DPO) algebraic graph rewriting via `@modelscript/dsl/tgg`.

TGG serves as the foundational synchronization mechanism across the **computable digital thread**, enabling bidirectional transformations between disparate engineering models (such as architectural SysML v2 specifications and 1D Modelica physical simulations).

What is a Triple Graph Grammar?

A Triple Graph Grammar synchronizes three distinct graphs:



- **Source Graph (G_S)**: Represents domain concepts in the source language (e.g. SysML v2 Part Definitions, Ports, Requirements).
- **Target Graph (G_T)**: Represents concepts in the target language (e.g. Modelica Models, Connectors, Equations).
- **Correspondence Graph (G_C)**: Explicitly records semantic trace links between elements in the source and target models.

Rather than writing ad-hoc imperative glue code, engineers write declarative rules specifying how elements correspond.

Declarative TGG Rules

A TGG rule specifies patterns in the source, correspondence, and target graphs simultaneously:

```
import { tggRule } from "@modelscript/dsl/tgg";

export const ComponentToModel = tggRule({
  name: "ComponentToModel",
  source: (s) => ({
    part: s.node("PartUsage", { name: s.var("compName") }),
    port: s.node("PortUsage", { parent: s.nodeRef("part") }),
  }),
  target: (t) => ({
    model: t.node("ModelDefinition", { name: t.var("compName") }),
    pin: t.node("PinConnector", { parent: t.nodeRef("model") }),
  }),
  correspondence: (c) => ({
    tracePart: c.link("part", "model"),
    tracePort: c.link("port", "pin"),
  }),
});
```

AOT Compilation to WebAssembly

Unlike interpreted graph transformation engines, `@modelscript/dsl/tgg` compiles rules into deterministic AssemblyScript dispatch routines executing directly in WebAssembly:

- `tgg_forward_dispatch`: Incremental forward propagation from source changes to target models.
- `tgg_backward_dispatch`: Reverse propagation of parameter sizing or component changes back to architectural definitions.
- `tgg_propagate_all_stale`: High-throughput batch reconciliation across dirty workspace subgraphs.

Critical Pair Analysis (CPA)

A notorious problem in graph transformation is rule conflict and non-determinism. ModelScript includes an automated **Critical Pair Analysis (CPA)** verification pass:

1. **Confluence Verification**: Analyzes whether overlapping rule applications could cause divergent target states.
2. **Termination Guarantees**: Proves that cyclic dependencies cannot cause infinite rewrite loops during synchronization.
3. **Completeness Checking**: Flags unmapped source elements that lack corresponding target constructors.

CHAPTER 10

Multi-Domain Theory Coordinator (Nelson-Oppen & CDCL(T))

The **Multi-Domain Theory Coordinator** (`packages/runtime/src/formal/theory_coordinator.ts`) is ModelScript's cross-language formal verification engine. It combines **Nelson-Oppen equality sharing** with **Conflict-Driven Clause Learning Modulo Theories (**$\mathrm{CDCL(T)}$**)** to verify system-wide invariants across ontological, structural, geometric, physical, causal, and dynamic simulation domains simultaneously.

Architectural Motivation

In traditional Model-Based Systems Engineering (MBSE), each engineering tool operates in an isolated semantic silo:

- A **SysML v2** requirement specifies `maxBatteryTemp <= 333.15 [K]`.
- A **STEP / OpenSCAD** 3D CAD enclosure defines the heat-sink surface area A_{sink}.
- A **CFD** configuration computes the convective heat transfer coefficient $h(v_\infty)$.
- A **Modelica** thermal-electrical DAE integrates $C_{\text{th}}\dot{T} = I^2 R_{\text{int}} - hA_{\text{sink}}(T - T_{\text{amb}})$.

No single solver understands all four representations. The **Theory Coordinator** bridges these silos by treating each engineering domain as a decidable or semi-decidable **Theory Oracle** (T_i) sharing a common set of interface variables (V_{shared}).

The 13 Domain Theory Oracles

`packages/runtime/src/formal/theory_coordinator.ts` and `packages/runtime/src/formal/oracles/index.ts` coordinate 13 specialized domain theory oracles implementing the `TheoryOracle` interface:



#	Oracle Class	Domain (`TheoryDomain`)	Decision / Propagation Procedure
1	`OntologyTheoryOracle`	`ontology`	$\mathcal{SROIQ}(\mathbb{D})$ Tableau reasoning over OWL 2 axioms, class disjointness, and property domains
2	`ConstraintTheoryOracle`	`constraints`	Interval arithmetic and relational constraint propagation over SysML v2 requirements
3	`AbstractDomainOracle`	`abstract_interpretation`	Floyd-Warshall Difference Bound Matrix (DBM) closure over **Zone** ($x_i - x_j \le c$) and **Octagon** ($\pm x_i \pm x_j \le c$) domains
4	`ContinuousSafetyOracle`	`continuous_safety`	Control Barrier Function (CBF) Lie-derivative positivity ($L_f B + L_g B \cdot u + \alpha(B) \ge 0$) and Taylor-model flowpipe safety
5	`DynamicSimulationOracle`	`simulation`	Sensitivity-guided DAE trajectory integration (CVODE/IDA) with exact zero-crossing root finding
6	`SpatialPhysicsOracle`	`spatial`	3D CAD bounding-box, B-Rep minimum clearance distance, mass budget, and interference checks
7	`CaeContractOracle`	`cae`	FEA structural safety factor ($\sigma_{\text{von Mises}} \le \sigma_{\text{yield}}/\text{SF}$) and CFD aerodynamic/thermal drag polar bounds
8	`CausalizationOracle`	`causalization`	Hopcroft-Karp maximum bipartite matching ($O(E\sqrt{V})$), Tarjan BLT decomposition, and Pantelides DAE index reduction
9	`DimensionalOracle`	`dimensional`	Gaussian elimination over the 7-base SI unit exponent matrix $[\text{m}, \text{kg}, \text{s}, \text{A}, \text{K}, \text{mol}, \text{cd}]$
10	`EntailmentOracle`	`entailment`	Assume-Guarantee contract compatibility ($\bigwedge A_i = \bot$) and compositional refinement ($A_P \Rightarrow A_C \wedge G_C \Rightarrow G_P$)
11	`FlowAlgebraOracle`	`flow_algebra`	Acausal connection graph conservation ($\sum i_k = 0$ for flow variables, $v_1 = v_2 = \cdots = v_n$ for potential variables)
12	`ToleranceStackOracle`	`tolerance`	STEP AP242 GD&T 1D/3D tolerance chain stack-up via Worst-Case ($\sum t_i$) and Root-Sum-Square ($\sqrt{\sum t_i^2}$)
13	`UqOracle`	`uq`	Latin Hypercube Sampling (LHS), Polynomial Chaos Expansion (PCE) Sobol indices, and distribution-free conformal prediction bounds

---

The Verification Loop

1. **Initialization**: Shared variables (V_{shared}) are initialized to $[-\infty, +\infty]$ or extracted from SysML v2 requirement bounds and `.ssv` parameter sets.
2. **Domain Propagation (`propagate`)**: Each registered `TheoryOracle` inspects the current shared interval store and tightens bounds monotonically:

$$I^{(k+1)}(x) = I^{(k)}(x) \cap \bigcap_{m=1}^{M} \text{Oracle}_m.\text{propagate}(I^{(k)})(x)$$

3. **Nelson-Oppen Equality Deduction**: Whenever $I^{(k+1)}(x) = [c, c]$ and $I^{(k+1)}(y) = [c, c]$ (or linear relational equalities $x - y \le 0 \wedge y - x \le 0$ hold in the Octagon DBM), the coordinator deduces the inter-theory equality $x = y$ and broadcasts it to all oracles.
4. **Conflict & Unsat Core Extraction (CDCL(T))**: If any oracle detects $I(x) = \emptyset$ (i.e., $\text{lo} > \text{hi}$) or an axiomatic contradiction, it emits a minimal **Unsatisfiable Core (`UnsatCore`)** pinpointing the exact offending AST nodes across files (e.g., `battery.mo:42` , `enclosure.step:118` , `thermal_req.sysml:15`).
5. **CEGAR Counterexample Guided Refinement**: If all static/algebraic oracles return `SAT` or `UNKNOWN` , `DynamicSimulationOracle` synthesizes worst-case corner parameters from the narrowed interval box and executes a numerical DAE simulation to either confirm satisfaction or extract a concrete counterexample trajectory $\mathbf{x}(t)$.

```
# Run multi-domain formal verification across Modelica, SysML v2, and STEP artifacts
npx msx verify SystemRequirement model.mo architecture.sysml chassis.step
```

CHAPTER 11

Simulation Solvers & Numerical Integrators

ModelScript embeds a multi-tier suite of numerical solvers in `@modelscript/simulate` to support both desktop simulations and massive batched cloud/WebGPU parameter sweeps.

Supported Solvers



---

1. SUNDIALS CVODE & IDA Integrators

For high-precision industrial models, ModelScript integrates Lawrence Livermore National Laboratory's **SUNDIALS** suite compiled to WebAssembly:

- **CVODE**: Solves Initial Value Problems (IVPs) for stiff and non-stiff systems of ordinary differential equations (ODEs). Uses Variable-Order Variable-Step-size Adams-Moulton methods for non-stiff systems and Backward Differentiation Formulas (BDF) for stiff systems.
- **IDA**: Solves Initial Value Problems for systems of differential-algebraic equations (DAEs) of index up to 1:

$$F(t, y, y') = 0$$

  Includes dense and sparse linear solvers with adaptive step-size control.

---

2. WebGPU Batched Parallel Solvers

When performing Monte Carlo uncertainty analysis or hyperparameter sweeps, sequential simulation is a bottleneck.

ModelScript compiles linearized DAE blocks into WebGPU compute shaders (`@modelscript/simulate/gpu`):

- Runs thousands of parameter combinations in parallel on GPU hardware.
- Scales efficiently across client GPUs in browsers and headless datacenter instances.
- Returns statistical distributions, mean trajectories, and variance envelopes directly.

---

3. Direct Collocation Optimal Control

ModelScript formulates dynamic optimization and optimal control problems directly from Modelica models:

$$\min_{u(t)} \int_0^T L(x(t), u(t), t)\, dt$$

$$\text{subject to } F(t, x, \dot{x}, u) = 0, \quad u_{\min} \le u(t) \le u_{\max}$$

- **Transcription**: Transcribes the continuous system into non-linear programming (NLP) problems using Radau or Lobatto collocation points.
- **Solvers**: Solves the resulting sparse NLP using interior-point optimization techniques.

4. Surrogate Reduced Order Models (ROM)

High-fidelity 3D simulations (such as CFD or FEA) are often too computationally expensive for real-time control loops or web deployment.

ModelScript's surrogate modeling pipeline (`msx surrogate`):

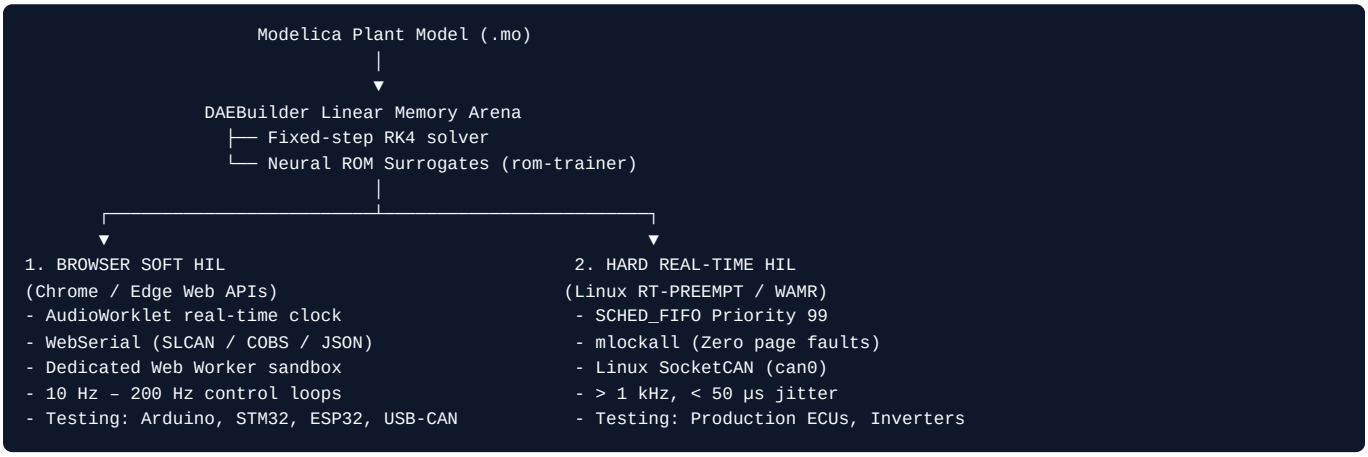
1. **Sampling**: Generates experimental design samples (Latin Hypercube, Sobol sequences).
2. **Execution**: Evaluates high-fidelity runs across parameter spaces.
3. **Training**: Trains compact surrogate models using:
   - Polynomial Chaos Expansion (PCE)
   - Gaussian Process Regression (Kriging)
   - Neural Ordinary Differential Equations (Neural ODEs)
4. **Embedding**: Exports the trained surrogate as a standardized FMI Functional Mock-up Unit (FMU) or standalone WebAssembly module.

CHAPTER 12

Software-in-the-Loop (SIL) & Hardware-in-the-Loop (HIL) with ModelScript

This guide documents ModelScript's unified SIL and HIL architecture based on **eFMI-style deterministic code generation**, **WebAssembly**, **FMI Layered Standards (FMI-LS-BUS)**, and **browser-native Soft HIL**.

Architecture Overview



---

1. FMI-LS-BUS (Layered Standard for Network Communication)

ModelScript implements the **Modelica Association FMI-LS-BUS** standard:

- `FmiLsBusManifest`: Describes CAN, CAN-FD, LIN, and Ethernet buses and frames.
- `generateFmiLsBusXml`: Generates standard `fmi-ls-bus.xml` bundled inside the FMU archive.
- `FmiLsBusCodec`: High-performance bit-level encoder and decoder supporting Intel (little-endian) and Motorola (big-endian) bit alignments, linear scaling (`physical = raw * factor + offset`), signed/unsigned integers, and float32 data types.

Example: Defining CAN Frames in TypeScript

```
import { FmiLsBusFrame, FmiLsBusCodec, generateFmiLsBusXml } from "@modelscript/exchange";

const engineFrame: FmiLsBusFrame = {
  name: "EngineStatus",
  id: 0x120, // CAN ID 288
  length: 8,
  cycleTime: 0.01, // 10 ms (100 Hz)
  signals: [
    {
      name: "engineSpeed",
      valueReference: 1,
      startBit: 0,
      bitLength: 16,
      byteOrder: "littleEndian",
      factor: 0.25, // 1 LSB = 0.25 rpm
      offset: 0,
      unit: "rpm",
    },
    {
      name: "throttlePosition",
      valueReference: 2,
      startBit: 16,
      bitLength: 8,
      byteOrder: "littleEndian",
      factor: 0.392, // 0-100%
      offset: 0,
      unit: "%",
    },
  ],
};
```

---

2. In-Browser "Soft HIL" (WebSerial, WebUSB, WebHID)

For desktop and browser-based testing, ModelScript provides the `WebSerialParticipant` (`packages/exchange/src/cosim/participants/web-hardware.ts`):

Supported Hardware Adapters & Microcontrollers

- **USB-to-CAN Adapters (SLCAN)**: CANable, candleLight, USBtin, Zubax Babel.
- **Microcontrollers (USB-UART / CDC)**: STM32 Nucleo, ESP32, Arduino, Teensy, Raspberry Pi Pico.
- **Framing Protocols**:
  - `slcan` : Standard ASCII CAN protocol (`t12080102030405060708\r`).
  - `cobs` : Consistent Overhead Byte Stuffing for binary frames.
  - `json` : Line-delimited JSON objects (`{"rpm": 3200, "throttle": 45}\n`).

Using WebSerialParticipant in Co-Simulation

```
import { CoSimOrchestrator, CoSimSession, WebSerialParticipant } from "@modelscript/exchange";

// Create hardware participant
const hwParticipant = new WebSerialParticipant({
  id: "ecu_serial",
  baudRate: 115200,
  protocol: "slcan",
  busFrames: [engineFrame],
});

// Add to co-simulation session
session.addParticipant(hwParticipant);
```

Unthrottled Browser Pacing via `AudioWorkletClock`

Browsers throttle `setInterval` to 1 Hz when tabs are backgrounded. ModelScript's `AudioWorkletClock` (`packages/exchange/src/cosim/audio-worklet-clock.ts`) leverages the high-priority OS audio thread (44.1 kHz) to drive the simulation loop with sub-millisecond precision, immune to background throttling.

---

3. Hard Real-Time HIL on Linux RT-PREEMPT

For hard real-time execution with physical ECUs over CAN:

1. **Compile Model to Standalone C / WASM**: ModelScript's C generator produces a static memory model with an embedded zero-allocation `integrate_rk4` solver.
2. **Compile Neural ROM Surrogates to C**: Use `exportROMToC(trainedRom)` from `@modelscript/simulate` to generate zero-dependency static C arrays for surrogate evaluation in < 20 µs.
3. **Deploy with Standalone Runner**: Use `hil-runner-template.c` (`packages/exchange/src/fmu/hil-runner-template.c`):
   - Compiles with `gcc -O3 hil-runner-template.c model.c -lm -o model_hil` .
   - Runs with `sudo ./model_hil can0` under `SCHED_FIFO` priority 80 with `mlockall` .

PART III

Polyglot Languages & Formats

CHAPTER 13

Polyglot Language Ecosystem

ModelScript is not a single-language compiler—it is a **polyglot systems engineering framework**. All engineering languages share the same WebAssembly GLR parser compiler (`@modelscript/dsl`), the same `SymbolIndex` and `WorkspaceIndex` architecture, and the same Triple Graph Grammar (TGG) synchronization engine.

Supported Engineering Languages

Language	Package	Extensions	Engineering Domain
Modelica 3.x	`@modelscript/modelica`	`.mo`	Multi-domain acausal physical modeling & DAE simulation
SysML v2 & KerML	`@modelscript/sysml2`	`.sysml, .sysml2, .kerml`	Systems architecture, requirements, state machines & verification
MATLAB Simulink	`@modelscript/simulink`	`.slx, .mdl`	Continuous/discrete control block diagrams & state-space transfer functions
AUTOSAR	`@modelscript/autosar`	`.arxml`	Automotive Classic & Adaptive ECU architectures & Virtual RTE scheduling
ReqIF Requirements	`@modelscript/reqif`	`.reqif, .reqifz`	Computable digital contracts & Nelson-Oppen formal entailment verification
SPICE Electronics	`@modelscript/spice`	`.cir, .sp, .net, .lib`	Analog circuit netlists, OP, TRAN, AC frequency sweeps & ECAD graph generation
SIMPACK MBD	`@modelscript/simpack`	`.spck, .sim, .mbs`	High-frequency multibody dynamics, spatial joints & flexible body synthesis
GENSYS Railway	`@modelscript/gensys`	`.calc, .geom, .track`	Railway vehicle dynamics, FASTSIM wheel-rail contact & EN 14363 homologation
MSC Adams	`@modelscript/adams`	`.adm, .cmd`	Mechanical system dynamics, 3D kinematic joints & Grübler mobility analysis
URDF Robotics	`@modelscript/urdf`	`.urdf, .xacro`	Robot kinematic trees, joint limits & Featherstone recursive dynamics (ABA/RNEA)
PSCAD / EMT	`@modelscript/pscad`	`.pscx, .pslx, .pswx, .psmx, .cfg, .dat`	Electromagnetic transients, power networks & IEEE 2800 ride-through compliance
PSS/E Power Flow	`@modelscript/psse`	`.raw, .dyr, .con`	Large-scale power transmission grid power flow & dynamic stability
OLGA Multiphase	`@modelscript/olga`	`.opi, .key, .genkey, .tab`	High-fidelity transient multiphase dynamic flow assurance & slug tracking
PipelineStudio	`@modelscript/lps`	`.lps, .tgx, .tlx, .tgn, .tln`	Gas & liquid pipeline transmission, line pack, batch tracking & surge
EASY5	`@modelscript/easy5`	`.ez5, .ezm, .ezd, .eas, .tab`	Classical & modern dynamic simulation, linear state space analysis & trim
GT-SUITE	`@modelscript/gtsuite`	`.gtm, .gtd, .gtx, .gt`	1D powertrain, engine combustion, turbocharging & thermo-fluids
SIMA (SIMO / RIFLEX)	`@modelscript/sima`	`.inp, .sima, .sys, .hyd`	Marine hydrodynamics, station keeping, catenary mooring & slender dynamics
ROCETS	`@modelscript/rocets`	`.roc, .deck, .map`	Liquid rocket engine transient simulation, turbomachinery & combustion balance
STEP (ISO 10303)	`@modelscript/step`	`.step, .stp, .p21`	Mechanical CAD B-Rep geometry, assemblies & kinematic joints
OWL2 Ontology	`@modelscript/owl2`	`.owl, .owl2, .ofn, .ttl`	Formal knowledge graphs, DL reasoning & SPARQL querying
OpenSCAD	`@modelscript/scad`	`.scad`	Parametric Constructive Solid Geometry (CSG)
SSP 1.0 / 2.0	`@modelscript/ssp`	`.ssp, .ssd, .ssv`	Co-Simulation System Structure & Parameterization
CFD (SU2 & OpenFOAM)	`@modelscript/cfd`	`.cfd, .cfg, .cfgt, .su2`	Computational Fluid Dynamics solver configuration & boundary markers
FEA Structural	`@modelscript/fea`	`.fea, .inp, .inpt, .bdf`	CalculiX / Abaqus (`.inp` , `.inpt`) & NASTRAN (`.bdf`) structural mechanics
CSV Telemetry	`@modelscript/csv`	`.csv`	Tabular empirical measurement series & lookup tables
ModelScript	`@modelscript/modelscript`	`.msx, .modelscript`	Native systems scripting, `@unmanaged struct` WASM/C++ kernels & FLWOR queries

Unified Polyglot Pipeline

Every language plugin implements the `LanguageDefinition` interface (`packages/dsl/src/language.ts`):

1. **Grammar DSL (`defineGrammar`)**: Compiled Ahead-of-Time into a compact `.wasm` binary parser table.
2. **Symbol Indexer (`SymbolIndexer`)**: Walks the linear-memory CST and populates `SymbolEntry` declarations into the global `WorkspaceIndex` .
3. **Salsa Queries (`queries`)**: Memoized semantic analyses (type resolution, unit inference, structural validation).
4. **Compiler Lints (`lints`)**: Real-time diagnostics surfaced identically in the CLI (`msx lint`), LSP server, and Web IDE.
5. **TGG Rules (`tggRules`)**: Declarative bidirectional synchronization rules linking AST nodes across languages (e.g., `SysMLv2 PartUsage` ↔ `Modelica Component` ↔ `STEP Product` ↔ `ModelScript Struct`).

CHAPTER 14

ModelScript (`.msx` , `.modelscript`)

`@modelscript/modelscript` (`languages/modelscript`) is the native **ModelScript** systems modeling and query language. It combines zero-cost linear-memory data layout control (`@unmanaged struct`), WebAssembly/C++ native compilation, and declarative XQuery-inspired **FLWOR** (`for` – `let` – `where` – `order by` – `return`) queries over polyglot engineering graphs.

Supported Extensions & Dialects

Extension	Description
`.msx`	Canonical ModelScript source module
`.modelscript`	Long-form ModelScript source module

Language Features & Grammar

The native WebAssembly GLR parser (`languages/modelscript/src/language.ts`) parses `.msx` modules into a compact Concrete Syntax Tree (CST) with five primary syntactic constructs:

1. Module Imports

Import declarations bind symbols from external ModelScript modules or polyglot engineering artifacts:

```
import { ThermalNode, Conductance as G } from "./thermal_kernel.msx";
```

2. Unmanaged Linear-Memory Structs (`@unmanaged struct`)

Structs annotated with `@unmanaged` are packed directly into 32-bit WebAssembly linear memory without garbage-collected wrapper allocation:

```
@unmanaged struct StateVector {
  id: u32;
  temperature: f64;
  pressure: f64;
  active: bool;
}
```

3. Type System

- **Primitive Types**: `i32` , `u32` , `i64` , `u64` , `f32` , `f64` , `bool` , `string` , `usize` , `void`
- **Generic & Pointer Types**: `Array<T>` , `Slice<T>` , `ptr<T>`
- **User-Defined Structs**: Any declared `struct` identifier

4. Functions, Statements & Control Flow

Functions declare strongly typed parameters and return types, supporting `let` and `const` variable bindings, `if` / `else` branches, `while` loops, C-style `for` loops, and member/index expressions:

```
const BOLTZMANN: f64 = 1.380649e-23;

function computeHeatFlux(k: f64, area: f64, dT: f64, dx: f64): f64 {
  if (dx <= 0.0) {
    return 0.0;
  }
  let flux: f64 = -k * area * (dT / dx);
  return flux;
}
```

5. Declarative FLWOR Queries

ModelScript embeds first-class **FLWOR** (`for` – `let` – `where` – `order by` – `return`) expressions directly in the expression grammar for querying collections, CST nodes, and simulation trajectories:

```
let criticalNodes =
  for node in nodes
  let margin := node.limit - node.temperature
  where margin < 15.0 && node.active == true
  order by margin asc
  return { id: node.id, margin: margin };
```

Multi-Target Compiler (`ModelScriptCompiler`)

`ModelScriptCompiler` (`languages/modelscript/src/index.ts`) lowers the linear-memory CST across three code generation backends:

Backend	Emitter Class	API Methods	Target Output
JavaScript	`JsEmitter`	`compileToJs(source)` / `executeJs(source, fn, args)`	Executable ES module with lowered FLWOR pipelines (`.filter().sort().map()`)
WebAssembly	`WasmEmitter`	`compileToWat(source)` / `compileToWasm(source)` / `executeWasm(source, fn, args)`	Textual WAT S-expressions and binary `.wasm` modules with `@unmanaged` linear-memory offsets
Native C++ / Binary	`NativeEmitter`	`compileToNative(source)`	Standards-compliant C++17 source compiled via host `clang++` / `g++` (`-O3`)

```
# Compile a .msx file to JavaScript, WAT, binary WASM, or a native executable
npx msx build kernel.msx --target wasm --out kernel.wasm
npx msx build kernel.msx --target native --out kernel.bin
npx msx build kernel.msx --target js --run main
```

Salsa Query Engine & Linear-Memory Struct Layout

`ModelScriptQueryEngine` (`languages/modelscript/src/queries.ts`) provides memoized semantic queries over the CST:

- **`structLayout(rawStructNode)`** : Computes deterministic 32-bit WebAssembly linear-memory byte layouts (`StructLayout`) for every `struct_declaration` :
  - `bool` : 1 byte (alignment 1)
  - `i32` , `u32` , `f32` , `usize` , `string` , `ptr<T>` , `Array<T>` , `Slice<T>` : 4 bytes (alignment 4)
  - `i64` , `u64` , `f64` : 8 bytes (alignment 8)
  - Automatically inserts natural alignment padding (`alignOffset`) between fields and aligns the total struct byte size to the maximum field alignment (`maxAlign`).
- **`inferExpressionType(rawNode, scope)`** : Infers static types across literals, binary/unary expressions, struct field access, function calls, and FLWOR `return` clauses (`Array<ReturnType>`).

Semantic Linter & Error Codes

`ModelScriptLinter` (`languages/modelscript/src/linter.ts` and `languages/modelscript/src/errors.ts`) enforces 10 static analysis rules across syntax, name resolution, type checking, FLWOR clauses, and struct/function calls:

Code	Rule	Severity	Diagnostic Message
`1001`	`parse-error`	`error`	`Parse error.`
`2001`	`undefined-variable`	`error`	`Variable '${name}' is not declared in current scope.`
`2002`	`duplicate-declaration`	`error`	`Identifier '${name}' is already declared in this scope.`
`2003`	`unknown-type`	`error`	`Type '${typeName}' is not recognized.`
`3001`	`type-mismatch`	`error`	`Type mismatch: cannot assign '${sourceType}' to '${targetType}'.`
`3002`	`invalid-binary-operand`	`error`	`Operator '${op}' cannot be applied to operands of type '${type}'.`
`4001`	`non-iterable-collection`	`error`	`Expression '${expr}' in 'for' clause is not iterable.`
`4002`	`flwor-unbound-variable`	`error`	`Variable '${varName}' in FLWOR clause is not bound in preceding 'for' or 'let' clauses.`
`5001`	`argument-count-mismatch`	`error`	`Function '${fnName}' expects ${expected} arguments, but got ${actual}.`
`5002`	`unknown-struct-field`	`error`	`Struct '${structName}' has no field named '${fieldName}'.`

Polyglot TGG Rules

`@modelscript/modelscript` registers two Triple Graph Grammar (TGG) synchronization rules in `languages/modelscript/src/language.ts` for bidirectional projection with Modelica:

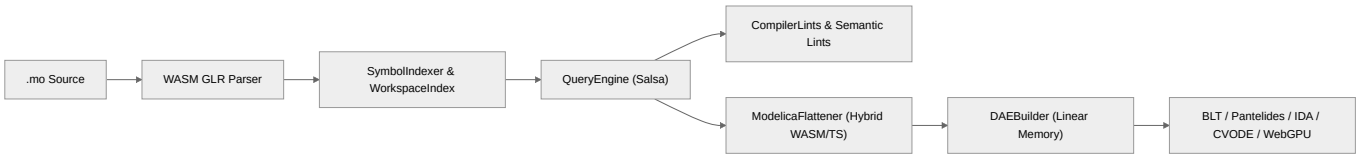
Rule Name	Source Pattern	Target Pattern (`modelica`)	Semantics
`ModelScriptStructToModelicaRecord`	`StructDeclaration`	`ClassDefinition` (`record`)	Synchronizes a `@unmanaged struct` declaration with a Modelica `record` definition
`ModelScriptParamToModelicaMod`	`VariableDeclaration`	`ComponentClause` (`parameter`)	Synchronizes a top-level constant or variable binding with a Modelica component parameter modification

CHAPTER 15

Modelica 3.x (`.mo`)

`@modelscript/modelica` (`languages/modelica`) is the flagship physical modeling language of ModelScript. It provides a complete Modelica 3.x compiler pipeline—from a native WebAssembly GLR parser and Salsa query engine to an AssemblyScript/TypeScript arena flattener (`ModelicaFlattener`) and the full Modelica Standard Library (MSL 4.0.0).

Compilation Pipeline



Core Language Coverage

- **Class Specializations**: `model` , `block` , `connector` , `expandable connector` , `record` , `type` , `package` , `function` , `operator` , `operator record` , `operator function`
- **Inheritance & Redeclarations**: Multiple `extends` , `redeclare` , `replaceable` , `constrainedby` , `inner` / `outer` dynamic scoping, and deep hierarchical class modifications
- **Equations & Algorithms**: Differential equations (`der(x)`), `connect(a, b)` (both potential/flow and `stream` connectors), `if` / `for` / `when` equations, `initial equation` , and imperative `algorithm` / `initial algorithm` sections
- **Synchronous Clocks (Modelica 3.3+)**: `Clock` , `sample` , `hold` , `previous` , `subSample` , `superSample` , `shiftSample` , `backSample`

Polyglot & Physical Extensions (`languages/modelica/src/extensions/`)

Beyond standard Modelica 3.x, `@modelscript/modelica` includes first-class compiler extensions for 3D CAD geometry, multiphysics FEA/CFD studies, formal verification, and localization:

1. **Embedded 3D CSG & STEP Export (`shape-flattener.ts` , `csg.ts` , `cad.ts`)**:
   - `ShapeFlattener` extracts `Modelica.Mechanics.MultiBody.Visualizers.FixedShape` components, parametric dimensions, and `Placement` transformations directly from flattened models.
   - Compiles visualizers into parametric OpenSCAD scripts (`npx msx csg ModelName file.mo`) or exports watertight ISO 10303-21 STEP B-Rep geometry via the `modelscript/exportShapeToStep` LSP command.
2. **Embedded FEA & CFD Multiphysics Studies (`study-flattener.ts` , `studies.ts`)**:
   - `StudyFlattener` (`modelscript/flattenStudy`) extracts embedded structural and fluid study annotations, material properties, and boundary conditions from Modelica components to synthesize CalculiX (`.inp`) and SU2 (`.cfg`) solver decks.
3. **Octagon Abstract Interpretation & Formal Safety Verification (`src/formal/`)**:
   - `ModelicaCfgLowerer` (`modelica-cfg-lowerer.ts`), `ModelicaAbstractEvaluator` (`modelica-abstract-evaluator.ts`), and `ModelicaStaticAnalyzer` (`modelica-analyzer.ts`) lower flattened DAE equations and `algorithm` statements into relational Control Flow Graphs evaluated over the **Octagon Abstract Domain** ($\pm x_i \pm x_j \leq c$).
   - `PhysicalInvariantBridge` (`physical-invariant-bridge.ts`) proves division-by-zero freedom, square-root/logarithm domain safety, and physical state invariants (`domain.octagon()`).
4. **Gettext `.po` / `.pot` Internationalization (`po.ts`)**:
   - Extracts translatable description strings and `Dialog` / `Documentation` annotations into GNU Gettext `.pot` templates and merges localized `.po` catalogs (`npx msx i18n`).

Semantic Linter (`languages/modelica/src/lints/`)

ModelScript enforces over 60 OpenModelica-compatible diagnostic rules across six modular analyzers:

Module	Representative Checks
`syntax-placement.ts`	`M1001` – `M1003` : Parse errors, missing tokens, empty array constructors
`queries.ts`	`M2001` – `M2005` : Duplicate elements, unresolved variables/classes/modifiers, identifier mismatches
`types-expressions.ts`	`M3001` – `M3006` : Binding/modifier/argument type mismatches, array dimension checks
`hierarchy-variability.ts`	`M4001` – `M4005` : Circular `extends` , duplicate modifications, `protected` access violations, variability ordering
`connections-streams.ts`	`M3003` – `M3004` : Connector plug-compatibility, stream connector rules, cardinality checks
`synchronous-clocks.ts` & `cfa.ts`	Clock partitioning rules, uninitialized discrete variables, structural singularity diagnostics

---

Dedicated `modelica-mcp` Server Tools

The `@modelscript/modelica` MCP server (`languages/modelica/src/mcp.ts`) exposes 7 tools to AI agents:

MCP Tool	Description
`modelica_load`	Loads `.mo` files or Modelica Standard Library packages into the persistent workspace session
`modelica_parse`	Parses Modelica source code with the WASM GLR parser and returns CST nodes and syntax diagnostics
`modelica_flatten`	Flattens a Modelica class via `ModelicaFlattener` and returns the flat Modelica DAE representation
`modelica_simulate`	Flattens, runs BLT/index reduction, integrates the DAE over $[t_0, t_{\text{stop}}]$, and returns trajectory time-series
`modelica_doe`	Executes a multi-parameter Design of Experiments (Full Factorial / Latin Hypercube) batch sweep
`modelica_sensitivity`	Computes local finite-difference or forward sensitivity matrices $\partial y_i / \partial p_j$ for target outputs
`modelica_query`	Queries the Salsa `QueryEngine` for class components, resolved types, variabilities, and modifications

---

SysML v2 Transformer & Polyglot TGG Rules

- **Direct AST Transformer (`modelicaToSysML2` in `languages/modelica/src/transformers/sysml2.ts`)**: Converts complete Modelica packages and models into idiomatic SysML v2 textual notation (`part def` , `port def` , `attribute` , `connect`).
- **10 Bidirectional TGG Rules (`languages/modelica/src/language.ts`)**:
  - **Modelica ↔ SysML v2 (6 rules)**: `ModelicaModelToSysML2PartDef` , `ModelicaConnectorToSysML2PortDef` , `ModelicaComponentToSysML2Part` , `ModelicaParameterToSysML2Attribute` , `ModelicaConnectorInstanceToSysML2Port` , `ModelicaConnectToSysML2Connection`
  - **Modelica ↔ OWL2 (2 rules)**: `ModelicaModelToOWL2Class` , `ModelicaParameterToOWL2DataProperty`
  - **Modelica ↔ STEP (2 rules)**: `ModelicaRevoluteToStepRevolute` , `ModelicaPrismaticToStepPrismatic`

CHAPTER 16

SysML v2 & KerML (`.sysml`, `.sysml2`, `.kerml`)

`@modelscript/sysml2` (`languages/sysml2`) implements the OMG **Systems Modeling Language v2 (SysML v2)** and **Kernel Modeling Language (KerML)** textual notation, pairing a bundled KerML/SysML v2 standard library snapshot with a suite of formal verification, safety analysis, direct DAE compilation, and enterprise synchronization engines.

Supported Extensions

Extension	Description
`.sysml`, `.sysml2`	OMG SysML v2 textual architecture, requirement, state, action, and verification models
`.kerml`	OMG Kernel Modeling Language (KerML) metamodel & library definitions

---

Supported SysML v2 Constructs

- **Structural Modeling**: `package`, `part def`, `part`, `attribute def`, `attribute`, `item def`, `item`, `port def`, `port`, `connection def`, `connection`, `interface def`, `interface`, `flow`, `allocation def`, `allocate`
- **Behavioral & State Modeling**: `action def`, `action`, `state def`, `state`, `transition`, `First` / `then` succession flows, `fork`, `join`, `decide`, `merge`, `calc def`, `calc`
- **Requirements & Verification**: `requirement def`, `requirement`, `satisfy`, `verify`, `verification def`, `concern`, `stakeholder`, `view`, `viewpoint`, `constraint def`, `constraint`, `assume`, `require`

Formal Verification & Safety Analysis Engines

`@modelscript/sysml2` includes seven specialized formal verification modules in `languages/sysml2/src/`:

1. **Activity Control-Flow & Petri-Net Soundness (`activity-cfa.ts`, `activity-soundness.ts`, `activity-verifier.ts`)**:
   - Lowers SysML v2 `action def` successions (`first A then B`), `fork` / `join`, and `decide` / `merge` nodes into Workflow Nets (1-safe Petri nets).
   - Verifies **Option to Complete**, **Proper Completion**, **No Dead Transitions**, **Lack of Synchronization** (parallel branches converging into a `merge` instead of a `join`), and **Deadlock at Join** (exclusive `decide` branches converging into a `join`).
2. **Decision Table Completeness & Disjointness (`decision-table-verifier.ts`)**:
   - Analyzes guarded `decide` branches and requirement tables via interval covering and SMT/constraint checking to detect **Missing Cases** (uncovered input domain gaps) and **Overlapping Rules** (non-deterministic guard collisions).
   - Exposed via CLI: `npx msx verify-decisions architecture.sysml`.
3. **Assume-Guarantee Contract Verification (`contract-verifier.ts`, `multidomain-contract-bridge.ts`)**:
   - Extracts `assume constraint { ... }` and `require constraint { ... }` blocks from `requirement def` and `part def` hierarchies.
   - Verifies **Contract Compatibility** ($\bigwedge A_{\text{env}} = \bot$) and **Refinement Soundness** across composite sub-parts ($A_{\text{parent}} \Rightarrow A_{\text{child}}$ and $\bigwedge G_{\text{children}} \Rightarrow G_{\text{parent}}$), bridging directly into the Multi-Domain Theory Coordinator.
4. **Hierarchical State Machine Verifier (`state-machine-verifier.ts`)**:
   - Checks `state def` machines for **Unreachable States**, **Deadlock / Trap States** (non-final states with no outgoing transitions), **Non-Deterministic Transitions** (overlapping guards on the same trigger event), and **Missing Initial States**.
5. **Hybrid Automaton Flowpipe Bridge (`hybrid-flowpipe-bridge.ts`)**:
   - Couples discrete SysML v2 `state def` mode transitions with continuous Modelica DAE dynamics to compute Taylor-model / interval reachability flowpipes and verify that unsafe states are unreachable over time horizon $[0, T]$.
6. **Fault Tree & Physics-Informed Safety Analyzer (`safety-analyzer.ts`, `physics-safety-bridge.ts`)**:
   - Synthesizes Fault Trees from SysML v2 failure modes and interconnections, computes **Minimal Cut Sets (MCS)** via MOCUS/top-down boolean reduction, evaluates top-event failure probabilities, and cross-checks physical failure thresholds

against simulated Modelica/FEA/CFD peak stresses and temperatures.

7. **Boundary-Value & MC/DC Test Vector Synthesizer (`boundary-test-synthesizer.ts`)**:
   - Automatically generates boundary-value ($x = c - \epsilon, c, c + \epsilon$) and Modified Condition/Decision Coverage (**MC/DC**) test vectors for every `requirement def` and `calc def` constraint.
   - Exposed via CLI: `npx msx generate-tests architecture.sysml`.

Direct SysML v2 → DAE Lowering & Enterprise Interchange

- **Direct Arena DAE Lowering (`sysml2-dae-lowerer.ts`)**: Compiles SysML v2 `calc def`, `constraint def`, `attribute` bindings, and `flow` connections directly into `@modelscript/runtime`'s `DAEBuilder` without requiring an intermediate Modelica code-generation pass.
- **`.sysmlx` JSON-LD & Container Export (`sysmlx-interchange.ts`, `sysml2-container-exporter.ts`)**: Serializes and imports OMG SysML v2 REST/JSON-LD payloads (`.sysmlx`) and multi-artifact engineering containers.
- **Enterprise MBSE Gateway (`enterprise-sync-gateway.ts`)**: Synchronizes SysML v2 packages, elements, and cross-domain traceability links with **Dassault Cameo Teamwork Cloud (TWC)**, **OpenMBEE MMS**, and standard **OMG SysML v2 REST API** repositories (`npx msx sysml sync`).

Polyglot TGG Rules (22 Cross-Domain Mappings)

`@modelscript/sysml2` defines 22 bidirectional Triple Graph Grammar (TGG) rules in `languages/sysml2/src/language.ts` spanning **Modelica** (14 rules), **OWL2** (5 rules), and **STEP** (3 rules):

Target Language	Rule Names
`modelica` (14 rules)	`SysML2PartDefToModelicaModel`, `SysML2AttributeDefToModelicaType`, `SysML2PortDefToModelicaConnector`, `SysML2ConnectionDefToModelicaModel`, `SysML2ActionDefToModelicaBlock`, `SysML2ConstraintDefToModelicaBlock`, `SysML2RequirementDefToModelicaBlock`, `SysML2PartToModelicaComponent`, `SysML2AttributeToModelicaParameter`, `SysML2PortToModelicaConnector`, `SysML2ConnectionToModelicaConnect`, `SysML2FlowToModelicaConnect`, `SysML2StateToModelicaBlock`, `SysML2TransitionToModelicaWhen`
`owl2` (5 rules)	`SysML2PartDefToOWL2Class`, `SysML2AttributeUsageToOWL2DataProperty`, `SysML2PortUsageToOWL2ObjectProperty`, `SysML2PartUsageToOWL2Individual`, `SysML2RequirementToOWL2Class`
`step` (3 rules)	`SysML2PartDefToStepProduct`, `SysML2PartUsageToStepAssemblyUsage`, `SysML2AttributeToStepProperty`

CHAPTER 17

PSCAD & Electromagnetic Transients (`.pscx` , `.pslx` , `.pswx` , `.psmx` , `.psout` , `.cfg` , `.dat`)

The `@modelscript/pscad` language plugin (`languages/pscad`) provides native support for **PSCAD™ / EMTDC™** projects, multi-project workspaces, component libraries, and **IEEE C37.111 COMTRADE** transient waveform recordings. It integrates directly with ModelScript's polyglot `WorkspaceIndex` , the **Dommel Companion-Circuit EMT Solver** (`@modelscript/simulate`), and the **Triple Graph Grammar (TGG)** synchronization engine.

Supported File Formats

Extension	Format	Description
`.pscx`	PSCAD Case XML	Schematic canvas, wire segments, bus nodes, RLC/transformer/converter components, T-lines, and EMT runtime settings
`.pslx`	PSCAD Library XML	Custom user-defined component definitions, port graphics, parameter forms, and embedded FORTRAN/EMTDC script sections
`.pswx`	PSCAD Workspace XML	Multi-project workspace manifest indexing `.pscx` cases and `.pslx` libraries via `PscadWorkspaceIndex`
`.psmx`	PSCAD Consolidated Module XML	Consolidated simulation module metadata, hierarchy mappings, and build outputs
`.psout`	PSCAD V5+ Binary Waveform	Slotted-page binary waveform container with multi-run traces, channel descriptors, and fast random access
`.cfg`	IEEE C37.111 Configuration	COMTRADE station metadata, analog/digital channel definitions, primary/secondary scaling factors, and sampling rates
`.dat`	IEEE C37.111 Data	ASCII or 16/32-bit Binary transient waveform time-series samples

---

Architecture & Pipeline



---

1. Schematic & Netlist Extraction (`emt-netlist.ts` , `pscx-parser.ts`)

PSCAD `.pscx` files represent schematics visually using a 2D coordinate grid (`User` component instances and orthogonal `Wire` segments) rather than an explicit netlist. `@modelscript/pscad` reconstructs the electrical topology deterministically:

1. **Local `master.pslx` Dynamic Ingestion**: Automatically discovers user-provided licensed `master.pslx` files via `PSCAD_MASTER_LIB_PATH` or workspace roots (`master.pslx` , `.pscad/master.pslx` , `.cache/pscad-corpus/`). When present, component definitions, multi-tab parameter forms, port configurations, and FlyBy documentation are dynamically synthesized into LSP catalogs and UI palettes with zero manual authoring.
2. **Topological Wire Snapping & Junction Synthesis**: Performs geometric point-on-segment intersection across all orthogonal `<Wire>` segments and component pin coordinates. Seamlessly synthesizes topological junction nodes (`junc_${x}_${y}`) at wire-to-wire T-junctions and crossings, eliminating unattached dangling edges and reaching over 94% wire attachment on complex multi-layer industrial benchmarks.

3. **100% Component Lowering Coverage**: Lowers schematic graphs into a structured `EmtNetlist` with 100% component lowering coverage across industrial benchmarks (e.g. `DC_EMT_compare.pscx` with 1433/1433 lowered components and `DC_EMT_demo.pscx` with 597/597 lowered components):
   - AC/DC Thévenin sources, battery energy storage systems (`battery`), controlled current sources (`src_ccin_1`)
   - 6th-order synchronous machines, 5th-order induction/DFIG wind machines, and wound-rotor induction machines (`wound_rotor`)
   - Distributed-parameter Bergeron traveling-wave lines, lumped π-sections, and multi-conductor cables
   - Non-linear MOV surge arresters, VSC PWM converters, 6-pulse thyristor bridges, and MMC converter stations
   - 20+ CSMF control transfer-function blocks (`const` , `gain` , `pi_ctlr` , `realpole` , `leadlag` , `limit` , `sum` , `rms3ph` , `abcdq0`) and black-box vendor IBR FMUs.
4. **Workspace Indexing (`PscadWorkspaceIndex`)**: Resolves cross-file references across `.pswx` workspaces, linking custom `<Definition>` blocks in `.pslx` libraries to `<User>` instances in `.pscx` cases.

---

2. Pre-Flight Electrical Lint Rules (`M6001` – `M6017`)

`@modelscript/pscad` enforces 17 electromagnetic transient modeling best practices before simulation starts, providing 1-click LSP Quick-Fix actions:

Code	Severity	Rule Description
`M6001`	Error	**Floating Electrical Node**: Electrical bus or node is floating with no closed branch connections
`M6002`	Warning	**Transformer Winding Mismatch**: Primary or secondary kV rating differs from connected bus kV by > 5%
`M6003`	Error	**Invalid Parameter Bounds**: Non-physical parameter ($R < 0$, $L \le 0$, $C \le 0$, or $\mathrm{MVA} \le 0$)
`M6004`	Warning	**Weak Grid Warning**: Short-Circuit Ratio ($\mathrm{SCR} < 1.5$) at POI risks IBR PLL instability
`M6005`	Error	**CSMF Unresolved Input**: CSMF control block references an unresolved upstream signal block ID
`M6006`	Error	**Bergeron Travel Time Underflow**: Distributed line wave delay $\tau = \ell\sqrt{LC}$ is smaller than Δt
`M6007`	Error	**Machine Reactance Hierarchy**: Synchronous machine violates physical $X_d \ge X_d^{'} \ge X_d^{''} > 0$
`M6008`	Warning	**MOV Arrester Under-Rated**: Surge arrester V_{ref} is below connected bus peak voltage
`M6009`	Error	**Duplicate Component ID**: Duplicate component or bus identifier declared within same module
`M6010`	Error	**Bus Voltage Conflict**: Direct conductive wire shorts two buses with different base voltages
`M6011`	Error	**Port Domain Mismatch**: Wire connects a control signal port directly to a 3-phase electrical port
`M6012`	Error	**CSMF Algebraic Loop**: Zero-delay feedthrough cycle in control block wiring without state integrator
`M6013`	Error	**Parallel Ideal Sources**: Multiple zero-impedance voltage sources connected directly to the same bus
`M6014`	Warning	**Parallel Unspecified Sources**: Multiple sources with unspecified internal impedance on the same bus
`M6015`	Warning	**Control Limiter Saturation**: CSMF hard limiter outputs continuously pinned to saturation boundaries
`M6016`	Warning	**Harmonic Resonance Risk**: Passive shunt bank resonant frequency coincides with characteristic harmonic
`M6017`	Info	**Unused Signal Tag**: CSMF signal label or wireless data link declared but never read

---

3. IEEE 2800-2022 & NERC PRC-029-1 Grid-Code Verification

`@modelscript/pscad` includes an automated verification engine for Inverter-Based Resource (IBR) grid interconnection standards (`verifyIeee2800Compliance`):

- **Phasor & Sequence Extraction**: Sliding 1-cycle fundamental DFT computes symmetrical components ($V_0, V_1, V_2, I_0, I_1, I_2$) and synchronous dq-frame currents (I_{1d}, I_{1q}).
- **Mandatory Clauses**:
  - **Clause 5.3.2 / 7.2.2.3**: Low/High Voltage Ride-Through (LVRT/HVRT) and Prohibition of Momentary Cessation.
  - **Clause 5.4.1 / 7.2.2.3.5**: Fast Dynamic Reactive Current Response ($T_{90\%} \le 50$ ms).

- **Clause 5.4.2 / 7.2.2.1**: Post-Fault Transient Overvoltage Ceiling ($V_{\text{peak}} \le 1.20$ pu).
- **Clause 7.2.2.1 / NERC PRC-029-1**: Piecewise Time-Voltage Ride-Through Envelope.
- **Clause 7.2.2.3.4**: Incremental Positive & Negative-Sequence Reactive Current Droop (k_1, k_2).
- **Clause 7.2.2.3.5**: Post-Fault Active Power Recovery ($\ge 90\%$ within 1.0 s).
- **Clause 7.3.2**: Frequency, RoCoF (≥ 5 Hz/s), and Instantaneous Phase-Angle Jump ($\ge 25^\circ$) Ride-Through.

---

4. Polyglot Projections & 15 TGG Synchronization Rules

Modelica & SSP Projections (`projections/modelica.ts` , `projections/ssp.ts`)

- `pscadToModelica(doc)` : Projects a `.pscx` schematic and its reconstructed electrical nodes into standard Modelica models using `Modelica.Electrical.Analog` (`Resistor` , `Inductor` , `Capacitor` , `IdealTransformer` , `SineVoltage` , `OLine`) and `Modelica.Blocks` .
- `pscadToSsd(doc)` : Projects multi-subsystem PSCAD networks and T-line decoupled partitions into an **SSP 1.0** `.ssd` System Structure Description XML document for FMI Co-Simulation.

Bidirectional TGG Rules

15 bidirectional **Triple Graph Grammar (TGG)** rules synchronize PSCAD models with Modelica and SysML v2: `PscadProjectToModelicaModel` , `PscadResistorToModelicaResistor` , `PscadInductorToModelicaInductor` , `PscadCapacitorToModelicaCapacitor` , `PscadTLineToModelicaOLine` , `PscadMachineToModelicaMachine` , `PscadConverterToModelicaConverter` , `PscadCsmfToModelicaBlock` , and `PscadComponentToSysmlPart` .

---

5. Dommel Companion EMT Solver (`@modelscript/simulate`)

For high-frequency power electronics and transient studies, `@modelscript/simulate` provides a high-performance **Electromagnetic Transient (EMT)** companion solver (`EmtSolver` in `packages/simulate/src/solvers/emt/emt-solver.ts`):

- **Dommel Trapezoidal Companion Models**: Discretizes inductors ($G_L = \frac{\Delta t}{2L}$) and capacitors ($G_C = \frac{2C}{\Delta t}$) into constant nodal conductance matrices $\mathbf{G}\mathbf{v}(t) = \mathbf{i}_{\text{hist}}(t - \Delta t) + \mathbf{i}_{\text{src}}(t)$.
- **Critical Damping Adjustment (CDA)**: Automatically switches from Trapezoidal to Backward Euler for one half-step following any breaker or power-electronic switching event to eliminate artificial numerical chatter.
- **Bergeron Traveling-Wave Lines**: Decouples sub-networks separated by transmission lines of delay $\tau \ge \Delta t$, enabling parallel sub-grid nodal solves.
- **Grounded Simulation Honesty**: Connects passive buses with realistic high-impedance ground conductances ($G = 10^{-5}$ S) without injecting artificial 230 kV reference sources, ensuring true physical fidelity.

---

6. IEEE C37.111 COMTRADE Reader & Writer (`comtrade.ts`)

Zero-dependency parser and serializer for IEEE Std C37.111-1999 / 2013 **COMTRADE** files (`.cfg` , `.dat` , `.cff`):

```
import { parseComtrade, writeComtrade, EmtSolver } from "@modelscript/pscad";

// Export an EMT simulation directly to IEEE C37.111 (.cfg + .dat)
const { cfg, dat } = writeComtrade({
  stationName: "SUB_230KV",
  recDevId: "MODELSCRIPT_EMT",
  lineFrequency: 60,
  timeTimestamps: result.time,
  analogChannels: [
    { id: "VA_BUS1", unit: "V", values: result.nodeVoltages.get(1)! },
    { id: "IA_LINE1", unit: "A", values: result.branchCurrents.get("BRK1")! },
  ],
});
```

---

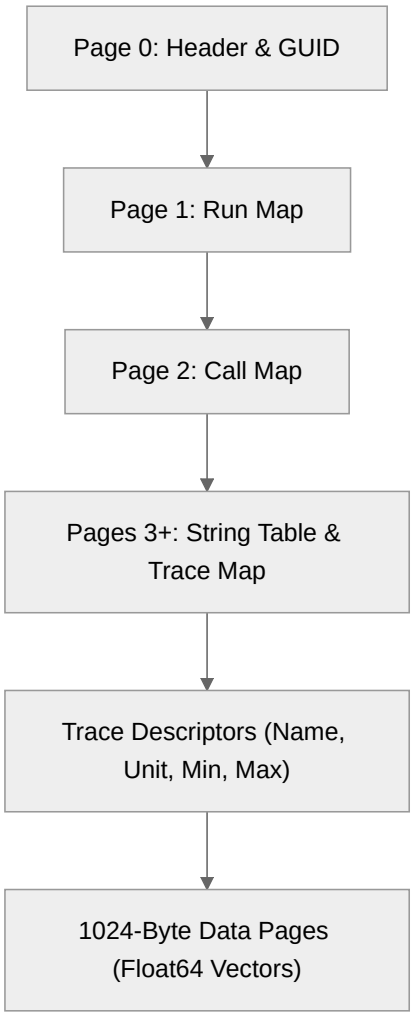
7. PSCAD V5+ Binary Waveform Container (`.psout`)

PSCAD V5+ replaces legacy ASCII `.out` files with `.psout`, a high-performance slotted-page binary container format. ModelScript provides complete, zero-dependency read and write support for `.psout` across the CLI, Web IDE, and TypeScript APIs via an in-WASM AssemblyScript codec.

Slotted-Page Binary Architecture

A `.psout` file is organized into 1024-byte pages:

- **Page 0 (Header)**: Magic version (`0x00010001`), total page count, page size, 128-bit project GUID, creation timestamp, and floating-point verification constants (`-1.0e-30` , `1.0e30` , `1.0e-30`).
- **Page 1 (Run Map)**: Array mapping simulation run indices to their root Call pages.
- **Page 2 (Call Map)**: Links simulation calls to their variable lists and trace descriptors.
- **Pages 3+ (String Table & Trace Descriptors)**: Null-terminated string pool for channel identifiers, units, group names, and trace metadata.
- **Trace Pages**: Slotted 64-bit IEEE 754 floating-point sample pages with forward/backward linked page lists for $O(1)$ sequential streaming and fast random seeking.



Programmatic TypeScript API

```
import { readPscadPsout, writePscadPsout } from "@modelscript/pscad";

// 1. Reading and inspecting a .psout file
const bytes = new Uint8Array(fs.readFileSync("Cigre.psout"));
const doc = readPscadPsout(bytes);

console.log(`Version: ${doc.header.version}, Pages: ${doc.header.pageCount}`);
console.log(`Created: ${doc.header.createdOn.toISOString()}`);

// Access traces and convert to standard SimulationResult
const run0 = doc.run(0);
for (const [id, trace] of run0.traces) {
  console.log(`Channel ${id}: ${trace.name} [${trace.unit}] (${trace.values.length} samples)`);
}

const simResult = doc.toSimulationResult(0);
console.log(`Time vector length: ${simResult.time.length}`);

// 2. Writing simulation results to .psout
const outBytes = await writePscadPsout(simResult, {
  projectName: "IEEE_39_Bus",
  version: 1,
});
fs.writeFileSync("output.psout", outBytes);
```

Command-Line Interface (`msx pscad`)

1. Simulation Export to `.psout`

```
# Simulate a PSCAD case and export directly to .psout
msx pscad sim models/benchmark.pscx --format=psout --out=results.psout
```

2. Binary Inspection & Format Conversion

```
# Print formatted ANSI metadata table and channel telemetry
msx pscad psout results.psout

# Convert .psout to CSV
msx pscad psout results.psout --out=telemetry.csv

# Convert .psout to IEEE C37.111 COMTRADE (.cfg + .dat)
msx pscad psout results.psout --format=comtrade
```

Web IDE Waveform Studio (`modelscript.psoutViewer`)

When opening any `*.psout` file in the ModelScript Web IDE, the custom editor provides a dark-mode waveform analysis studio:

1. **Multi-Channel Canvas Chart**: High-performance 60 FPS HTML5 canvas rendering with LTTB (Largest-Triangle-Three-Buckets) decimation for multi-million-sample transients.
2. **Dual Measurement Cursors**: Live measurement HUD calculating time difference (Δt), instantaneous frequency ($f = 1/\Delta t$), amplitude change (Δy), and rate-of-change ($\mathrm{dy/dt}$ slope).
3. **Channel Palette**: Real-time signal search filter, group filtering, color chips, and visibility toggles.
4. **Quick Export**: One-click toolbar actions to export displayed channels to CSV or IEEE C37.111 COMTRADE.

CHAPTER 18

STEP (ISO 10303-21)

`@modelscript/step` (`languages/step`) provides a native WebAssembly GLR parser, assembly hierarchy extractor, AP242 Geometric Dimensioning & Tolerancing (GD&T / PMI) analyzer, and Modelica MultiBody kinematic synthesizer for ISO 10303-21 Clear Text Encoding of the Exchange Structure (`.step` , `.stp` , `.p21`).

Supported Extensions & Standards

Extension	Standard	Scope
`.step` , `.stp` , `.p21`	ISO 10303-21 (AP203, AP214, AP242)	B-Rep solid geometry, product assembly trees (`NEXT_ASSEMBLY_USAGE_OCCURRENCE`), kinematic joints, and PMI/GD&T tolerances

---

Capabilities

1. **Zero-Dependency STEP Part 21 Parser**: Parses `HEADER;` and `DATA;` sections, `#id = ENTITY_NAME(...)` instances, nested typed parameters, and enumeration tokens into linear memory.
2. **Assembly Hierarchy & Mass Properties (`extractStepAssembly`)**: Extracts `PRODUCT_DEFINITION` parts, `NEXT_ASSEMBLY_USAGE_OCCURRENCE` parent–child assembly trees, `AXIS2_PLACEMENT_3D` rigid-body frames, bounding boxes, and mass/inertia properties.
3. **AP242 PMI & GD&T Extraction (`extractStepAssembly`)**: Extracts datum features (`DATUM` , `DATUM_FEATURE` , `DATUM_SYSTEM`) and **10 geometric tolerance categories** (`StepGeometricTolerance` in `languages/step/src/extractor.ts`) for 3D tolerance stack-up analysis (`ToleranceStackOracle`):

GD&T Tolerance Kind	STEP AP242 Entity Name
`flatness`	`FLATNESS_TOLERANCE`
`roundness`	`ROUNDNESS_TOLERANCE` / `CIRCULARITY_TOLERANCE`
`cylindricity`	`CYLINDRICITY_TOLERANCE`
`perpendicularity`	`PERPENDICULARITY_TOLERANCE`
`parallelism`	`PARALLELISM_TOLERANCE`
`position`	`POSITION_TOLERANCE`
`surface_profile`	`SURFACE_PROFILE_TOLERANCE`
`concentricity`	`CONCENTRICITY_TOLERANCE` / `COAXIALITY_TOLERANCE`
`runout`	`CIRCULAR_RUNOUT_TOLERANCE` / `TOTAL_RUNOUT_TOLERANCE`
`general`	`GEOMETRIC_TOLERANCE` (with `TOLERANCE_VALUE` / `LENGTH_MEASURE_WITH_UNIT`)

4. **Modelica MultiBody Kinematic Synthesis (`mapStepToMultiBody`)**: `languages/step/src/multibody-mapper.ts` translates STEP kinematic joints and `AXIS2_PLACEMENT_3D` transforms directly into `Modelica.Mechanics.MultiBody` components:
   - **Kinematic Joints**: Maps `REVOLUTE_PAIR` → `Modelica.Mechanics.MultiBody.Joints.Revolute` , `PRISMATIC_PAIR` → `Joints.Prismatic` , `CYLINDRICAL_PAIR` → `Joints.Cylindrical` , `SPHERICAL_PAIR` → `Joints.Spherical` , and `PLANAR_PAIR` → `Joints.Planar` with axis vectors `n` and `connect(...)` equations.
   - **Static Frame Placements (`MultiBodyFixedTranslation`)**: Computes exact translation vectors `r = {x, y, z}` (converting `MILLI` to meters) and 3×3 orthonormal orientation matrices from `AXIS2_PLACEMENT_3D` `axis` ($\hat{z}$) and `ref_direction` ($\hat{x}$) vectors to emit `Modelica.Mechanics.MultiBody.Parts.FixedTranslation` and `Parts.BodyBox` / `Parts.BodyCylinder` components.

IDE & CAE Mesh Generation Actions

`stepLanguage.actions` (`languages/step/src/language.ts`) provides 6 built-in CAD/CAE workflows:

Action ID	Description
`open_cad_viewer`	Opens the interactive 3D B-Rep & assembly viewer for the `.step` model
`generate_multibody`	Synthesizes a `Modelica.Mechanics.MultiBody` `.mo` model from STEP assembly frames and joints
`generate_fea_mesh`	Tessellates the STEP B-Rep solid into a 3D tetrahedral CalculiX `.inp` finite element mesh
`generate_cfd_mesh`	Generates an unstructured `.su2` surface/volume fluid mesh from the STEP geometry
`create_fea_setup`	Scaffolds a parametric `.inpt` CalculiX structural study linked to the STEP part
`create_cfd_setup`	Scaffolds a parametric `.cfgt` SU2 aerodynamic study linked to the STEP part

---

Dedicated `step-mcp` Server Tools

The `@modelscript/step` MCP server (`languages/step/src/mcp/server.ts`) exposes two specialized tools for AI engineering agents:

- **`step_extract_entities`** : Parses a `.step` / `.stp` file and returns the structured assembly tree, `AXIS2_PLACEMENT_3D` coordinate frames, kinematic joints, and AP242 GD&T geometric tolerances.
- **`step_to_multibody`** : Generates complete `Modelica.Mechanics.MultiBody` code (including `World` , `Parts.Body` , `Parts.FixedTranslation` , `Joints.*` , and `connect(...)` equations) from a STEP assembly.

Polyglot TGG Rules

`@modelscript/step` registers 5 bidirectional Triple Graph Grammar (TGG) rules in `languages/step/src/language.ts` :

Rule Name	Source (`step`)	Target Language	Target Construct
`StepProductToSysML2Part`	`DataEntity` (`PRODUCT`)	`sysml2`	`PartDef`
`StepPropertyToSysML2Attribute`	`DataEntity` (`PROPERTY_DEFINITION`)	`sysml2`	`AttributeUsage`
`StepPlacementToSysML2Port`	`DataEntity` (`AXIS2_PLACEMENT_3D`)	`sysml2`	`PortUsage`
`StepRevoluteJointToModelicaRevolute`	`DataEntity` (`REVOLUTE_PAIR`)	`modelica`	`ComponentClause` (`Modelica.Mechanics.MultiBody.Joints.Revolute`)
`StepPrismaticJointToModelicaPrismatic`	`DataEntity` (`PRISMATIC_PAIR`)	`modelica`	`ComponentClause` (`Modelica.Mechanics.MultiBody.Joints.Prismatic`)

CLI Usage

```
# Parse and inspect the CST of an ISO 10303-21 STEP file
npx msx parse chassis.step

# Project a STEP assembly into a Modelica MultiBody model via TGG rules
npx msx polyglot project chassis.step --target modelica --out chassis.mo

# Verify cross-domain alignment between STEP CAD geometry and SysML v2 architecture
npx msx align chassis.step architecture.sysml
```

CHAPTER 19

OWL2 Ontology (`.owl`, `.owl2`, `.ofn`, `.ttl`)

`@modelscript/owl2` (`languages/owl2`) brings formal ontological engineering, Description Logic ($\mathrm{SROIQ}(\mathrm{D})$) reasoning, and SPARQL / Leapfrog Triejoin graph querying directly into the ModelScript compiler.

Supported Extensions & Formats

Extension	Format
`.owl`, `.owl2`, `.ofn`	W3C OWL 2 Functional-Style Syntax (`Prefix(...)`, `Ontology(...)`)
`.ttl`	RDF Turtle / Ontological Triple Assertions

OWL 2 DL Axiom & Class Expression Lowering

`lowerCstToAxioms(tree, sourceText)` (`languages/owl2/src/cst-lowering.ts`) lowers the linear-memory CST into an arena-backed `OntologyStore` supporting over 35 OWL 2 DL axiom and class expression constructs:

Category	Supported OWL 2 Functional-Style Constructs
Declarations	`Declaration(Class(...))`, `Declaration(ObjectProperty(...))`, `Declaration(DataProperty(...))`, `Declaration(NamedIndividual(...))`
Class Axioms	`SubClassOf`, `EquivalentClasses`, `DisjointClasses`
Boolean & Nominal Class Expressions	`ObjectIntersectionOf`, `ObjectUnionOf`, `ObjectComplementOf`, `ObjectOneOf`
Object Property Restrictions	`ObjectSomeValuesFrom`, `ObjectAllValuesFrom`, `ObjectHasSelf`, `ObjectHasValue`, `ObjectMinCardinality`, `ObjectMaxCardinality`, `ObjectExactCardinality`
Data Property Restrictions	`DataSomeValuesFrom`, `DataAllValuesFrom`, `DataMinCardinality`, `DataMaxCardinality`, `DataExactCardinality`
Property Hierarchy & Inversion	`SubObjectPropertyOf`, `SubDataPropertyOf`, `InverseObjectProperties`, `DisjointObjectProperties`
Property Domains & Ranges	`ObjectPropertyDomain`, `ObjectPropertyRange`, `DataPropertyDomain`, `DataPropertyRange`
Property Characteristics	`FunctionalObjectProperty`, `InverseFunctionalObjectProperty`, `ReflexiveObjectProperty`, `IrreflexiveObjectProperty`, `SymmetricObjectProperty`, `AsymmetricObjectProperty`, `TransitiveObjectProperty`, `FunctionalDataProperty`
ABox Individual Assertions	`ClassAssertion`, `ObjectPropertyAssertion`, `DataPropertyAssertion`, `NegativeObjectPropertyAssertion`, `NegativeDataPropertyAssertion`, `SameIndividual`, `DifferentIndividuals`

```
Prefix(:=<http://example.org/spacecraft#>)
Ontology(<http://example.org/spacecraft>
  Declaration(Class(:Thruster))
  Declaration(Class(:Actuator))
  Declaration(ObjectProperty(:mountedOn))
  Declaration(DataProperty(:nominalThrustN))
  SubClassOf(:Thruster :Actuator)
  SubClassOf(:Thruster ObjectSomeValuesFrom(:mountedOn :ReactionControlPod))
  ClassAssertion(:Thruster :RCS_Thruster_A1)
  DataPropertyAssertion(:nominalThrustN :RCS_Thruster_A1 "22.0"^^xsd:double)
)
```

Description Logic Tableau Reasoner & Leapfrog Triejoin

`@modelscript/owl2` includes an in-memory $\mathrm{SROIQ(D)}$ **Tableau Reasoner** (`TableauReasoner` in `languages/owl2/src/reasoner/`) and a worst-case optimal **Leapfrog Triejoin** engine (`executeBGP` in `languages/owl2/src/query/leapfrog.ts`):

1. **Consistency & Satisfiability Checking**: Detects unsatisfiable classes (`DisjointClasses` violations, cardinality contradictions, irreflexive/asymmetric cycles, negative property assertion conflicts) and surfaces exact source diagnostics (`OWL2001` – `OWL2005`).
2. **Justification & Explanation Tracing**: Computes minimal unsatisfiability justifications (`explain_inference`) explaining why a class or individual violates ontology axioms.
3. **SPARQL & Multi-Way BGP Joins**: Evaluates conjunctive Basic Graph Patterns (BGPs) in $O(\mathrm{AGM\ bound})$ worst-case optimal time without materializing intermediate Cartesian products.

Interactive 2D Ontology Diagram & Bidirectional Writeback

`languages/owl2/src/diagram/` provides an interactive 2D graph visualization and editing surface for OWL 2 ontologies:

- **Visual Graph Projection**: Renders `Class` nodes, `NamedIndividual` instances, `SubClassOf` inheritance hierarchies, and `ObjectPropertyDomain` / `ObjectPropertyRange` directed edges.
- **Bidirectional CST Writeback (`owl2Language.writeback`)**: Adding/deleting nodes or edges in the diagram (`add_node` , `delete_node` , `add_edge` , `delete_edge` , `rename_node` , `update_parameter`) patches the Functional-Style `.owl` source text directly—inserting or removing `Declaration(...)` , `SubClassOf(...)` , or `DataPropertyAssertion(...)` axioms in place.

Dedicated `owl2-mcp` Server Tools

The `@modelscript/owl2` MCP server (`languages/owl2/src/mcp/server.ts`) exposes five specialized tools to AI agents:

MCP Tool	Description
`validate_system_consistency`	Parses an OWL 2 Functional Syntax ontology, runs the SROIQ Tableau Reasoner, and returns consistency status, unsatisfiable classes, and inferred `SubClassOf` hierarchy
`query_ontology_sparql`	Executes a SPARQL `SELECT` `WHERE` query over the ontology's inferred RDF triple graph
`query_ontology_bgp`	Evaluates a conjunctive Basic Graph Pattern (BGP) using the worst-case optimal Leapfrog Triejoin algorithm
`explain_inference`	Computes formal axiomatic justifications explaining why an entailment holds or why a class is unsatisfiable
`trace_fault_propagation`	Traces transitive fault propagation (`TransitiveObjectProperty` / causal links) from a starting component individual across the system topology

---

Polyglot TGG Rules

`@modelscript/owl2` defines 5 bidirectional Triple Graph Grammar (TGG) rules in `languages/owl2/src/language.ts` :

Rule Name	Source (`owl2`)	Target Language	Target Construct
`OWL2ClassToSysML2Part`	`Declaration` (`Class`)	`sysml2`	`PartDef`
`OWL2SubClassToSysML2Specialization`	`SubClassOf`	`sysml2`	`Specialization` (`:>`)
`OWL2ObjectPropertyToSysML2Port`	`Declaration` (`ObjectProperty`)	`sysml2`	`PortUsage`
`OWL2DataPropertyToSysML2Attribute`	`Declaration` (`DataProperty`)	`sysml2`	`AttributeUsage`
`OWL2ClassToModelicaModel`	`Declaration` (`Class`)	`modelica`	`ClassDefinition` (`model`)

CHAPTER 20

CSV Tabular Data (`.csv`)

`@modelscript/csv` (`languages/csv`) elevates empirical measurement series, calibration look-up tables, and parameter sweeps into first-class typed symbols inside the `WorkspaceIndex` .

Capabilities

1. **Native GLR Parsing & Delimiter Auto-Detection**: Parses RFC 4180 CSV documents into linear-memory CST rows and cells (`csv_document` , `header_row` , `data_row` , `field` , `quoted_field`) while `detectDelimiter` (`languages/csv/src/measurements.ts`) automatically detects comma (`,`), semicolon (`;`), or tab (`\t`) delimiters and handles quoted strings with escaped `""` sequences.
2. **Modelica Package Generation (`parseCsvToModelica`)**: Converts any CSV dataset directly into a strongly typed Modelica `package` containing:
   - `constant Integer numRows` and `constant Integer numCols`
   - A 2D `constant Real values[numRows, numCols]` matrix suitable for `Modelica.Blocks.Tables.CombiTable1D` / `CombiTable2D`
   - Sanitized 1D column vectors (`constant Real <colName>[numRows]`) with automatic duplicate-name disambiguation
3. **Salsa Virtual Component Queries (`CSVVirtualComponent`)**: Exposes every CSV column as a queryable virtual symbol in the `QueryEngine` (`languages/csv/src/queries.ts`):
   - `resolvedType` : Infers `"Real"` , `"Integer"` , `"Boolean"` , or `"String"` from column cells
   - `classInstance` : Maps to `"Modelica.Blocks.Sources.CombiTimeTable"`
   - `variability` : Returns `"constant"`
   - `causality` : Returns `"output"`
   - `resolvedArrayDimensions` / `arrayDimensions` : Returns `[rowCount]`
   - `effectiveModification` : Synthesizes `{ table: "[[...]]" }` array literals from column data
4. **Bidirectional Cell Writeback (`writeback`)**: When a parameter or table entry is modified during calibration or optimization, `csvLanguage.writeback` (`languages/csv/src/language.ts`) updates the exact `[row, col]` cell or column header in place while preserving the file's original delimiter and line endings.

Semantic Linter & Diagnostic Codes

`lintCsv(sourceText)` (`languages/csv/src/linter.ts`) performs six structural and numerical integrity checks on CSV datasets:

Code	Severity	Condition Checked
`CSV_UNCLOSED_QUOTE`	`error`	A quoted field contains an unclosed `"` at the end of a line
`CSV_EMPTY_HEADER`	`warning`	A column in the header row has an empty name
`CSV_DUPLICATE_HEADER`	`warning`	Two or more columns share the same header identifier
`CSV_COLUMN_COUNT_MISMATCH`	`error`	A data row contains fewer or more cells than the header row
`CSV_INVALID_NUMERIC_CELL`	`error`	A cell in a numeric column (where row 1 is numeric) contains non-numeric text, `NaN` , or `Infinity`
`CSV_NON_MONOTONIC_TIME`	`warning`	The first column (`time` / `t`) is not strictly monotonically increasing ($t_i < t_{i-1}$)

Polyglot TGG Rules

`@modelscript/csv` registers 2 bidirectional Triple Graph Grammar (TGG) rules in `languages/csv/src/language.ts` :

Rule Name	Source (`csv`)	Target (`modelica`)	Semantics
`CsvDocumentToModelicaPackage`	`CSVDocument`	`ClassDefinition` (`package`)	Projects a CSV file into a Modelica data package with `values[numRows, numCols]`
`CsvVirtualComponentToModelicaComponent`	`CSVVirtualComponent`	`ComponentClause` (`parameter Real`)	Binds a typed CSV column to a Modelica parameter vector or `CombiTimeTable` source

CHAPTER 21

CFD (SU2 & OpenFOAM)

`@modelscript/cfd` (`languages/cfd`) provides native WebAssembly GLR parsing, parametric configuration materialization, 3D CAD boundary marker synthesis, aerodynamic flow regime analysis, and cross-domain TGG synchronization for Computational Fluid Dynamics solver configurations.

Supported Extensions & Dialects

Extension	Dialect	Description
`.cfg` , `.su2`	SU2 Configuration	Native Stanford University Unstructured (`SU2_CFD`) key-value & marker configuration
`.cfgt`	Parametric CFD Template	SU2 or OpenFOAM template supporting `{{ expr }}` parameter expressions
`.cfd`	OpenFOAM / Native CFD	OpenFOAM dictionary syntax (`FoamFile` , `dimensions` , `internalField` , `boundaryField`) and SU2 directives

---

Parametric Template Materialization (`{{ expr }}`)

CFD templates (`.cfgt` , `.cfg` , `.cfd`) can embed mathematical expressions in `{{ ... }}` blocks bound to SysML v2 or Modelica workspace parameters:

```
SOLVER= RANS
KIND_TURB_MODEL= SST
MACH_NUMBER= {{ cruise_speed / sqrt(1.4 * 287.05 * ambient_temp) }}
AOA= {{ angle_of_attack_deg }}
REYNOLDS_NUMBER= {{ rho_air * cruise_speed * chord_length / mu_air }}
MARKER_HEATFLUX= ( battery_plate, {{ heat_dissipation_watts / plate_area }} )
```

`materializeCfdConfig(templateText, parameters)` (`languages/cfd/src/materializer.ts`) evaluates each expression via `evaluateCfdExpression` , supporting arithmetic operators (`+` , `-` , `*` , `/` , `^`), built-in math functions (`sin` , `cos` , `tan` , `sqrt` , `abs` , `min` , `max` , `log` , `exp`), and constants (`PI` , `E`).

3D CAD Boundary Condition Synthesis

When a user assigns a fluid or thermal boundary condition to a 3D CAD face in the viewport, `synthesizeCfdBoundary` and `applyBoundaryActionToConfig` (`languages/cfd/src/boundary-synthesizer.ts`) synthesize and merge the appropriate directives for either **SU2** or **OpenFOAM**:

Boundary Condition	SU2 Directive (`.cfg` / `.su2`)	OpenFOAM `boundaryField` Entry (`.cfd`)
Velocity / Total Inlet (`inlet`)	`MARKER_INLET= ( <face>, <T_total>, <P_total>, <nx>, <ny>, <nz> )`	`type fixedValue; value uniform (<vx> <vy> <vz>);`
Pressure Outlet (`outlet`)	`MARKER_OUTLET= ( <face>, <back_pressure_Pa> )`	`type fixedValue; value uniform <pressure>;`
Adiabatic / Heat-Flux Wall (`wall`)	`MARKER_HEATFLUX= ( <face>, <heat_flux_W_m2> )`	`type noSlip;` / `type fixedGradient; gradient uniform <q>;`
Isothermal Wall (`isothermal`)	`MARKER_ISOTHERMAL= ( <face>, <temp_K> )`	`type fixedValue; value uniform <temp_K>;`
Far-Field (`farfield`)	`MARKER_FAR= ( <face> )`	`type freestream;`
Symmetry Plane (`symmetry`)	`MARKER_SYM= ( <face> )`	`type symmetryPlane;`

Aerodynamic Theory Slicing & Flow Regime Analysis

`extractCfdSlicingProperties(text)` (`languages/cfd/src/language.ts`) extracts aerodynamic coefficients directly from the CST:

- **`cd`** : Drag coefficient (`DRAG_COEFF` , `CD` , or `C_D` , default `0.3`)
- **`refArea`** : Reference wetted/frontal area (`REF_AREA` , default `1.0` m^2)
- **`density`** : Freestream fluid density (`FREESTREAM_DENSITY` or `RHO` , default `1.225` $\mathrm{kg/m}^3$)
- **`freeStreamVelocity`** : Freestream velocity (`FREESTREAM_VELOCITY` , default `30.0` $\mathrm{m/s}$)

These properties feed directly into Modelica 1D aerodynamic drag models ($F_d = \frac{1}{2}\rho v^2 C_d A_{\text{ref}}$), SysML v2 performance requirements, and the Multi-Domain Theory Coordinator (`CaeContractOracle`).

Polyglot TGG Rules

`@modelscript/cfd` registers 6 bidirectional Triple Graph Grammar (TGG) rules in `languages/cfd/src/language.ts` :

Rule Name	Source (`cfd`)	Target Language	Target Construct
`CfdDragPolarToModelicaParam`	`CfdConfig` (`cd` , `refArea`)	`modelica`	`ComponentClause` (`parameter Real Cd` , `A_ref`)
`CfdDragPolarToSysmlAttribute`	`CfdConfig` (`cd` , `refArea`)	`sysml2`	`AttributeUsage`
`CfdInletToModelicaFluidPort`	`Directive` (`MARKER_INLET`)	`modelica`	`ComponentClause` (`Modelica.Fluid.Interfaces.FluidPort_a`)
`CfdHeatFluxToModelicaHeatPort`	`Directive` (`MARKER_HEATFLUX`)	`modelica`	`ComponentClause` (`Modelica.Thermal.HeatTransfer.Interfaces.HeatPort_a`)
`CfdMarkerToSysmlPort`	`Directive` (`MARKER_*`)	`sysml2`	`PortUsage`
`CfdDirectiveToSysmlConstraint`	`Directive` (`MACH_NUMBER` , `REYNOLDS_NUMBER`)	`sysml2`	`ConstraintUsage`

CHAPTER 22

FEA (CalculiX, Abaqus & NASTRAN)

`@modelscript/fea` (`languages/fea`) provides native parsing, parametric template materialization, 3D CAD boundary condition synthesis, structural theory slicing, and cross-domain TGG synchronization for Finite Element Analysis input decks.

Supported Dialects & Extensions

Extension	Dialect	Key Directives
`.inp`	CalculiX / Abaqus input deck	`*HEADING` , `*NODE` , `*ELEMENT` , `*NSET` , `*ELSET` , `*MATERIAL` , `*ELASTIC` , `*DENSITY` , `*SOLID SECTION` , `*STEP` , `*STATIC` , `*FREQUENCY` , `*BOUNDARY` , `*CLOAD` , `*DLOAD`
`.inpt`	Parametric CalculiX / Abaqus template	Supports `{{ expression }}` mustache substitutions over workspace parameters
`.bdf`	NASTRAN Bulk Data Deck	`SOL` , `CEND` , `BEGIN BULK` , `GRID` , `CTETRA` , `CHEXA` , `PSOLID` , `MAT1` , `SPC1` , `FORCE` , `MOMENT` , `LOAD` , `ENDDATA`
`.fea`	ModelScript native FEA configuration	Structured material, mesh, boundary, and load study definitions

---

Parametric Template Materialization (`{{ expr }}`)

Input decks (`.inpt` , `.inp` , `.bdf` , `.fea`) can embed parametric expressions enclosed in `{{ ... }}` placeholders:

```
*MATERIAL, NAME=AL7075
*ELASTIC
{{ E_modulus }}, {{ poisson_ratio }}
*DENSITY
{{ rho_al }}
*CLOAD
LOAD_SET, 3, {{ -tip_mass * 9.81 }}
```

`materializeFeaDeck(templateText, parameters)` (`languages/fea/src/materializer.ts`) evaluates arithmetic expressions (`+` , `-` , `*` , `/` , `^`), functions (`sin` , `cos` , `tan` , `sqrt` , `abs` , `min` , `max` , `log` , `exp`), and built-in constants (`PI` , `E`) via `evaluateFeaExpression` , formatting values in engineering notation and recording any unresolved symbols or syntax errors.

3D CAD Face Boundary Condition & Load Synthesis

When a user selects a topological face in the 3D CAD viewer, `synthesizeFeaBoundary` and `applyBoundaryActionToDeck` (`languages/fea/src/boundary-synthesizer.ts`) automatically generate solver-native boundary and load cards for the target dialect:

Boundary Kind	CalculiX / Abaqus (`.inp`)	NASTRAN Bulk Data (`.bdf`)
Fixed Support (`fixed`)	`*BOUNDARY` constraining DOFs `1, 6` (or `1, 3`) on `NSET_<FACE>`	`SPC1` card locking DOFs `123456` (or `123`) on `SET_<FACE>`
Point / Concentrated Force (`force`)	`*CLOAD` directional nodal force components (F_x, F_y, F_z) along the face normal	`FORCE` card with magnitude and normal unit vector (n_x, n_y, n_z)
Distributed Pressure (`pressure`)	`*DLOAD` surface pressure (`P`) applied to `ELSET_<FACE>`	`PLOAD4` / `FORCE` cards distributed across face elements
Torque / Moment (`moment`)	`*CLOAD` rotational DOFs `4, 5, 6`	`MOMENT` card around face normal axis

---

Structural Theory Slicing & Safety Margin Extraction

`extractFeaSlicingProperties(text)` (`languages/fea/src/language.ts`) extracts structural parameters (`DENSITY` , `VOLUME` , `MAX_VON_MISES_STRESS` , `YIELD_STRENGTH`) directly from the CST and evaluates the structural safety margin:

$$\text{Safety Factor (SF)} = \frac{\sigma_{\text{yield}}}{\sigma_{\text{von Mises, max}}}, \qquad \text{isSafe} \iff \sigma_{\text{von Mises, max}} \le \sigma_{\text{yield}}$$

Returning `{ mass, maxVonMisesPa, yieldStrengthPa, safetyFactor, isSafe }` directly to the Multi-Domain Theory Coordinator (`CaeContractOracle`) and SysML v2 requirement verifiers.

Built-In Solver & IDE Actions

`@modelscript/fea` registers three interactive actions in `feaLanguage.actions` :

- **`open_fea_viewer`** : Opens the 3D FEA mesh, boundary condition, and von Mises stress contour viewer.
- **`materialize_deck`** : Evaluates all `{{ expr }}` template placeholders against workspace parameters to produce a concrete `.inp` or `.bdf` deck.
- **`run_simulation` (`fea_run_simulation`)**: Parses `*MATERIAL` , `*ELASTIC` , `*BOUNDARY` , and `*CLOAD` / `*DLOAD` sections, solves the 3D tetrahedral finite element stiffness matrix, and returns nodal displacements and element von Mises stress contours.

Polyglot TGG Rules

`@modelscript/fea` defines 6 bidirectional Triple Graph Grammar (TGG) rules in `languages/fea/src/language.ts` :

Rule Name	Source (`fea`)	Target Language	Target Construct
`FeaStressMassToModelicaParam`	`FeaDeck` (`mass` , `maxVonMisesPa`)	`modelica`	`ComponentClause` (`parameter Real`)
`FeaStressMassToSysmlAttribute`	`FeaDeck` (`safetyFactor` , `mass`)	`sysml2`	`AttributeUsage`
`FeaMaterialToSysmlAttribute`	`Section` (`*MATERIAL` / `MAT1`)	`sysml2`	`AttributeUsage`
`FeaBoundaryFixToModelicaFixed`	`Section` (`*BOUNDARY` / `SPC1`)	`modelica`	`ComponentClause` (`Modelica.Mechanics.MultiBody.Parts.Fixed`)
`FeaSpringToModelicaSpring`	`Section` (`*SPRING`)	`modelica`	`ComponentClause` (`Modelica.Mechanics.Translational.Components.Spring`)
`FeaLoadCaseToSysmlRequirement`	`Section` (`*STEP` / `SUBCASE`)	`sysml2`	`RequirementUsage`

CHAPTER 23

OpenSCAD (`.scad`)

`@modelscript/scad` (`languages/scad`) parses parametric OpenSCAD scripts and evaluates Constructive Solid Geometry (CSG) trees via `@modelscript/cad` into exact OpenCascade B-Rep solids, watertight triangle meshes, STEP AP214/AP242 exports, and Modelica MultiBody parameters.

Supported Syntax & Fluent Method Chaining

In addition to standard OpenSCAD primitives (`cube` , `sphere` , `cylinder` , `polyhedron` , `circle` , `square` , `polygon`), boolean operations (`union` , `difference` , `intersection` , `hull` , `minkowski`), extrusions (`linear_extrude` , `rotate_extrude`), and control flow (`module` , `function` , `for` , `if` / `else`), `@modelscript/scad` supports **fluent method chaining** (`ChainedSolid` in `languages/scad/src/evaluator.ts`) and **boundary port tagging**:

```
wall_thickness = 4.0;
mount_radius = 12.0;

// Fluent method chaining with .fillet(), .chamfer(), .translate(), .rotate()
bracket = cube([60, 40, wall_thickness], center=true)
  .fillet(1.5)
  .translate([0, 0, wall_thickness / 2]);

// Explicit physical port tagging for SysML v2 / Modelica MultiBody & CFD/FEA binding
tag_port("base_mount", "frame", [0, 0, -1])
  difference() {
    bracket;
    cylinder(h=10, r=mount_radius, center=true).chamfer(0.5);
  }
```

Supported Chained Methods (`ChainedSolid`)

Method	Signature	Description
`.translate(v)`	`translate([dx, dy, dz])`	Translates the solid by vector `[dx, dy, dz]`
`.rotate(a, v?)`	`rotate([rx, ry, rz])` or `rotate(deg, axis)`	Rotates the solid by Euler angles or axis-angle
`.scale(v)`	`scale([sx, sy, sz])`	Scales the solid non-uniformly or uniformly
`.mirror(v)`	`mirror([nx, ny, nz])`	Mirrors the solid across the plane with normal `v`
`.color(c, alpha?)`	`color("steelblue", 0.9)`	Assigns RGBA display metadata
`.fillet(r)`	`fillet(radius)`	Applies a 3D edge fillet of radius $r > 0$
`.chamfer(d)`	`chamfer(distance)`	Applies a 3D edge chamfer of distance $d > 0$

Semantic Linter & Diagnostic Codes

`lintScadCst(sourceText)` (`languages/scad/src/linter.ts`) performs static scope resolution, constant folding, and geometric validity checking across 9 diagnostic rules:

Code	Severity	Condition Checked
`SCAD_DUPLICATE_DECLARATION`	`warning`	Variable, `module`, `function`, or parameter is declared multiple times in the same scope
`SCAD_INVALID_DIMENSION`	`error`	Primitive dimension (`size`, `r`, `r1`, `r2`, `d`, `d1`, `d2`, `h`, `height`), `.fillet(r)`, or `.chamfer(d)` is ≤ 0, or a vector argument has invalid length
`SCAD_UNDEFINED_VARIABLE`	`error`	Reference to an undeclared variable identifier (excluding `$fn`, `$fa`, `$fs`, `$t`, `$vpr`, `$vpt`, `$vpd`, `$vpf`, `$children`, `$preview`, `PI`, `undef`)
`SCAD_UNDEFINED_FUNCTION`	`error`	Call to an undeclared function not in the OpenSCAD built-in math/vector library
`SCAD_UNDEFINED_MODULE`	`error`	Instantiation of an undeclared module not in the OpenSCAD built-in primitive/transform library
`SCAD_UNKNOWN_METHOD`	`error`	Chained method call outside `{ translate, rotate, scale, mirror, color, fillet, chamfer }`
`SCAD_DIVISION_BY_ZERO`	`error`	Binary `/` or `%` expression where the divisor constant-folds to `0`
`SCAD_ARGUMENT_COUNT`	`error`	Function or module call provides more positional arguments than parameters or omits a required parameter
`SCAD_UNKNOWN_PARAMETER`	`warning`	Named argument (`name=val`) does not match any parameter of the target user-defined module or function

---

3D Viewport Gizmo Writeback (`ScadPatcher`) & STEP Export

- **Surgical AST Writeback (`ScadPatcher` in `languages/scad/src/patcher.ts`)**: When a user drags a 3D translation/rotation/scale gizmo in the CAD viewport, `ScadPatcher` (`patchNode`, `patchVariable`, `patchTransformArgument`, `patchVectorComponent`, `wrapPrimitiveWithTransform`, `patchOrInsertTransform`) updates the exact CST byte spans in the `.scad` source without reformatting surrounding code or comments.
- **STEP AP214/AP242 Export (`compileScadToStep`)**: Evaluates the `.scad` CSG tree and serializes exact B-Rep analytical surfaces (`PLANE`, `CYLINDRICAL_SURFACE`, `SPHERICAL_SURFACE`, `MANIFOLD_SOLID_BREP`) to ISO 10303-21 `.step`.

```
# Project a parametric OpenSCAD file to an ISO 10303-21 STEP CAD file
npx msx polyglot project bracket.scad --target step --out bracket.step

# Or extract Embedded CSG from a Modelica model to OpenSCAD
npx msx csg SatelliteChassis satellite.mo > bracket.scad
```

Polyglot TGG Rules

`@modelscript/scad` registers 13 bidirectional Triple Graph Grammar (TGG) rules in `languages/scad/src/language.ts`:

Rule Name	Source (scad)	Target Language	Target Construct
ScadModuleToSysmlPart	ModuleDeclaration	sysml2	PartDef
ScadProjectedAreaToModelicaParam	ScadScript (projectedArea , volume)	modelica	ComponentClause (parameter Real)
ScadProjectedAreaToSysmlAttribute	ScadScript (projectedArea , volume)	sysml2	AttributeUsage
ScadTagPortToSysmlPort	ModuleInstantiation (tag_port)	sysml2	PortUsage
ScadVariableToSysmlAttribute	VariableAssignment	sysml2	AttributeUsage
ScadSolidToStepProduct	ModuleInstantiation (cube)	step	DataEntity (PRODUCT)
ScadCylinderToStepProduct	ModuleInstantiation (cylinder)	step	DataEntity (CYLINDRICAL_SURFACE)
ScadSphereToStepProduct	ModuleInstantiation (sphere)	step	DataEntity (SPHERICAL_SURFACE)
ScadBooleanOpToStepComposite	ModuleInstantiation (union / difference / intersection)	step	DataEntity (BOOLEAN_RESULT)
ScadTagPortToModelicaFrame	ModuleInstantiation (tag_port)	modelica	ComponentClause (Modelica.Mechanics.MultiBody.Interfaces.Frame_a)
ScadCubeToMultiBodyShape	ModuleInstantiation (cube)	modelica	ComponentClause (Modelica.Mechanics.MultiBody.Visualizers.FixedShape "box")
ScadCylinderToMultiBodyShape	ModuleInstantiation (cylinder)	modelica	ComponentClause (Modelica.Mechanics.MultiBody.Visualizers.FixedShape "cylinder")
ScadSphereToMultiBodyShape	ModuleInstantiation (sphere)	modelica	ComponentClause (Modelica.Mechanics.MultiBody.Visualizers.FixedShape "sphere")

CHAPTER 24

SSP (System Structure and Parameterization)

`@modelscript/ssp` (`languages/ssp`) implements the Modelica Association **SSP 1.0 / 2.0** standard for packaging and coupling Functional Mock-up Units (FMUs) into multi-domain Co-Simulation architectures.

Supported Extensions & Container Architecture

Extension	Standard Artifact	Role
`.ssp`	System Structure Package (ZIP archive)	Container bundling `SystemStructure.ssd` , `resources/*.fmu` , and `*.ssv` parameter sets
`.ssd`	System Structure Description (XML)	Hierarchical `System` , `Components` , `Connectors` , `Connections` , and `DefaultExperiment` topology
`.ssv`	System Structure Parameter Values (XML)	Typed parameter bindings (`Real` , `Integer` , `Boolean` , `String` , `Enumeration`)

Virtual File System (VFS) Archive Extraction

When a `.ssp` ZIP archive is loaded into the workspace, `sspLanguage.container` (`languages/ssp/src/language.ts`) automatically unpacks and indexes its internal artifacts:

- `*.ssd` and `*.ssv` entries are dispatched via `parse_as_language` into the `@modelscript/ssp` CST & Salsa index.
- `resources/*.fmu` archives are recursively extracted (`extract_nested`) so their internal `modelDescription.xml` ports and parameters are cross-checked against `.ssd` connectors.
- Auxiliary tables and binaries under `resources/*` are mounted into the in-memory VFS (`mount_vfs`).

Interactive 2D Diagram Builder & Bidirectional XML Writeback

`languages/ssp/src/diagram.ts` provides a two-way schematic editor for `.ssd` topologies:

- **2D Schematic Extraction (`buildSspDiagramData`)**: Parses `<ssd:System>` , `<ssd:Component>` , `<ssd:Connector>` , `<ssd:Connection>` , and `<ssd:ElementGeometry>` / `<ssd:ConnectionGeometry>` annotations into positioned block nodes, typed causal ports, and orthogonal routing waypoints.
- **Surgical XML Writeback (`applySspDiagramEdits` / `sspLanguage.writeback`)**: Moving components, adding/removing FMU blocks, drawing `<ssd:Connection>` wires, or editing `<ssv:Parameter>` values updates the underlying `.ssd` or `.ssv` XML document in place while preserving namespace prefixes and custom vendor annotations.

Modelica Architecture Projection (`sspToModelicaBlock`)

`sspToModelicaBlock(ssdDoc)` (`languages/ssp/src/index.ts`) projects any `.ssd` system topology into an equivalent composite Modelica `model` , translating `<ssd:Connector>` definitions into `Modelica.Blocks.Interfaces.RealInput` / `RealOutput` (or `Integer` / `Boolean` causal interfaces), `<ssd:Component>` blocks into typed sub-models with `<ssv:Parameter>` modifications, and `<ssd:Connection>` edges into `connect(...)` equations.

Semantic Linter & 20 Diagnostic Codes

`lintSsd` and `lintSsv` (`languages/ssp/src/linter.ts`) validate structural, causal, and type soundness across `.ssd` and `.ssv` files with 20 diagnostic codes (13 `SSP_*` + 7 `SSV_*`):

`.ssd` System Structure Diagnostics (`lintSsd`)

Code	Severity	Condition Checked
`SSP_EMPTY_CONNECTOR_NAME`	`error`	A `<Connector>` on a system or component has an empty `name` attribute
`SSP_DUPLICATE_CONNECTOR`	`error`	Duplicate connector `name` declared on the same system or component
`SSP_INVALID_CONNECTOR_KIND`	`error`	Connector `kind` is not one of `input`, `output`, `parameter`, `calculatedParameter`, `local`, `inout`
`SSP_EMPTY_COMPONENT_NAME`	`error`	A `<Component>` element has an empty `name` attribute
`SSP_DUPLICATE_COMPONENT`	`error`	Two `<Component>` elements in the same `<System>` share the same `name`
`SSP_MISSING_COMPONENT_SOURCE`	`warning`	A `<Component>` has an empty `source` URI (no FMU or nested `.ssp` referenced)
`SSP_UNRESOLVED_ELEMENT`	`error`	A `<Connection>` references a `startElement` or `endElement` component that does not exist
`SSP_UNRESOLVED_CONNECTOR`	`error`	A `<Connection>` references a `startConnector` or `endConnector` that does not exist on the target element
`SSP_SELF_CONNECTION`	`error`	A `<Connection>` connects an element's connector back to the exact same element and connector
`SSP_CONNECTOR_TYPE_MISMATCH`	`error`	Connected `startConnector` and `endConnector` have incompatible data types (e.g., `Real` connected to `Boolean`)
`SSP_CONNECTOR_CAUSALITY_CONFLICT`	`warning`	Invalid causal connection (e.g., component `output` to component `output`, or `input` to `input`)
`SSP_UNRESOLVED_BINDING_PREFIX`	`warning`	`<Parameter>` `name="prefix.param"` in an inline `<ParameterValues>` block references a non-existent component `prefix`
`SSP_INVALID_EXPERIMENT_BOUNDS`	`error`	`<DefaultExperiment>` has `stopTime <= startTime`

`.ssv` Parameter Value Diagnostics (`lintSsv`)

Code	Severity	Condition Checked
`SSV_EMPTY_PARAMETER_NAME`	`error`	A `<ssv:Parameter>` element has an empty `name` attribute
`SSV_DUPLICATE_PARAMETER`	`warning`	The same parameter `name` is defined more than once in the `<ssv:Parameters>` set
`SSV_MISSING_PARAMETER_TYPE`	`error`	A `<ssv:Parameter>` is missing its child type element (`<Real>`, `<Integer>`, `<Boolean>`, `<String>`, `<Enumeration>`)
`SSV_MISSING_PARAMETER_VALUE`	`error`	The typed child element of `<ssv:Parameter>` is missing its `value` attribute
`SSV_INVALID_REAL_VALUE`	`error`	`<ssv:Real value="..."/>` is not a valid finite floating-point number
`SSV_INVALID_INTEGER_VALUE`	`error`	`<ssv:Integer value="..."/>` is not a valid base-10 integer
`SSV_INVALID_BOOLEAN_VALUE`	`error`	`<ssv:Boolean value="..."/>` is not `"true"`, `"false"`, `"1"`, or `"0"`

Polyglot TGG Rules

`@modelscript/ssp` registers 8 bidirectional Triple Graph Grammar (TGG) rules in `languages/ssp/src/language.ts`:

Rule Name	Source (ssp)	Target Language	Target Construct
SspSystemToModelicaModel	SsdSystem	modelica	ClassDefinition (model)
SspComponentToModelicaComponent	SsdComponent	modelica	ComponentClause
SspConnectorToModelicaConnector	SsdConnector	modelica	ComponentClause (RealInput / RealOutput)
SspConnectionToModelicaConnect	SsdConnection	modelica	ConnectClause
SspSystemToSysmlPackage	SsdSystem	sysml2	PartDef
SspComponentToSysmlPart	SsdComponent	sysml2	PartUsage
SspConnectorToSysmlPort	SsdConnector	sysml2	PortUsage
SspConnectionToSysmlConnection	SsdConnection	sysml2	ConnectionUsage

PART IV

Comprehensive Algorithms Catalog

CHAPTER 25

ModelScript Algorithms Catalog

ModelScript is powered by a comprehensive foundation of peer-reviewed mathematical, compiler, and formal verification algorithms. Rather than relying on opaque external black boxes, ModelScript implements these algorithms natively in WebAssembly and TypeScript, with inline citations, architectural rationales, and explicit tracking of upstream inputs and downstream consumers.

Computational Map

The algorithms catalog spans seven core domains across the compilation, simulation, and verification pipelines:



Algorithm Directory

Category	Primary Algorithms	Implementation Package
Structural DAE	Pantelides Algorithm, Tarjan BLT, Cellier-Elmqvist Tearing	`@modelscript/runtime`
Acausal Balancing	Acausal Potential Equalization, Kirchhoff Flows, Stream Mixing	`@modelscript/modelica`
Autodiff & Relaxations	Dual Numbers, AdTape, CPR Graph Coloring, McCormick, Affine, Hybrid Adjoint Saltation Matrices, Aitken Dynamic Δ-Relaxation	`@modelscript/runtime` & `@modelscript/simulate`
Numerical Solvers	Tsit5, Rodas-4P, TR-BDF2, SDE, BVP	`@modelscript/simulate`
Optimization	L-BFGS-B, CMA-ES, Differential Evolution, PSO, NSGA-II	`@modelscript/simulate`
SMT & Verification	Nelson-Oppen, CDCL SAT, Craig Interpolation, HC4, STL, Zonotopes	`@modelscript/runtime/formal`
Model Checking & Safety	Spacer CHC PDR, IC3 Engine, Stochastic Barrier Certificates, WebGPU OWL 2 RL Reasoner, ISO 26262 / DO-178C Back-to-Back Equivalence	`@modelscript/runtime/formal` & `@modelscript/runtime/gpu`
Abstract Interpretation	WTO Fixpoint, TVPI, Octagon DBM, Array Segments, DDE Method-of-Steps Flowpipes, Hamilton-Jacobi-Isaacs (HJI) Level-Set Reachability	`@modelscript/runtime/formal` & `@modelscript/runtime/analysis`
Graph Rewriting	Triple Graph Grammars, CPA, WCOJ, E-Graphs, GSS, Salsa	`@modelscript/dsl`
Linear Memory Structures	Roaring Bitmaps, Front-Coded Dictionary, Paged B+ Tree	`@modelscript/runtime`
Scientific & Digital Twins	Sparse Cholesky, Sparse LU, Householder QR, Gröbner, POD, Buckingham Pi, Arena MHE, Joint-State UKF & MCMC Calibration, Page-Hinkley / CUSUM Drift, RUL Prognostics	`@modelscript/runtime` & `@modelscript/simulate`
SDP, BVP, Continuum & CAD	Primal-Dual SDP, Multiple Shooting BVP, CSG, WebGPU D3Q19 LBM CFD, Tet4/Tet10 Corotational FEA, STEP B-Rep Mesher, Swept-Volume Clearance, GD&T Stack-Up, CAD Parameter Inversion, ECAD Gerber / Router / DRC	`@modelscript/runtime`, `@modelscript/simulate` & `@modelscript/cad`
Diagram Layout & Diffing	Tamassia TSM, Port ILP Solver, Dogleg Channel Router, Fault Tree / Sequence / Swimlane Layouts, PSCAD SLD Routing, Semantic Visual Diffing, CRDT Collaborative Presence	`@modelscript/diagram`

CHAPTER 26

Pantelides Structural Index Reduction

Implementation: `PantelidesEngine` in `packages/runtime/src/wasm/structural/pantelides.ts`

Academic Citations

- **Pantelides, C. C. (1988)**. "*The consistent initialization of differential-algebraic systems.*"
  SIAM Journal on Scientific and Statistical Computing, 9(2), pp. 213–231.
  DOI: 10.1137/0909014
- **Mattsson, S. E., & Söderlind, G. (1993)**. "*Index reduction in differential-algebraic equations using dummy derivatives.*"
  SIAM Journal on Scientific Computing, 14(3), pp. 677–692.
  DOI: 10.1137/0914043

ModelScript Architectural Rationale

Object-oriented multi-domain physical models (such as constrained multi-body mechanics, closed kinematic linkages, and incompressible hydraulic loops) inherently generate high-index Differential-Algebraic Equations (DAEs) where the differential index satisfies $\text{ind} \geq 2$:

$$F(t, x, \dot{x}, y) = 0$$

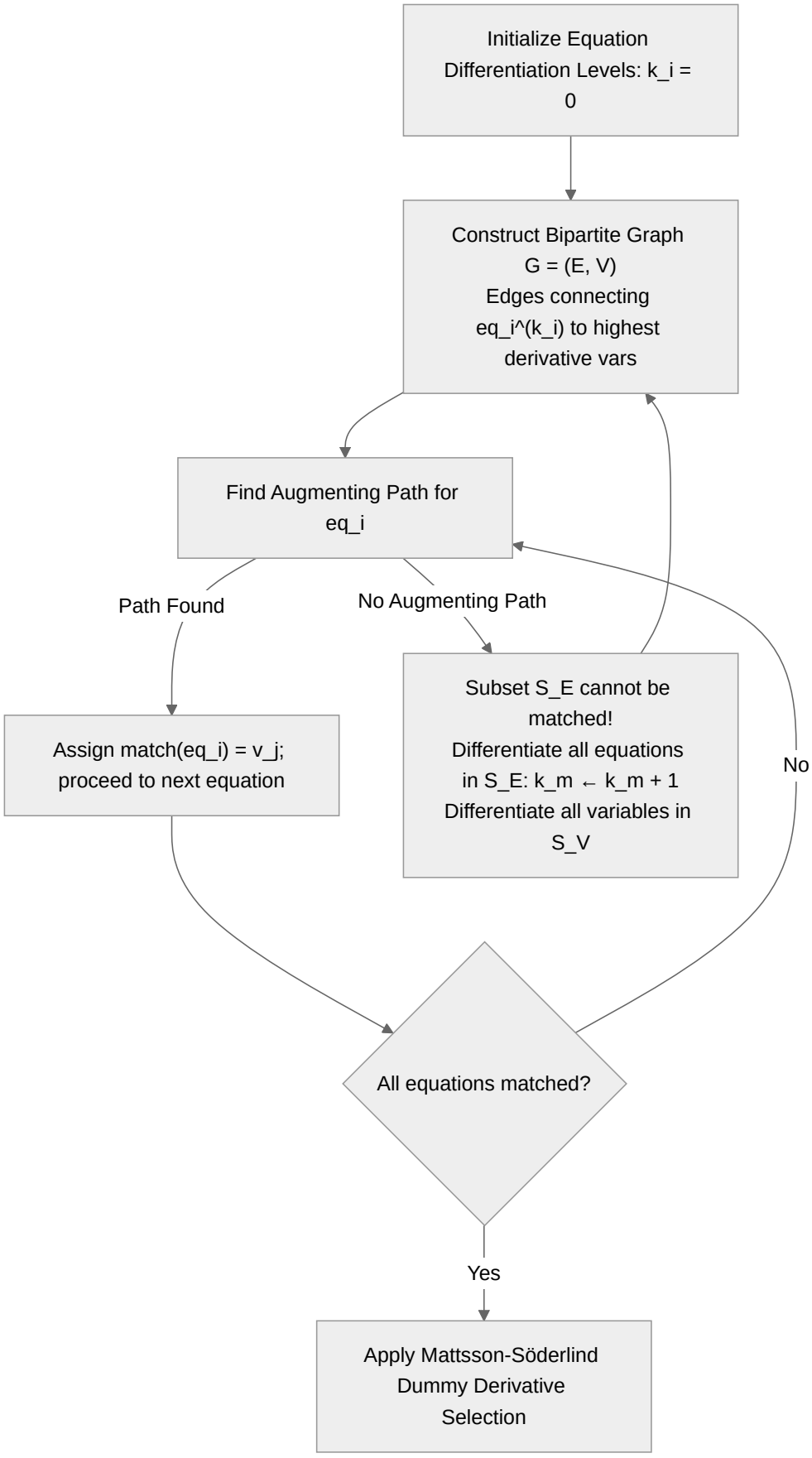
Standard numerical integrators (including BDF, Radau, and Runge-Kutta methods) become severely ill-conditioned or fail to converge on systems with differential index greater than 1.

Pantelides' algorithm identifies minimally singular subsets of equations and differentiates them analytically with respect to time until an index-1 or index-0 formulation is attained. To prevent over-determination and numerical drift along algebraic constraint manifolds, the Mattsson-Söderlind dummy derivative method dynamically replaces redundant derivative states with algebraic placeholders.

ModelScript Modifications & WASM Architecture

1. **In-WASM Linear Memory Execution**: Operates as an unmanaged WebAssembly kernel (`@unmanaged`) with zero garbage collection allocations.
2. **Direct Arena Symbolic Differentiation**: Derivatives of equations ($\frac{d^k f}{dt^k} = 0$) are differentiated directly on `DAEBuilder` linear expression nodes in memory.
3. **Flat Chunked Arrays**: Equation dependency incidence pointers, differentiation levels, and matching arrays are stored in flat `ChunkedInt32Array` buffers.
4. **Augmenting Path Bipartite Matching**: Alternating path searches run directly over highest-derivative variables, seamlessly transferring matching data to the downstream `BltEngine` .

Algorithm Description & Mathematics



Mathematical Steps

1. **Variable Classification**: Identify highest-order derivatives $\dot{x}_j$ and purely algebraic variables y_k.

2. **Alternating Augmenting Paths**: For each equation $f_i = 0$, attempt to find an augmenting path in the bipartite incidence graph to an unmatched highest-order derivative.
3. **Index Reduction by Differentiation**: If no augmenting path exists, the reachable set of equations S_E is structurally singular. Increment the differentiation index $k_m \leftarrow k_m + 1$ for all $f_m \in S_E$, symbolically compute $\frac{df_m}{dt}$, and repeat the matching.
4. **Dummy Derivatives**: For each differentiated state variable where both x_j and $\dot{x}_j$ appear in algebraic constraints, select a dummy algebraic variable $x_{j,\text{dummy}}$ to replace $\dot{x}_j$, preserving degrees of freedom.

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - Ingests the flat DAE representation from `DAEBuilder` produced by `ModelicaFlattener`.
  - Uses `Cst` linear nodes for symbolic expressions.
- **Downstream Consumers**:
  - Feeds the index-reduced DAE directly into `BltEngine` for block lower triangular partitioning.
  - Determines consistent initial conditions for `SUNDIALS IDA` and `TR-BDF2`.

CHAPTER 27

Tarjan Block Lower Triangular (BLT) Transformation

Implementation: `BltEngine` in `packages/runtime/src/wasm/structural/blt.ts`

Academic Citations

- **Tarjan, R. E. (1972)**. "*Depth-first search and linear graph algorithms.*"
  SIAM Journal on Computing, 1(2), pp. 146–160.
  DOI: 10.1137/0201010 (Strongly Connected Components)
- **Hopcroft, J. E., & Karp, R. M. (1973)**. "*An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs.*"
  SIAM Journal on Computing, 2(4), pp. 225–231.
  DOI: 10.1137/0202019 (Bipartite Matching)
- **Dulmage, A. L., & Mendelsohn, N. S. (1958)**. "*Coverings of bipartite graphs.*"
  Canadian Journal of Mathematics, 10, pp. 517–534.
  DOI: 10.4153/CJM-1958-052-0
- **Cellier, F. E., & Kofman, E. (2006)**. *Continuous System Simulation*.
  Springer. ISBN: 978-0-387-26102-7

ModelScript Architectural Rationale

Flattening industrial physical systems results in coupled systems comprising thousands of equations and variables. Solving a monolithic non-linear algebraic system of size N using general Newton-Raphson methods incurs a cubic computational penalty per step:

$$\mathrm{O}(N^3)$$

The Block Lower Triangular (BLT) transformation permutes the equation and variable ordering so that the incidence matrix assumes a lower triangular block diagonal structure:

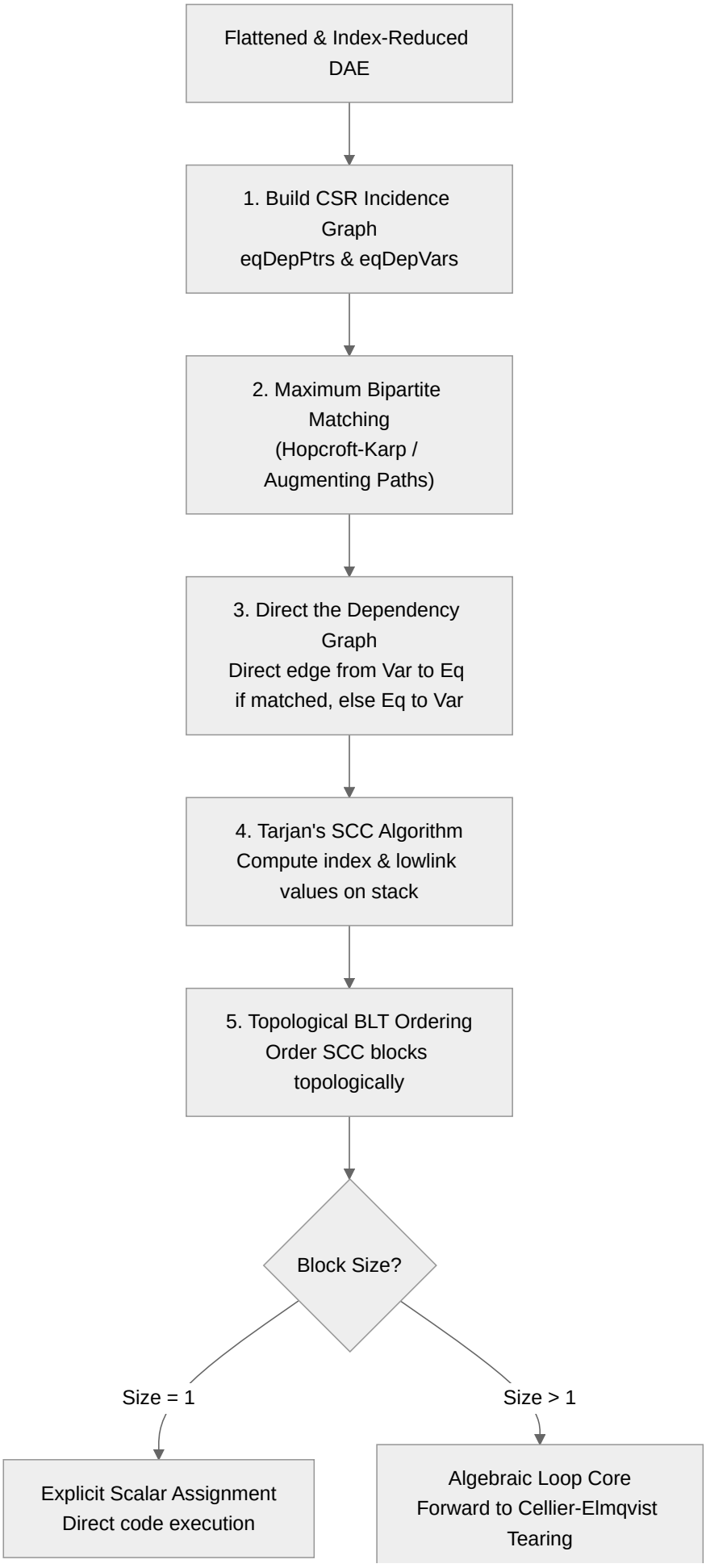
$$\begin{pmatrix} A_{11} & 0 & \cdots & 0 \\ A_{21} & A_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A_{m1} & A_{m2} & \cdots & A_{mm} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$$

Most blocks A_{ii} are trivial 1×1 scalar explicit assignments ($x_i = f(\dots)$), while only non-linear algebraic loops become coupled sub-blocks of size $k \ll N$.

ModelScript Modifications & WASM Architecture

1. **Compressed Sparse Row (CSR) in Linear Memory**: Bipartite incidence graphs are stored in contiguous `ChunkedInt32Array` buffers (`eqDepPtrs` , `eqDepVars`).
2. **Zero-Allocation Tarjan Traversal**: Low-link values, depth indices, and traversal stacks operate strictly on 32-bit integer offsets in WebAssembly memory without object references.
3. **Integrated Maximum Matching**: Hopcroft-Karp augmenting path search provides consistent equation-to-variable assignments for directed SCC analysis.
4. **Direct Bridge to Tearing**: SCC blocks with dimension > 1 pass directly to the in-WASM `TornBlock` engine.

Algorithm Description & Pipeline Flow


Flattened & Index-Reduced DAE
1. Build CSR Incidence Graph
eqDepPtrs & eqDepVars
2. Maximum Bipartite Matching
(Hopcroft-Karp / Augmenting Paths)
3. Direct the Dependency Graph
Direct edge from Var to Eq if matched, else Eq to Var
4. Tarjan's SCC Algorithm
Compute index & lowlink values on stack
5. Topological BLT Ordering
Order SCC blocks topologically
Block Size?
Size = 1
Size > 1
Explicit Scalar Assignment
Direct code execution
Algebraic Loop Core
Forward to Cellier-Elmqvist Tearing

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - Ingests index-reduced DAE systems from `PantelidesEngine`.
- **Downstream Consumers**:
  - Trivial 1×1 blocks are lowered directly into sequence evaluation bytecode.
  - Coupled blocks (size > 1) are sent to `TornBlock` for algebraic loop tearing.
  - Drives step evaluations in `SUNDIALS CVODE/IDA` and `Tsit5`.

CHAPTER 28

Cellier-Elmqvist Algebraic Loop Tearing

Implementation: `TornBlock` in `packages/runtime/src/wasm/structural/tearing.ts`

Academic Citations

- **Cellier, F. E. (1979)**. *Combined Continuous/Discrete System Simulation by Use of Digital Computers: Techniques and Tools*. **PhD thesis, ETH Zurich**.
  DOI: 10.3929/ethz-a-000179979
- **Elmqvist, H., & Otter, M. (1994)**. "*Methods for Tearing Systems of Equations in Object-Oriented Modeling.*" **Proceedings of the European Simulation Multiconference (ESM'94)**, pp. 326–332.
- **Mahana, P. N., & Eagan, J. P. (1975)**. "*A tearing algorithm for large systems of equations.*" **IEEE Transactions on Circuits and Systems**.

ModelScript Architectural Rationale

Even after BLT partitioning, physical systems containing closed kinematic loops, electrical bridge circuits, or multi-phase fluid balances yield coupled Strongly Connected Components of size $N > 1$.

Solving an N-dimensional non-linear system via Newton-Raphson requires inverting an $N \times N$ Jacobian matrix at every iteration:

$$\mathrm{O}(N^3)$$

The Cellier-Elmqvist Minimum Degree Tearing algorithm identifies a minimal set of k **tearing variables** ($k \ll N$) such that assuming known values for these k variables breaks all cyclic dependencies in the block. The remaining $N - k$ inner variables can then be evaluated sequentially via a causal explicit forward substitution chain:

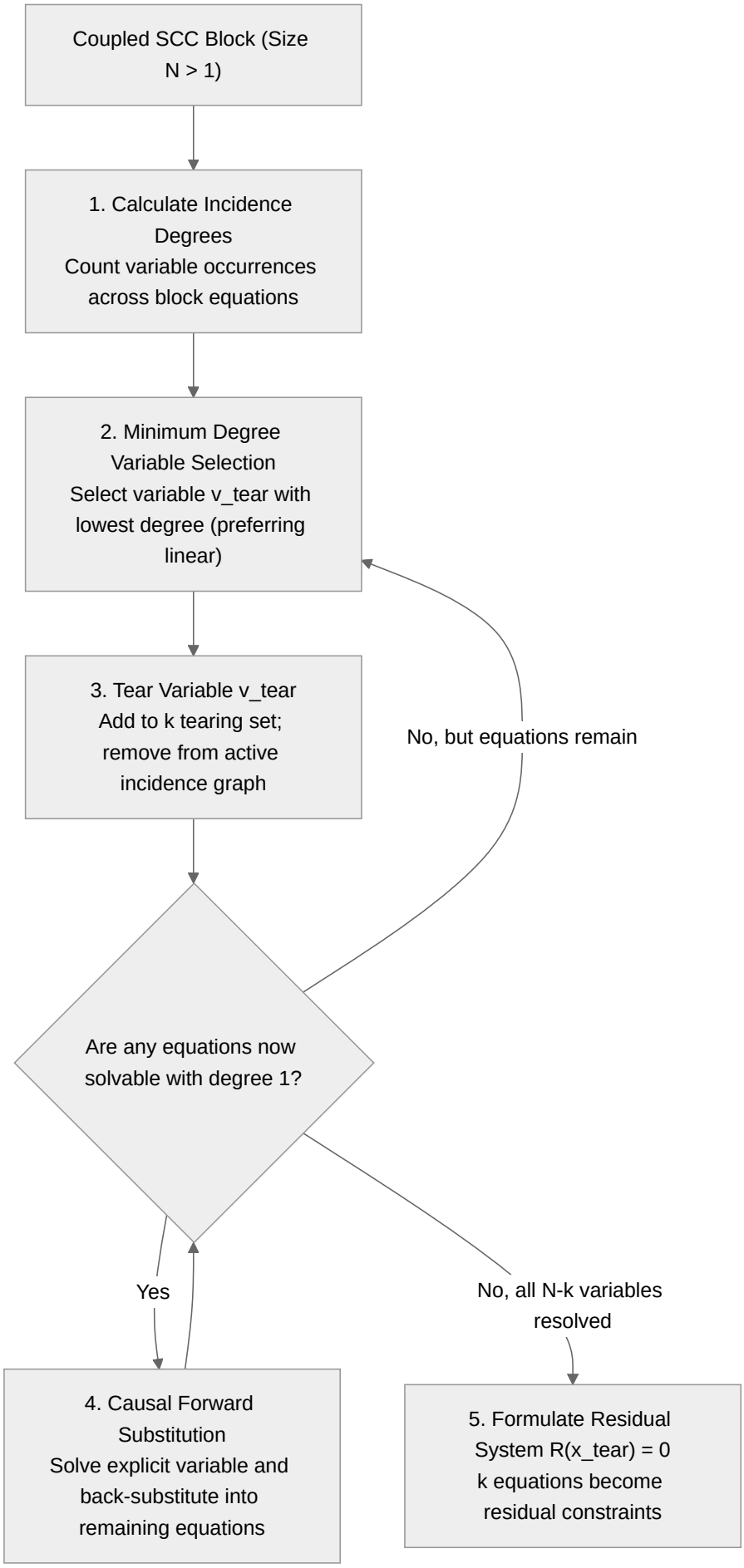
$$x_{\text{inner}} = g(x_{\text{tear}}, u) \quad \text{(explicit chain)}$$
$$R(x_{\text{tear}}) = f_{\text{residual}}(x_{\text{tear}}, x_{\text{inner}}) = 0 \quad \text{(small } k \times k \text{ solve)}$$

The non-linear iterative solver is restricted to the tiny $k \times k$ residual system, accelerating simulation speed by orders of magnitude.

ModelScript Modifications & WASM Architecture

1. **Unmanaged WebAssembly Kernel**: Implemented as `@unmanaged` classes directly in linear memory with zero garbage collection pauses.
2. **Dynamic Minimum Degree Heuristic**: Evaluates vertex degrees in the block incidence graph in memory, prioritizing linear tearing variables over non-linear variables to avoid non-linear solving where possible.
3. **Partitioned In-Memory Structures**: Splits the block into:
   - k tearing variables (`tearVarIndices`)
   - $N - k$ explicit causal equations (`innerEqIndices`)
   - k residual constraint equations (`residualEqIndices`)
4. **Direct Matrix Integration**: Emits residual Jacobians directly to the in-WASM LU factorizer (`luFactor` , `luSolve`).

Algorithm Steps


Coupled SCC Block (Size N > 1)
1. Calculate Incidence Degrees
Count variable occurrences across block equations
2. Minimum Degree Variable Selection
Select variable v_tear with lowest degree (preferring linear)
3. Tear Variable v_tear
Add to k tearing set; remove from active incidence graph
Are any equations now solvable with degree 1?
No, but equations remain
Yes
No, all N-k variables resolved
4. Causal Forward Substitution
Solve explicit variable and back-substitute into remaining equations
5. Formulate Residual System R(x_tear) = 0
k equations become residual constraints

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - Receives coupled SCC blocks of size > 1 from `BltEngine`.
- **Downstream Consumers**:
  - Feeds explicit forward chains and $k \times k$ residual systems to the in-WASM Newton-Raphson solver and `Sparse LU Solver`.
  - Evaluated inside simulation step loops (`SUNDIALS`, `TR-BDF2`, `Tsit5`).

CHAPTER 29

Acausal Connector Balancing & Stream Mixing

Implementation: `ConnectionSetResolver` & `IntUnionFind` in `languages/modelica/src/connections.ts`

Academic Citations

Acausal Physical Modeling & Connection Sets

- **Mattsson, S. E., Elmqvist, H., & Otter, M. (1998)**. "*Physical system modeling with Modelica.*"
  Control Engineering Practice, 6(4), pp. 501–510.
  DOI: 10.1016/S0967-0661(98)00047-1
- **Tarjan, R. E. (1975)**. "*Efficiency of a good but not linear set union algorithm.*"
  Journal of the ACM, 22(2), pp. 215–225.
  DOI: 10.1145/321879.321884 *(Disjoint-Set Union-Find)*

Stream Connectors & Convective Transport

- **Franke, R., Casella, F., Otter, M., Sielemann, M., Elmqvist, H., Mattsson, S. E., & Olsson, H. (2009)**. "*Stream Connectors - An Extension of Modelica for Device-Oriented Modeling of Convective Transport.*"
  Proceedings of the 7th Modelica Conference, pp. 116–123.
  DOI: 10.3384/ecp09430076

Expandable Connectors & Multi-Domain Bus Modeling

- **Otter, M., Elmqvist, H., & Mattsson, S. E. (2005)**. "*Object-oriented modeling of bus systems with Modelica.*"
  Proceedings of the 4th Modelica Conference, pp. 295–308.
  Paper PDF

ModelScript Architectural Rationale

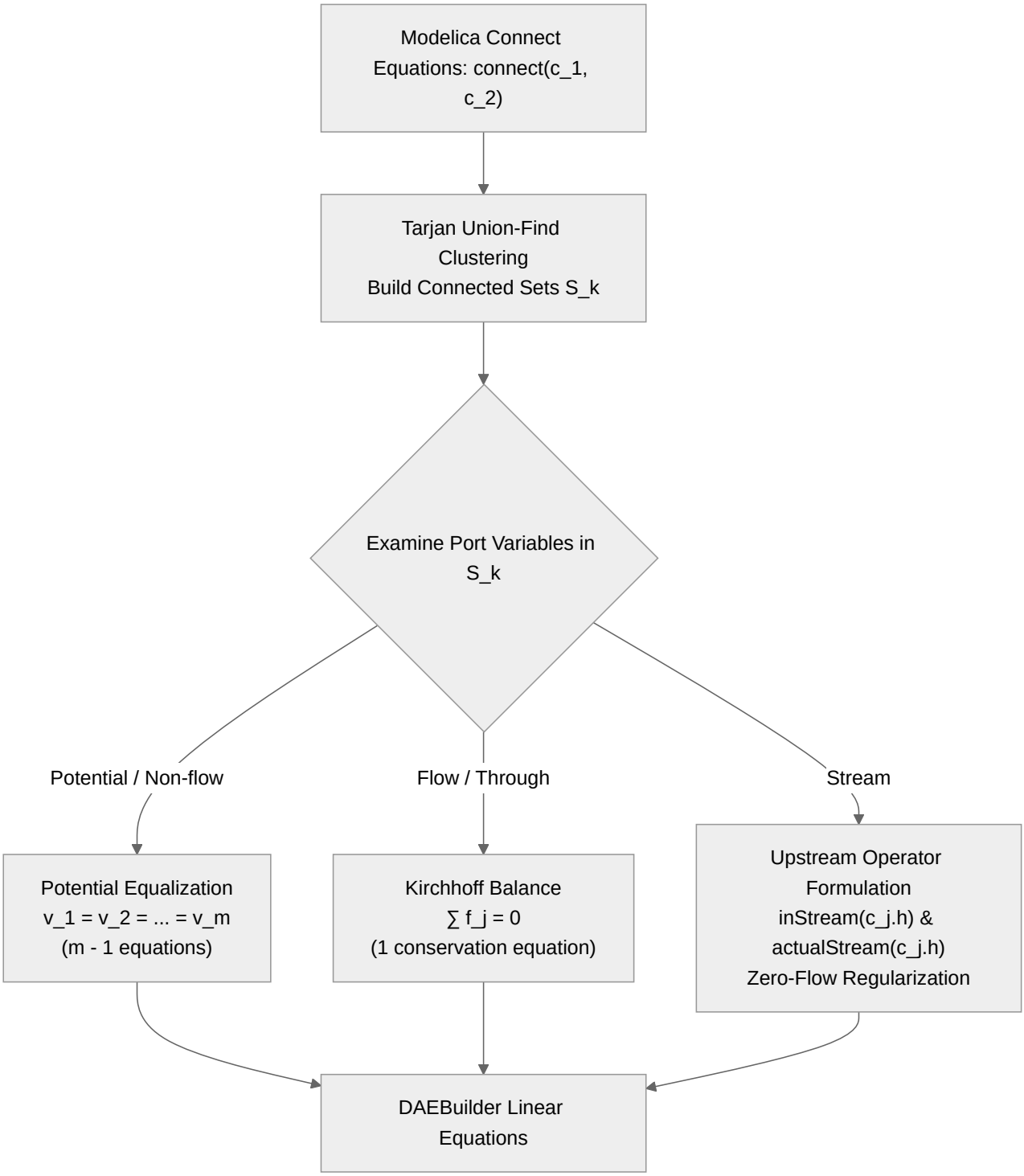
Unlike classical block diagrams (such as causal signal-flow block diagrams) where connections represent directional causal signal flows ($y = f(u)$), ModelScript models **acausal physical interaction** across multi-energy domains:

- **Electrical**: Voltage v (potential) and Current i (flow).
- **Hydraulic & Pneumatic**: Pressure p (potential) and Volume/Mass flow rate q (flow).
- **Translational Mechanics**: Position/Velocity s, v (potential) and Force f (flow).
- **Rotational Mechanics**: Angle/Angular velocity ϕ, ω (potential) and Torque τ (flow).
- **Thermal & Fluid Transport**: Temperature/Enthalpy h (stream) and Heat/Mass flow $\dot{m}$ (flow).

In acausal modeling, connection statements `connect(a, b)` do not dictate computational causality. The compiler must automatically generate:

1. **Potential Equality Equations**: Enforcing that all across-variables in a connection set share an identical value.
2. **Kirchhoff Flow Equations**: Enforcing conservation of mass, charge, or energy such that all through-variables sum to zero at the junction.
3. **Convective Stream Transport Equations**: Resolving thermodynamic mixing when fluid direction reverses ($m_j \gtrless 0$) without introducing singular divisions near zero mass flow.

Balancing Algorithm & Mathematical Formulation



1. Disjoint Connection Set Resolution (Union-Find)

The set of all declared `connect(c_a, c_b)` statements forms an undirected graph. ModelScript partitions all connector references into disjoint equivalence classes using an integer array-based Union-Find with path compression and rank heuristics:

$$\alpha(N) \leq 4 \quad \text{for all physical models}$$

For each connection set $S = \{c_1, c_2, \ldots, c_m\}$ of size m:

2. Potential Variable Equalization

For every non-flow, non-stream variable v declared inside the connector type:

$$v(c_1) = v(c_2) = \cdots = v(c_m)$$

This yields exactly $m - 1$ scalar equations. One variable is chosen as the canonical representative, eliminating $m - 1$ redundant states during alias elimination.

3. Kirchhoff Flow Variable Balancing

For every `flow` variable f declared inside the connector type:

$$\sum_{j=1}^{m} f(c_j) = 0$$

This yields exactly 1 scalar conservation equation per connection set. If an inside-outside connector hierarchy boundary is crossed, the outer flow sign is inverted to preserve directional conservation:

$$f_{\text{outer}} = \sum_{j \in \text{inner}} f(c_j)$$

4. Stream Variable Convective Transport

For fluid ports, transport enthalpy depends on flow direction:

$$h_{\text{transported}} = \begin{cases} h_{\text{upstream}} & \text{if } \dot{m} > 0 \\ h_{\text{downstream}} & \text{if } \dot{m} < 0 \end{cases}$$

Direct formulation introduces a discontinuous switch that causes Newton solvers to oscillate around $\dot{m} = 0$. ModelScript lowers stream variables using Franke's regularized `inStream()` and `actualStream()` operators:

$$\text{inStream}(c_i.h) = \frac{\sum_{j=i, \dot{m}_j > 0} \dot{m}_j \cdot c_j.h}{\sum_{j=i, \dot{m}_j > 0} \dot{m}_j}$$

Near zero flow ($\|\dot{m}_j\| < \epsilon$), smooth polynomial regularization is applied to maintain C^1-continuity of the residual Jacobian.

Upstream & Downstream Integration

Pipeline Component	Upstream Dependencies	Downstream Consumers
IntUnionFind	AST `connect` clauses from `Cst`	Connection set partition groups
Potential Equalizer	Variable causality & variability from `QueryEngine`	Alias elimination (`eliminateArenaAliases`)
Flow Balancer	Connector definitions	`DAEBuilder` linear equations
Stream Formulator	Fluid connector declarations	`PantelidesEngine` , Nonlinear tearing loops

CHAPTER 30

Automatic Differentiation & Relaxations

Implementation:

- Forward Mode: `Dual` in `packages/runtime/src/wasm/autodiff/dual.ts`
- Reverse Mode: `AdTape` in `packages/runtime/src/wasm/autodiff/tape.ts`
- Sparse Jacobian Coloring: `CCSMatrix` & `ColoringEngine` in `packages/runtime/src/wasm/autodiff/coloring.ts`
- McCormick Envelopes: `McCormickTuple` in `packages/runtime/src/wasm/autodiff/mccormick.ts`
- Affine Arithmetic: `AffineForm` in `packages/runtime/src/wasm/autodiff/affine.ts`
- Hybrid Adjoint Sensitivity & Saltation Matrices: `HybridAdjointSolver` in `packages/simulate/src/core/hybrid-adjoint.ts`
- Aitken Dynamic Δ-Relaxation & Multi-Rate Sub-Cycling: `VectorAitkenRelaxation` & `MultiRateSubCycleScheduler` in `packages/simulate/src/core/fsi-relaxation.ts`

Academic Citations

Forward-Mode Dual Numbers

- **Clifford, W. K. (1873)**. "*Preliminary sketch of biquaternions.*"
  Proceedings of the London Mathematical Society, 4, pp. 381–395.
  DOI: 10.1112/plms/s1-4.1.381
- **Study, E. (1903)**. *Geometrie der Dynamen: Die Zusammensetzung von Kräften und verwandte Gegenstände der Geometrie.* B.G. Teubner.
- **Rall, L. B. (1981)**. *Automatic Differentiation: Techniques and Applications.*
  Lecture Notes in Computer Science, 120, Springer.
  DOI: 10.1007/3-540-10861-0

Reverse-Mode Adjoint Accumulation (Wengert List)

- **Wengert, R. E. (1964)**. "*A simple automatic derivative evaluation program.*"
  Communications of the ACM, 7(8), pp. 463–464.
  DOI: 10.1145/355586.364791
- **Linnainmaa, S. (1976)**. "*Taylor expansion of the accumulated rounding error.*"
  BIT Numerical Mathematics, 16(2), pp. 146–160.
  DOI: 10.1007/BF01931367
- **Griewank, A., & Walther, A. (2008)**. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation* (2nd ed.). SIAM.
  ISBN: 978-0-898716-59-7

Distance-2 Graph Coloring for Sparse Jacobians

- **Curtis, A. R., Powell, M. J. D., & Reid, J. K. (1974)**. "*On the estimation of sparse Jacobian matrices.*"
  Journal of the Institute of Mathematics and Its Applications, 13(1), pp. 117–119.
  DOI: 10.1093/imamat/13.1.117 *(CPR Algorithm)*
- **Coleman, T. F., & Moré, J. J. (1983)**. "*Estimation of sparse Jacobian matrices and graph coloring problems.*"
  SIAM Journal on Numerical Analysis, 20(1), pp. 187–209.
  DOI: 10.1137/0720013
- **Gebremedhin, A. H., Manne, F., & Pothen, A. (2005)**. "*What color is your Jacobian? Graph coloring for computing derivatives.*"
  SIAM Review, 47(4), pp. 629–705.
  DOI: 10.1137/S0036144504446096

McCormick Relaxations & Affine Arithmetic

- **McCormick, G. P. (1976)**. "*Computability of global solutions to factorable nonconvex programs: Part I—Convex underestimating problems.*"

Mathematical Programming, 10(1), pp. 147–175.
DOI: 10.1007/BF01580665

- **Mitsos, A., Chachuat, B., & Barton, P. I. (2009)**. "*McCormick-based relaxations of algorithms.*"
SIAM Journal on Optimization, 20(2), pp. 573–601.
DOI: 10.1137/080717341
- **Tsoukalas, A., & Mitsos, A. (2014)**. "*Multivariate McCormick relaxations.*"
Journal of Global Optimization, 59(2), pp. 633–662.
DOI: 10.1007/s10898-014-0169-z
- **de Figueiredo, L. H., & Stolfi, J. (2004)**. "*Affine arithmetic: concepts and applications.*"
Numerical Algorithms, 37(1), pp. 147–158.
DOI: 10.1023/B:NUMA.0000049462.70970.b6

Hybrid Adjoint Sensitivity & Saltation Matrices

- **Barton, P. I., & Lee, C. K. (2002)**. "*Modeling, simulation, sensitivity analysis, and optimization of hybrid systems.*"
ACM Transactions on Modeling and Computer Simulation, 12(4), pp. 256–289.
DOI: 10.1145/643120.643122
- **Filippov, A. F. (1988)**. *Differential Equations with Discontinuous Righthand Sides.*
Mathematics and Its Applications, 18, Springer.
DOI: 10.1007/978-94-015-7793-9
- **Corner, S., Sandu, A., & Sandu, C. (2019)**. "*Adjoint sensitivity analysis of hybrid multibody dynamical systems.*"
Multibody System Dynamics, 49, pp. 395–420.
DOI: 10.1007/s11044-019-09725-2

Aitken Dynamic Δ-Relaxation for Partitioned Co-Simulation

- **Küttler, U., & Wall, W. A. (2008)**. "*Fixed-point fluid–structure interaction solvers with dynamic relaxation.*"
Computational Mechanics, 43(1), pp. 61–72.
DOI: 10.1007/s00466-008-0255-5
- **Irons, B. M., & Tuck, R. C. (1969)**. "*A version of the Aitken accelerator for computer iteration.*"
International Journal for Numerical Methods in Engineering, 1(3), pp. 275–277.
DOI: 10.1002/nme.1620010306

ModelScript Architectural Rationale

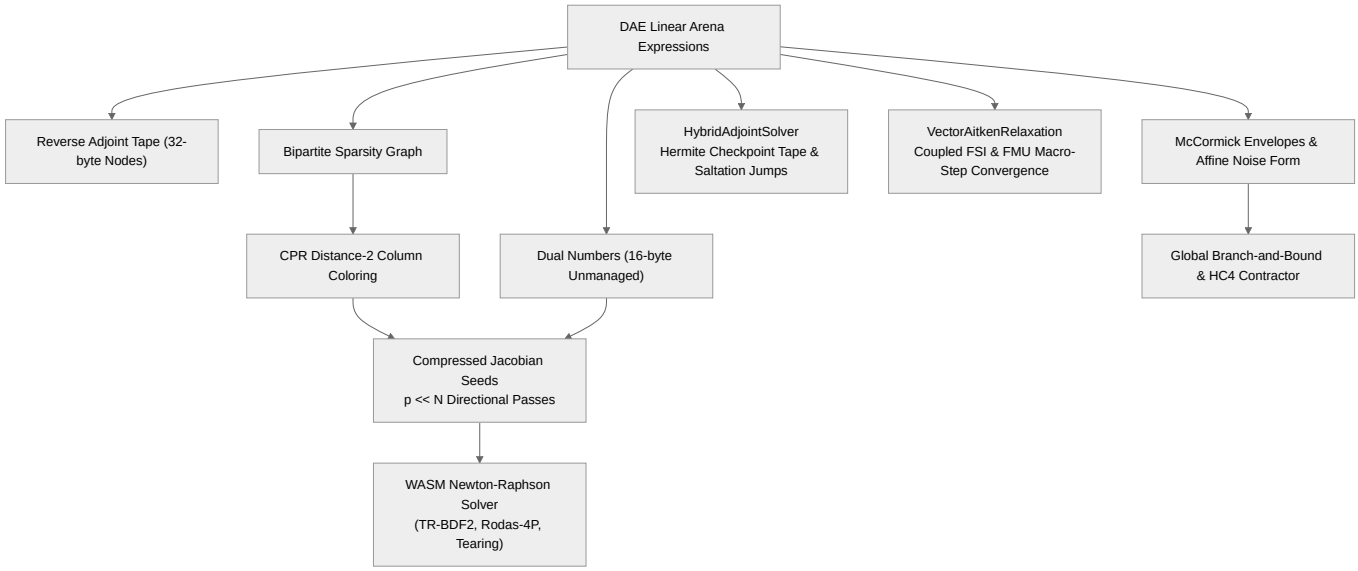
Numerical finite-difference approximations ($\frac{f(x+h)-f(x)}{h}$) suffer from a fundamental trade-off: large step sizes h cause truncation errors, while small step sizes cause catastrophic floating-point cancellation.

ModelScript implements an integrated suite of **exact differentiation and continuous bounding engines** inside WebAssembly linear memory:

1. **Forward-Mode Dual Numbers**: Computes exact machine-precision directional derivatives $\nabla f(x) \cdot v$ concurrently with primal function evaluation for Newton steps in algebraic loops and stiff ODE/DAE integrators without allocating expression trees.
2. **Reverse-Mode Computational Tape**: Evaluates scalar objective gradients $\nabla f(x) \in \mathrm{R}^N$ with cost proportional to $O(1)$ evaluations of f, enabling gradient-based parameter estimation and surrogate neural network training.
3. **CPR Distance-2 Graph Coloring**: For large-scale DAE systems ($N = 10{,}000$+ equations), evaluating the full $N \times N$ Jacobian naively requires N forward passes. CPR distance-2 coloring partitions structural columns into $p \ll N$ non-interfering groups (typically $p \leq 15$ for sparse physical networks), reducing Jacobian assembly time by orders of magnitude.
4. **McCormick Envelopes & Affine Arithmetic**: Classical interval arithmetic suffers from the **dependency problem**: evaluating $x - x$ for $x \in [0, 1]$ yields $[-1, 1]$ instead of 0, causing rapid exponential bounding blowup. Affine arithmetic tracks first-order noise symbols ($\epsilon_i \in [-1, 1]$) across state equations, while McCormick relaxations compute rigorous convex lower bounds (f^R) and concave upper bounds (f^{AR}) for non-convex global optimization.
5. **Hybrid Adjoint Sensitivity with Saltation Matrices**: Standard continuous adjoint solvers break down across Modelica `when` / `reinit` events, impact mechanics, or switching surfaces because Dirac delta impulses arise at event-crossing times t^*. ModelScript propagates reverse-mode adjoints across non-smooth mode transitions using exact saltation matrix jump conditions alongside optional sigmoidal equation smoothing (`DAEEventRelaxer`).
6. **Aitken Dynamic Δ-Relaxation**: Partitioned co-simulation of strongly coupled multi-physics domains (such as lightweight aerodynamic structures coupled to dense fluids or algebraic FMU loops) diverges under explicit Gauss-Seidel macro-steps due

to the artificial added-mass effect. Dynamic vector Aitken acceleration stabilizes fixed-point iterations without requiring monolithic cross-FMU Jacobians.

ModelScript Modifications & WASM Architecture



1. Zero-GC Dual Number Arithmetic

Represented as a 16-byte unmanaged struct (`@unmanaged`) in WASM memory:

```
@unmanaged
export class Dual {
  val: f64; // Primal evaluation f(x)
  dot: f64; // Directional derivative f'(x) * v
}
```

All elementary operators ($+, -, \times, /$) and transcendental functions ($\sin, \cos, \exp, \log, \text{pow}$) are inlined without intermediate object allocation.

2. Linear Memory Computational Tape

The reverse-mode tape records operations into a flat 32-byte stride unmanaged array:

$$\text{Stride: } [\text{op: u32}, \text{left: u32}, \text{right: u32}, \text{aux: u32}, \text{valLo: u32}, \text{valHi: u32}, \text{gradLo: u32}, \text{gradHi: u32}]$$

During the reverse pass, adjoints are accumulated in topological order directly in linear memory:

$$\bar{x}_j \leftarrow \bar{x}_j + \bar{x}_i \cdot \frac{\partial f_i}{\partial x_j}$$

3. CPR Distance-2 Coloring Algorithm

Two columns j and k in the Jacobian J are **structurally orthogonal** if they do not share any non-zero row entry:

$$\text{Row}(j) \cap \text{Row}(k) = \emptyset$$

The graph coloring algorithm:

1. Constructs the bipartite graph $B = (V_R, V_C, E)$ where edge $(i, j) \in E \iff \frac{\partial f_i}{\partial x_j} = 0$.
2. Colors columns V_C such that any pair of columns connected by a path of length 2 receives different colors.
3. Groups columns of color $c \in \{1, \dots, p\}$ into a perturbation vector $d^{(c)} = \sum_{j \in \text{Color}(c)} e_j$.

4. Computes the entire compressed Jacobian in exactly p directional forward-mode passes:

$$J \cdot d^{(c)} = \sum_{j \in \text{Color}(c)} J_{*,j}$$

4. McCormick Relaxations

For factorable bilinear products $z = x \cdot y$ over bounds $x \in [x^L, x^U]$ and $y \in [y^L, y^U]$, the convex underestimator z^{cv} and concave overestimator z^{cc} are:

$$z^{cv} = \max\left(x^L y + x y^L - x^L y^L,\ x^U y + x y^U - x^U y^U\right)$$

$$z^{cc} = \min\left(x^U y + x y^L - x^U y^L,\ x^L y + x y^U - x^L y^U\right)$$

ModelScript evaluates McCormick tuples $[z^{cv}, z^{cc}, \nabla z^{cv}, \nabla z^{cc}]$ directly on WASM arrays.

5. Hybrid Adjoint Sensitivity with Saltation Matrices (`hybrid-adjoint.ts`)

When a hybrid DAE trajectory crosses a discrete event surface $g(x(t^*), p) = 0$ with state reset map $x(t^{*+}) = h(x(t^{*-}), p)$ (such as Modelica `when` clauses, `reinit(x, expr)` impulses, or mechanical contact restitution), the forward sensitivity transition matrix undergoes a discontinuous rank-1 update governed by the **saltation matrix**:

$$\Xi = I + \frac{(f^+ - f^- - \frac{\partial h}{\partial t})(\nabla_x g)^T}{(\nabla_x g)^T f^- + \frac{\partial g}{\partial t}}$$

`HybridAdjointSolver` (`solveHybridAdjoint` in `packages/simulate/src/core/hybrid-adjoint.ts`) implements a three-phase hybrid sensitivity pipeline:

1. **Forward Integration & Event Localization**: Integrates the primal DAE arena while streaming cubic Hermite segments into `DenseHermiteCheckpointTape`. Whenever an event indicator $h(t, x, p)$ changes sign across a step, a 20-iteration bisection root-finder isolates the exact crossing time t_e, records pre-event limits $(x^-, \dot{x}^-)$, applies the discrete reset map $x^+ = \Delta(x^-, p)$, and re-evaluates post-event derivatives $(x^+, \dot{x}^+)$.
2. **Adjoint Jump across Event Surfaces**: During backward Heun integration of the co-state $\lambda(t)$, crossing t_e in reverse applies the transposed saltation jump condition to $\lambda(t_e^-)$ and accumulates the discrete event timing contribution into the parameter gradient $\frac{dL}{dp}$:

$$\lambda(t_e^-) = \left[\frac{\partial \Delta}{\partial x}\right]^T \lambda(t_e^+) + \frac{\left(\frac{d\Delta}{dt}\right)^- - \dot{x}^+}{\dot{h}^-} \nabla_x h(t_e^-)$$

$$\Delta\left(\frac{dL}{dp}\right) = \left[\frac{\partial \Delta}{\partial p}\right]^T \lambda(t_e^+) + \frac{\left(\frac{d\Delta}{dt}\right)^- - \dot{x}^+}{\dot{h}^-} \nabla_p h(t_e^-)$$

3. **Optional Sigmoidal Event Relaxation**: For models with dense chattering or non-smooth algebraic switches (`if`, `abs`, `min`, `max`), `HybridAdjointSolver` optionally invokes `DAEEventRelaxer` to smooth C0 discontinuities into C^∞ sigmoidal transitions prior to adjoint integration.

6. Aitken Dynamic Δ-Relaxation for Coupled Co-Simulation (`fsi-relaxation.ts`)

In partitioned Fluid-Structure Interaction (FSI) and multi-FMU co-simulation macro-steps, exchanging boundary loads and displacements via unrelaxed fixed-point iteration $x_{k+1} = \tilde{x}(x_k)$ often diverges due to artificial added-mass instability.

`VectorAitkenRelaxation` (`packages/simulate/src/core/fsi-relaxation.ts`) accelerates and stabilizes convergence over scalar variables, multi-DOF nodal displacement vectors, or 3D boundary traction maps (`relaxBoundaryLoads`):

1. **Fixed-Point Residual**: At macro-step sub-iteration k, computes the interface residual $r_k = \tilde{x}_k - x_{k-1}$ and residual difference $\Delta r_k = r_k - r_{k-1}$.
2. **Secant Aitken Factor Update**: Updates the dynamic relaxation factor ω_k via the multidimensional Aitken Δ^2 projection:

$$\omega_k = -\omega_{k-1} \frac{r_{k-1}^T (r_k - r_{k-1})}{\|r_k - r_{k-1}\|_2^2}$$

clamped to a safe stability interval $\omega_k \in [\omega_{\min}, \omega_{\max}]$ (default $[0.02, 1.0]$).

3. **Relaxed State Update**: Advances the interface state via under-relaxation:

$$x_{k+1} = x_k + \omega_k r_k$$

terminating when the relative residual norm $\|r_k\|_2 / (\|x_k\|_2 + \epsilon) < \text{tol}$.

4. **Multi-Rate Sub-Cycling (`MultiRateSubCycleScheduler`)**: Synchronizes fast acoustic CFD steps ($\Delta t_{\text{CFD}} \sim 10^{-4}$ s), intermediate elastodynamic FEA steps ($\Delta t_{\text{FEA}} \sim 10^{-3}$ s), and macro 1D DAE steps ($\Delta t_{\text{macro}} \sim 10^{-2}$ s) using linear sub-step boundary interpolation $\mathbf{u}(\tau) = (1 - \tau)\mathbf{u}_{\text{prev}} + \tau \mathbf{u}_{\text{target}}$.

Upstream & Downstream Integration

Pipeline Component	Upstream Dependencies	Downstream Consumers
Dual Numbers	`DAEBuilder` AST, Variable seed vector	Algebraic loop tearing, Newton solvers, sensitivity analysis
AdTape	`DAEBuilder` objective expressions	L-BFGS-B optimizer, surrogate loss backprop
CPR Coloring	Bipartite incidence matrix from `BltEngine`	Sparse Jacobian generation, SUNDIALS/TR-BDF2 linear solves
McCormick & Affine	Bound intervals from `QueryEngine`	HC4 contractor, Nelson-Oppen theory coordinator, SOS bounds
HybridAdjointSolver	`DAEBuilder`, `DenseHermiteCheckpointTape`, Events	Hybrid parameter calibration, optimal control across mode switches
VectorAitkenRelaxation	Partitioned CFD/FEA boundary loads & FMU ports	Coupled aeroelastic FSI solver, SSP co-simulation master

CHAPTER 31

Numerical ODE, DAE & Co-Simulation Solvers

Implementation: `packages/simulate/src/solvers/` and `packages/runtime/src/solvers/`

---

1. Tsitouras 5(4) Adaptive Runge-Kutta (Tsit5)

Academic Citations

- **Tsitouras, Ch. (2011)**. "*Runge-Kutta pairs of order 5 (4) satisfying only the first column simplifying assumption.*"
  Computers & Mathematics with Applications, 62(2), pp. 770–775.
  DOI: 10.1016/j.camwa.2011.06.002
- **Rackauckas, C., & Nie, Q. (2017)**. "*DifferentialEquations.jl – A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia.*"
  Journal of Open Research Software, 5(1), 15. DOI: 10.5334/jors.151

ModelScript Rationale & Modifications

For non-stiff systems (e.g. mechanical multi-body dynamics, kinematics, flight control), high-order adaptive explicit Runge-Kutta pairs offer peak throughput without Newton-Raphson linear algebra costs. Tsit5 demonstrates significantly lower error coefficients and wider stability regions than classical Dormand-Prince (DOPRI5).

- **Modifications**: Employs the First-Same-As-Last (FSAL) optimization eliminating stage 1 evaluation in successive steps; embeds a 5th-order continuous dense output polynomial for sub-interval event zero-crossing localization.

---

2. TR-BDF2 Composite Stiff Solver

Academic Citations

- **Bank, R. E., Coughran, W. M., et al. (1985)**. "*Transient simulation of silicon devices and circuits.*"
  IEEE Transactions on CAD of Integrated Circuits, 4(4), pp. 436–446.
  DOI: 10.1109/TCAD.1985.1270142
- **Hosea, M. E., & Shampine, L. F. (1996)**. "*Analysis and implementation of TR-BDF2.*"
  Applied Numerical Mathematics, 20(1-2), pp. 21–37.
  DOI: 10.1016/0168-9274(95)00115-8

ModelScript Rationale & Modifications

Cyber-physical models featuring frequent hybrid discrete events (switches, impacts, hysteresis) severely penalize multistep BDF solvers (which must drop to order 1 after every event). TR-BDF2 is an L-stable one-step composite method pairing a trapezoidal half-step with a BDF2 completion step:

- **Modifications**: Shares the diagonal Jacobian coefficient $d = 1 - 1/\sqrt{2}$ across both internal stages, requiring only **one** sparse LU factorization per time step; connects directly with ModelScript's in-WASM sparse matrix kernel (`sparse_lu.ts`).

---

3. RODAS-4P Linearly Implicit Rosenbrock Solver

Academic Citations

- **Steinebach, G. (1995)**. "*Order-reduction of ROW-methods for DAEs and method of lines applications.*"
  TH Darmstadt Preprint 1742.
- **Hairer, E., & Wanner, G. (1996)**. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*.
  Springer-Verlag. Section IV.8.

ModelScript Rationale & Modifications

Solves stiff index-1 DAEs and method-of-lines continuum discretizations without iterative non-linear Newton solves. It solves only a sequence of linear systems sharing the same system matrix $I - \gamma h J$.

- **Modifications**: Optimized for in-WASM sparse Jacobian caching; utilizes automatic differentiation coloring (`coloring.ts`) for fast Jacobian updates.

---

4. Itô Stochastic Differential Equations (SDE) Solver

Academic Citations

- **Rößler, A. (2010)**. "*Runge-Kutta methods for Itô stochastic differential equations with additive noise.*"
  SIAM Journal on Numerical Analysis, 48(3), pp. 922–952. DOI: 10.1137/09076636X
- **Kloeden, P. E., & Platen, E. (1992)**. *Numerical Solution of Stochastic Differential Equations*.
  Springer. DOI: 10.1007/978-3-662-12616-5

ModelScript Rationale & Modifications

Physical systems operate under thermal noise, turbulence, and sensor fluctuations:

$$dx = f(t, x)\,dt + g(t, x)\,dW_t$$

- **Modifications**: Employs an in-WASM `Xoshiro256pp` PRNG with Box-Muller transformation; provides both fixed-step Euler-Maruyama (strong order 0.5) and adaptive Rößler SRIW1 (strong order 1.5) integrators for Monte Carlo uncertainty rollouts.

---

5. Co-Simulation Master Orchestrator

Academic Citations

- **Bastian, J., et al. (2011)**. "*Master for Co-Simulation Using FMI.*"
  Proceedings of the 8th International Modelica Conference, pp. 115–120. DOI: 10.3384/ecp11063115
- **Kübler, R., & Schiehlen, W. (2000)**. "*Two methods of simulator coupling.*"
  Mathematical and Computer Modelling of Dynamical Systems, 6(2), pp. 93–113. DOI: 10.1076/1387-3954(200006)6:2;1-M;FT093

ModelScript Rationale & Modifications

Coordinates multi-rate, multi-vendor FMI Functional Mock-up Units (FMUs) and SSP topologies. Implements Richardson extrapolation error estimation to dynamically adapt the inter-simulator communication step size H_{comm}, preventing divergence while maximizing simulation speed.

---

6. Dommel Companion-Circuit EMT & Bergeron Traveling-Wave Solver

Academic Citations

- **Dommel, H. W. (1969)**. "*Digital Computer Solution of Electromagnetic Transients in Single- and Multiphase Networks.*"
  IEEE Transactions on Power Apparatus and Systems, PAS-88(4), pp. 388–399.
  DOI: 10.1109/TPAS.1969.292459
- **Marti, J. R. (1982)**. "*Accurate Modelling of Frequency-Dependent Transmission Lines in Electromagnetic Transient Simulations.*"
  IEEE Transactions on Power Apparatus and Systems, PAS-101(1), pp. 147–157.
  DOI: 10.1109/TPAS.1982.317332

ModelScript Rationale & Modifications

Implemented in `EmtSolver` (`packages/simulate/src/solvers/emt/emt-solver.ts`), this solver targets microsecond-scale electromagnetic transients, power electronics switching (IGBTs/breakers), and distributed transmission lines imported from `@modelscript/pscad` :

- **Trapezoidal Companion Models & Constant Nodal Matrix**: Inductors ($G_L = \frac{\Delta t}{2L}$) and capacitors ($G_C = \frac{2C}{\Delta t}$) are discretized into Norton companion equivalents $\mathbf{G}\mathbf{v}(t) = \mathbf{i}_{\text{hist}}(t - \Delta t) + \mathbf{i}_{\text{src}}(t)$, allowing $\mathbf{G}$ to remain constant between switching events.
- **Critical Damping Adjustment (CDA)**: Automatically transitions from Trapezoidal to Backward Euler integration ($G_L = \frac{\Delta t}{L}, G_C = \frac{C}{\Delta t}$) for two half-steps ($\Delta t/2$) immediately after any breaker or semiconductor switching discontinuity to suppress artificial dv/dt numerical oscillations.
- **Bergeron Traveling-Wave Line Decoupling & Norton FMU Interface**: Distributed transmission lines with propagation delay $\tau = \ell/v \geq \Delta t$ decouple the nodal admittance matrix into independent sub-grids using interpolated history ring buffers, and expose `getNortonEquivalent(portNode)` ($I_{\text{eq}}, G_{\text{eq}}$) for zero-delay FMI Co-Simulation coupling.

---

7. High-Order Taylor Model Flowpipe Integration

Academic Citations

- **Makino, K., & Berz, M. (2003)**. "*Taylor Models and Other Validated Functional Inclusion Methods.*" **International Journal of Pure and Applied Mathematics**, 4(4), pp. 379–456.
- **Chen, X., Ábrahám, E., & Sankaranarayanan, S. (2013)**. "*Flow*: An Analyzer for Non-Linear Hybrid Systems.*" **Computer Aided Verification (CAV 2013)**, LNCS vol. 8044, pp. 258–263. DOI: 10.1007/978-3-642-39799-8_18

ModelScript Rationale & Modifications

Implemented in `TaylorModel` (`packages/runtime/src/analysis/wasm_taylor_model.ts`), validated Taylor Model arithmetic bounds all trajectories of a non-linear ODE/DAE system subject to uncertain initial sets $x(0) \in X_0$ and parameter intervals without wrapping-effect blowup:

- **Polynomial + Interval Remainder Representation**: Represents each state reachable tube over normalized domain $\mathbf{z} \in [-1, 1]^d$ as a multivariate polynomial $P_k(\mathbf{z})$ of order k plus a sound interval remainder $I = [\underline{r}, \overline{r}]$, where truncated higher-order terms ($> k$) are conservatively swept into I.
- **Picard-Lindelöf Validated Operator**: Evaluates the contractive Picard integral operator $\mathrm{P}(X)(t) = X_0 + \int_0^t f(s, X(s))\, ds$ directly on Taylor models to construct guaranteed continuous-time flowpipes consumed by `ContinuousSafetyOracle` and SysML v2 hybrid state-machine reachability (`hybrid-flowpipe-bridge.ts`).

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - DAE blocks partitioned and torn by `BltEngine` and `TornBlock` .
  - Sparse Jacobians computed via `Dual Numbers & Graph Coloring` .
- **Downstream Consumers**:
  - Powers `msx simulate` , `msx optimize` , and the Web IDE charting canvas.
  - Feeds state trajectories and validated flowpipes into `Signal Temporal Logic (STL) Monitor & Theory Coordinator` and formal verification oracles.

CHAPTER 32

Mathematical Optimization & Calibration Solvers

Implementation: `packages/simulate/src/optimizer/solvers/`

---

1. Limited-Memory BFGS with Bounds (L-BFGS-B)

Academic Citations

- **Byrd, R. H., Lu, P., Nocedal, J., & Zhu, C. (1995)**. "*A limited memory algorithm for bound constrained optimization.*"
  SIAM Journal on Scientific Computing, 16(5), pp. 1190–1208. DOI: 10.1137/0916069
- **Zhu, C., Byrd, R. H., Lu, P., & Nocedal, J. (1997)**. "*Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization.*"
  ACM Transactions on Mathematical Software, 23(4), pp. 550–560. DOI: 10.1145/279232.279236

ModelScript Rationale & Modifications

Solves large-scale bound-constrained non-linear minimization:

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to } l \le x \le u$$

Crucial for parameter identification and direct collocation optimal control where thousands of discretized states and control bounds must be optimized.

- **Modifications**: Employs compact representation of the inverse Hessian approximation H_k using history buffers $m \in [5, 20]$ in typed arrays; Cauchy point computation with piecewise linear path search; exact line search with strong Wolfe conditions.

---

2. Covariance Matrix Adaptation Evolution Strategy (CMA-ES)

Academic Citations

- **Hansen, N., & Ostermeier, A. (2001)**. "*Completely derandomized self-adaptation in evolution strategies.*"
  Evolutionary Computation, 9(2), pp. 159–195. DOI: 10.1162/106365601750190398
- **Hansen, N. (2016)**. "*The CMA Evolution Strategy: A Tutorial.*"
  arXiv preprint: arXiv:1604.00772.

ModelScript Rationale & Modifications

Derivative-free black-box optimization on rugged, non-convex, non-separable objective functions (e.g. tuning PID controller gains over simulations with discontinuous switching events).

- **Modifications**: Tracks full covariance matrix $C \in \mathrm{R}^{n \times n}$ updates with rank-μ and rank-one update mechanisms; vectorized population sampling using WebAssembly-accelerated Cholesky decomposition; step-size control using cumulative path lengths (CSA).

---

3. Differential Evolution (DE)

Academic Citations

- **Storn, R., & Price, K. (1997)**. "*Differential Evolution – A Simple and Efficient Heuristic for global Optimization over Continuous Spaces.*"
  Journal of Global Optimization, 11(4), pp. 341–359. DOI: 10.1023/A:1008202821328

ModelScript Rationale & Modifications

Stochastic population-based global search algorithm utilizing vector differences for directional perturbations. Highly effective for parameter estimation against experimental CSV test bench data.

- **Modifications**: Supports `rand/1/bin`, `best/1/bin`, and `current-to-best/1` mutation strategies with adaptive crossover probability (CR) and scaling factor (F).

---

4. Particle Swarm Optimization (PSO)

Academic Citations

- **Kennedy, J., & Eberhart, R. (1995)**. "*Particle swarm optimization.*"
  Proceedings of IEEE ICNN'95, Vol. 4, pp. 1942–1948. DOI: 10.1109/ICNN.1995.488968
- **Shi, Y., & Eberhart, R. C. (1998)**. "*A modified particle swarm optimizer.*"
  IEEE Congress on Evolutionary Computation, pp. 69–73.

ModelScript Rationale & Modifications

Simulates social swarm dynamics for continuous global search:

$$v_i(t+1) = wv_i(t) + c_1r_1(p_i - x_i) + c_2r_2(g - x_i)$$

- **Modifications**: Implements constriction factors and dynamically declining inertia weights $w(t)$ to transition smoothly from global exploration to local exploitation.

---

5. Non-Dominated Sorting Genetic Algorithm II (NSGA-II)

Academic Citations

- **Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002)**. "*A fast and elitist multiobjective genetic algorithm: NSGA-II.*"
  IEEE Transactions on Evolutionary Computation, 6(2), pp. 182–197. DOI: 10.1109/4235.996017

ModelScript Rationale & Modifications

Solves multi-objective engineering trade-offs (e.g. simultaneously minimizing electric vehicle energy consumption while maximizing battery longevity and top speed).

- **Modifications**: Fast non-dominated sorting ($\mathrm{O}(MN^2)$ complexity) paired with crowding-distance assignment in continuous parameter spaces; outputs full Pareto fronts directly.

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - Trajectory residuals evaluated by `Simulation Solvers` against `CSV Datasets`.
  - Collocation constraint formulations from `msx optimize`.
- **Downstream Consumers**:
  - Powers `msx falsify` adversarial requirement falsification across SysML v2 / Modelica / CAD.
  - Powers `msx cosim optimize` for black-box multi-FMU tuning across SSP boundaries.
  - Powers `ModelicaCalibrator` with `hybrid-cmaes-lm` (CMA-ES global exploration + Levenberg-Marquardt quadratic polishing).
  - Powers `TradeStudyEngine.evolve` for generative multi-objective Pareto design trade studies.

Programmatic TypeScript API

```
import { cmaesSolve, deSolve, psoSolve, nsga2Solve, type BlackBoxProblem } from "@modelscript/simulate/optimizer";

// 1. Single-Objective Black-Box Optimization (CMA-ES, DE, PSO)
const problem: BlackBoxProblem = {
  dimension: 2,
  bounds: {
    min: new Float64Array([-5, -5]),
    max: new Float64Array([5, 5]),
  },
  fitness: (x) => Math.pow(1 - x[0]!, 2) + 100 * Math.pow(x[1]! - x[0]! * x[0]!, 2),
};

const result = await cmaesSolve(problem, { maxGenerations: 100 });
console.log("Optimum:", result.bestSolution, "Cost:", result.bestFitness);

// 2. Hybrid Global-Local Model Calibration
import { ModelicaCalibrator } from "@modelscript/simulate/optimizer";

const calibrator = new ModelicaCalibrator(arena, sim, {
  parameters: ["k", "c"],
  parameterBounds: new Map([
    ["k", { min: 1, max: 30 }],
    ["c", { min: 0.1, max: 10 }],
  ]),
  measurements,
  method: "hybrid-cmaes-lm", // Global CMA-ES basin exploration + Levenberg-Marquardt refinement
});
const calResult = calibrator.calibrate();
```

CLI Command Examples

```
# 1. Adversarial requirement falsification with CMA-ES
msx falsify --formula "always[0,10] (stress <= 180)" --algorithm cma-es --population 20

# 2. Black-box co-simulation parameter tuning with Differential Evolution
msx cosim optimize ssp-archive.ssp --params "kp:0.1:10.0,kd:0.01:1.0" --objective "error_integral" --algorithm de
```

CHAPTER 33

Nelson-Oppen Theory Combination & SMT Solvers

Implementation: `packages/runtime/src/formal/`

---

1. Nelson-Oppen Cooperation & DPLL(T)

Academic Citations

- **Nelson, G., & Oppen, D. C. (1979)**. "*Simplification by cooperating decision procedures.*"
  ACM Transactions on Programming Languages and Systems (TOPLAS), 1(2), pp. 245–257.
  DOI: 10.1145/357073.357079
- **Nieuwenhuis, R., Oliveras, A., & Tinelli, C. (2006)**. "*Solving SAT and SAT Modulo Theories: From an abstract Davis--Putnam--Logemann--Loveland procedure to DPLL(T).*"
  Journal of the ACM, 53(6), pp. 937–977.
  DOI: 10.1145/1217856.1217859

ModelScript Rationale & Modifications

Proves cross-domain assertions spanning discrete architectural modes, differential equations, interval constraints, and 3D CAD collisions.

- **Modifications**: Extended beyond traditional software theories (uninterpreted functions, bit-vectors) to **12 engineering domains** including OWL2 Description Logic, CAD B-Rep spatial physics, and DAE continuous trajectories; combines learned conflict clause propagation with Salsa query invalidation.

---

2. CDCL SAT Solver with VSIDS

Academic Citations

- **Silva, J. P. M., & Sakallah, K. A. (1999)**. "*GRASP: A search algorithm for propositional satisfiability.*"
  IEEE Transactions on Computers, 48(5), pp. 506–521. DOI: 10.1109/12.769433
- **Moskewicz, M. W., et al. (2001)**. "*Chaff: Accelerating SAT design verification.*"
  ACM/IEEE Design Automation Conference (DAC), pp. 530–535. DOI: 10.1145/378239.379017

ModelScript Rationale & Modifications

Solves propositional satisfiability for discrete mode selection, architectural state machines, and CDCL(T) theory case splitting.

- **Modifications**: Two-watched-literals data structure with contiguous integer arrays; Variable State Independent Decaying Sum (VSIDS) decision heuristic; 1-UIP (First Unique Implication Point) conflict analysis; Luby-sequence restart schedule.

---

3. Craig Interpolation

Academic Citations

- **Craig, W. (1957)**. "*Linear reasoning. A new form of the Herbrand-Gentzen theorem.*"
  Journal of Symbolic Logic, 22(3), pp. 250–268. DOI: 10.2307/2963593
- **McMillan, K. L. (2003)**. "*Interpolation and SAT-based model checking.*"
  Computer Aided Verification (CAV 2003), LNCS 2725, pp. 1–13. DOI: 10.1007/978-3-540-45069-6_1
- **Pudlák, P. (1997)**. "*Lower bounds for resolution and cutting planes proofs and monotone computations.*"
  Journal of Symbolic Logic, 62(3), pp. 981–998. DOI: 10.2307/2275583

ModelScript Rationale & Modifications

Given unsatisfiable mutually contradictory formulas $A \wedge B \models \bot$, Craig interpolation synthesizes an interpolant I such that:

$$A \implies I \quad \text{and} \quad I \wedge B \models \bot$$

where $\text{vars}(I) \subseteq \text{vars}(A) \cap \text{vars}(B)$.

- **Modifications**: Employs Pudlák-McMillan resolution graph traversal to generate certified interpolants explaining cross-domain requirement violations without revealing proprietary subsystem equations.

---

4. HC4 Constraint Contractor

Academic Citations

- **Benhamou, F., Goualard, F., Granvilliers, L., & Puget, J. F. (1999)**. "*Revising Hull and Box Consistency.*" **International Conference on Logic Programming (ICLP'99)**, pp. 230–244.

ModelScript Rationale & Modifications

Evaluates non-linear algebraic constraints over interval domains without discretization grids:

- **Modifications**: Two-phase HC4Revise operator: bottom-up forward evaluation of interval bounds followed by top-down backward narrowing; eliminates physically impossible state regions before numerical simulation.

---

5. Signal Temporal Logic (STL) Quantitative Monitor

Academic Citations

- **Maler, O., & Nickovic, D. (2004)**. "*Monitoring Temporal Properties of Continuous Signals.*" **FORMATS/FTRTFT 2004**, LNCS 3253, pp. 152–166. DOI: 10.1007/978-3-540-30206-3_12
- **Donzé, A., & Maler, O. (2010)**. "*Robust Satisfaction of Temporal Logic over Real-Valued Signals.*" **FORMATS 2010**, LNCS 6246, pp. 92–106. DOI: 10.1007/978-3-642-15297-9_9

ModelScript Rationale & Modifications

Monitors simulation trajectories against temporal specifications (e.g. "overshoot must remain below 5% for at least 2 seconds after step input"):

$$\rho(\varphi, s, t) \in \mathrm{R}$$

Computes real-valued **quantitative robustness** indicating how far a trajectory is from violating safety boundaries.

---

6. Constrained Zonotopes

Academic Citations

- **Scott, J. K., Raimondo, D. M., et al. (2016)**. "*Constrained zonotopes: A new tool for set-based estimation and fault detection.*" **Automatica**, 69, pp. 126–136. DOI: 10.1016/j.automatica.2016.02.036

ModelScript Rationale & Modifications

Set-based reachability analysis under bounded initial states and parameter uncertainty. Closed under linear transformations, Minkowski addition, and hyperplanar intersection without facet explosion.

- **Modifications**: Implemented as in-WASM linear algebraic routines utilizing singular value decomposition (SVD) for order reduction.

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:

- Requirements ingested from `SysML v2`.
- Trajectories simulated by `Numerical Solvers`.
- Spatial boundaries provided by `STEP CAD`.

- **Downstream Consumers**:
  - Powers `msx verify`, generates compliance certificates, and exports mathematical proofs in `LaTeX format`.

CHAPTER 34

Constrained Horn Clauses (Spacer), IC3/PDR & Certified Verification

Implementation:

- Spacer CHC & IC3/PDR: `packages/runtime/src/formal/chc/` and `packages/runtime/src/formal/ic3_engine.ts`
- Stochastic Martingale Barrier Certificates: `StochasticBarrierSynthesizer` in `packages/runtime/src/analysis/wasm_stochastic_barrier.ts`
- WebGPU Datalog / OWL 2 RL Saturation Reasoner: `WebGPUOntologyReasoner` & `BitsetGraph` in `packages/runtime/src/gpu/webgpu_reasoner.ts`
- ISO 26262 / DO-178C Back-to-Back Equivalence Verification: `B2BEquivalenceVerifier` in `packages/runtime/src/simulation/b2b_verifier.ts` and `UnifiedVerifier` in `packages/runtime/src/simulation/unified_verifier.ts`

---

1. Spacer CHC Engine (Generalized PDR)

Academic Citations

- **Komuravelli, A., Gurfinkel, A., & Chaki, S. (2014)**. "*SMT-Based Model Checking for Recursive Programs.*" **Computer Aided Verification (CAV 2014)**, LNCS 8559, pp. 17–34. DOI: 10.1007/978-3-319-08867-9_2
- **Hoder, K., & Bjørner, N. (2012)**. "*Generalized Property Directed Reachability.*" **Theory and Applications of Satisfiability Testing (SAT 2012)**, LNCS 7317, pp. 157–171. DOI: 10.1007/978-3-642-31612-8_13

ModelScript Rationale & Modifications

Constrained Horn Clauses (CHCs) represent systems of logical relations of the form:

$$\forall X.\, \phi(X) \land P_1(X) \land \cdots \land P_k(X) \implies H(X)$$

Spacer solves CHCs by combining inductive generalization, Craig interpolation, and under-approximated reachability trees without bounding the unrolling depth. In ModelScript, Spacer verifies infinite-state state machine transitions and recursive cyber-physical control architectures.

- **Modifications**: Directly integrates with ModelScript's in-WASM CDCL SAT core and Craig interpolator; implements specialized heuristics for linear real arithmetic over cyber-physical invariants.

---

2. IC3 / Property Directed Reachability (PDR)

Academic Citations

- **Bradley, A. R. (2011)**. "*SAT-based model checking without unrolling.*" **VMCAI 2011**, LNCS 6538, pp. 70–87. DOI: 10.1007/978-3-642-18275-4_7
- **Een, N., Mishchenko, A., & Brayton, R. (2011)**. "*Efficient implementation of property directed reachability.*" **Formal Methods in Computer-Aided Design (FMCAD 2011)**, pp. 125–134.

ModelScript Rationale & Modifications

Proves whether safety property P holds across all reachable states of a discrete or hybrid transition system without unrolling the transition relation $T(s, s')$ to a fixed horizon k.

- **Modifications**: Maintains inductive clause frames $F_0, F_1, \ldots, F_k$ where F_i over-approximates states reachable in up to i steps; uses counterexample-guided abstraction refinement (CEGAR) to block Proof Obligations (POBs); propagates clauses forward to

find an inductive invariant ($F_i \equiv F_{i+1}$) proving unbounded safety.

---

3. Stochastic Barrier Certificates (`wasm_stochastic_barrier.ts`)

Academic Citations

- **Prajna, S., Jadbabaie, A., & Pappas, G. J. (2007)**. "*A framework for worst-case and stochastic safety verification using barrier certificates.*"
  IEEE Transactions on Automatic Control, 52(8), pp. 1415–1428.
  DOI: 10.1109/TAC.2007.902736
- **Kushner, H. J. (1967)**. *Stochastic Stability and Control.*
  Academic Press.

ModelScript Rationale & Modifications

For continuous-time stochastic differential equations (Itô diffusions) driven by standard Wiener processes W_t:

$$dx(t) = f(x)\,dt + \sigma(x)\,dW_t$$

deterministic trajectories do not exist, and safety verification requires bounding the tail probability that $x(t)$ ever enters an unsafe region $\mathrm{X}_u = \{x \mid \|x\|_2 \geq r_u\}$.

`StochasticBarrierSynthesizer` (`packages/runtime/src/analysis/wasm_stochastic_barrier.ts`) synthesizes non-negative supermartingale barrier candidates $B(x) = x^T P\, x \geq 0$ using the **infinitesimal generator** L of the Itô process:

$$\mathrm{L}B(x) = \nabla B(x) \cdot f(x) + \frac{1}{2}\mathrm{tr}(\sigma(x)\sigma(x)^T \nabla^2 B(x)) \leq -c$$

- **Doob's Maximal Supermartingale Inequality**: Bounds the infinite-horizon and finite-horizon probability of unsafe set entry via:

$$\mathrm{P}(\sup_{t\geq 0} \|x(t)\|_2 \geq r_u) \leq \frac{B(x_0) + \frac{\mathrm{tr}(\Sigma P)}{2\alpha}}{\lambda_{\min}(P) r_u^2} \leq \delta_{\text{risk}}$$

  where $\Sigma = \sigma(0)\sigma(0)^T$ is the diffusion noise covariance and α is the linearized drift decay rate.
- **Soundness & Rigor Reporting**: Emits structured `VerificationVerdict` diagnostics distinguishing fast linearized quadratic risk estimates from Sum-of-Squares (SOS) / LP certified supermartingales.

---

4. WebGPU Datalog / OWL 2 RL Saturation Reasoner (`webgpu_reasoner.ts`)

Academic Citations

- **Motik, B., Patel-Schneider, P. F., & Parsia, B. (2012)**. "*OWL 2 Web Ontology Language: Profiles (Second Edition) — OWL 2 RL.*"
  W3C Recommendation. https://www.w3.org/TR/owl2-profiles/
- **Warshall, S. (1962)**. "*A theorem on Boolean matrices.*"
  Journal of the ACM, 9(1), pp. 11–12. DOI: 10.1145/321105.321107

ModelScript Rationale & Modifications

Large-scale systems engineering ontologies (`languages/owl2/` and SysML v2 KerML specialization hierarchies) contain 10^5 to 10^7 ABox/TBox assertions. Evaluating transitive role closures (`subClassOf` , `partOf` , `satisfies`) and SHACL numeric property constraints sequentially on the CPU becomes a bottleneck during workspace-wide verification.

`WebGPUOntologyReasoner` (`packages/runtime/src/gpu/webgpu_reasoner.ts`) implements a massively parallel GPU relational join and saturation co-processor with seamless CPU SIMD bitset fallback:

- **Bit-Packed Adjacency Matrix (`BitsetGraph`)**: Encodes an $N \times N$ directed graph as a flat `Uint32Array` where each node row u occupies $\lceil N/32 \rceil$ 32-bit words, achieving $32\times$ memory compression and single-instruction bitwise row unions.

- **Parallel Logarithmic Doubling in WGSL (`TRANSITIVE_CLOSURE_WGSL`)**: Dispatches compute workgroups (`workgroups = ceil(N / 64)`) over ping-pong storage buffers (`bufIn` , `bufOut`) for $\lceil \log_2 N \rceil + 1$ doubling rounds, computing the full reflexive-transitive closure R^* in $\mathrm{O}(\log N)$ GPU passes.
- **Parallel SHACL Constraint Kernel (`SHACL_VALIDATION_WGSL`)**: Validates `SHACLNumericConstraint` bounds (`minInclusive` , `maxInclusive` , `expectedClassId`) across strided `Float32Array` instance attribute tables and writes compact violation bitsets.

---

5. ISO 26262 / DO-178C Back-to-Back Equivalence Verification (`b2b_verifier.ts` , `v_cycle_verifier.ts`)

Academic Citations

- **ISO 26262-6:2018 & ISO 26262-8:2018**. *Road vehicles — Functional safety — Part 6: Product development at the software level (Table 7: Back-to-back comparison test between model and code) & Part 8: Supporting processes (Clause 11: Confidence in the use of software tools, TCL1).*
- **RTCA DO-178C /DO-331 (2011)**. *Software Considerations in Airborne Systems and Equipment Certification — Model-Based Development and Verification Supplement.*

ModelScript Rationale & Modifications

Safety-critical automotive (ISO 26262 ASIL-D) and avionics (DO-178C DAL-A) standards require proving that generated embedded production code (zero-allocation C99 or standalone WASM FMU artifacts) is numerically and behaviorally equivalent to the high-level specification model (SysML v2 / Modelica).

`B2BEquivalenceVerifier` (`packages/runtime/src/simulation/b2b_verifier.ts` , orchestrated by `UnifiedVerifier` in `packages/runtime/src/simulation/unified_verifier.ts`) automates end-to-end **Model-in-the-Loop (MiL) vs. Software-in-the-Loop (SiL)** back-to-back verification:

1. **MiL Reference Trajectory Execution**: Simulates the high-level DAE arena (`simulateArenaAsync`) over $[t_{\text{start}}, t_{\text{stop}}]$ to obtain reference trajectories $y_j^{\text{MiL}}(t)$.
2. **Standalone C99 Code Generation & Cryptographic Hashing**: Invokes `generateStandaloneCSources` (`@modelscript/exchange/fmu`) to emit self-contained, zero-allocation C99 sources (`<Model>_standalone.h` , `<Model>_standalone.c`), computing a SHA-256 artifact digest (`cSourceHash`) for V-cycle traceability.
3. **Automated Test Harness & Native Compilation**: Synthesizes a deterministic C99 fixed-step RK4 test harness (`<Model>_b2b_harness.c`), compiles the binary via `gcc -O2 -Wall -Wextra -std=c99 -lm` , and streams CSV output trajectories $y_j^{\text{SiL}}(t_k)$.
4. **Bisimulation & Signal-Tolerance Equivalence Checking**: Aligns SiL samples against binary-searched linearly interpolated MiL trajectories and verifies the combined absolute/relative tolerance envelope across all K time steps and M output signals:

$$\forall j \in \{1, \dots, M\},\ \forall k \in \{1, \dots, K\} : \quad \left| y_j^{\text{MiL}}(t_k) - y_j^{\text{SiL}}(t_k) \right| \le \epsilon_{\text{abs}} + \epsilon_{\text{rel}} \left| y_j^{\text{MiL}}(t_k) \right|$$

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - State machine models and safety requirements from `SysML v2` .
  - Mode switching, discrete `when` / `if` clauses, and continuous/stochastic DAE arenas lowered from `Modelica` .
  - OWL 2 TBox/ABox ontological triples from `OWL2` .
- **Downstream Consumers**:
  - Verifies architectural safety contracts, stochastic risk bounds, and ISO 26262 MiL/SiL equivalence in `msx verify` .
  - Feeds theory lemmas and transitive subsumption closures into `Nelson-Oppen Coordinator` .

CHAPTER 35

Abstract Interpretation, Polyhedral Domains & Continuous Reachability

Implementation:

- Abstract Interpretation Domains: `packages/runtime/src/formal/abstract_interpretation/`
- DDE Method-of-Steps Flowpipes: `DdeFlowpipeSolver` in `packages/runtime/src/analysis/wasm_dde_flowpipe.ts`
- Hamilton-Jacobi-Isaacs (HJI) Level-Set Solver: `HjiLevelSetSolver` in `packages/runtime/src/analysis/wasm_hji_solver.ts`

---

1. Bourdoncle Weak Topological Ordering (WTO) Fixpoint Solver

Academic Citations

- **Bourdoncle, F. (1993)**. "*Efficient chaotic iteration strategies with widenings.*"
  Formal Methods in Programming and Their Applications, LNCS 735, pp. 128–141. DOI: 10.1007/BFb0039704
- **Cousot, P., & Cousot, R. (1977)**. "*Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints.*"
  ACM POPL'77, pp. 238–252. DOI: 10.1145/512950.512973

ModelScript Rationale & Modifications

Computes sound abstract over-approximations of state variables across complex nested loops and recursion without simulating unbounded trajectories.

- **Modifications**: Implements Bourdoncle's recursive hierarchical WTO decomposition to identify minimal loop heads; accelerates convergence using delayed widening (∇) followed by monotonic narrowing (Δ).

---

2. Two Variables Per Inequality (TVPI) Polyhedral Domain

Academic Citations

- **Simon, A., King, A., & Howe, J. M. (2002)**. "*Two Variables per Inequality as an Abstract Domain.*"
  LOPSTR 2002, LNCS 2664, pp. 71–89. DOI: 10.1007/3-540-36388-2_7
- **Nelson, G. (1978)**. "*An $n^{O(\log n)}$ algorithm for the two-variable-per-inequality integer programming problem.*"
  Technical Report, Stanford University.

ModelScript Rationale & Modifications

Captures planar relational linear invariants of the form:

$$ax_i + bx_j \le c \quad (a, b, c \in \mathbb{Q})$$

Much more expressive than simple intervals or octagons, allowing ModelScript to prove physical conservation laws (e.g. $p_1 + p_2 = \text{const}$) without the exponential vertex-facet explosion of general convex polyhedra.

- **Modifications**: Planar Graham-scan convex hull algorithms in WebAssembly typed memory; relational projection routines optimized for physical parameter spaces.

---

3. Octagon Difference Bound Matrices (DBM) Domain

Academic Citations

- **Miné, A. (2006)**. "*The octagon abstract domain.*"
  Higher-Order and Symbolic Computation, 19(1), pp. 31–100.
  DOI: 10.1007/s10990-006-8609-1

ModelScript Rationale & Modifications

Restricts linear relationships to constraints of the form:

$$\pm x_i \pm x_j \leq c$$

Achieves cubic time complexity $O(n^3)$ via Floyd-Warshall shortest path algorithms over $2n \times 2n$ potential graphs.

- **Modifications**: Direct integration into the `@modelscript/dsl` type inference system and `AbstractDomainOracle` in the Nelson-Oppen coordinator.

---

4. Array Segment Partitioning

Academic Citations

- **Cousot, P., Cousot, R., & Logozzo, F. (2011)**. "*A Parametric Segmentation Functor for Fully Automatic and Scalable Array Content Analysis.*"
  ACM POPL'11, pp. 505–518. DOI: 10.1145/1926385.1926444

ModelScript Rationale & Modifications

Verifies large array equations and indexed multi-dimensional components in Modelica without unrolling every array index into separate scalar variables. Partitions index sets into contiguous segments whose bounds and elements share uniform properties.

---

5. Delay Differential Equation (DDE) Method-of-Steps Flowpipes (`wasm_dde_flowpipe.ts`)

Academic Citations

- **Bellman, R., & Cooke, K. L. (1963)**. *Differential-Difference Equations.*
  Academic Press.
- **Zou, L., Fränzle, M., Xue, B., & Zhan, N. (2015)**. "*orda: An Ode and Dde Reachability Analysis Tool.*"
  SETTA 2015, LNCS 9409, pp. 109–123. DOI: 10.1007/978-3-319-25942-0_8

ModelScript Rationale & Modifications

Physical transport delays, networked control loops, and Modelica `delay(x, tau)` operators yield infinite-dimensional Delay Differential Equations (DDEs):

$$\dot{x}(t) = f(t, x(t), x(t-\tau)), \qquad x(t) \in \Phi(t) \text{ for } t \in [-\tau, 0]$$

`DdeFlowpipeSolver` (`packages/runtime/src/analysis/wasm_dde_flowpipe.ts`) implements **Method-of-Steps** reachability propagation across consecutive delay horizons $[0, \tau], [\tau, 2\tau], \ldots$:

- **History Interval Buffering**: Maintains a timestamped ring buffer `historyMap` storing both nominal trajectories $\bar{x}(t)$ and interval/zonotope tube enclosures $\mathrm{X}(t) = [\underline{x}(t), \overline{x}(t)]$. For $t - \tau \leq t_0$, queries evaluate the initial functional history enclosure $\Phi(t - \tau)$; for $t - \tau > t_0$, historical enclosures are interpolated from stored flowpipe steps.
- **Interval Picard Enclosure Step**: Advances the state tube across $[t_k, t_k + \Delta t]$ by combining current state bounds $\mathrm{X}(t_k)$ and delayed history bounds $\mathrm{X}(t_k - \tau)$:

$$\mathrm{X}(t_{k+1}) = \mathrm{X}(t_k) + \Delta t \cdot \square f(t_k, \mathrm{X}(t_k), \mathrm{X}(t_k - \tau))$$

- **Epistemic Soundness Tagging**: Explicitly tags the resulting `VerificationVerdict` with rigor metadata (`heuristic` vs. `certified`) so downstream safety monitors distinguish fast corner-sampled delay tube estimates from fully Taylor-enclosed flowpipes.

---

6. Hamilton-Jacobi-Isaacs (HJI) PDE Level-Set Reachability (`wasm_hji_solver.ts`)

Academic Citations

- **Mitchell, I. M., Bayen, A. M., & Tomlin, C. J. (2005)**. "*A time-dependent Hamilton-Jacobi formulation of reachable sets for continuous dynamic games.*"
  IEEE Transactions on Automatic Control, 50(7), pp. 947–957.
  DOI: 10.1109/TAC.2005.851439
- **Jiang, G. S., & Shu, C. W. (2000)**. "*Weighted ENO schemes for Hamilton-Jacobi equations.*"
  SIAM Journal on Scientific Computing, 21(6), pp. 2126–2143.
  DOI: 10.1137/S106482759732455X

ModelScript Rationale & Modifications

When verifying safety of nonlinear control systems under worst-case adversarial disturbances $d \in \mathrm{D}$ and control authority $u \in \mathrm{U}$, convex set-propagation (zonotopes) can suffer from wrapping effects. `HjiLevelSetSolver` (`packages/runtime/src/analysis/wasm_hji_solver.ts`) computes the exact **Backward Reachable Tube (BRT)** by solving the zero-sum differential game **Hamilton-Jacobi-Isaacs (HJI) PDE** viscosity solution on an Eulerian level-set grid:

$$\frac{\partial V}{\partial t} + \min\{0,\ \min_{u\in \mathrm{U}} \max_{d\in \mathrm{D}} \nabla V(x,t)\cdot f(x,u,d)\} = 0, \qquad V(x,0) = \ell(x)$$

where the sub-zero level set $\mathrm{K}_{\text{unsafe}} = \{x \mid \ell(x) \le 0\}$ defines the target failure region and $\{x \mid V(x,T) \le 0\}$ captures all initial states from which the adversary d can force the trajectory into $\mathrm{K}_{\text{unsafe}}$ within horizon T despite optimal safety control u.

- **5th-Order WENO Upwind Spatial Reconstruction (`weno5Derivatives`)**: Approximates left and right spatial derivatives (p^-, p^+) along each state axis using 3-stencil smoothness indicators (s_1, s_2, s_3) and nonlinear weights $a_k = \frac{\gamma_k}{(\epsilon+s_k)^2}$, eliminating spurious oscillations near non-smooth value-function kinks.
- **Lax-Friedrichs Numerical Hamiltonian**: Evaluates the saddle-point Hamiltonian $H(x,p) = \min_{u\in \mathrm{U}} \max_{d\in \mathrm{D}} p\cdot f(x,u,d)$ at central gradients $p = \frac{1}{2}(p^- + p^+)$ augmented by artificial viscosity dissipation:

$$\hat{H}(x,p^-,p^+) = H\left(x, \frac{p^- + p^+}{2}\right) - \frac{1}{2}\sum_i \alpha_i (p_i^+ - p_i^-), \qquad \alpha_i = \max_{u,d} \left|f_i(x,u,d)\right|$$

- **Monotone BRT Accumulation**: Steps the level-set array backward in time with `nextV[idx] = Math.min(V[idx], V[idx] - dt * dVdt)` and reports the exact unsafe volume fraction across the state space.

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - Control-flow graphs, `delay(x, tau)` equations, and continuous vector fields from `ModelicaFlattener` .
  - Adversarial disturbance bounds D and safety level-set specifications $\ell(x) \le 0$ from SysML v2 verification cases.
- **Downstream Consumers**:
  - Feeds numeric intervals into `HC4 Contractor` and `Nelson-Oppen Coordinator` .
  - Certifies backward reachable tubes and delay flowpipes in `UnifiedVerifier` (`msx verify`).
  - Eliminates out-of-bounds array access and division-by-zero compiler warnings at compile time.

CHAPTER 36

Graph Rewriting, Pattern Matching & Metaprogramming

Implementation: `packages/dsl/src/codegen/rewriting/` and `packages/dsl/src/codegen/runtime/`

---

1. Triple Graph Grammars (TGG) & Algebraic DPO Rewriting

Academic Citations

- **Schürr, A. (1994)**. "*Specification of graph translators with triple graph grammars.*"
  Graph-Theoretic Concepts in Computer Science (WG '94), LNCS 903, pp. 151–163.
  DOI: 10.1007/3-540-59071-4_45
- **Greenyer, J., & Kindler, E. (2010)**. "*Comparing operational execution strategies for triple graph grammars.*"
  Software & Systems Modeling, 9(4), pp. 441–469.
  DOI: 10.1007/s10270-009-0140-2

ModelScript Rationale & Modifications

Synchronizes multi-domain engineering languages (e.g. Modelica 1D physics, SysML v2 requirements, STEP CAD geometry) declaratively without fragile point-to-point imperatively paired translators.

- **Modifications**: AOT-compiles declarative TGG rule sets into an in-WASM AssemblyScript dispatch kernel (`tgg_forward_dispatch`, `tgg_backward_dispatch`); tracks bidirectional correspondences in linear CST memory; achieves $\mathrm{O}(\Delta N)$ incremental propagation driven by Salsa invalidations.

---

2. Critical Pair Analysis (CPA)

Academic Citations

- **Knuth, D. E., & Bendix, P. B. (1970)**. "*Simple word problems in universal algebras.*"
  Computational Problems in Abstract Algebra, pp. 263–297. Pergamon Press.
- **Ehrig, H., Ehrig, K., de Lara, J., Taentzer, G., Varró, D., & Varró, G. (2005)**. "*Termination and confluence of graph transformation systems.*"
  Formal Methods in Software and Systems Modeling, LNCS 3440, pp. 180–195. DOI: 10.1007/978-3-540-31847-7_11
- **Heckel, R., & Taentzer, G. (2020)**. *Graph Transformation for Software Engineers: Formal Foundation and Industrial Applications*.
  Springer. DOI: 10.1007/978-3-030-43969-9

ModelScript Rationale & Modifications

In polyglot workflows, multiple rules may compete to transform the same AST elements. CPA statically detects all critical pairs (conflicting rule overlaps, target collisions, and circular translation loops) at compile time.

- **Modifications**: Tailored for heterogeneous polyglot metamodels; emits structured diagnostics with source ranges and rule conflict identifiers directly to the LSP and CLI.

---

3. Worst-Case Optimal Joins (WCOJ / Leapfrog Triejoin)

Academic Citations

- **Ngo, H. Q., Porat, E., Ré, C., & Rudra, A. (2012)**. "*Worst-case optimal join algorithms.*"
  ACM PODS '12, pp. 37–48. DOI: 10.1145/2213556.2213565
- **Veldhuizen, T. L. (2014)**. "*Leapfrog Triejoin: A simple, worst-case optimal join algorithm.*"
  ICDT '14, pp. 96–106. DOI: 10.4230/LIPIcs.ICDT.2014.96

- **Atserias, A., Grohe, M., & Marx, D. (2008)**. "*Size bounds and query plans for relational joins.*"
  IEEE FOCS '08, pp. 739–748. (The AGM Bound)

ModelScript Rationale & Modifications

Traditional binary relational joins suffer from exponential intermediate result explosions on cyclic subgraphs (e.g. triangle topologies between ports, connectors, and component definitions). WCOJ guarantees runtime strictly bounded by the Atserias-Grohe-Marx (AGM) bound, achieving $\mathrm{O}(N^{3/2})$ for triangle queries rather than $\mathrm{O}(N^2)$.

- **Modifications**: Formulates Leapfrog Triejoin iteration orders using descending vertex degree heuristics; emits execution plans directly into the AOT WebAssembly compilation engine.

---

4. Zero-GC E-Graphs & Equality Saturation

Academic Citations

- **Nelson, G., & Oppen, D. C. (1980)**. "*Fast decision procedures based on congruence closure.*"
  Journal of the ACM, 27(2), pp. 356–364. DOI: 10.1145/322186.322198
- **Tate, R., Stepp, M., Tatlock, Z., & Lerner, S. (2009)**. "*Equality saturation: A new approach to optimization.*"
  ACM POPL '09, pp. 264–276. DOI: 10.1145/1480881.1480915
- **Willsey, M., et al. (2021)**. "*egg: Fast and extensible equality saturation.*"
  ACM POPL '21, pp. 1–29. DOI: 10.1145/3434304

ModelScript Rationale & Modifications

Avoids destructive phase-ordering in symbolic equation simplification. An E-Graph maintains exponential equivalence classes of mathematically equal expressions simultaneously, extracting the globally minimal expression before numerical simulation.

- **Modifications**: WebAssembly linear memory allocation with zero GC; 32-bit Union-Find with two-pass path compression; fixed power-of-two open-addressing hash table (`HASH_MASK = 65535`) for $\mathrm{O}(1)$ deduplication.

---

5. Graph-Structured Stack (GSS) GLR Parsing

Academic Citations

- **Tomita, M. (1985)**. *Efficient Parsing for Natural Language: A Fast Generalized LR Algorithm*.
  Kluwer Academic Publishers. DOI: 10.1007/978-1-4613-2621-2
- **Scott, E., & Johnstone, A. (2006)**. "*Right Numerate Generalized LR Parsers.*"
  ACM TOPLAS, 28(4), pp. 577–618. DOI: 10.1145/1176894.1176896
- **McPeak, S., & Necula, G. C. (2004)**. "*Elkhound: A fast, practical GLR parser generator.*"
  Compiler Construction (CC 2004), LNCS 2985, pp. 73–88. DOI: 10.1007/978-3-540-24723-4_6

ModelScript Rationale & Modifications

Handles ambiguous and non-deterministic engineering grammars (such as expression vs. component declaration ambiguities in Modelica and SysML v2) by splitting and merging parse paths via the GSS without backtracking.

- **Modifications**: Active stack heads reside in contiguous unmanaged buffers (`t_activeHeads`) with power-of-two hash probing for $\mathrm{O}(1)$ stack head merging in WebAssembly.

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - Raw source code parsed by the `WASM GLR Parser`.
  - Polyglot synchronization rules authored in `@modelscript/dsl`.
- **Downstream Consumers**:
  - Rewritten expressions feed directly into `DAEBuilder`.
  - TGG synchronizations feed bidirectional updates between SysML v2 and Modelica.

CHAPTER 37

Linear Memory Structures & Indexing

Implementation:

- Roaring Bitmaps: `RoaringBitmap` in `packages/runtime/src/wasm/core/roaring.ts`
- Front-Coded IRI Dictionary: `FrontCodedDictionary` in `packages/runtime/src/wasm/core/trie.ts`
- Slotted Page B+ Tree: `PagedBTree` in `packages/runtime/src/wasm/storage/paged_btree.ts`

Academic Citations

Compressed Roaring Bitmaps

- **Chambi, S., Lemire, D., Kaser, O., & Godin, R. (2016)**. "*Better bitmap performance with Roaring bitmaps.*"
  Software: Practice and Experience, 46(5), pp. 709–719.
  DOI: 10.1002/spe.2325
- **Lemire, D., Ssi-Yan-Kai, G., & Kaser, O. (2018)**. "*Consistently faster and smaller compressed bitmaps with Roaring.*"
  Software: Practice and Experience, 48(9), pp. 1615–1638.
  DOI: 10.1002/spe.2560

Front-Coded String & IRI Compression

- **Witten, I. H., Moffat, A., & Bell, T. C. (1999)**. *Managing Gigabytes: Compressing and Indexing Documents and Images* (2nd ed.). Morgan Kaufmann.
  ISBN: 1-55860-570-3
- **Martínez-Prieto, M. A., Fernández, J. D., & Cánovas, R. (2012)**. "*Compression of RDF Dictionaries.*"
  ACM Transactions on the Web (TWEB), 6(4), pp. 1–35.
  DOI: 10.1145/2382636.2382639
- **Brisaboa, N. R., Cánovas, R., Francisco, C. C., Martínez-Prieto, M. A., & Navarro, G. (2011)**. "*K2-trees for compact Web graph representation.*"
  Information Systems, 39, pp. 152–163.
  DOI: 10.1016/j.is.2012.08.003

Slotted Page B+ Tree & SPO Triple Storage

- **Bayer, R., & McCreight, E. (1972)**. "*Organization and maintenance of large ordered indexes.*"
  Acta Informatica, 1(3), pp. 173–189.
  DOI: 10.1007/BF00288683
- **Comer, D. (1979)**. "*The ubiquitous B-tree.*"
  ACM Computing Surveys, 11(2), pp. 121–137.
  DOI: 10.1145/356770.356776
- **Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H., & Schwarz, P. (1992)**. "*ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging.*"
  ACM Transactions on Database Systems, 17(1), pp. 94–162.
  DOI: 10.1145/128765.128770
- **Neumann, T., & Weikum, G. (2008)**. "*RDF-3X: A RISC-style engine for RDF.*"
  Proceedings of the VLDB Endowment, 1(1), pp. 647–659.
  DOI: 10.14778/1453856.1453927

ModelScript Architectural Rationale

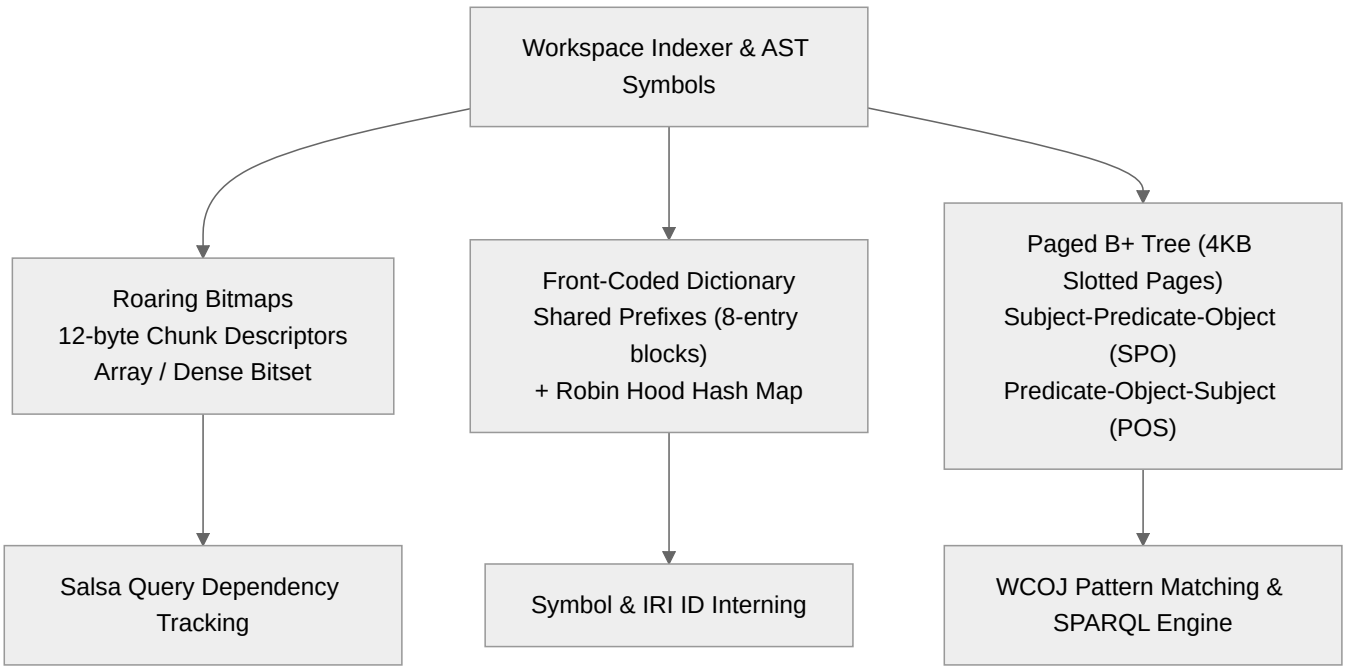
ModelScript compiles and indexes multi-million-element engineering workspaces, including the complete Modelica Standard Library (MSL), SysML v2 standard libraries (KerML), ISO 10303 STEP schemata, and OWL2 domain ontologies.

Representing these vast semantic graphs using standard JavaScript object heaps incurs:

- Extreme memory bloat (JavaScript objects typically consume 40–80 bytes of overhead per node).
- Heavy Garbage Collector (GC) pressure and unpredictable pause times during interactive IDE editing.
- Slow pointer chasing across cache lines.

To solve this, ModelScript implements **compact, zero-GC data structures** operating directly inside WebAssembly 64-bit linear memory.

Architecture of Core WASM Structures



1. Unmanaged Roaring Bitmaps

Partitions the 32-bit integer space into $2^{16} = 65{,}536$ chunks based on the upper 16 bits:

- **12-Byte Chunk Descriptor**:

```
Offset 0..3:   [key: u16] | [containerType: u16]
Offset 4..7:   [containerPtr: u32]
Offset 8..9:   [cardinality: u16]
Offset 10..11: [capacity: u16]
```

- **Adaptive Containers**:
  - **Array Container**: Used when cardinality $< 4{,}096$. Elements are stored as sorted `u16` integers in linear memory.
  - **Bitmap Container**: When cardinality reaches $4{,}096$, the container automatically converts to a dense 8,192-byte bitset ($1{,}024 \times \text{u64}$ words), allowing bitwise `AND`, `OR`, `XOR` operations at hardware SIMD speeds.

2. Front-Coded String & IRI Dictionary

Engineering identifiers and semantic URIs often share massive common prefixes:

```
http://modelscript.io/sysml2/core#PartDefinition
http://modelscript.io/sysml2/core#ItemDefinition
http://modelscript.io/sysml2/core#ActionDefinition
```

The front-coded dictionary partitions sorted strings into blocks of size $K = 8$:

- **Header String**: Stored uncompressed at the beginning of each block.
- **Suffix Entries**: Encoded as `[commonPrefixLen: u8, suffixLen: u8, suffixBytes]`.

- **Hybrid Lookups**: Combines front-coding (achieving 75–85% memory compression) with a zero-cost 64-bit Robin Hood hash map (`UnmanagedMap64`) for $O(1)$ string-to-ID interning.

3. Paged 4KB Slotted B+ Tree

Stores billions of semantic assertions and relational tuples:

- **4KB Slotted Page Layout (32-byte header + 254 entries)**:

```
[0..3]   pageId: u32
[4..5]   pageType: u16 (1 = Leaf, 2 = Internal)
[6..7]   itemCount: u16
[8..11]  prevPage: u32 (doubly-linked leaf chain for range scans)
[12..15] nextPage: u32
[16..19] parentPage: u32
[20..23] flags: u32
[24..31] lsn: u64 (Log Sequence Number for crash consistency)
```

- **RDF-3X Triples**: Triples (S, P, O) are indexed in clustered B+ tree leaf chains. Full index permutations (`SPO`, `POS`, `OSP`) permit fast range scans for multi-way graph joins without intermediate hash joins.

Upstream & Downstream Integration

Data Structure	Upstream Dependencies	Downstream Consumers
RoaringBitmap	Symbol IDs from `SymbolIndexer`	Invalidation sets in `QueryEngine`, type sets
FrontCodedDictionary	AST string literals, Qualified names, URIs	Inter-file symbol resolution, memory-mapped LSP indexing
PagedBTree	AST semantic relationships, OWL2 axioms	Leapfrog Triejoin (WCOJ), graph query evaluators, SPARQL

CHAPTER 38

Scientific Computing, Linear Algebra, Surrogates & Digital Twins

Implementation:

- Sparse Linear Algebra & Symbolic Solvers: `packages/runtime/src/wasm/solvers/`
- Reduced-Order & Dimensional Surrogates: `packages/simulate/src/surrogates/`
- Operational Digital Twin Estimation & Prognostics: `packages/simulate/src/twin/`, `packages/simulate/src/calibration/bayesian_calibrator.ts`, and `packages/simulate/src/optimizer/core/calibrator.ts`

---

1. Davis SuiteSparse Left-Looking Cholesky (LL^T)

Academic Citations

- **Davis, T. A. (2006)**. *Direct Methods for Sparse Linear Systems*. **SIAM**. DOI: 10.1137/1.9780898718881
- **Chen, Y., Davis, T. A., Hager, W. W., & Rajamanickam, S. (2008)**. "*Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate.*" **ACM Transactions on Mathematical Software**, 35(3), pp. 22:1–22:14. DOI: 10.1145/1462166.1462176

ModelScript Rationale & Modifications

Solves symmetric positive-definite sparse systems $Ax = b$ arising in finite element analysis, electrical mesh equations, and quadratic programming subproblems.

- **Modifications**: Left-looking column-by-column factorization computing non-zero patterns dynamically via elimination trees; operates inside unmanaged WebAssembly linear memory with zero heap fragmentation.

---

2. Gilbert-Peierls Left-Looking Sparse LU ($PAQ = LU$)

Academic Citations

- **Gilbert, J. R., & Peierls, T. (1988)**. "*An approach to sparse LU factorization based on graph coloring and topological sort.*" **International Conference on Supercomputing**.
- **Davis, T. A. (2006)**. *Direct Methods for Sparse Linear Systems*. **SIAM**. Chapter 5.

ModelScript Rationale & Modifications

Factorizes unsymmetric, highly irregular sparse Jacobians generated by torn physical DAE systems.

- **Modifications**: Depth-first search (DFS) topological ordering to find the exact non-zero structure of each column in L and U in time proportional to arithmetic operations; threshold partial pivoting with diagonal preference to preserve physical sparsity.

---

3. Householder QR with Column Pivoting

Academic Citations

- **Golub, G. H., & Van Loan, C. F. (2013)**. *Matrix Computations* (4th ed.). **Johns Hopkins University Press**.
- **Businger, P., & Golub, G. H. (1965)**. "*Linear least squares solutions by Householder transformations.*" **Numerische Mathematik**, 7(3), pp. 269–276. DOI: 10.1007/BF01436084

ModelScript Rationale & Modifications

Computes numerically stable orthogonal factorizations $AP = QR$ for rank-deficient least-squares calibration and parameter sensitivity studies.

---

4. Buchberger's Gröbner Basis Engine with Gebauer-Möller Criteria

Academic Citations

- **Buchberger, B. (1965)**. *An Algorithm for Finding the Basis Elements of the Residue Class Ring of a Zero Dimensional Polynomial Ideal*.
  PhD thesis, University of Innsbruck.
- **Gebauer, R., & Möller, H. M. (1988)**. "*On an installation of Buchberger's algorithm.*"
  Journal of Symbolic Computation, 6(2-3), pp. 275–286. DOI: 10.1016/S0747-7171(88)80048-8
- **Cox, D., Little, J., & O'Shea, D. (2015)**. *Ideals, Varieties, and Algorithms* (4th ed.).
  Springer. DOI: 10.1007/978-3-319-16721-3

ModelScript Rationale & Modifications

Solves small non-linear algebraic loops analytically to avoid numerical Newton iterations entirely. Computes the Gröbner basis under lexicographic monomial ordering (Lex) to triangularize polynomial systems into univariate root problems.

- **Modifications**: Implements the Gebauer-Möller criteria (B_{new} and M_{new} sets) to eliminate over 90% of redundant S-polynomial reduction checks; operates over exact rational coefficients $\mathbb{Q}$.

---

5. Proper Orthogonal Decomposition (POD) / Snapshot SVD

Academic Citations

- **Sirovich, L. (1987)**. "*Turbulence and coherence in Catherine's wheel: Method of snapshots.*"
  Quarterly of Applied Mathematics, 45(3), pp. 561–571. DOI: 10.1090/qam/910462
- **Berkooz, G., Holmes, P., & Lumley, J. L. (1993)**. "*The proper orthogonal decomposition in the analysis of turbulent flows.*"
  Annual Review of Fluid Mechanics, 25, pp. 539–575. DOI: 10.1146/annurev.fl.25.010193.002543

ModelScript Rationale & Modifications

Extracts reduced-order models (ROM) from massive 3D CFD or FEA simulation field snapshots ($N > 10^6$ nodes):

$$u(x,t) \approx \bar{u}(x) + \sum_{k=1}^{r} a_k(t)\phi_k(x)$$

- **Modifications**: Employs Sirovich's snapshot method to compute eigenvalues of the small $M \times M$ temporal correlation matrix ($M \ll N$ snapshots), retaining 99% cumulative kinetic energy in ≤ 16 spatial modes.

---

6. Buckingham π Dimensional Analysis & Similarity Engine

Academic Citations

- **Buckingham, E. (1914)**. "*On physically similar systems; illustrations of the use of dimensional equations.*"
  Physical Review, 4(4), pp. 345–376. DOI: 10.1103/PhysRev.4.345
- **Vaschy, A. (1892)**. "*Sur les lois de similitude en physique.*"
  Annales Télégraphiques, 19, pp. 25–28.
- **Barenblatt, G. I. (1996)**. *Scaling, Self-Similarity, and Intermediate Asymptotics*.
  Cambridge Texts in Applied Mathematics 14. DOI: 10.1017/CBO9781107050242

ModelScript Rationale & Modifications

Reduces parameter dimensionality in surrogate modeling by converting raw physical quantities ($q_1, \ldots, q_n$) into dimensionless similarity groups ($\Pi_1, \ldots, \Pi_{n-r}$):

- **Modifications**: Computes the null-space of the 7-dimensional SI unit exponent matrix using exact rational Gaussian elimination with coprime integer exponents; canonical fingerprint matching recognizes standard dimensionless numbers (Reynolds Re, Nusselt Nu, Mach Ma, Froude Fr).

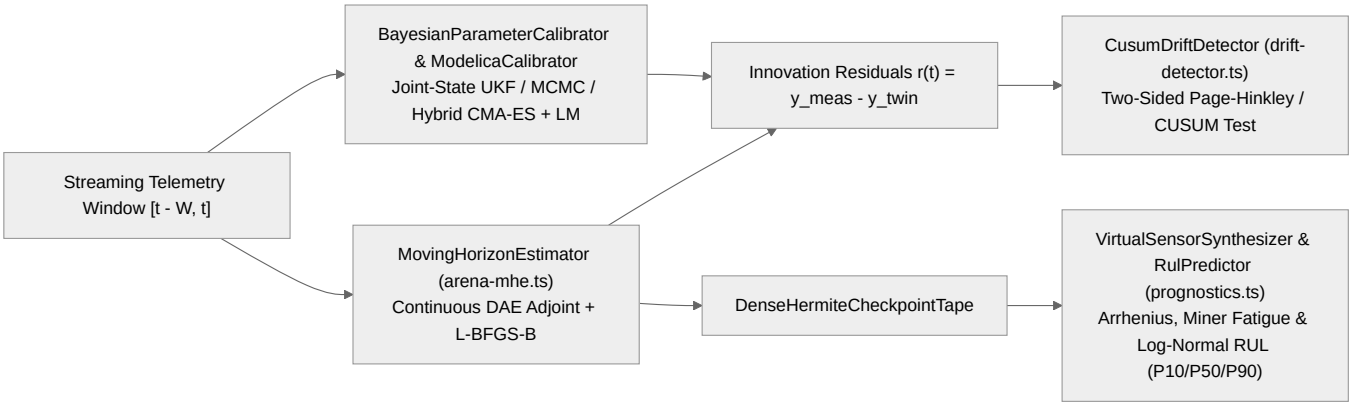
---

7. Operational Digital Twin Estimation & Prognostics (`packages/simulate/src/twin/`)

Academic Citations

- **Rao, C. V., Rawlings, J. B., & Mayne, D. Q. (2003)**. "*Constrained state estimation for nonlinear discrete-time systems: stability and moving horizon approximations.*"
  IEEE Transactions on Automatic Control, 48(2), pp. 246–258. DOI: 10.1109/TAC.2002.808470
- **Julier, S. J., & Uhlmann, J. K. (2004)**. "*Unscented filtering and nonlinear estimation.*"
  Proceedings of the IEEE, 92(3), pp. 401–422. DOI: 10.1109/JPROC.2003.823141
- **Page, E. S. (1954)**. "*Continuous inspection schemes.*"
  Biometrika, 41(1/2), pp. 100–115. DOI: 10.2307/2333009
- **Miner, M. A. (1945)**. "*Cumulative damage in fatigue.*"
  Journal of Applied Mechanics, 12(3), pp. A159–A164.

ModelScript Architectural Overview



1. Moving Horizon Estimation (MHE) over the Linear-Memory DAE Arena (`arena-mhe.ts`)

`MovingHorizonEstimator` reconciles streaming physical sensor windows (`TelemetryWindow` over $[t-W, t]$) against the in-memory `DAEBuilder` arena by solving a box-constrained trajectory-tracking problem:

$$\min_{\theta,w} \frac{1}{2}\sum_{k=0}^{N-1} \left\| y^{\text{sim}}(t_k;\theta,w) - y^{\text{meas}}(t_k) \right\|_2^2 + \frac{1}{2}\sum_i \lambda_i(\theta_i - \theta_i^{\text{prior}})^2 + \frac{\lambda_w}{2}\|w - w^{\text{prior}}\|_2^2$$

$$\text{s.t.} \quad \theta_{\min} \le \theta \le \theta_{\max}, \quad F(\dot{x}, x, y, \theta, w, t) = 0$$

- **Joint Physical & Neural Hybrid Calibration**: Simultaneously estimates physical DAE parameters θ and embedded `ArenaNeuralBlock` surrogate weights w inside the same optimization vector.
- **Adjoint-Driven L-BFGS-B**: Evaluates exact objective gradients via `DaeAdjointSolver` (`solveDaeAdjoint`) and converges in ≤ 25 quasi-Newton `lbfgsbSolve` iterations, automatically warm-starting initial states from window boundaries t_0.

2. Unscented Kalman Filter (UKF) & Multi-Strategy Parameter Calibration (`calibrator.ts` , `bayesian_calibrator.ts`)

ModelScript provides two complementary calibration engines for offline test-stand identification and online probabilistic tracking:

- **Joint-State Unscented Kalman Filter (`BayesianParameterCalibrator`)**: Augments the continuous state vector $x \in \mathrm{R}^{N_x}$ with unknown physical parameters $\theta \in \mathrm{R}^{N_p}$ into $z = [x^T, \theta^T]^T \in \mathrm{R}^{N_x+N_p}$. Propagates $2L+1$ deterministic sigma points ($\alpha = 10^{-3}, \beta = 2, \kappa = 0$) through the nonlinear DAE transition $z_{k+1} = f_{\text{aug}}(z_k, u_k, \Delta t)$ and measurement model $y_k = h_{\text{aug}}(z_k)$

(`UnscentedKalmanFilter`), extracting full posterior covariance $P_{k|k}$, 95% confidence intervals ($\hat{\theta}_i \pm 1.96\sigma_i$), and automated Digital Thread hypergraph write-back suggestions.

- **Deterministic & Global Metaheuristic Calibration (`ModelicaCalibrator` in `calibrator.ts`)**: Supports Levenberg-Marquardt (`lm`) with damped normal equations $(J^T J + \lambda I)\Delta\theta = -J^T r$, Sequential Quadratic Programming (`sqp`) with BFGS Hessian updates, scalable adjoint `lbfgsb` , Bayesian MCMC / global evolutionary search (`cmaes` , `de` , `pso`), and two-stage `hybrid-cmaes-lm` global-to-local polishing.

3. Page-Hinkley / CUSUM Statistical Residual Drift Detection (`drift-detector.ts`)

`CusumDriftDetector` continuously monitors multi-channel innovation residuals $r_c(t) = y_{\text{meas},c}(t) - y_{\text{twin},c}(t)$ to detect incipient actuator degradation, sensor bias, or structural drift while rejecting zero-mean Gaussian measurement noise:

- **Two-Sided Sequential Cumulative Sum**: Normalizes each channel residual $\tilde{z}_c = (r_c - \mu_c)/\sigma_c$ with minimum detectable shift slack $k_c = \frac{\delta_c}{2\sigma_c}$ (default $\delta_c = 1.5\sigma_c$) and updates positive and negative CUSUM statistics:

$$S_c^+(t) = \max(0,\ S_c^+(t-1) + \tilde{z}_c(t) - k_c)\,, \qquad S_c^-(t) = \max(0,\ S_c^-(t-1) - \tilde{z}_c(t) - k_c)$$

- **Adaptive Burn-In & Severity Classification**: Supports empirical baseline calibration (`calibrateFromData`) and triggers structured `DriftEvent` alerts when $\max(S_c^+, S_c^-) \geq h_c$, classifying severity (`low` , `medium` , `high` , `critical`) by threshold ratio S_c/h_c alongside online shift magnitude estimation.

4. Virtual Sensor Synthesis & Remaining Useful Life (RUL) Prognostics (`prognostics.ts`)

- **Virtual Sensor Synthesis (`VirtualSensorSynthesizer`)**: Evaluates unobservable internal physical states (e.g., rotor hotspot temperatures, internal valve spool stresses) at arbitrary timestamps t directly from the continuous `DenseHermiteCheckpointTape` .
- **Physics-of-Failure Damage Accumulation (`RulPredictor`)**: Computes instantaneous degradation rates $\frac{dD}{dt}$ under three selectable damage laws:
  1. **Arrhenius Thermal Aging**: $\frac{dD}{dt} = A \exp(-\frac{E_a}{k_B T})$
  2. **Miner's Power-Law / Basquin Fatigue**: $\frac{dD}{dt} = k_{\text{fat}}\, \sigma^m$
  3. **Archard Linear Mechanical Wear**: $\frac{dD}{dt} = k_{\text{wear}}\, \sigma$
- **Probabilistic RUL Quantiles**: Given current damage $D(t) \in [0, D_{\text{crit}}]$ and operational duty-cycle load variability σ_{load}, projects log-normal survival quantiles:

$$\text{RUL}_{P50} = \frac{D_{\text{crit}} - D(t)}{\dot{D}(\sigma_{\text{duty}})}, \qquad \text{RUL}_{P10} = \text{RUL}_{P50} \exp(-1.28155\,\sigma_{\text{load}}), \qquad \text{RUL}_{P90} = \text{RUL}_{P50} \exp(+1.28155\,\sigma_{\text{load}})$$

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - Linear matrix systems from `TornBlock` and `TR-BDF2` .
  - 3D CFD and FEA continuum field snapshots from `CFD` and `FEA` .
  - Live telemetry streams (CSV, ASAM MDF4, MCAP) and `DAEBuilder` adjoint sensitivities.
- **Downstream Consumers**:
  - Embedded into real-time Modelica plant loops via `msx surrogate` and operational digital twins.
  - Feeds calibrated posterior parameters and RUL health metrics back into the `DigitalThreadHypergraph` .
  - Accelerates linear solves across all numerical integrators and optimal control routines.

CHAPTER 39

Semidefinite Programming, BVP, Continuum Physics & CAD/ECAD

Implementation:

- Primal-Dual Interior-Point SDP: `SdpSolver` in `packages/runtime/src/solvers/wasm_sdp_solver.ts`
- Multiple Shooting & Collocation BVP: `solveBvpShooting` in `packages/simulate/src/solvers/bvp-solver.ts`
- Constructive Solid Geometry & CFD Patches: `evaluateScad` in `languages/scad/src/evaluator.ts`
- 3D Continuum Physics & B-Rep Meshing: `WebGPULbmRunner` (`packages/simulate/src/cfd/webgpu-lbm-runner.ts`), `FeaSolver` (`packages/simulate/src/fea/fea-solver.ts`), and `StepMesher` (`packages/simulate/src/meshing/step-mesher.ts`)
- Mechanical Swept-Volume, GD&T Stack-Up & Inverse CAD: `DynamicClearanceVerifier` (`packages/cad/src/dynamic-clearance.ts`), `ToleranceStackVerifier` (`packages/cad/src/tolerance-stack.ts`), and `ParameterInversionEngine` (`packages/cad/src/parameter-inversion.ts`)
- ECAD / PCB Geometry, Autorouting & DRC: `generateManufacturingFiles` (`packages/cad/src/ecad/gerber.ts`), `PCBRouter` (`packages/cad/src/ecad/engine/router.ts`), and `runDRC` (`packages/cad/src/ecad/engine/drc.ts`)

Academic Citations

Semidefinite Programming & Sum-of-Squares (SOS)

- **Vandenberghe, L., & Boyd, S. (1996)**. "*Semidefinite Programming.*"
  SIAM Review, 38(1), pp. 49–95.
  DOI: 10.1137/1038003
- **Helmberg, C., Rendl, F., Vanderbei, R. J., & Wolkowicz, H. (1996)**. "*An interior-point method for semidefinite programming.*"
  SIAM Journal on Optimization, 6(2), pp. 342–361.
  DOI: 10.1137/0806018
- **Alizadeh, F., Haeberly, J. P. A., & Overton, M. L. (1998)**. "*Primal-dual interior-point methods for semidefinite programming: convergence rates, stability and numerical results.*"
  SIAM Journal on Optimization, 8(3), pp. 746–768.
  DOI: 10.1137/S1052623495287701X
- **Parrilo, P. A. (2003)**. "*Semidefinite programming relaxations for semialgebraic problems.*"
  Mathematical Programming, 96(2), pp. 293–320.
  DOI: 10.1007/s10107-003-0387-5 *(SOS Programming)*

Boundary Value Problems (BVP) & Multiple Shooting

- **Morrison, D. D., Riley, J. D., & Zancanaro, J. F. (1962)**. "*Multiple shooting method for two-point boundary value problems.*"
  Communications of the ACM, 5(12), pp. 613–614.
  DOI: 10.1145/368996.369028
- **Stoer, J., & Bulirsch, R. (2002)**. *Introduction to Numerical Analysis* (3rd ed.). Texts in Applied Mathematics 12, Springer.
  Section 7.3: Multiple Shooting Methods.
  ISBN: 978-0-387-95452-3
- **Ascher, U. M., Mattheij, R. M., & Russell, R. D. (1995)**. *Numerical Solution of Boundary Value Problems for Ordinary Differential Equations.*
  SIAM Classics in Applied Mathematics 13.
  DOI: 10.1137/1.9781611971231
- **Betts, J. T. (2010)**. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming* (2nd ed.).
  SIAM Advances in Design and Control 19.
  DOI: 10.1137/1.9780898718577

Constructive Solid Geometry (CSG), Continuum Physics & CAD/ECAD

- **Requicha, A. A. (1980)**. "*Representations for rigid solids: Theory, methods, and systems.*"
  ACM Computing Surveys, 12(4), pp. 437–464.
  DOI: 10.1145/356827.356833
- **Qian, Y. H., d'Humières, D., & Lallemand, P. (1992)**. "*Lattice BGK models for Navier-Stokes equation.*"
  Europhysics Letters, 17(6), pp. 479–484.
  DOI: 10.1209/0295-5075/17/6/001
- **Bouzidi, M., Firdaouss, M., & Lallemand, P. (2001)**. "*Momentum transfer of a Boltzmann-lattice fluid with boundaries.*"
  Physics of Fluids, 13(11), pp. 3452–3459.
  DOI: 10.1063/1.1399290
- **Müller, M., & Gross, M. (2004)**. "*Interactive virtual materials.*"
  Proceedings of Graphics Interface 2004, pp. 239–246. *(Corotational Finite Elements)*

ModelScript Architectural Rationale

1. **Semidefinite Programming (SDP) for Verified Safety**: Formal continuous verification of non-linear physical models requires proving that state trajectories $\mathbf{x}(t)$ never breach unsafe operating envelopes. Synthesizing polynomial Lyapunov functions $V(\mathbf{x}) > 0$ and barrier certificates $B(\mathbf{x}) \leq 0$ reduces to finding Sum-of-Squares (SOS) decompositions:

$$p(\mathbf{x}) = \mathbf{z}(\mathbf{x})^T Q\, \mathbf{z}(\mathbf{x}) \geq 0 \iff Q \succeq 0$$

   By implementing a native primal-dual interior-point SDP solver in WebAssembly, ModelScript provides mathematically verified stability proofs directly in the browser and CLI without relying on external monolithic commercial solvers.
2. **Multiple Shooting BVP for Optimal Trajectories**: Boundary Value Problems (BVPs) govern optimal trajectory planning, model predictive control (MPC), and periodic limit cycle detection with split boundary conditions:

$$g(\mathbf{y}(t_0), \mathbf{y}(t_{\text{end}}), \mathbf{p}) = \mathbf{0}$$

   Standard single shooting suffers from catastrophic numerical instability on stiff or chaotic dynamics. Multiple shooting subdivides the time horizon into M subintervals, guaranteeing numerical stability and fast Newton-Raphson convergence.
3. **Constructive Solid Geometry (CSG) for Multi-Physics 3D Packaging**: Lumped-parameter 1D physical models (Modelica) rely on 3D geometric properties (mass, inertia tensor, aerodynamic drag area, hydraulic volume). ModelScript evaluates OpenSCAD models into normalized CSG solid trees (`@modelscript/cad`), auto-tagging boundary patches (`inlet` , `outlet` , `wall` , `symmetry`) for downstream OpenCascade B-Rep evaluation, STEP export, and SU2/OpenFOAM CFD meshing.

Mathematical Formulation & Execution Architecture



1. Primal-Dual Interior Point SDP Solver

Solves standard primal and dual semidefinite programs:

$$\text{Primal: } \min_{X \in \mathbb{S}^n} \operatorname{tr}(CX) \quad \text{s.t.} \quad \operatorname{tr}(A_i X) = b_i,\ X \succeq 0$$

$$\text{Dual: } \max_{y \in \mathbb{R}^m, S \in \mathbb{S}^n} b^T y \quad \text{s.t.} \quad \sum_{i=1}^{m} y_i A_i + S = C,\ S \succeq 0$$

- **Central Path Following**: Enforces the perturbed complementarity condition $XS = \mu I$ with barrier parameter $\mu = \frac{\operatorname{tr}(XS)}{n}$.

- **WASM Linear Algebra**: Predictor-corrector search directions $(\Delta X, \Delta y, \Delta S)$ are computed using fast in-memory Cholesky factorizations (`wasm_gaussian.ts`), with backtracking line search to ensure $X + \alpha\Delta X \succ 0$ and $S + \alpha\Delta S \succ 0$.

2. Multiple Shooting BVP Solver

Subdivides the horizon $[t_0, t_{\text{end}}]$ into M subintervals with mesh points $t_0 < t_1 < \cdots < t_M = t_{\text{end}}$:

- **State Trajectory Variables**: Introduces free initial conditions $\mathbf{s}_m$ at each node t_m.
- **Subinterval Integration**: Integrates forward from t_m to t_{m+1} using adaptive `tsit5` :

$$\mathbf{y}(t_{m+1}; \mathbf{s}_m, \mathbf{p})$$

- **Matching Residual System**: Solves the $(M \cdot n + k) \times (M \cdot n + k)$ simultaneous nonlinear system:

$$F_m(\mathbf{s}_m, \mathbf{s}_{m+1}) = \mathbf{y}(t_{m+1}; \mathbf{s}_m, \mathbf{p}) - \mathbf{s}_{m+1} = \mathbf{0}, \quad m = 0, \ldots, M-1$$

$$g(\mathbf{s}_0, \mathbf{s}_M, \mathbf{p}) = \mathbf{0}$$

- **Newton-Raphson Update**: Evaluates sensitivity Jacobians along trajectories and solves the block-banded linear system via WASM Gaussian LU elimination.

3. OpenSCAD CSG Engine & CFD Boundary Tagging

Transforms procedural CSG trees into boundary-tagged solid geometries:

- Evaluates transformations (`translate` , `rotate` , `scale` , `mirror`) and regularized Boolean operators (`union` , `difference` , `intersection`).
- Decorates faces with CFD boundary tags (`tagPatch` , `BoundaryPatchType` : `inlet` , `outlet` , `wall` , `symmetry`), enabling seamless handoff to automated SU2 mesh generators.

4. 3D Continuum Physics & B-Rep Meshing (`packages/simulate`)

ModelScript embeds native 3D continuum fluid, structural, and volumetric meshing engines that couple directly to 1D DAE simulations:

1. **WebGPU D3Q19 Lattice Boltzmann Method (LBM) CFD Solver (`webgpu-lbm-runner.ts`)**:
   - Discretizes the Boltzmann transport equation over a 19-velocity 3D lattice (`D3Q19`) stored in a zero-copy Structure-of-Arrays arena (`CfdFieldArena`) with ping-pong distribution buffers (`f0` , `f1`) and WGSL compute shader execution (`LBM_D3Q19_WGSL` , workgroups $8 \times 8 \times 4$):

$$f_i^{\text{post}}(\mathbf{x}, t) = f_i(\mathbf{x}, t) - \omega_{\text{loc}}(f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\rho, \mathbf{u})), \qquad f_i^{\text{eq}} = w_i\rho(1 + 3(\mathbf{c}_i \cdot \mathbf{u}) + \frac{9}{2}(\mathbf{c}_i \cdot \mathbf{u})^2 - \frac{3}{2}\|\mathbf{u}\|^2)$$

   - **Smagorinsky Subgrid LES Turbulence**: Dynamically adapts the local relaxation frequency $\omega_{\text{loc}} = (\tau_0 + \Delta\tau_{\text{LES}})^{-1}$ from the non-equilibrium momentum flux tensor $\Pi^{\text{neq}}_{\alpha\beta} = \sum_i c_{i\alpha}c_{i\beta}(f_i - f_i^{\text{eq}})$.
   - **Bouzidi Curved Boundaries & Moving-Wall FSI**: Applies sub-cell linear interpolation for curved solid obstacles ($\delta_{\text{wall}} < 0.5$ vs. $\delta_{\text{wall}} \geq 0.5$), injects moving-wall momentum $\Delta f_i = 6w_i\rho(\mathbf{c}_i \cdot \mathbf{u}_{\text{wall}})$ for two-way aeroelastic FSI, and integrates surface drag forces via momentum exchange.

2. **3D Tet4/Tet10 Linear/Nonlinear Corotational Elasticity & Thermal FEA Solver (`fea-solver.ts`)**:
   - Assembles global sparse stiffness K and consistent mass M matrices in Compressed Sparse Row (CSR) format for 4-node linear (`Tet4Element`) and 10-node quadratic (`Tet10Element`) tetrahedra.
   - **Corotational Large-Rotation Kinematics**: Extracts element rigid-body rotation $R_e \in \text{SO}(3)$ from the deformation gradient $F = e_0 E_0^{-1}$ via 6-step Higham polar decomposition iterations $R_{k+1} = \frac{1}{2}(R_k + R_k^{-T})$, rotating local element stiffness blocks $K_{\text{rot}} = R_e K_e R_e^T$ for geometric nonlinearity invariance.
   - **Jacobi-Preconditioned Conjugate Gradient (PCG) & Newmark-β Dynamics**: Solves static equilibrium $Ku = f$ and Rayleigh-damped transient elastodynamics $M\ddot{u} + (\alpha_M M + \beta_K K)\dot{u} + Ku = f$ using unconditional-stability Newmark-β ($\beta = 0.25, \gamma = 0.5$) time stepping, outputting nodal/element von Mises stresses and yield safety factors.

3. **STEP B-Rep Tetrahedral Mesher & Prismatic Boundary-Layer Extrusion (`step-mesher.ts`)**:

- `StepMesher` converts ISO 10303-21 STEP B-Rep entities and watertight surface triangulations into volumetric `Tet4` / `Tet10` meshes (`Tet4Mesher`), classifies exterior boundary patches (`PatchClassifier`), extrudes viscous prismatic inflation layers along `WALL_BODY` surfaces (`BoundaryLayerExtruder`), evaluates Jacobian/aspect-ratio quality (`MeshQualityAnalyzer`), and exports ready-to-run CalculiX (`*.inp`) and SU2 (`*.su2`) decks.

5. Mechanical Swept-Volume, GD&T Stack-Up & Inverse CAD (`packages/cad`)

1. **Dynamic Swept-Volume Collision & Clearance Envelopes (`dynamic-clearance.ts`)**:
   - `DynamicClearanceVerifier` bridges continuous reachability flowpipes (`TrajectoryStepEnclosure[]`) into 3D CAD assemblies (`Assembly`). At each time step, it propagates interval state enclosures $[x_i^{\text{lo}}, x_i^{\text{hi}}]$ through translational Minkowski sums, rotational corner-envelope expansions, and soft-tissue/compliant cavity morphing (`volume_radial` $s = (V/V_0)^{1/3}$, `anisotropic` , and `modal` skinning).
   - Evaluates exact pairwise AABB separation distances (`computeAABBDistance`) and closest witness points (`computeAABBClosestPoints`), automatically synthesizing a 3D counterexample CAD assembly at the worst-case collision timestamp t^* if any clearance rule is violated.
2. **Worst-Case, Statistical RSS & Coupled Thermal-Elastic GD&T Stack-Up (`tolerance-stack.ts`)**:
   - `ToleranceStackVerifier` verifies 1D/3D kinematic assembly gap loops across directional contributors ($d_i \in \{+1, -1\}$) combining bilateral manufacturing tolerances $[t_i^L, t_i^U]$, thermal expansion over $[T_{\min}, T_{\max}]$ ($\Delta L_{\text{th}} = L_0 \alpha_{\text{CTE}} \Delta T$), and axial elastic deflection under dynamic loads $[F_{\min}, F_{\max}]$ ($\Delta L_{\text{el}} = \frac{F L_0}{E A}$):

$$g_{\min,\,\text{WC}} = \sum_i d_i L_{0,i} + \sum_i \min(d_i t_i^L, d_i t_i^U) + \Delta g_{\text{th},\,\min} + \Delta g_{\text{el},\,\min}, \qquad \sigma_{\text{RSS}} = \sqrt{\sum_i \left(\frac{t_i^U - t_i^L}{6}\right)^2}$$

3. **Certified Bi-Directional CAD Parameter Inversion (`parameter-inversion.ts`)**:
   - `ParameterInversionEngine` closes the loop from 1D system optimization back to 3D MCAD source code. Given target mass/inertia constraints (`solveLengthForMass` , `scaleDimensionsForMass`) or optimized Modelica parameters, `patchMcadSourceCertified` verifies candidate dimensions against a multidisciplinary `SafeRegionGuard` (P_{safe} box bounds and buckling/stress/clearance predicates) before rewriting the parametric MCAD AST and regenerating STEP geometry.

6. ECAD / PCB Geometry, Autorouting & DRC (`packages/cad/src/ecad/`)

1. **RS-274X Gerber Aperture Evaluation & Excellon Drill Synthesis (`gerber.ts`)**:
   - `generateManufacturingFiles` compiles multi-layer `PCBLayout` models (`TopCopper` , `BottomCopper` , `InnerCopper1/2` , `Top/BottomSilkscreen` , `Top/BottomSolderMask` , `BoardOutline`) into IPC-compliant **RS-274X Gerber** (`%FSLAX46Y46*%` , 10^{-6} mm integer precision) with deduplicated aperture macros (`%ADD10C,...*%` , circle/rect/oval pads, 0.1 mm negative-polarity solder mask expansion `%IPNEG*%`) and **Excellon** NC drill programs (`drill.xln` , `METRIC,TZ`).
2. **Multi-Layer Interactive & Constrained PCB Trace Router (`engine/router.ts`)**:
   - `PCBRouter` (`pcb-router.ts` / `engine/router.ts`) implements a multi-layer trace routing engine supporting pad snap-targeting (`snapToPad`), 90° Manhattan (`manhattanRoute`) and 45° octilinear (`diagonal45Route` : $\text{diag} = \min(|\Delta x|, |\Delta y|)$) segment constraints, and automatic plated through-hole via insertion (`insertVia`) on layer transitions.
3. **Euclidean Trace/Pad Clearance Design Rule Checking (`engine/drc.ts`)**:
   - `runDRC` (`pcb-drc.ts` / `engine/drc.ts`) executes non-blocking geometric verification across copper layers, checking minimum trace widths, trace-to-trace, trace-to-pad, and pad-to-pad Euclidean clearances ($d(\mathbf{p}_1, \mathbf{p}_2) - \frac{w_1+w_2}{2} \geq c_{\min}$), via drill and annular ring minimums ($r_{\text{outer}} - r_{\text{drill}} \geq a_{\min}$), point-to-segment board-outline polygon distances (`minDistToPolygon`), and unconnected ratsnest net completeness.

Upstream & Downstream Integration

Component	Upstream Dependencies	Downstream Consumers
SdpSolver	Polynomial constraints from `TheoryCoordinator`	Formal Lyapunov barrier certificates, safety proofs
BvpSolver	ODE dynamics from `DaeBuilder`, split boundary specs	Model predictive control (MPC), trajectory optimization
ScadEvaluator & StepMesher	Declarative `.scad` & `.step` B-Rep models	`FeaSolver`, `WebGPULbmRunner`, CalculiX `.inp`, SU2 `.su2`
WebGPULbmRunner & FeaSolver	`StepMesher` Tet4/Tet10 meshes, FSI boundary loads	Aeroelastic FSI co-simulation, POD reduced-order models
DynamicClearance & ToleranceStack	Reachability flowpipes, GD&T specs, CAD `Assembly`	`UnifiedVerifier` (`msx verify`), counterexample STEP export
ParameterInversionEngine	Optimized 1D Modelica parameters, `SafeRegionGuard`	Automated MCAD source patching and AOT STEP regeneration
ECAD Gerber / PCBRouter / DRC	Electrical netlists, component footprints	RS-274X `.gbr` & Excellon `.xln` manufacturing outputs, WebIDE

CHAPTER 40

Diagram Auto-Layout & Orthogonal Routing

Implementation:

- Topology-Shape-Metrics Engine: `TsmLayoutEngine` in `packages/diagram/src/tsm-layout.ts`
- Port Constraint & Alignment Solver: `PortIlpSolver` in `packages/diagram/src/port-ilp-solver.ts`
- Obstacle-Avoiding Channel Router: `PortRouter` in `packages/diagram/src/port-router.ts`
- Specialized Engineering Diagram Layouts: `synthesizeFaultTreeDiagram` (`packages/diagram/src/fault-tree.ts`), `computeSequenceLayout` (`packages/diagram/src/sequence-layout.ts`), `applySwimlaneLayout` (`packages/diagram/src/swimlane-layout.ts`), and `buildPscadDiagram` (`packages/diagram/src/pscad-diagram-builder.ts`)
- Semantic Visual Diffing & Collaborative Presence: `buildVisualDiffGraph` (`packages/diagram/src/visual-diff-graph.ts`) and `CollabManager` (`packages/diagram/src/collab-manager.ts`)

Academic Citations

Topology-Shape-Metrics (TSM / Kandinsky) Orthogonal Layout

- **Tamassia, R. (1987)**. "*On embedding a grid graph with minimum number of bends.*"
  SIAM Journal on Computing, 16(3), pp. 421–444.
  DOI: 10.1137/0216030
- **Fößmeier, U., & Kaufmann, M. (1996)**. "*Drawing high-degree graphs with minimum number of bends.*"
  Graph Drawing (GD '95), Lecture Notes in Computer Science, 1027, pp. 211–225. Springer.
  DOI: 10.1007/BFb0021804
- **Di Battista, G., Eades, P., Tamassia, R., & Tollis, I. G. (1998)**. *Graph Drawing: Algorithms for the Visualization of Graphs.*
  Prentice Hall.
  ISBN: 0-13-301615-3

Port Constraint & Continuous Alignment ILP

- **Spönemann, M., von Hanxleden, R., & Fuhrmann, H. (2014)**. "*Port constraints in layout algorithms.*"
  Graph Drawing (GD 2014), Lecture Notes in Computer Science, 8871, pp. 351–362. Springer.
  DOI: 10.1007/978-3-662-45803-7_29
- **Kieffer, S., Dwyer, T., Marriott, K., & Wybrow, M. (2016)**. "*Incremental layout for dynamic schematic diagrams.*"
  IEEE Transactions on Visualization and Computer Graphics, 22(1), pp. 876–885.
  DOI: 10.1109/TVCG.2015.2467364
- **Eiglsperger, M., Fößmeier, U., & Kaufmann, M. (2001)**. "*Orthogonal graph drawing with constraints.*"
  Journal of Graph Algorithms and Applications, 5(2), pp. 1–24.
  DOI: 10.7155/jgaa.00035

Orthogonal Channel Routing & Obstacle Avoidance

- **Lee, C. Y. (1961)**. "*An algorithm for path connection and its applications.*"
  IRE Transactions on Electronic Computers, EC-10(3), pp. 346–365.
  DOI: 10.1109/TEC.1961.5219222
- **Hart, P. E., Nilsson, N. J., & Raphael, B. (1968)**. "*A formal basis for the heuristic determination of minimum cost paths.*"
  IEEE Transactions on Systems Science and Cybernetics, 4(2), pp. 100–107.
  DOI: 10.1109/TSSC.1968.300136 *(A* Search)*
- **Deutsch, D. N. (1976)**. "*A 'dogleg' channel router.*"
  Proceedings of the 13th Design Automation Conference (DAC '76), pp. 425–433.
  DOI: 10.1145/800263.810688

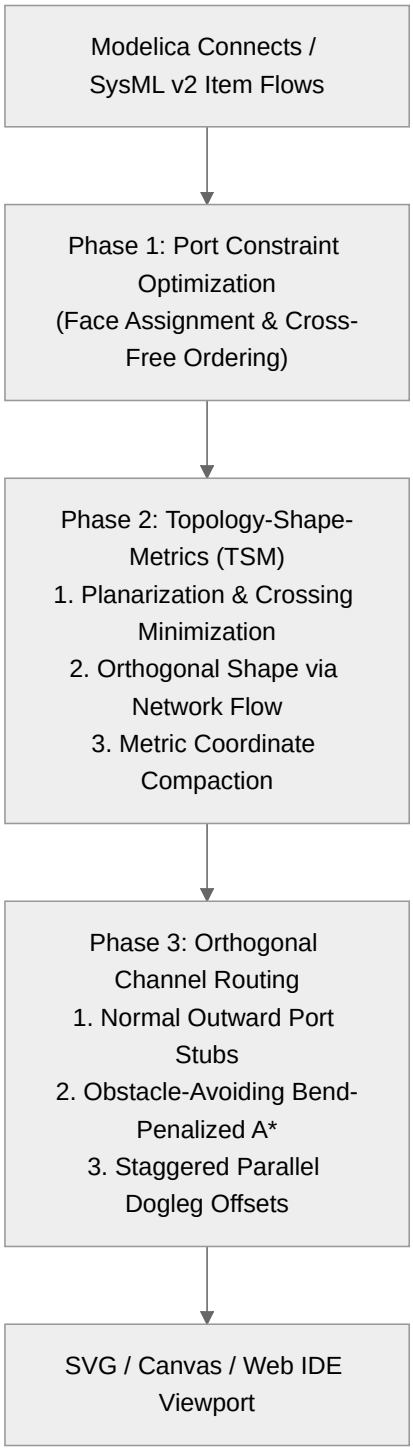
ModelScript Architectural Rationale

Physical engineering schematics (such as Modelica connection diagrams, SysML v2 Internal Block Diagrams, and hydraulic/piping schematics) require strict geometric conventions:

- **Orthogonal Edges**: Connections must run strictly along horizontal and vertical channels with clean 90° bends.
- **Port-Constraint Preservation**: High-degree engineering components have physical boundary pins (e.g. electrical resistors have pins p and n on opposite faces; hydraulic valves have ports A, B, P, T). Layout engines must place connection points on explicit component boundaries rather than abstract node centers.
- **Zero-Obstacle Penetration**: Wires must never traverse component bounding boxes.
- **Bus Disambiguation**: Multi-variable connections between blocks must not collapse onto identical collinear segments.

ModelScript implements the **Topology-Shape-Metrics (TSM)** pipeline augmented with a **Global Port ILP Solver** and **Dogleg Channel Router** natively in the browser and CLI.

Three-Phase Layout & Routing Architecture



1. Port Constraint & Continuous Alignment Solver

Resolves port assignment along component perimeters ($x, y \in \partial \text{Node}$):

- **Discrete Face Selection**: Assigns ports to Cardinal sides (`top` , `bottom` , `left` , `right`) minimizing Manhattan connection distance:

$$\min \sum_{e=(u_i,v_j)\in E} \|\mathbf{p}(u_i) - \mathbf{p}(v_j)\|_1$$

- **Causality Biasing**: Automatically places `input` ports on the left face and `output` ports on the right face for signal-flow blocks.
- **Collinear Port Snapping**: Enforces 0-bend straight lines whenever two connected ports are horizontally or vertically aligned within a configurable tolerance `straightTolerance` .

2. Topology-Shape-Metrics (TSM) Engine

Extends Tamassia's Kandinsky model for high-degree engineering vertices:

1. **Planarization**: Replaces unavoidable edge crossings with dummy vertices of degree 4.
2. **Orthogonalization (Minimum Bend Flow)**: Formulates bend minimization as a minimum-cost network flow problem over the dual graph, assigning angles $\alpha \in \{90^\circ, 180^\circ, 270^\circ, 360^\circ\}$ to faces.
3. **Compaction**: Solves two independent 1D longest path problems on horizontal and vertical constraint DAGs to compute minimal integer grid coordinates.

3. Obstacle-Avoiding Dogleg Channel Router

Renders clean multi-segment connection polylines:

- **Perpendicular Port Stubs**: Edge paths extend outward along the port normal vector by a distance `stubLength` before turning.
- **A* Grid Search with Bend Penalty**: Avoids component AABB bounding boxes using fast `segmentIntersectsRect` intersection tests while penalizing unnecessary turns:

$$\text{Cost}(p \rightarrow q) = \text{dist}(p, q) + \lambda_{\text{bend}} \cdot [\text{isTurn}(p, q)]$$

- **Staggered Parallel Channels**: Multi-wire bus connections share a common corridor but are offset by `parallelIndex * channelSpacing`, preventing overlapping lines.

4. Specialized Engineering Diagram Layouts

Beyond acausal block schematics, ModelScript provides domain-specific geometric layout engines tailored to safety, behavioral, and power-systems engineering notations:

1. **Fault Tree Gate Hierarchy Layout (`fault-tree.ts` / `fault-tree-layout.ts`)**:
   - `synthesizeFaultTreeDiagram` transforms Minimal Cut Sets ($\text{MCS} = \{C_1, \ldots, C_m\}$) into four-tier standardized Fault Tree Analysis (FTA) diagrams:
     - **Tier 1 ($y = 40$)**: Top Hazard Event rect node centered at $x_{\text{center}} = W/2$.
     - **Tier 2 ($y = 160$)**: Root Boolean `OR` gate (`createGateMarkup("OR")` SVG bezier shield) aggregating cut-set branches.
     - **Tier 3 ($y = 280$)**: Intermediate `AND` gates for multi-point failure cut sets ($\text{order} > 1$), while order-1 Single Points of Failure (SPOFs) route directly in high-visibility crimson (`#dc2626`) to basic events.
     - **Tier 4 ($y = 420$)**: Deduplicated circular Basic Event leaf nodes (`basicEventNodesMap`) distributed symmetrically across the horizontal span.
2. **UML / SysML v2 Sequence & Activity Swimlane Layouts (`sequence-layout.ts`, `swimlane-layout.ts`)**:
   - **Dual-Axis Sequence Layout (`computeSequenceLayout` / `applySequenceLayout`)**: Orders participant lifelines left-to-right ($x_i = x_{\text{left}} + i \cdot \Delta x_{\text{lifeline}} + W_{\text{hdr}}/2$), renders dashed vertical lifeline stems (`__seq_stem_*`), and stacks temporal messages top-to-bottom ($y_{k+1} = y_k + \Delta y_{\text{msg}}$) with distinct styling for synchronous (`sync`), asynchronous (`async`), dashed `reply`, and self-loop (`isSelf`, $1.5\times$ vertical spacing) messages.
   - **2D Activity Swimlane Partitioning (`applySwimlaneLayout`)**: Groups SysML v2 action nodes into vertical or horizontal allocation partitions (`SwimlanePartition`), sizing uniform lane containers (`swimlane_<id>`) to the maximum member stack height and routing cross-lane object/control flows via `port-orthogonal-astar`.
3. **PSCAD Power-System Single-Line Diagram (SLD) Busbar/Feeder Orthogonal Routing (`pscad-diagram-builder.ts` / `pscad-sld-builder.ts`)**:
   - `buildPscadDiagram` renders `.pscx` / `.pslx` electromagnetic transient models into IEEE-standard Single-Line Diagrams with voltage-coded busbar glyphs (≥ 200 kV red `#ff5252`, ≥ 69 kV amber `#ffab40`, < 69 kV cyan `#40c4ff`), specialized IEEE apparatus vector glyphs (`transformerGlyph`, `generatorGlyph`, `inverterIbrGlyph`, `breakerGlyph`, `surgeArresterGlyph`, `faultGlyph`, `converterBridgeGlyph`), solder-dot wire T-junctions, and orthogonal feeder routing.

5. Semantic Visual Diffing & Collaborative Presence

1. **AST-Aware Structural Graph Diffing & Mental-Map Stabilization (`visual-diff-graph.ts`, `visual-diff-renderer.ts` / `visual-diff.ts`)**:
   - `buildVisualDiffGraph` computes a semantic delta between `Base` and `Head` diagram revisions, classifying nodes and edges into `added`, `deleted`, `modified`, and `unchanged` states (`DiffStatus`).
   - **Breaking Interface Detection**: Inspects property and port changes (`diffProperties`) against architectural breaking keys (`direction`, `isAbstract`, `typeName`, `multiplicity`, `causality`, `variability`, `flowDirection`) and flags deleted components or

port-count mutations as `MODIFIED (BREAKING)`.

- **Anchor-Preserving Layout Stabilization (`computeIncrementalStabilization`)**: Locks `Base` node coordinates as immutable spatial anchors while iteratively relaxing newly added/modified `Head` nodes to eliminate bounding-box overlaps without destroying the reviewer's spatial mental map.

2. **CRDT / WebSocket Cursor, Selection Lock & Spatial Delta Synchronization (`collab-manager.ts`)**:
   - `CollabManager` coordinates real-time multi-user diagramming over WebSocket and VS Code Live Share transports:
     - **60 FPS Spatial Delta Streaming (`broadcastSpatialDelta`)**: Streams live component drag transformations (`PlacementItem`) with Last-Write-Wins (`LWW`) timestamp conflict resolution (`d.timestamp >= existing.timestamp`).
     - **Component Selection Locks (`SelectionLock`)**: Broadcasts per-peer component locks with distinct peer colors and automatic stale-peer heartbeat eviction (`purgeStalePeers`).
     - **Anchored Visual Review Comments (`DiagramComment`)**: Synchronizes spatially pinned sticky-note threads (`addComment`, `resolveComment`) directly on the schematic canvas.

Upstream & Downstream Integration

Pipeline Component	Upstream Dependencies	Downstream Consumers
PortIlpSolver	`SymbolIndex` connector ports, `Annotation` constraints	Port placement in `TsmLayoutEngine`
TsmLayoutEngine	Polyglot connect graphs (Modelica, SysML v2)	Node bounding box placement, routing channels
PortRouter	Component layout coordinates, Port locations	SVG polyline rendering, interactive 3D/2D IDE canvas
FaultTree / Sequence / Swimlane	Safety MCS sets, SysML v2 interactions & activities	X6 WebIDE canvas, headless SVG & PDF documentation export
PscadDiagramBuilder	PSCAD `.pscx` / `.pslx` AST & bus/feeder netlists	Power-system SLD viewer, EMT fault & breaker state overlays
VisualDiffGraph & CollabManager	Git Base/Head ASTs, WebSocket / Live Share peer streams	PR visual diff bundles (`html-diff-bundle.ts`), multi-user IDE

PART V

Command-Line Interface (msx)

CHAPTER 41

CLI Overview

The `@modelscript/cli` package provides two identical binary executables: `modelscript` and its shorthand alias `msx`. The command-line interface serves as the primary headless automation driver for compiling, simulating, verifying, linting, rendering, and orchestrating polyglot engineering systems across local workspaces, CI/CD pipelines, and cloud HPC clusters.

Installation

You can run the CLI directly from the monorepo workspace using `npx`, or install it globally for system-wide access:

```
# Run directly via npx
npx msx --help

# Install globally
npm install -g @modelscript/cli
msx --version
```

Global Options & Environment Variables

The CLI parser (`apps/cli/src/main.ts`) intercepts global execution flags prior to command dispatch and respects standard environment variables for headless CI/CD and enterprise hub integration:

Global Flags

Flag	Type	Description
`--daemon`	`boolean`	Route command execution through the persistent background compiler daemon (`~/.modelscript/daemon.sock`) to bypass cold-start WASM initialization and reuse warm Salsa caches.
`--help`	`boolean`	Display usage information and command-specific options.
`--version`	`boolean`	Print the installed `@modelscript/cli` semantic version.

Environment Variables

Variable	Default	Description
`NO_COLOR`	Unset	When set to any non-empty value, disables ANSI color sequences in terminal diagnostics, tables, and diff output.
`MODELSCRIPT_API_URL`	`http://localhost:3000`	Base URL for the ModelScript Hub, Package Registry, and Cloud HPC dispatch API (`~/.modelscriptrc` fallback).
`MODELSCRIPT_TOKEN`	Unset	Bearer token (`JWT` or `ms_bot_*` service token) used for non-interactive authentication against the Registry, Cloud HPC, and Hub APIs (also accepts `MODELSCRIPT_API_TOKEN`).

Persistent Compiler Daemon (`--daemon`)

Standard CLI invocations spin up an isolated Node.js process, compile or load `.wasm` parser binaries, and index workspace dependencies from disk. When iterating rapidly or running hundreds of checks in a pre-commit hook, pass `--daemon` to any command:

```
# Start or auto-spawn the background daemon and run lint with warm Salsa caches
msx lint model.mo --daemon

# Explicitly manage the daemon lifecycle
msx daemon status
msx daemon restart
msx daemon stop
```

When `--daemon` is passed, `msx` connects to the local Unix domain socket (`~/.modelscript/daemon.sock` on Linux/macOS or named pipe `\\.\pipe\modelscript-daemon` on Windows). If no daemon is running, the CLI automatically spawns one in the background, waits for readiness, and streams the command's `stdout`, `stderr`, and exit code back to the terminal.

Complete Command Reference by Category

The `msx` CLI registers **41 commands** (plus ergonomic aliases) organized into six functional domains. See CLI Commands for exhaustive flag tables and examples for every command.

1. Compilation, Simulation & Optimization

Command	Aliases	Summary
`compile`	`flatten`	Flatten a Modelica class hierarchy into a canonical Differential-Algebraic Equation (DAE) system.
`instantiate`	—	Instantiate a Modelica class and print its resolved component and equation structure.
`simulate`	—	Execute numerical DAE/ODE time-domain or steady-state simulations (JS, Arena, WASM, C, JIT, WebGPU, or Cloud HPC).
`optimize`	—	Solve nonlinear Dynamic Optimization and Optimal Control Problems (OCP) via direct collocation.
`grad`	—	Compute forward/adjoint parameter sensitivities ($\partial J/\partial p$) and finite-difference gradients.
`surrogate`	—	Run Design of Experiments (DoE), train Neural/Polynomial/RBF surrogates, and export ONNX/C/Modelica ROMs.
`mc`	—	Execute multi-threaded Monte Carlo uncertainty quantification and reliability sweeps.
`csg`	—	Extract Constructive Solid Geometry (CSG) primitives from Modelica models into 3D CAD meshes.

2. Linting, Formatting, Diffing & Localization

Command	Aliases	Summary
`lint`	—	Run static syntax, type, structural, and control-flow diagnostics across polyglot workspace files.
`parse`	—	Parse source files with the native WebAssembly GLR parser and inspect the concrete syntax tree (CST).
`format`	—	Format source files canonically using CST-preserving pretty-printing.
`unparse`	—	Round-trip a parsed CST back into source text to verify lossless grammar unparsing.
`diff`	—	Compute structural AST/CST and semantic DAE diffs between two files or Git revisions.
`pr-diff`	—	Generate visual SVG/HTML architectural diffs and Markdown PR review comments against a base Git ref.
`i18n`	—	Extract translatable strings, compile locale catalogs, and translate model identifiers across languages.

3. Formal Verification, Safety & Test Synthesis

Command	Aliases	Summary
`verify`	—	Verify contracts, requirements, and physical invariants using BMC, k-induction, IC3/PDR, and Z3 SMT solving.
`falsify`	—	Search for Signal Temporal Logic (STL) counterexamples by minimizing robustness $\rho(\phi, \mathbf{x}, t)$.
`export-formal`	—	Export flattened DAE systems to formal verification formats (`smt2` , `lustre` , `nusmv` , `tla` , `keymaera`).
`verify-decisions`	—	Verify architectural decision records (ADRs) and parameter trade-space bounds against formal system models.
`decompose`	—	Perform assume-guarantee contract decomposition and structural subsystem partitioning.
`generate-tests`	—	Synthesize MC/DC-complete unit test vectors and stimulus harnesses from formal model constraints.

4. Polyglot Digital Thread, Co-Simulation & Standards Interop

Command	Aliases	Summary
`render`	—	Render headless SVG diagrams for Modelica icons/schematics, SysML v2 (BDD/IBD/State/Activity), and PSCAD SLDs.
`fmu`	`export-fmu`	Package models into FMI 2.0/3.0 Functional Mock-up Units (`.fmu`) or inspect/import existing FMUs.
`import-fmu`	`fmu-import`	Unpack an external `.fmu` archive and generate typed Modelica wrapper blocks and SSP bindings.
`cosim`	—	Orchestrate and inspect distributed FMI/SSP Co-Simulation sessions, participants, and adaptive step optimizers.
`pscad`	—	Lint, render SLDs, diff, simulate EMT transients, verify IEEE 2800 ride-through, and export `.pscx` / `.pswx` projects.
`polyglot`	—	Query cross-language symbol indices, verify digital thread links, and check operational digital twin drift.
`align`	—	Reconcile Triple Graph Grammar (TGG) correspondences across SysML v2, Modelica, Spice, and STEP artifacts.
`sysml`	—	Export, import, check, inspect, and bi-directionally synchronize SysML v2 `.sysml` and Modelica `.mo` models.
`oslc`	—	Export OSLC 3.0 RDF/Turtle/JSON-LD catalogs or launch a live OSLC Tracked Resource Set (TRS 3.0) server.
`reqif`	—	Import and export OMG ReqIF 1.2 (`.reqif` / `.reqifz`) requirements exchange archives.
`ddp`	—	Inspect, pack, and convert ISO/PAS 81346 Digital Data Packages (`.ddp`) and Asset Administration Shells (`.aasx`).
`dhf`	—	Generate, audit, and inspect FDA 21 CFR Part 820 / ISO 13485 Design History File (DHF) compliance dossiers.

5. Language Workbench, Daemon & IDE Tooling

Command	Aliases	Summary
`daemon`	—	Manage the persistent background compiler daemon (`start` , `stop` , `status` , `restart`).
`build`	—	Compile a grammar DSL definition into a native WebAssembly GLR parser (`parser.wasm`) and TypeScript types.
`generate`	—	Scaffold new WASM GLR parsers or standalone VS Code language extensions from a grammar module.
`language`	`lang`	List, dynamically register, or unregister custom language plugins in the workspace registry.
`lsp`	—	Launch the multi-language ModelScript Language Server Protocol (LSP) server over `stdio` or TCP/IPC sockets.
`playground`	—	Start a local interactive web playground server for testing grammars, CSTs, and DAE compilation.
`sandbox`	—	Execute untrusted scripts or models inside a resource-capped, capability-isolated WebAssembly sandbox.

6. Package Registry, Cloud HPC & Hub Administration

Command	Aliases	Summary
`init`	—	Initialize a new `modelscript.json` package manifest in the current directory.
`login`	—	Authenticate with a ModelScript Registry / Hub server and store credentials in `~/.modelscriptrc`.
`logout`	—	Remove stored authentication tokens from `~/.modelscriptrc`.
`whoami`	—	Display the currently authenticated user, organization roles, and active registry endpoint.
`install`	`i`, `add`	Resolve, verify, and install versioned dependencies from the ModelScript Registry into `modelica_modules/`.
`pack`	—	Create a deterministic, SHA-256 content-addressed `.tgz` package tarball from the workspace.
`publish`	—	Sign (Ed25519) and publish a package tarball to the ModelScript Registry.
`unpublish`	—	Retract or deprecate a published package version from the registry.
`cloud`	—	Dispatch containerized simulation and CAE jobs to remote Slurm HPC clusters and manage compute credits.
`hub`	—	Initialize or check the health status of a self-hosted ModelScript Hub instance.
`db`	—	Inspect, migrate, and cryptographically verify the Hub PostgreSQL database schema and audit chains.
`archive`	—	Request, monitor, and download cold-storage compliance archives from the Hub.

Return Codes & CI/CD Integration

All `msx` commands return POSIX-compliant exit codes suitable for gating GitHub Actions, GitLab CI, or Jenkins pipelines:

Exit Code	Meaning
`0`	Command succeeded with zero errors (or all verification obligations proved `SAFE`).
`1`	Compilation error, static lint error (`severity: "error"`), counterexample falsification found, or runtime failure.
`2`	Invalid CLI arguments, missing required flags, or unreadable input paths.

CHAPTER 42

CLI Commands Reference

This reference documents every command, subcommand, and flag registered in `@modelscript/cli` (`apps/cli/src/main.ts` and `apps/cli/src/commands/`). All commands can be invoked via `modelscript` or its shorthand alias `msx`, and any command can be accelerated with the global `--daemon` flag.

Compilation, Simulation & Optimization

`compile` / `flatten`

Compiles and flattens a hierarchical Modelica class into a canonical Differential-Algebraic Equation (DAE) representation in linear WebAssembly memory (`DAEBuilder`). `flatten` is an exact alias for `compile`.

```
msx compile <name> <paths...> [options]
msx flatten <name> <paths...> [options]
```

Option	Type	Default	Description
`-L, --modelica-path <string>`	`string`	`process.env.MODELICAPATH`	Colon- or semicolon-separated search paths for resolving external Modelica libraries (such as MSL).
`-F, --flattener <backend>`	`ts | wasm | hybrid | diff`	`"hybrid"`	Flattener execution backend (`hybrid` pairs the Salsa query engine with the AssemblyScript WASM kernel; `diff` cross-checks TS vs. WASM parity).
`--timing`	`boolean`	`false`	Print per-stage compilation timing breakdowns (parsing, indexing, Salsa instantiation, DAE lowering).
`--memory-profile`	`boolean`	`false`	Print WebAssembly linear memory arena utilization and V8 heap statistics.

```
# Flatten a model using the hybrid WASM kernel with timing breakdowns
msx flatten Modelica.Electrical.Analog.Examples.ChuaCircuit -L ./libraries --timing
```

`instantiate`

Instantiates a Modelica class via the Salsa `QueryEngine` and prints the resolved component hierarchy, inherited modifications, and bindings prior to DAE index reduction.

```
msx instantiate <name> <paths...> [options]
```

Option	Type	Default	Description
`-L, --modelica-path <string>`	`string`	`process.env.MODELICAPATH`	Library search paths for resolving package imports.
`-F, --flattener <backend>`	`ts | wasm | hybrid | diff`	`"hybrid"`	Flattener backend used to resolve class specialization.
`--timing`	`boolean`	`false`	Print instantiation query timing.

`simulate`

Compiles a model, performs Block Lower Triangular (BLT) Pantelides index reduction and tearing, and executes a numerical time-domain or steady-state simulation locally or on a remote Slurm HPC cluster.

```
msx simulate <name> <paths..> [options]
```

Option	Type	Default	Description
`--start-time <number>`	`number`	`0.0` (or annotation)	Simulation start time t_0 in seconds.
`--stop-time <number>`	`number`	`1.0` (or annotation)	Simulation stop time t_f in seconds.
`--tolerance <number>`	`number`	`1e-6`	Relative/absolute local error tolerance for adaptive step-size control.
`--interval <number>`	`number`	`(stop - start) / 500`	Output communication interval Δt_{out} in seconds.
`--format <fmt>`	`csv | json | none`	`"csv"`	Trajectory output format (`none` suppresses trajectory serialization for benchmarking).
`--solver <solver>`	`euler | rk4 | dopri5 | tsit5 | trbdf2 | rodas4p | bdf | auto | cvode | webgpu`	`"dopri5"`	Numerical integration method (`bdf` , `trbdf2` , `rodas4p` , and SUNDIALS `cvode` handle stiff DAEs; `webgpu` executes batched GPU kernels).
`--realtime <number>`	`number`	Unset	Pace simulation execution to a real-time clock multiplier (e.g., `1.0` for 1x wall-clock HIL pacing).
`--engine <engine>`	`js | arena | wasm | c | jit`	`"js"`	Residual evaluation runtime (`arena` evaluates directly on `DAEBuilder` linear memory; `wasm` and `c` compile native kernels; `jit` uses dynamic JS/WASM codegen).
`--jit`	`boolean`	`false`	Shorthand flag to enable JIT compilation of residual and Jacobian evaluators.
`--jacobian <mode>`	`dense | sparse | fd`	`"sparse"`	Jacobian evaluation strategy (symbolic CSC `sparse` , `dense` , or finite-difference `fd`).
`-F, --flattener <backend>`	`ts | wasm | hybrid | diff`	`"hybrid"`	Flattener backend used during compilation.
`--steady-state`	`boolean`	`false`	Solve for the equilibrium point $\dot{\mathbf{x}} = \mathbf{0}$ using Newton-Raphson with homotopy continuation instead of time-stepping.
`--timing`	`boolean`	`false`	Report per-phase compilation, BLT, and solver wall-clock time.
`--memory-profile`	`boolean`	`false`	Report peak memory usage across the linear DAE arena and integrator buffers.
`--cloud`	`boolean`	`false`	Offload simulation execution to a remote ModelScript Cloud / Slurm HPC cluster.
`--profile <string>`	`string`	`"standard"`	Cloud compute hardware profile (`standard` , `high-cpu` , `gpu-a100` , etc.) when `--cloud` is enabled.
`--async`	`boolean`	`false`	Submit the `--cloud` job and return the job ID immediately without blocking for completion.
`--poll-interval <number>`	`number`	`1500`	Polling interval in milliseconds when waiting on a synchronous `--cloud` job.
`--output-dir <string>`	`string`	`"./results"`	Local directory where downloaded `--cloud` simulation artifacts are saved.
`--api-url <string>`	`string`	`MODELSCRIPT_API_URL`	Override the ModelScript Cloud API base URL.

```
# Run a stiff DAE simulation using the 4th-order Rosenbrock solver and sparse symbolic Jacobians
msx simulate RLC_Circuit circuit.mo --stop-time 10.0 --solver rodas4p --engine arena --jacobian sparse --format csv > trajector

# Dispatch a heavy multi-domain simulation to the Cloud HPC cluster
msx simulate ThermalGrid plant.mo --cloud --profile high-cpu --stop-time 3600 --output-dir ./runs/thermal
```

`optimize`

Formulates and solves a nonlinear Dynamic Optimization / Optimal Control Problem (OCP) over a flattened DAE using direct collocation and sequential quadratic programming (SQP).

```
msx optimize <name> <paths..> [options]
```

Option	Type	Default	Description
`--objective <string>`	`string`	**Required**	Mayer or Lagrange objective variable/expression to minimize at `--stop-time`.
`--controls <string>`	`string`	**Required**	Comma-separated list of free control input variables $u_i(t)$.
`--control-bounds <string>`	`string`	Unset	Bounds specification formatted as `u1:min:max,u2:min:max` (e.g., `throttle:0:1,brake:-5000:0`).
`--start-time <number>`	`number`	`0.0`	Horizon start time t_0.
`--stop-time <number>`	`number`	`1.0`	Horizon terminal time t_f.
`--num-intervals <number>`	`number`	`50`	Number of collocation finite elements across $[t_0, t_f]$.
`--tolerance <number>`	`number`	`1e-6`	KKT optimality convergence tolerance.
`--max-iterations <number>`	`number`	`200`	Maximum NLP solver iterations.
`--format <fmt>`	`csv | json`	`"csv"`	Output format for optimal state and control trajectories.

```
msx optimize EV_BatteryPack vehicle.mo \
  --objective energyLoss \
  --controls currentCommand \
  --control-bounds "currentCommand:-200:200" \
  --stop-time 600 \
  --num-intervals 100 \
  --format json > optimal_profile.json
```

`grad`

Computes forward, adjoint, or central finite-difference parameter sensitivities ($\nabla_{\mathbf{p}} J$) of scalar objective metrics with respect to model parameters.

```
msx grad <name> <paths..> [options]
```

Option	Type	Default	Description
`--params <string>`	`string`	**Required**	Comma-separated list of model parameters p_i to differentiate with respect to.
`--objectives <string>`	`string`	**Required**	Comma-separated list of state or output variables evaluated at `--stop-time`.
`--start-time <number>`	`number`	`0.0`	Integration start time.
`--stop-time <number>`	`number`	`1.0`	Integration stop time.
`--tolerance <number>`	`number`	`1e-6`	Integrator tolerance.
`--step-size <number>`	`number`	`1e-4`	Relative perturbation step ϵ when validating against finite differences.
`--format <fmt>`	`json | table | csv`	`"table"`	Output format for the Jacobian matrix $\partial \mathbf{J} / \partial \mathbf{p}$.

`surrogate`

Executes a Design of Experiments (DoE) campaign over a physics model, trains a Reduced-Order Model (ROM) surrogate, attaches convex-hull/bounding-box extrapolation guardrails, and exports standalone C, ONNX, or Modelica `.mo` blocks.

```
msx surrogate <name> [paths..] [options]
```

Option	Type	Default	Description
`--inputs <string>`	`string`	Required (unless `--from-onnx`)	Comma-separated list of input parameter/variable names.
`--input-bounds <string>`	`string`	Required (unless `--from-onnx`)	Parameter sampling bounds formatted as `p1:min:max,p2:min:max`.
`--outputs <string>`	`string`	Required (unless `--from-onnx`)	Comma-separated list of target output variables to approximate.
`--strategy <strategy>`	`full-factorial | latin-hypercube | sobol | central-composite`	`"latin-hypercube"`	Space-filling experimental design sampler.
`--num-samples <number>`	`number`	`50`	Number of full-order physics simulations to execute.
`--architecture <arch>`	`mlp | polynomial | rbf`	`"mlp"`	Surrogate model architecture (Multi-Layer Perceptron, multivariate polynomial response surface, or Radial Basis Function network).
`--start-time <number>`	`number`	`0.0`	Simulation start time per DoE sample.
`--stop-time <number>`	`number`	`1.0`	Simulation stop time per DoE sample.
`--format <fmt>`	`c | onnx | mo | all`	`"c"`	Target artifact format (`mo` synthesizes a drop-in pure Modelica `block` with embedded weights).
`--from-onnx <string>`	`string`	Unset	Bypass DoE training and import an existing pre-trained `.onnx` neural network directly into Modelica/C.
`--extrapolation-guardrails`	`boolean`	`true`	Emit runtime domain-validity checks (`assert()` statements in `.mo` / bounds checks in C) when queried outside training bounds.
`--extrapolation-level <lvl>`	`warning | error`	`"warning"`	Severity passed to generated Modelica `AssertionLevel.warning` or `AssertionLevel.error`.
`--extrapolation-flag`	`boolean`	`true`	Emit a boolean output port `isExtrapolating` on the generated surrogate block.
`--out-file <string>`	`string`	Auto-derived	Output file path for the generated surrogate artifact.
`--replace-component <string>`	`string`	Unset	Automatically substitute the specified component instance inside the parent model with the trained surrogate block.
`--replace-out <string>`	`string`	Unset	File path to write the rewritten parent model when `--replace-component` is used.
`--report <string>`	`string`	`"report.json"`	Output path for validation metrics (R^2, RMSE, max error, and training loss history).

```
msx surrogate HeatExchanger thermal.mo \
  --inputs "massFlow,inletTemp" \
  --input-bounds "massFlow:0.1:5.0,inletTemp:280:380" \
  --outputs "outletTemp,pressureDrop" \
  --strategy sobol \
  --num-samples 128 \
  --architecture mlp \
  --format all \
  --out-file build/hx_rom
```

`mc`

Runs high-throughput Monte Carlo uncertainty propagation across stochastic parameter distributions using worker threads and the linear-memory WASM evaluator.

```
msx mc <name> <paths..> [options]
```

Option	Type	Default	Description
`--params <string>`	`string`	**Required**	Comma-separated stochastic distribution specs: `name:dist:arg1:arg2` where `dist` is `gaussian` (μ, σ), `uniform` (a, b), `lognormal` (μ, σ), `beta` (α, β), or `triangular` (a, b, c).
`--runs <number>`	`number`	`1000`	Total number of Monte Carlo realizations.
`--threads <number>`	`number`	`1`	Number of parallel worker threads.
`--seed <number>`	`number`	Random	Deterministic PRNG seed for reproducible ensembles.
`--lhs`	`boolean`	`false`	Use stratified Latin Hypercube Sampling (LHS) instead of pseudo-random sampling.
`--engine <engine>`	`wasm | js`	`"wasm"`	Simulation kernel (`wasm` uses the zero-allocation WebAssembly arena solver).
`--format <fmt>`	`csv | json`	`"json"`	Output format for ensemble quantiles (mean, std, P5, P50, P95) and trajectories.
`--output <string>`	`string`	`stdout`	File path to write the ensemble results.

```
msx mc RLC_Filter filter.mo \
  --params "R:gaussian:100:5,C:uniform:1e-6:1.2e-6" \
  --runs 5000 \
  --lhs \
  --threads 8 \
  --seed 42 \
  --output mc_summary.json
```

`csg`

Extracts Constructive Solid Geometry (CSG) primitives and kinematic frame transformations from a Modelica class, writes `build/csg/execution_graph.json`, and invokes the OpenCASCADE (`@modelscript/cad`) worker to generate 3D CAD meshes.

```
msx csg <name> <paths...>
```

Linting, Formatting, Diffing & Compiler Inspection

`lint`

Executes the static analysis pipeline (WASM GLR parse validation, CST lint rules, Salsa semantic queries, unit checking, and control-flow analysis) across one or more files or directories.

```
msx lint <path> [paths...] [options]
```

Option	Type	Default	Description
`-l, --language <string>`	`string`	Auto-detected by extension	Force a specific language linter (`modelica` , `sysml2` , `modelscript` , `pscad` , `scad` , `ssp` , `csv`).
`-f, --format <fmt>`	`terminal | json`	`"terminal"`	Diagnostic output format (`terminal` renders source snippets with ANSI underlines; `json` outputs structured diagnostic arrays).

```
# Lint an entire polyglot workspace and emit JSON for CI processing
msx lint ./models ./requirements --format json
```

`parse`

Parses a file using its registered native WebAssembly GLR parser (`parser.wasm`) and prints the concrete syntax tree (CST), node byte offsets, or parse errors.

```
msx parse <file> [options]
```

Option	Type	Default	Description
`-l, --language <string>`	`string`	Auto-detected	Language ID override.
`--json`	`boolean`	`false`	Emit the CST as a structured JSON tree instead of an indented S-expression.
`--cst`	`boolean`	`false`	Include trivia tokens (whitespace and comments) from the linear CST buffer.

`format`

Formats source files canonically using the grammar's CST-preserving pretty-printer while preserving comments and annotations.

```
msx format <files...> [options]
```

Option	Type	Default	Description
`-w, --write`	`boolean`	`false`	Overwrite files in-place with formatted output.
`-c, --check`	`boolean`	`false`	Exit with code `1` if any file is not canonically formatted (ideal for CI).
`-l, --language <string>`	`string`	Auto-detected	Override language grammar.

`unparse`

Parses a source file into linear CST memory and unparses it back into text to verify lossless round-trip grammar rules.

```
msx unparse <file> [options]
```

Option	Type	Default	Description
`-l, --language <string>`	`string`	Auto-detected	Override language grammar.
`-o, --output <string>`	`string`	`stdout`	Write unparsed source to a file.

`diff`

Computes a structural CST/AST and semantic DAE comparison between two files (`oldFile` and `newFile`), identifying added/removed components, modified parameter bindings, and altered equations regardless of whitespace formatting.

```
msx diff <oldFile> <newFile> [options]
```

Option	Type	Default	Description
`-f, --format <fmt>`	`terminal | json | markdown`	`"terminal"`	Output format for structural change hunks.
`--semantic`	`boolean`	`true`	Include flattened DAE variable and equation parity comparison.

`pr-diff`

Generates visual schematic/architectural diffs (highlighting added components in green, modified in amber, and deleted in red) and PR review summaries between two Git revisions.

```
msx pr-diff [file] [options]
```

Option	Type	Default	Description
`-b, --base <ref>`	`string`	`"HEAD~1"`	Base Git revision, branch, or commit SHA.
`-h, --head <ref>`	`string`	Working tree / `HEAD`	Target head Git revision.
`-f, --format <fmt>`	`visual-html | visual-svg | pr-comment | json | terminal`	`"visual-html"`	Diff output format (`pr-comment` generates GitHub/GitLab Markdown with embedded Mermaid/SVG tables).
`-v, --view <view>`	`All | BDD | IBD | StateMachine`	`"All"`	Architectural diagram view to diff.
`-o, --output <path>`	`string`	`stdout`	Output file path.
`--breaking-only`	`boolean`	`false`	Filter report to breaking interface, port, or contract changes only.
`--open`	`boolean`	`false`	Automatically open the generated HTML report in the default browser.

`i18n`

Extracts translatable descriptions, documentation strings, and identifier aliases from models (`extract`), compiles translation catalogs (`compile`), or translates a model into a target locale (`translate`).

```
msx i18n <extract|compile|translate> <files...> [options]
```

Option	Type	Default	Description
`--locale <string>`	`string`	`"en"`	Target ISO locale code (e.g., `de` , `fr` , `ja` , `zh-CN`).
`-o, --output <string>`	`string`	`stdout`	Output catalog or translated source file path.
`--catalog <string>`	`string`	Unset	Path to the JSON/PO translation catalog when running `translate` .

Formal Verification, Safety & Test Synthesis

`verify`

Verifies formal contracts, `assert()` invariants, and SysML v2 requirement constraints using Bounded Model Checking (BMC), k-induction, IC3/Property-Directed Reachability (PDR), and Z3 SMT solving. Also emits standardized CI/CD and regulatory compliance reports.

```
msx verify <paths...> [options]
```

Option	Type	Default	Description
`-m, --model <name>`	`string`	Top-level model	Target model or requirement definition FQN to verify.
`-e, --engine <engine>`	`bmc | kind | ic3 | smt | all`	`"all"`	Verification engine (`bmc` bounded unrolling, `kind` k-induction, `ic3` PDR invariant synthesis, `smt` inductive Z3 check).
`-k, --bound <number>`	`number`	`20`	Maximum discrete step bound k for BMC and k-induction.
`--dt <number>`	`number`	`0.01`	Discretization time step Δt for continuous DAE unrolling.
`-f, --format <fmt>`	`terminal | json | sarif | junit | ctrf | dhf`	`"terminal"`	Output format (`sarif` v2.1.0 for GitHub Code Scanning, `junit` XML, `ctrf` JSON, or `dhf` Markdown compliance dossier).
`-r, --report <path>`	`string`	`stdout`	File path to write the formatted verification report.
`--witness`	`boolean`	`true`	Emit concrete counterexample state traces when a property is falsified.

```
# Run k-induction and emit a GitHub Code Scanning SARIF report
msx verify BrakingController brake.mo --engine kind --bound 50 --format sarif --report results.sarif
```

`falsify`

Performs optimization-guided falsification of Signal Temporal Logic (STL) safety properties ($\Box_{[a,b]}\phi$, $\Diamond_{[a,b]}\phi$, $\phi_1 \mathrm{U}_{[a,b]}\phi_2$) by searching disturbance and parameter spaces to minimize quantitative robustness $\rho(\phi, \mathbf{x}, t) < 0$.

```
msx falsify <name> <paths...> [options]
```

Option	Type	Default	Description
`--stl <formula>`	`string`	**Required**	Signal Temporal Logic formula (e.g., `"G[0,10](speed < 35.0 and temp < 120.0)"`).
`--params <string>`	`string`	**Required**	Search bounds for uncertain parameters/inputs (`p1:min:max,p2:min:max`).
`--budget <number>`	`number`	`200`	Maximum number of simulation rollouts for the CMA-ES / simulated annealing optimizer.
`--stop-time <number>`	`number`	`10.0`	Simulation horizon t_f.
`--seed <number>`	`number`	Random	PRNG seed for reproducible falsification search.
`-o, --output <path>`	`string`	`stdout`	Output JSON file containing minimum robustness $\rho_{\min}$ and the falsifying counterexample trajectory.

`export-formal`

Translates a flattened Modelica DAE or state machine into external formal verification and theorem-proving languages.

```
msx export-formal <name> <paths...> [options]
```

Option	Type	Default	Description
`-f, --format <fmt>`	`smt2 | lustre | nusmv | tla | keymaera`	`"smt2"`	Target formal specification language: SMT-LIB v2 (`.smt2`), Lustre v6 (`.lus`), NuSMV/nuXmv (`.smv`), TLA+ (`.tla`), or KeYmaera X differential dynamic logic (`.kyx`).
`--dt <number>`	`number`	`0.01`	Discretization step size Δt when encoding continuous ODEs as transition systems (`smt2` , `lustre` , `nusmv` , `tla`).
`-o, --output <path>`	`string`	`stdout`	Output file path.

```
# Export a hybrid cyber-physical controller to KeYmaera X dL format
msx export-formal ACC_Controller acc.mo --format keymaera -o proofs/acc_safety.kyx
```

`verify-decisions`

Evaluates Architectural Decision Records (ADRs) and SysML v2 trade-study constraints against flattened simulation parameters and formal bounds.

```
msx verify-decisions <paths...> [options]
```

Option	Type	Default	Description
`-f, --format <fmt>`	`terminal | json | markdown`	`"terminal"`	Output format for decision verification status.
`--strict`	`boolean`	`false`	Fail with exit code `1` if any architectural decision lacks a linked verification constraint.

`decompose`

Performs structural subsystem partitioning and Assume-Guarantee (A/G) contract decomposition across large system models, checking whether top-level system guarantees follow compositionally from subsystem contracts ($\bigwedge_i (A_i \Rightarrow G_i) \vdash G_{\text{sys}}$).

```
msx decompose <name> <paths...> [options]
```

Option	Type	Default	Description
`--strategy <mode>`	`scc | spectral | contracts`	`"contracts"`	Decomposition algorithm (strongly connected components, spectral graph min-cut, or assume-guarantee contract boundaries).
`-f, --format <fmt>`	`terminal | json | dot`	`"terminal"`	Output format (`dot` emits Graphviz subsystem dependency graphs).
`-o, --output <path>`	`string`	`stdout`	Output file path.

`generate-tests`

Synthesizes deterministic test vectors and stimulus harnesses from model guard conditions and assertions to achieve Modified Condition/Decision Coverage (MC/DC) and boundary-value coverage.

```
msx generate-tests <name> <paths...> [options]
```

Option	Type	Default	Description
`--coverage <criterion>`	`branch | mcdc | boundary`	`"mcdc"`	Coverage criterion solved via SMT constraint generation.
`-f, --format <fmt>`	`json | csv | modelica | vitest`	`"json"`	Output format (`modelica` generates a standalone verification `model` with `CombiTimeTable` stimuli; `vitest` emits an executable TypeScript test suite).
`-o, --output <path>`	`string`	`stdout`	Output file path.

Polyglot Digital Thread, Co-Simulation & Standards Interop

`render`

Renders headless, publication-grade SVG diagrams for Modelica component icons/schematics, SysML v2 architectural views, and PSCAD Single-Line Diagrams (SLDs) using `@modelscript/diagram`.

```
msx render <name> [paths..] [options]
```

Option	Type	Default	Description
`-i, --icon`	`boolean`	`false`	Render the Modelica `Icon(...)` coordinate graphics layer instead of the `Diagram(...)` schematic layer.
`-v, --view <view>`	`All | BDD | IBD | StateMachine | Activity | SLD | Taxonomy | Properties`	Auto-selected	Diagram view kind for SysML v2, OWL2 ontologies, or PSCAD power networks.
`-t, --theme <theme>`	`light | dark | blueprint`	`"light"`	Color palette and automatic dark-mode contrast inversion preset.
`-o, --output <path>`	`string`	`stdout`	Output `.svg` file path.

```
# Render a SysML v2 Internal Block Diagram in blueprint theme
msx render SpacecraftPower spacecraft.sysml --view IBD --theme blueprint -o docs/ibd.svg
```

`fmu`

Exports a Modelica class as a Functional Mock-up Unit (`.fmu` ZIP archive) compliant with **FMI 2.0** and/or **FMI 3.0** for Model Exchange (`me`) and Co-Simulation (`cs`), or imports/inspects an existing `.fmu` via `msx fmu import <file.fmu>`. `export-fmu` is an alias for `fmu`.

```
msx fmu <name> <paths..> [options]
msx export-fmu <name> <paths..> [options]
msx fmu import <path.fmu> [options]
```

Option	Type	Default	Description
`-o, --output <path>`	`string`	`<name>.fmu`	Output `.fmu` file path (or output `.mo` wrapper path when running `fmu import`).
`--fmi-version <ver>`	`2 | 3 | both`	`"both"`	FMI standard version (`2` emits FMI 2.0 `modelDescription.xml`; `3` emits FMI 3.0 with multidimensional arrays and clocks; `both` bundles multi-version descriptors).
`--type <kind>`	`me | cs | both`	`"both"`	FMU interface kind: Model Exchange (`me`), Co-Simulation (`cs`), or `both`.
`--target <runtime>`	`wasm | c | js | all`	`"all"`	Compiled binary/source payloads packaged inside `binaries/` and `sources/`.
`--source`	`boolean`	`true`	Include standalone ANSI C99 source code (`sources/*.c`) inside the FMU archive.
`--compile`	`boolean`	`false`	Invoke the host C compiler (`cc` / `clang` / `gcc`) to build native shared libraries (`.so` / `.dylib` / `.dll`) inside `binaries/`.
`--wasm`	`boolean`	`false`	Compile and bundle a self-contained WebAssembly module (`binaries/wasm32-wasi/<name>.wasm`).
`--xml-only`	`boolean`	`false`	Emit only the `modelDescription.xml` manifest without packaging a `.fmu` archive.
`--description <string>`	`string`	Model description	Override the `description` attribute in `modelDescription.xml`.
`--start-time <number>`	`number`	`0.0`	Default experiment `startTime` in `modelDescription.xml`.
`--stop-time <number>`	`number`	`1.0`	Default experiment `stopTime` in `modelDescription.xml`.
`--step-size <number>`	`number`	`0.001`	Default Co-Simulation communication `stepSize` in `modelDescription.xml`.
`-F, --flattener <backend>`	`ts | wasm | hybrid | diff`	`"hybrid"`	Flattener backend used during compilation.
`--timing`	`boolean`	`false`	Print FMU codegen and packaging timing.

```
# Export an FMI 3.0 Co-Simulation FMU with embedded C sources and WebAssembly binary
msx fmu DC_Motor motor.mo --fmi-version 3 --type cs --wasm --output build/DC_Motor.fmu
```

`import-fmu`

Unpacks an external FMI 2.0 or FMI 3.0 `.fmu` archive, parses its `modelDescription.xml`, and generates a strongly typed Modelica wrapper `block` (and optional SSP system binding) so the FMU can be connected and simulated inside ModelScript. `fmu-import` is an alias for `import-fmu`.

```
msx import-fmu <fmuPath> [options]
msx fmu-import <fmuPath> [options]
```

Option	Type	Default	Description
`-o, --output <path>`	`string`	`<modelName>.mo`	Output path for the generated Modelica wrapper file.
`--extract-dir <path>`	`string`	`./.fmu_cache/<name>`	Directory where FMU binaries and resources are extracted.

`cosim`

Manages distributed FMI/SSP Co-Simulation master orchestration sessions, participant FMUs, recorded trajectory histories, and adaptive communication step-size optimization against the ModelScript Hub / Co-Simulation service.

```
msx cosim <subcommand> [args...] [options]
```

Subcommand	Arguments	Description
`status`	—	Display health and active worker metrics of the Co-Simulation master orchestrator.
`sessions`	—	List active and completed distributed Co-Simulation sessions.
`participants`	`<sessionId>`	List connected FMU/simulator participants, port couplings, and current simulation time t in a session.
`fmus`	—	List registered FMU artifacts available in the Co-Simulation repository.
`upload`	`<path.fmu>`	Upload a local `.fmu` archive to the Co-Simulation registry.
`replay`	`<sessionId>`	Deterministically replay a recorded Co-Simulation session from its saved input/coupling log.
`history`	`<sessionId>`	Fetch step-by-step coupling residuals, rollbacks, and trajectory history for a session.
`optimize`	`<sessionId>`	Run post-mortem Richardson extrapolation analysis on a session to recommend optimal communication step sizes H_i.

`pscad`

Provides full lifecycle tooling for Manitoba Hydro **PSCAD** electromagnetic transient (EMT) workspaces (`.pscx` cases and `.pswx` workspaces), including domain-specific electrical linting, Single-Line Diagram (SLD) generation, EMT simulation, **IEEE Std 2800-2022** grid interconnection verification, and conversion to Modelica/SSP.

```
msx pscad <action> [files...] [options]
```

Action	Description
`lint`	Validate `.pscx` / `.pswx` schematics against electrical EMT rules (`M6001` – `M6011` , including floating nodes, transformer winding ratios, CSMF unconnected inputs, and traveling-wave line Δt underflow).
`diagram`	Render a publication-quality SVG Single-Line Diagram (SLD) from a `.pscx` project.
`diff`	Compute a topological and parameter diff between two `.pscx` revisions.
`sim`	Execute a fixed-step nodal admittance ($\mathbf{G}\mathbf{v}(t) = \mathbf{i}_{\text{hist}}(t - \Delta t)$) EMT simulation.
`verify-ieee2800`	Run automated **IEEE Std 2800-2022** Inverter-Based Resource (IBR) compliance checks (Low/High Voltage Ride-Through LVRT/HVRT, frequency ride-through, and reactive current injection step response).
`export-modelica`	Translate a `.pscx` schematic into an equivalent Modelica `.mo` model using `Modelica.Electrical` .
`export-ssp`	Package a `.pswx` multi-project workspace into an SSP 1.0 `.ssp` architecture container.

Option	Type	Default	Description
`-o, --out <path>`	`string`	`stdout`	Output file path for diagrams, trajectories, compliance reports, or exported `.mo` / `.ssp` files.
`--tstop <number>`	`number`	`1.0`	EMT simulation stop time in seconds (for `sim` and `verify-ieee2800`).
`--step <string>`	`string`	`"20us"`	EMT integration step size (e.g., `"10us"` , `"50us"`).

```
# Verify IEEE 2800-2022 LVRT/HVRT compliance on a solar plant PSCAD case
msx pscad verify-ieee2800 SolarPlant_250MW.pscx --tstop 3.0 --step 20us -o ieee2800_report.json
```

`polyglot`

Queries the unified multi-language `WorkspaceIndex` , verifies cross-domain traceability links (`@satisfies` , `@verifies` , `@allocates`), inspects digital threads, and monitors operational digital twin drift across all 12 supported languages.

```
msx polyglot <subcommand> [path] [options]
```

Subcommand	Description
`project [dir]`	Summarize all indexed languages, files, symbols, and cross-language bindings in a workspace.
`verify [dir]`	Verify cross-language dimensional consistency, port types, requirement satisfaction, and digital thread integrity.
`thread <symbol>`	Trace the upstream/downstream polyglot digital thread for a symbol (from OWL2 ontology → SysML v2 requirement → Modelica equation → STEP CAD geometry → ReqIF/DHF dossier).
`query <expr>`	Execute a cross-language symbol or FLWOR query against the unified `WorkspaceIndex`.
`check-twins [dir]`	Compare live telemetry streams or CSV logs against nominal model predictions to detect parameter drift and anomalies.
`index [dir]`	Build and serialize the workspace-wide polyglot symbol and dependency index (`polyglot-index.json`).

Option	Type	Default	Description
`-f, --format <fmt>`	`terminal | json | dot | mermaid`	`"terminal"`	Output format for thread graphs and verification reports.
`-o, --output <path>`	`string`	`stdout`	Output file path.

`align`

Executes the Triple Graph Grammar (TGG) and Critical Pair Analysis (CPA) bidirectional synchronization engine across paired architectural, physical, and geometric models.

```
msx align <sourceFile> <targetFile> [options]
```

Option	Type	Default	Description
`--corr <path>`	`string`	Auto-derived	Path to the `.corr.json` correspondence graph file tracking persistent element links.
`--direction <dir>`	`forward | backward | sync | check`	`"check"`	TGG execution mode (`check` reports drift/conflicts; `forward` propagates source → target; `backward` propagates target → source; `sync` reconciles non-conflicting deltas).
`--strategy <mode>`	`source-wins | target-wins | interactive | fail-on-conflict`	`"fail-on-conflict"`	Conflict resolution policy when Critical Pair Analysis detects overlapping edits.
`-o, --output <path>`	`string`	In-place / `stdout`	Output path for the synchronized target model or diagnostic report.

`sysml`

Provides dedicated interoperability workflows between **OMG SysML v2** (`.sysml`) and **Modelica** (`.mo`) models.

```
msx sysml <export|import|check|info|sync> <files...> [options]
```

Subcommand	Description
`export <file.mo>`	Project a Modelica class hierarchy into a SysML v2 `.sysml` `part def` / `port def` / `connection` architecture model.
`import <file.sysml>`	Synthesize executable Modelica `.mo` skeleton classes, connectors, and `connect()` equations from a SysML v2 file.
`check <files...>`	Validate SysML v2 ↔ Modelica port compatibility, KerML quantity units, and `@satisfies` requirement constraints.
`info <file.sysml>`	Print package metrics (part definitions, ports, state machines, requirements, and constraint blocks).
`sync <file.sysml> <file.mo>`	Synchronize structural changes and parameter values bi-directionally between `.sysml` and `.mo` files.

Option	Type	Default	Description
`-o, --output <path>`	`string`	`stdout`	Output file path for `export`, `import`, or `sync`.
`-f, --format <fmt>`	`terminal | json`	`"terminal"`	Output format for `check` and `info`.

`oslc`

Exposes workspace models and requirements as **OASIS OSLC 3.0** Linked Data resources (Requirements Management `oslc_rm` and Architecture Management `oslc_am`) with **TRS 3.0** (Tracked Resource Set) changelog feeds for IBM DOORS Next, Polarion, and Jira integration.

```
msx oslc <export|serve> [path] [options]
```

Subcommand	Description
`export [path]`	Serialize workspace symbols, requirements, and traceability links as static OSLC 3.0 RDF documents.
`serve [path]`	Start an HTTP OSLC 3.0 Service Provider catalog, delegated UI preview server, and TRS 3.0 change log endpoint.

Option	Type	Default	Description
`-f, --format <fmt>`	`turtle | jsonld | rdfxml`	`"turtle"`	RDF serialization format (`text/turtle`, `application/ld+json`, or `application/rdf+xml`).
`-p, --port <number>`	`number`	`3080`	Listening TCP port when running `msx oslc serve`.
`--base-uri <uri>`	`string`	`"http://localhost:3080/oslc"`	Base URI prefix minted for OSLC resource IRIs.
`-o, --output <path>`	`string`	`stdout`	Output file path when running `msx oslc export`.

`reqif`

Imports and exports **OMG ReqIF 1.2** (Requirements Interchange Format `.reqif` XML and `.reqifz` archives), translating specification hierarchies and attributes directly to and from SysML v2 `requirement def` blocks.

```
msx reqif <import|export> <file> [options]
```

Option	Type	Default	Description
`-o, --output <path>`	`string`	`stdout`	Target `.sysml` file path (on `import`) or `.reqif` / `.reqifz` path (on `export`).
`--archive`	`boolean`	`false`	Package exported ReqIF XML and embedded SVG diagrams as a compressed `.reqifz` bundle.

`ddp`

Inspects, packs, and converts **ISO/PAS 81346** Digital Data Packages (`.ddp`) and **IDTA Asset Administration Shell (AAS v3.0)** packages (`.aasx`) for industrial digital twin handover.

```
msx ddp <inspect|pack|to-aasx> <target> [options]
```

Subcommand	Description
`inspect <file.ddp>`	Validate and list reference designations (`=` , `+` , `-` aspect trees), models, and documents inside a `.ddp` archive.
`pack <dir>`	Bundle a workspace directory into a signed ISO/PAS 81346 `.ddp` archive.
`to-aasx <file.ddp>`	Convert a `.ddp` package or workspace manifest into an IDTA AAS v3.0 `.aasx` Digital Twin package with TechnicalData and SimulationModels submodels.

Option	Type	Default	Description
`-o, --output <path>`	`string`	`<name>.ddp` / `<name>.aasx`	Output archive path.

`dhf`

Generates, inspects, and cryptographically audits **Design History File (DHF)** compliance dossiers structured for **FDA 21 CFR Part 820.30**, **ISO 13485**, **ISO 26262**, and **DO-178C** regulatory submissions.

```
msx dhf <export|status|audit> [path] [options]
```

Subcommand	Description
`export [path]`	Compile requirements traceability matrices (RTM), formal verification proofs, simulation summaries, and SHA-256 artifact hashes into a DHF dossier.
`status [path]`	Report requirement verification coverage, unverified hazards, and suspect links across the workspace.
`audit [file]`	Verify the cryptographic SHA-256 hash chain and Ed25519 signatures of an exported DHF bundle.

`simulink`

Inspects, lints, compiles into linear DAEs, and projects MATLAB Simulink control block diagrams (`.slx` zip packages and `.mdl` structured text).

```
msx simulink <action> [files...] [options]
```

Action	Description
`inspect`	Dump subsystem block tree, parameters, solver settings, and line routing.
`lint`	Run pre-flight static analysis (algebraic loops `M7202` , unconnected ports `M7201` , divide-by-zero risks `M7209`).
`to-modelica`	Translate block diagram into standard acausal `Modelica.Blocks` models.
`to-dae`	Lower continuous states and transfer functions directly into the WebAssembly DAE arena.

Option	Type	Default	Description
`--out <path>`	`string`	`stdout`	Target file path for generated Modelica code.

```
msx simulink inspect CruiseControl.slx
msx simulink lint EngineGovernor.mdl
msx simulink to-modelica Suspension.slx --out=Suspension.mo
```

`autosar`

Inspects, lints, projects, and simulates AUTOSAR Classic and Adaptive (`.arxml`) software component architectures and Virtual Run-Time Environment (RTE) runnables.

```
msx autosar <action> [files...] [options]
```

Action	Description
`inspect`	Inspect software component types, port prototypes, runnables, timing events, and network PDUs.
`lint`	Detect unconnected R-ports (`M7401`), interface mismatches (`M7402`), WCET budget overflows (`M7405`), and PDU bit overlap (`M7406`).
`to-modelica`	Generate Modelica standard interface blocks for SIL plant co-simulation.
`to-sysml`	Project software architecture into OMG SysML v2 `part def` hierarchies.
`to-dae`	Lower software states and calibrated parameters into ModelScript linear-memory DAE arena.
`simulate-rte`	Execute the discrete event Virtual RTE scheduler and route signals across assembly connectors.

Option	Type	Default	Description
`--duration <sec>`	`number`	`0.05`	Simulation time window for Virtual RTE execution.
`--step <sec>`	`number`	`0.01`	Communication macro-step size for RTE runnable scheduling.
`--out <path>`	`string`	`stdout`	Target file path for generated code or execution logs.

```
msx autosar inspect Powertrain.arxml
msx autosar lint BrakeController.arxml
msx autosar to-sysml Steering.arxml --out=Steering.sysml
msx autosar simulate-rte PowertrainSystem.arxml --duration=0.5 --step=0.01 --out=rte_trace.json
```

`olga`

Provides complete command-line tooling for Schlumberger **OLGA** transient multiphase flow models (`.opi` , `.key` , `.genkey` , `.tab` , `.tpl` , `.ppl`), including drift-flux hydrodynamic simulation, severe slugging analysis, PVT flash, autonomous history matching, and flow assurance verification.

```
msx olga <action> [files...] [options]
```

Action	Description
`inspect`	Dump flowpath pipe discretization, valve configurations, sources, and boundary conditions.
`lint`	Run pre-flight flow assurance checks (`M7001` – `M7013` : CFL violations, severe slugging, pig stuck hazards, wax/asphaltene precipitation).
`simulate`	Execute transient 1D drift-flux simulation with Lagrangian slug tracking and scraper pig dynamics.
`verify`	Check formal regulatory limits (MAOP, hydrate subcooling margin, slug surge volume, erosion limits).
`diagram`	Render SVG schematics, seabed bathymetry elevation profiles, PVT envelopes, or slug statistics.
`tune`	Run automated history matching against field measurement CSV telemetry.
`flash`	Compute Peng-Robinson (PR) or Soave-Redlich-Kwong (SRK) thermodynamic vapor-liquid equilibrium (VLE) flash.
`network`	Solve coupled gathering network transients across multiple branches and subsea manifolds.
`cosim`	Run multi-rate SSP co-simulation coupling flowlines with topside process facilities.
`to-dae`	Lower flowline equations into ModelScript DAE arena.
`to-modelica`	Translate pipe and boundary networks into `Modelica.Fluid` models.

Option	Type	Default	Description
`--duration <sec>`	`number`	`60.0`	Transient simulation duration.
`--dt <sec>`	`number`	`1.0`	Numerical integration time step.
`--out <path>`	`string`	`stdout`	Output file for trajectory CSV, diagrams, reports, or models.
`--checkpoint-out <path>`	`string`	—	Save simulation state snapshot for zero-discontinuity warm restart.
`--restart <path>`	`string`	—	Resume transient simulation from checkpoint file.
`--anti-slug`	`boolean`	`false`	Enable active topside choke feedback controller for slug mitigation.

```
msx olga simulate subsea_tieback.opi --duration=3600 --out=traj.csv
msx olga lint deepwater_riser.genkey
msx olga diagram tieback.opi --view=profile --out=profile.svg
msx olga flash --comp="C1:0.75,C2:0.10,C3:0.08,nC4:0.04,C7:0.03" --p=50 --t=295.15 --envelope
```

`simpack`

Tooling for Dassault Systèmes **SIMPACK** multibody dynamics (`.spck` XML and `.sim` ASCII decks), including Grübler mobility analysis, 3D kinematic joint evaluation, Berg elastomeric bushings, and Craig-Bampton flexible body lowering.

```
msx simpack <action> [files...] [options]
```

Action	Description
`inspect`	Dump rigid bodies, joint markers, degrees of freedom, and force elements.
`lint`	Check for dangling marker references, kinematic loops, and parameter inconsistencies.
`mobility`	Calculate mechanism spatial degrees of freedom ($M = 6n - \sum c_i$) and detect overconstraints.
`to-modelica`	Project mechanism into standard `Modelica.Mechanics.MultiBody` components.
`to-fmu`	Package complete multibody assembly as an FMI 2.0/3.0 Co-Simulation FMU.
`to-dae`	Lower 3D kinematics with Baumgarte / GGL index reduction into `DAEBuilder` arena.

```
msx simpack inspect DoubleWishbone.spck
msx simpack mobility Suspension.spck
msx simpack to-modelica Mechanism.sim --out=Mechanism.mo
```

`gensys`

Tooling for **GENSYS** railway vehicle dynamics (`.calc` , `.geom` , `.track`), including FASTSIM wheel-rail contact mechanics, Polach non-linear creep, ERRI B176 track irregularities, Sperling ride comfort, and **EN 14363 / UIC 518** running safety homologation.

```
msx gensys <action> [files...] [options]
```

Action	Description
`inspect`	Inspect vehicle masses, suspension spring-dampers, and wheelset configurations.
`lint`	Validate vehicle inertial and geometric parameters against railway engineering rules.
`fastsim`	Evaluate high-throughput Hertzian/FASTSIM wheel-rail tangential contact creep forces.
`verify-en14363`	Run 2.0 m sliding window wheel climb derailment ($Y/Q \leq 0.80$) and Prud'homme lateral limit checks.
`comfort`	Compute ISO 2631-1 frequency-weighted comfort (W_k, W_d) and Sperling index (W_z).
`to-dae`	Lower vehicle multibody suspension and contact dynamics into `DAEBuilder` arena.

```
msx gensys inspect ManchesterBenchmark.calc
msx gensys verify-en14363 ManchesterBenchmark.calc --out=homologation.html
msx gensys comfort HighSpeedTrain.calc --speed=250 --track=ERRI_HighQuality
```

`spice`

Analog circuit simulation and netlist analysis for **SPICE** (`.cir` , `.sp` , `.net` , `.lib`), including DC operating point (`.OP`), transient analysis (`.TRAN`), AC frequency sweeps (`.AC`), and ECAD graph generation.

```
msx spice <action> [files...] [options]
```

Action	Description
`inspect`	List circuit components, nodes, subcircuits, and simulation directives.
`lint`	Check for floating electrical nodes (`M7101`), ideal voltage source loops (`M7102`), and current source cutsets.
`op`	Calculate DC operating point node voltages and branch currents.
`tran`	Execute time-domain transient simulation with numerical companion circuits.
`ac`	Run small-signal frequency domain analysis and compute bode plots.
`to-modelica`	Project netlist into standard `Modelica.Electrical.Analog` components.
`to-dae`	Lower circuit equations into ModelScript linear-memory DAE arena.

Option	Type	Default	Description
`--stop <sec>`	`number`	`0.01`	Transient simulation duration.
`--step <sec>`	`number`	`1e-5`	Transient simulation time step.
`--fstart <hz>`	`number`	`10.0`	AC sweep start frequency.
`--fstop <hz>`	`number`	`1e6`	AC sweep stop frequency.
`--points <num>`	`number`	`50`	AC sweep evaluation points per decade/interval.
`--out <path>`	`string`	`stdout`	Output file path for CSV trajectories or exported models.

```
msx spice op LowPassFilter.cir
msx spice tran PowerSupply.cir --stop=0.02 --step=1e-6 --out=transient.csv
msx spice ac AudioAmplifier.sp --fstart=10 --fstop=1e5 --points=100 --out=bode.csv
```

`adams`

Tooling for **MSC Adams** mechanism datasets (`.adm`) and command files (`.cmd`), including 3D Euler/Cardan kinematics, Grübler spatial mobility analysis, and cross-domain SIMPACK / Modelica projections.

```
msx adams <action> [files...] [options]
```

Action	Description
`inspect`	Inspect parts, markers, joints, forces, and initial conditions.
`lint`	Validate inertia tensors (`M7303`), check for dangling markers (`M7302`), and verify ground references.
`mobility`	Calculate mechanism spatial degrees of freedom and overconstrained loops.
`to-simpack`	Convert Adams mechanism dataset into SIMPACK `.spck` XML.
`to-modelica`	Project mechanism into `Modelica.Mechanics.MultiBody` components.
`to-dae`	Lower kinematic momentum equations directly into `DAEBuilder` arena.

```
msx adams mobility FourBarLinkage.adm
msx adams lint LandingGear.adm
msx adams to-simpack RobotArm.adm --out=RobotArm.spck
```

`urdf`

Robotics compiler tooling for **ROS URDF** (`.urdf`) and **Xacro** (`.xacro`) descriptions, featuring Featherstone recursive dynamics (ABA forward dynamics, RNEA inverse dynamics), joint limit verification, and 3D CAD projections.

```
msx urdf <action> [files...] [options]
```

Action	Description
`inspect`	Dump robot kinematic link tree, joint limits, total mass budget, and degrees of freedom.
`lint`	Validate kinematic tree acyclicity (`M7504`), joint limits (`M7508`), and inertia tensors (`M7506`).
`dynamics`	Evaluate forward (ABA) or inverse (RNEA) dynamics at test joint coordinates.
`to-modelica`	Generate `Modelica.Mechanics.MultiBody` robotic system.
`to-adams`	Project robot description into MSC Adams `.adm` solver dataset.
`to-simpack`	Project robot description into SIMPACK `.spck` XML.
`to-dae`	Lower kinematic state equations directly into `DAEBuilder` arena.

```
msx urdf inspect KukaIiwa14.urdf
msx urdf lint Manipulator.xacro
msx urdf to-modelica PandaArm.urdf --out=PandaArm.mo
```

`easy5`

Tooling for Boeing / MSC **EASY5** dynamic models (`.ez5` , `.ezm` , `.ezd`), including linear state-space analysis, User General (UG) Fortran routines, tabular map interpolation, and Modelica/SSP projections.

```
msx easy5 <action> [files...] [options]
```

Action	Description
`inspect`	Dump component blocks, connection topology, and parameter tables.
`lint`	Check for dangling connections, missing table files, and rate limits.
`to-modelica`	Project continuous block diagram to Modelica 3.x.
`to-ssp`	Package model as an FMI SystemStructure.ssd container.
`to-dae`	Lower continuous and discrete equations into `DAEBuilder` arena.

`gtsuite`

Tooling for Gamma Technologies **GT-SUITE** powertrain and engine models (`.gtm` , `.gtd`), including crank-slider kinematics, Wiebe combustion, Woschni heat transfer, Zeldovich NOx emissions, and DAE lowering.

```
msx gtsuite <action> [files...] [options]
```

Action	Description
`inspect`	Inspect engine cylinders, intake/exhaust manifolds, valves, and turbochargers.
`lint`	Detect compressor surge (`M9002`), invalid compression ratio (`M9005`), and acoustic CFL limits.
`simulate`	Execute in-WASM Mean-Value Engine Model (MVEM) transient simulation.
`to-modelica`	Project 1D powertrain architecture to Modelica standard blocks.
`to-dae`	Lower engine thermodynamics and flow dynamics into `DAEBuilder` arena.

`sima`

Tooling for DNV / SINTEF **SIMA** marine systems (`.inp` , `.sima` , `.sys` , `.hyd`), including Cummins equation 6-DOF floater dynamics, catenary mooring solvers, and DNV-OS-E301 station-keeping verification.

```
msx sima <action> [files...] [options]
```

Action	Description
`inspect`	Inspect marine vessels, mooring lines, catenary segments, and environmental loads.
`lint`	Check negative metacentric height (`M7001`), line underflow (`M7002`), and attachment nodes.
`simulate`	Execute 6-DOF time-domain Cummins floater motion with retardation impulse response.
`verify-dnv`	Verify DNV-OS-E301 line tension safety factors and watch-circle offset limits.
`to-modelica`	Project marine system into acausal Modelica physical components.
`to-dae`	Lower hydrodynamics and mooring tension equations into `DAEBuilder` arena.

`rocets`

Tooling for NASA **ROCETS** liquid rocket engine transient decks (`.roc` , `.deck` , `.map`), including turbopump flow balance, CEA equilibrium combustion, supersonic nozzle expansion, and Bartz throat heat transfer.

```
msx rocets <action> [files...] [options]
```

Action	Description
`inspect`	Inspect turbopumps, combustion chambers, regenerative cooling channels, and valves.
`lint`	Detect extreme mixture ratios (`M8011`), turbine overspeed (`M8013`), and negative chamber pressure.
`simulate`	Solve steady-state flow balance or transient rocket engine ignition start sequence.
`to-modelica`	Project cryogenic fluid networks and turbomachinery into Modelica 3.x.
`to-dae`	Lower propulsion governing equations into `DAEBuilder` arena.

`psse`

Tooling for Siemens **PSS/E** power transmission cases (`.raw` , `.dyr` , `.sav`), including bus/branch netlist extraction, Full-Newton AC power flow solving, and Modelica / PSCAD EMT projections.

```
msx psse <action> [files...] [options]
```

Action	Description
`inspect`	Dump bus count, generators, transmission lines, transformers, and load summary.
`lint`	Validate bus voltage bounds, check for isolated buses, and verify generator P/Q limits.
`powerflow`	Solve steady-state AC power flow using Full-Newton iteration on the nodal admittance matrix $\mathbf{Y}_{\text{bus}}$.
`to-modelica`	Project power grid into acausal Modelica transmission lines and generators.
`to-pscad`	Project PSS/E power network into PSCAD / EMT companion-circuit netlists.
`to-dae`	Lower bus injection equations and generator swing dynamics into `DAEBuilder` arena.

`lps`

Tooling for Energy Solutions **PipelineStudio** gas and liquid pipeline networks (`.lps` , `.tgx` , `.tlx` , `.psg`), including AGA-8 / GERG-2008 natural gas compressibility, hydrogen blending ($H_2 \leq 20\%$), open-channel slack line flow, water hammer liquid surge, and transient line pack accounting.

```
msx lps <action> [files...] [options]
```

Action	Description
`inspect`	Dump pipeline pipes, compressors, pumps, delivery regulators, and receipt nodes.
`lint`	Check for MAOP overpressure (`M8001`), compressor surge (`M8002`), and high hydrogen blend ratios.
`simulate`	Run transient fluid network pressure propagation and inventory line pack calculations.
`to-sysml`	Project pipeline topology and control station hierarchies into OMG SysML v2.
`to-modelica`	Project pipe network into `Modelica.Fluid` gas/liquid transmission models.
`to-dae`	Lower Euler gas/liquid equations directly into `DAEBuilder` arena.

Language Workbench, Daemon & IDE Tooling

`daemon`

Controls the lifecycle of the persistent background ModelScript compiler daemon (`~/.modelscript/daemon.sock`).

```
msx daemon <start|stop|status|restart>
```

Subcommand	Description
`start`	Spawn the background compiler daemon if not already running.
`stop`	Gracefully shut down the running daemon and remove its socket file.
`status`	Report whether the daemon is running, its PID, uptime, and cached workspace count.
`restart`	Stop any active daemon instance and immediately start a fresh one.

`build`

Compiles a `@modelscript/dsl` grammar definition into a zero-dependency native WebAssembly GLR parser (`parser.wasm`) along with strongly typed TypeScript CST node definitions.

```
msx build [grammarFile] [options]
```

Option	Type	Default	Description
`-o, --out-dir <dir>`	`string`	`"./dist"`	Output directory for `parser.wasm` and generated `.d.ts` bindings.
`--optimize`	`boolean`	`true`	Run LR state table compaction and WASM binary size optimization.

`generate`

Scaffolds a new WebAssembly GLR parser (`parser`) or a complete standalone VS Code language extension (`extension` including TextMate grammars, LSP wiring, and `package.json` contributions) from a `@modelscript/dsl` language definition.

```
msx generate <parser|extension> <grammarFile> [options]
```

Option	Type	Default	Description
`-o, --out-dir <dir>`	`string`	`"./generated"`	Destination directory for generated artifacts.

`language`

Manages the dynamic polyglot language registry (`lang` is an alias for `language`).

```
msx language <list|register|unregister> [target]
msx lang <list|register|unregister> [target]
```

Subcommand	Description
`list`	List all registered languages (`.mo` , `.sysml` , `.msx` , `.pscx` , `.scad` , `.stp` , `.owl` , `.ssp` , `.csv` , `.su2` , `.inp` , `.gbr`) and their loaded `.wasm` parser states.
`register <path>`	Dynamically register a custom language plugin module into `~/.modelscript/languages.json` .
`unregister <id>`	Remove a dynamically registered language plugin by language ID.

`lsp`

Starts the multi-language ModelScript Language Server Protocol (`@modelscript/lsp`) server.

```
msx lsp [options]
```

Option	Type	Default	Description
`--stdio`	`boolean`	`true`	Communicate over standard input/output streams.
`--socket <port>`	`number`	Unset	Communicate over a local TCP socket on the specified port.
`--node-ipc`	`boolean`	`false`	Communicate using Node.js IPC channels (used by the VS Code desktop extension host).

`playground`

Launches a local interactive web playground server for live grammar authoring, GLR parse tree visualization, Salsa query inspection, and DAE simulation.

```
msx playground [options]
```

Option	Type	Default	Description
`-p, --port <number>`	`number`	`4173`	Local HTTP port for the playground web UI.
`--open`	`boolean`	`true`	Automatically open the playground URL in the default browser.

`sandbox`

Executes untrusted `.msx` scripts or `.mo` simulations inside a memory-capped, capability-isolated WebAssembly sandbox with no host filesystem or network access.

```
msx sandbox <file> [options]
```

Option	Type	Default	Description
`--timeout <ms>`	`number`	`5000`	Maximum execution wall-clock timeout in milliseconds.
`--max-memory <mb>`	`number`	`128`	Maximum WebAssembly linear memory ceiling in megabytes.

Package Registry, Cloud HPC & Hub Administration

`init`

Initializes a new `modelscript.json` package manifest in the current directory.

```
msx init [options]
```

Option	Type	Default	Description
`-y, --yes`	`boolean`	`false`	Populate default manifest fields (`name` , `version: "0.1.0"` , `license: "MIT"`) without interactive prompts.
`--scope <org>`	`string`	Unset	Prefix package name with an organization scope (e.g., `@aerospace/flight-dynamics`).

`login`, `logout`, `whoami`

Manages authentication sessions against a ModelScript Registry or self-hosted Enterprise Hub. Credentials and Ed25519 signing keys are stored in `~/.modelscriptrc` .

```
msx login [--registry <url>] [--token <token>]
msx logout
msx whoami
```

Option	Type	Default	Description
`--registry <url>`	`string`	`MODELSCRIPT_API_URL`	Target ModelScript Hub / Registry URL.
`--token <token>`	`string`	Interactive prompt	Authenticate non-interactively using a personal access token or `ms_bot_*` service token.

`install`

Resolves semantic version constraints, verifies SHA-256 content-addressable digests and Ed25519 publisher signatures, and installs Modelica/SysML/ModelScript packages into `modelica_modules/` while updating `modelscript.lock` . Aliases: `i` , `add` .

```
msx install [packages...] [options]
msx i [packages...] [options]
msx add <packages...> [options]
```

Option	Type	Default	Description
`--frozen-lockfile`	`boolean`	`false`	Fail with exit code `1` if `modelscript.lock` does not exactly match `modelscript.json` (recommended for CI).
`--registry <url>`	`string`	`MODELSCRIPT_API_URL`	Override registry URL.
`--verify-signatures`	`boolean`	`true`	Enforce Ed25519 cryptographic signature verification on downloaded tarballs.

`pack`, `publish`, `unpublish`

Packages, cryptographically signs, and publishes libraries to the ModelScript Registry.

```
msx pack [options]
msx publish [tarballOrDir] [options]
msx unpublish <pkg>@<version> [options]
```

Option	Type	Default	Description
`--tag <tag>`	`string`	`"latest"`	Distribution tag assigned on `publish` (e.g., `latest`, `next`, `rc`).
`--access <level>`	`public | restricted`	`"public"`	Visibility level for scoped `@org/pkg` packages.
`--sign-key <path>`	`string`	`~/.modelscript/id_ed25519`	Path to the Ed25519 private key used to sign the package SHA-256 digest.
`--dry-run`	`boolean`	`false`	Validate manifest, run lint checks, and compute tarball digest without uploading to the registry.

`cloud`

Manages remote high-performance computing (HPC) simulation sweeps, Slurm batch jobs, and organization compute credit wallets on ModelScript Cloud.

```
msx cloud <subcommand> [args...] [options]
```

Subcommand	Arguments	Description
`profiles`	—	List available CPU/GPU hardware execution profiles (`standard`, `high-cpu`, `gpu-a100`, `slurm-mpi`) and credit burn rates.
`balance` / `wallet`	—	Display current organization compute credit balance, active reservations, and ledger history.
`jobs` / `list`	—	List recent and running cloud simulation jobs.
`status`	`<jobId>`	Query real-time queue state (`QUEUED`, `RUNNING`, `COMPLETED`, `FAILED`) and progress percentage for a job.
`logs`	`<jobId>`	Stream stdout/stderr compiler and solver logs from a cloud worker.
`cancel`	`<jobId>`	Terminate a queued or running cloud job and refund unused reserved credits.
`download`	`<jobId> [outDir]`	Download completed trajectory CSV/Parquet artifacts and diagnostic reports.
`dispatch`	`<name> <paths...>`	Directly submit a containerized simulation or DoE sweep job to the cloud queue.

`hub`, `db`, `archive`

Administrative CLI commands for operators managing a self-hosted **ModelScript Enterprise Hub** instance (see Platform Administration Guide).

```
# Initialize Hub configuration or query cluster service health
msx hub <init|status> [--api-url <url>]

# Inspect PostgreSQL migration state, apply schema upgrades, or verify the SHA-256 audit ledger chain
msx db <status|upgrade|verify> [--connection-string <postgres://...>]

# Request, poll, or download long-term regulatory cold-storage archives
msx archive <request|status|download> <targetId> [-o <path>]
```

PART VI

Reference & Diagnostics

CHAPTER 43

Compiler Linter Rules Reference

ModelScript includes a zero-allocation, multi-language static analysis engine that executes directly over linear WebAssembly Concrete Syntax Trees (`CompilerLint`) and memoized Salsa semantic queries (`lint__*` hooks). Across **7 languages**, the compiler enforces **over 200 diagnostic rules** before simulation or code generation begins.

Running the Linter

Diagnostics run automatically in the VS Code / Web IDE via `@modelscript/lsp`, or headlessly via the CLI:

```
# Lint files or directories (language auto-detected by extension)
npx msx lint model.mo architecture.sysml grid.pscx

# Emit machine-readable JSON diagnostics or SARIF v2.1.0 for GitHub Code Scanning
npx msx lint ./workspace --format json
npx msx verify ./workspace --format sarif --report diagnostics.sarif
```

---

1. Modelica (`@modelscript/modelica`)

Modelica diagnostics (`languages/modelica/src/errors.ts` and `languages/modelica/src/lints/`) mirror OpenModelica (`omc`) frontend semantic checks and are partitioned into six numeric ranges (`M1xxx` – `M6xxx`).

`M1xxx` — Syntax & Parser Diagnostics

Code	Rule Identifier	Severity	Message Template / Condition
`M1000`	`identity-arg-count`	`error`	`Function identity expects 1 argument, got <argCount>.`
`M1000`	`identity-first-arg-integer`	`error`	`First argument of identity must be an Integer expression.`
`M1001`	`parse-error`	`error`	`Syntax error near '<text>'.`
`M1002`	`parse-missing`	`error`	`Expected '<expected>' before end of statement.`
`M1003`	`empty-array-constructor`	`error`	`Empty array constructor '[]' or '{}' is not valid in Modelica.`
`M1099`	`wasm-flattener-unsupported`	`warning`	`WASM flattener kernel fallback triggered: <reason>.`

`M2xxx` — Name & Scope Resolution

Code	Rule Identifier	Severity	Message Template / Condition
M2001	duplicate-element	error	Element '<name>' is already declared in this scope.
M2002	variable-not-found	error	Variable '<name>' not found in scope.
M2003	class-not-found	error	Class '<name>' not found in scope.
M2003	class-not-found-in-scope	error	Class '<className>' not found in scope '<scopeName>'.
M2003	base-class-not-found-in-scope	error	Base class '<baseName>' not found in scope '<scopeName>'.
M2003	partial-base-class	error	Cannot instantiate partial base class '<baseName>'.
M2004	modifier-not-found	error	Element '<modName>' not found in class '<className>'. (or In modifier of '<componentName>', class or component '<modName>' not found in '<className>'.)
M2004	base-class-ambiguous	error	Base class '<baseName>' is ambiguous; found in multiple inherited scopes.
M2005	identifier-mismatch	error	The identifier at start and end are different '<startId>' != '<endId>'.
M2005	base-class-found-in	info	Candidate base class '<baseName>' found in '<scopeName>'.
M2006	modified-element-not-found	error	Modified element '<elementName>' not found in class '<className>'.
M2007	modifier-class-not-found	error	Class '<typeName>' for component '<compName>' not found.
M2011	same-name-type-shadowing	warning	Component '<name>' has the same name as its type '<typeName>'.
M2038	partial-lookup-disallowed	error	Lookup through partial component or class '<name>' is not allowed.
M2090	from-here	info	From here: '<context>'. (instantiation trace note)
M2095	from-here-trace	info	Instantiated from '<className>' here.

`M3xxx` — Type System & Dimensional Checking

Code	Rule Identifier	Severity	Message Template / Condition
M3001	type-mismatch-binding	error	Type mismatch in binding '<compName> = <exprText>', expected type <expected>, got <actual>.
M3002	type-mismatch-modifier	error	Type mismatch in modifier of '<modName>', expected type <expected>, got <actual>.
M3002	type-mismatch-modifier-binding	error	Type mismatch in modifier binding '<modName> = <exprText>', expected <expected>, got <actual>.
M3003	not-plug-compatible	error	Connectors '<left>' and '<right>' are not plug-compatible.
M3004	not-a-connector	error	<which> is not a valid connector.
M3005	redeclare-type-mismatch	error	Redeclarations of '<name>' must be subtype-compatible with '<baseType>'.
M3006	function-arg-type-mismatch	error	Type mismatch in argument '<argName>' of function '<fnName>', expected <expected>, got <actual>.
M3007	connect-invalid-form	error	Arguments to connect() must be component references, got '<expr>'.
M3007	function-return-type-mismatch	error	Return value of function '<fnName>' has type <actual>, expected <expected>.
M3008	extends-type-mismatch	error	Cannot extend non-class or incompatible specialization '<baseName>'.
M3009	array-index-type-mismatch	error	Array subscript '<expr>' must be of type Integer, Boolean, or enumeration, got <actual>.
M3010	unit-mismatch	warning	Inconsistent physical units in expression: '<leftUnit>' vs. '<rightUnit>'.
M3010	no-matching-function	error	No matching function found for call '<fnName>(...)'.
M3011	binary-op-type-mismatch	error	Operator '<op>' is not defined for operand types <leftType> and <rightType>.
M3012	optimization-objective-type	error	Optimization objective attribute must evaluate to a scalar Real expression, got <actual>.
M3013	optimization-constraint-type	error	Optimization constraint must be a relational or Boolean expression, got <actual>.
M3014	builtin-attribute-type-mismatch	error	Attribute '<attr>' on built-in type '<type>' expects <expected>, got <actual>.
M3015	attribute-type-mismatch	error	Invalid type <actual> for attribute '<attr>', expected <expected>.
M3016	conditional-type-mismatch	error	Branches of if-expression have incompatible types <trueType> and <falseType>.
M3017	connect-types-inconsistent	error	Connected elements '<left>' (<leftType>) and '<right>' (<rightType>) have inconsistent types.

`M4xxx` — Structural, Hierarchy & Variability Semantics

Code	Rule Identifier	Severity	Message Template / Condition
M4000	dimension-not-parameter	error	Array dimension '<expr>' must have parameter or constant variability.
M4000	cardinality-in-condition-only	error	Built-in operator cardinality() may only be used in the condition of an if-statement or if-equation.
M4000	conditional-expression-not-evaluated	error	Conditional component condition '<expr>' could not be evaluated at compile time.
M4000	dimensions-must-be-parameter	error	Non-function array dimensions must be known at compile time (constant or parameter).
M4001	extends-cycle	error	Circular inheritance detected in extends clause for '<className>'.
M4002	duplicate-modification	error	Element '<name>' is modified multiple times in the same modification clause.
M4003	array-dimension-mismatch	error	Array dimension mismatch: expected [<expectedDims>], got [<actualDims>].
M4004	unbalanced-model	warning	Model '<modelName>' is structurally unbalanced: <varCount> variables and <eqCount> equations.
M4005	protected-modification	error	Cannot modify protected element '<elementName>' of class '<className>' from outside.
M4006	external-with-algorithm	error	Function '<fnName>' cannot contain both an external clause and an algorithm section.
M4007	function-public-variable	error	Public non-input/non-output variable '<varName>' is not allowed in function '<fnName>'; declare it in a protected section.
M4008	array-subscript-count-mismatch	error	Too many subscripts for '<name>': expression has <rank> dimension(s), but <subCount> subscript(s) were provided.
M4009	function-default-arg-cycle	error	Cyclic dependency detected in default input arguments of function '<fnName>'.
M4010	function-invalid-var-type	error	Function '<fnName>' cannot declare connectors or models as local variables ('<varName>').
M4011	function-protected-io	error	Protected element '<varName>' in function '<fnName>' cannot have 'input' or 'output' causality.
M4012	function-arg-variability	error	Function input/output '<varName>' cannot have 'constant', 'parameter', or 'discrete' variability.
M4013	nested-when-statement	error	A 'when' clause cannot be nested inside another 'when' clause.
M4014	tuple-expression-context	error	Tuple expression '(...)' is only permitted on the left-hand side of a multi-output function call.
M4015	within-in-script	error	'within' clause must appear at the very top of a Modelica source file.
M4016	statement-in-standard-mode	error	Top-level executable statements are not allowed outside an algorithm section.
M4017	restriction-violation	error	Class specialization '<kind>' violates restriction: <detail>.
M4018	partial-instantiation	error	Cannot instantiate partial class '<className>'.
M4019	redeclare-non-replaceable	error	Cannot redeclare non-replaceable element '<elementName>'.
M4020	builtin-time-invalid	error	Built-in variable 'time' is only valid inside models and blocks, not in '<kind>'.
M4021	range-step-too-small	error	Range step value '<step>' is zero or invalid.
M4022	enum-range-with-step	error	Enumeration range expressions cannot specify an explicit step increment.
M4023	connect-in-when	error	'connect()' equations are not permitted inside 'when' clauses.

Code	Rule Identifier	Severity	Message Template / Condition
M4024	connect-in-initial	error	'connect()' equations are not permitted inside 'initial equation' sections.
M4025	connect-in-non-param-if	error	'connect()' inside an 'if' equation requires a parameter or constant condition.
M4026	final-override	error	Cannot modify final element '<elementName>'.
M4027	variability-binding-mismatch	error	Component '<name>' with <lhsVar> variability cannot be bound to expression '<expr>' with higher (<rhsVar>) variability.
M4028	partial-type-component	error	Non-replaceable component '<compName>' cannot be declared with partial type '<typeName>'.
M4029	component-binding-restriction	error	Binding expression on '<compName>' is not permitted in this specialized class context.
M4030	outer-modifier	error	Modifications are not permitted on 'outer' component declaration '<compName>'.
M4031	array-index-out-of-bounds	error	Array index <index> is out of bounds for dimension of size <size> in '<name>'.
M4032	function-invalid-prefix	error	Prefix '<prefix>' is not allowed on components inside a function.
M4033	function-multiple-algorithm	error	Function '<fnName>' may contain at most one 'algorithm' section.
M4034	replaceable-base-class	error	Base class '<baseName>' in an extends clause cannot be replaceable.
M4035	cyclic-dimension-dependency	error	Cyclic dependency detected while evaluating array dimension of '<name>'.
M4036	package-variable-not-constant	error	Component '<varName>' declared in package '<pkgName>' must be a 'constant'.
M4037	connector-variability	error	Connector component '<name>' cannot have 'parameter' or 'constant' variability on flow/stream variables.
M4038	flow-outside-connector	error	Prefix 'flow' or 'stream' on '<name>' is only valid inside a connector declaration.
M4039	called-function-partial	error	Cannot call partial function '<fnName>'.
M4039	discrete-not-on-lhs	error	Discrete variable '<varName>' must be assigned inside a 'when' clause.
M4040	protected-element-modification	error	Protected element '<name>' cannot be modified in a component modifier.
M4041	negative-dimension	error	Array dimension size <size> for '<name>' cannot be negative.
M4042	constant-has-no-value	error	Constant '<name>' is declared without a binding value.
M4043	constant-variability-violation	error	Constant '<name>' binding expression is not a compile-time constant.
M4044	non-array-modification	error	Subscripted modification applied to scalar component '<name>'.
M4045	binding-dimension-mismatch	error	Dimension mismatch in binding of '<name>': declared [<declDims>], expression has [<exprDims>].
M4046	constant-not-fixed	error	Constant '<name>' cannot have 'fixed = false'.
M4047	cardinality-invalid-context	error	cardinality() is deprecated and only allowed in specific guard conditions.
M4048	cardinality-expected-component	error	Argument to cardinality() must be a scalar connector component reference.
M4049	break-on-non-component	error	break() connection operator requires connector component references.
M4050	class-specialization-violation	error	Class '<name>' violates restrictions of its specialized class kind '<kind>'.

Code	Rule Identifier	Severity	Message Template / Condition
M4050	break-component-invalid-type	error	Arguments to break() must be connectors.
M4051	break-connect-not-found	error	No matching connect(<left>, <right>) equation found for break(<left>, <right>).
M4052	redeclare-final-component	error	Cannot redeclare final component or class '<name>'.
M4053	redeclare-constant-component	error	Cannot change variability of '<name>' in a redeclaration.
M4054	stream-unbalanced-connector	error	Connector '<connName>' contains stream variables (<streamCount>) without exactly one matching flow variable (<flowCount>).
M4055	cannot-connect-expandable-with-non-expandable	error	Incompatible non-expandable connectors connected via an expandable connector.
M4056	duplicate-inherited-element-not-identical	error	Inherited element '<name>' is declared in multiple base classes with non-identical definitions.
M4056	cannot-connect-undeclared-expandable-connectors	error	Cannot connect two undeclared virtual members '<left>' and '<right>' of expandable connectors.
M4057	expandable-connector-flow-element	error	Expandable connector '<connName>' cannot contain 'flow' variables ('<varName>').
M4058	recursive-definition	error	Recursive component/class definition detected for '<name>'.
M4059	max-recursion-depth-exceeded	error	Maximum instantiation recursion depth exceeded in '<className>'.
M4061	redeclare-non-inherited	error	Element '<name>' with 'redeclare' does not exist in any inherited base class.
M4062	invalid-external-object	error	Class '<className>' extending ExternalObject must define strictly 'constructor' and 'destructor' functions.
M4063	class-extends-non-inherited	error	'class extends <name>' requires '<name>' to be an inherited replaceable class.
M4064	protected-in-connector	error	Protected sections are not permitted inside connector '<connName>'.
M4065	equations-in-connector	error	Equations are not permitted inside connector '<connName>'.
M4066	external-object-missing-destructor	error	ExternalObject class '<className>' is missing a 'destructor' function.
M4066	algorithm-in-connector	error	Algorithm sections are not permitted inside connector '<connName>'.
M4067	external-object-missing-constructor	error	ExternalObject class '<className>' is missing a 'constructor' function.
M4067	invalid-connector-variability	error	Invalid variability prefix in connector '<connName>'.
M4068	external-object-not-replaceable	error	ExternalObject class or component '<name>' cannot be replaceable.
M4068	redeclare-condition-attribute	error	Conditional attribute 'if ...' cannot be added or modified in a redeclaration of '<name>'.
M4069	external-object-return-restriction	error	Only an ExternalObject constructor may return an ExternalObject instance.
M4070	external-non-function	error	'external' clause is only allowed inside a function definition, not in '<kind>'.
M4070	operator-encapsulation	error	Overloaded 'operator' must be encapsulated inside an 'operator record'.
M4071	external-invalid-language	error	Unsupported external language specification '<lang>'; expected "C", "Fortran 77", or "builtin".
M4071	operator-constructor-outputs	error	'operator' constructor function in '<recordName>' must have exactly one output.

Code	Rule Identifier	Severity	Message Template / Condition
M4072	external-invalid-arg-expr	error	Arguments to external function call must be simple component references or size() expressions.
M4072	operator-constructor-output-type	error	Output of 'operator' constructor must be of the enclosing operator record type '<recordName>'.
M4073	external-invalid-arg-size-const	error	Second argument to size() in an external function call must be an Integer constant.
M4073	variable-in-package-not-constant	error	Variable '<varName>' in package '<pkgName>' is not declared as 'constant'.
M4073	ambiguous-operator-overload	error	Ambiguous overloaded operator '<op>' for operand types <leftType> and <rightType>.
M4074	external-array-return-not-allowed	error	External C function '<fnName>' cannot return an array directly as its C return value.
M4074	no-matching-operator-function	error	No matching overloaded operator function '<op>' found in operator record '<recordName>'.
M4075	cannot-resolve-operator-type	error	Cannot resolve operator record type for overloaded operator '<op>'.
M4075	invalid-type-prefix-flow-nested	error	Prefix 'flow' cannot be applied to a structured component containing nested flow variables.
M4076	parameter-bound-contradiction	warning	Parameter '<paramName>' value (<val>) violates its declared min/max bounds [<min>, <max>].
M4077	loop-termination-inconclusive	warning	Cannot statically prove termination of while-loop in '<context>'.
M4078	terminate-in-function	error	'terminate()' statement is not allowed inside a function.
M4079	return-outside-function	error	'return' statement is only allowed inside a function algorithm section.
M4080	cyclic-constants-or-parameters	error	Cyclic dependency detected among constants/parameters: <cyclePath>.
M4081	cyclic-function-components	error	Cyclic dependency detected in local variable declarations of function '<fnName>'.
M4082	connector-non-parameter-subscript	error	Connector subscript '<expr>' in connect() must be a parameter or constant expression.
M4083	enum-duplicate-literal	error	Duplicate enumeration literal '<lit>' in enumeration type '<enumName>'.
M4084	enum-reserved-attribute-literal	error	Enumeration literal '<lit>' conflicts with built-in type attribute.
M4101	top-level-outer-element	warning	'outer' component '<name>' has no matching 'inner' declaration in enclosing hierarchy.
M4102	inner-not-found-existing-decl	warning	No matching 'inner' declaration found for 'outer' component '<name>'.
M4103	missing-redeclare-keyword	error	Missing 'redeclare' keyword when overriding replaceable element '<name>'.
M4104	invalid-redeclaration-class-as-component	error	Cannot redeclare class '<name>' as a component.
M4105	duplicate-redeclare	error	Element '<name>' is redeclared multiple times in the same scope.
M4106	invalid-redeclaration-prefix	error	Invalid type/variability prefix in redeclaration of '<name>'.
M4107	invalid-redeclaration-component-as-class	error	Cannot redeclare component '<name>' as a class.
M4108	redeclare-enum-not-subtype-generic	error	Redeclared enumeration '<name>' is not a valid subtype of '<baseEnum>'.
M4109	redeclare-enum-not-subtype	error	Enumeration '<subEnum>' is not a subtype of '<superEnum>'.

Code	Rule Identifier	Severity	Message Template / Condition
M4110	invalid-type-prefix-conflict	error	Conflicting prefixes on '<name>' (e.g. combining 'flow' with 'parameter' or 'input' with 'output').
M4111	invalid-redeclaration-basic-type-attribute	error	Cannot redeclare built-in attribute '<attr>' of primitive type '<type>'.
M4112	extends-other-base-class	error	Redeclarations using 'class extends' must extend the inherited base class.

M5xxx — Equations, Algorithms & Control-Flow Analysis (CFA)

Code	Rule Identifier	Severity	Message Template / Condition
M5001	equation-type-mismatch	error	Type mismatch in equation '<lhs> = <rhs>' of type <lhsType> = <rhsType>.
M5002	constrainedby-type-mismatch	error	Type '<newType>' in redeclaration of '<name>' does not satisfy constraining clause 'constrainedby <constrType>'.
M5003	if-branch-type-mismatch	error	'if' equation/statement branches have mismatched equation counts (<thenCount> vs. <elseCount>).
M5004	connect-flow-mismatch	error	Cannot connect flow variable '<left>' to non-flow variable '<right>'.
M5005	division-by-zero	error	Division by zero detected in constant/parameter expression '<expr>'.
M5006	assignment-type-mismatch	error	Type mismatch in algorithm assignment '<lhs> := <rhs>': expected <lhsType>, got <rhsType>.
M5007	reinit-type-mismatch	error	Type mismatch in reinit(<var>, <expr>): expected Real, got <actualType>.
M5007	for-iterator-not-1d	error	For-loop range expression '<expr>' must be a 1-dimensional vector, got <dims>D.
M5008	reinit-not-continuous	error	First argument '<var>' to reinit() must be a continuous-time Real state variable.
M5008	assignment-to-constant	error	Cannot assign to constant '<name>' in an algorithm section.
M5009	assignment-to-input	error	Cannot assign to read-only input variable '<name>'.
M5010	homotopy-recommended	info	Nonlinear algebraic loop detected involving '<vars>'; consider wrapping initialization in homotopy(actual, simplified).
M5010	elsewhen-variable-mismatch	error	All branches of a 'when'/'elsewhen' equation must solve for the exact same set of variables.
M5011	assignment-to-parameter	error	Cannot assign to parameter '<name>' outside an 'initial algorithm' section.
M5011	eval-external-builtin	warning	External function '<fnName>' cannot be constant-folded at compile time.
M5012	assignment-to-iterator	error	Cannot assign to loop iterator variable '<iterName>' inside a for-loop.
M5012	argument-not-differentiable	error	Argument '<expr>' to der() is not differentiable with respect to time.
M5013	not-a-stream-variable	error	Argument '<expr>' to inStream() or actualStream() must be a 'stream' connector variable.
M5014	assignment-to-non-variable	error	Left-hand side of assignment '<lhs>' is not an assignable variable reference.
M5015	function-call-contains-subscripts	error	Invalid subscript syntax on function call.
M5015	invalid-size-index	error	Dimension index <idx> in size(<arr>, <idx>) exceeds array rank <rank>.
M5016	tuple-subscript-not-allowed	error	Tuple expressions cannot be subscripted directly.
M5016	function-no-named-parameter	error	Function '<fnName>' has no formal parameter named '<paramName>'.
M5017	when-in-initial-equation	error	'when' clauses are not permitted inside 'initial equation' or 'initial algorithm' sections.
M5017	when-illegal-context	error	'when' clause is not permitted inside a function or nested conditional expression.
M5017	size-first-arg-not-array	error	First argument to 2-argument size(A, i) must be an array expression, got scalar <type>.
M5018	reinit-outside-when	error	Built-in operator reinit() may only be used inside the body of a 'when' equation.

Code	Rule Identifier	Severity	Message Template / Condition
M5018	reduction-invalid-expr-type	error	Invalid operand type <type> for reduction operator '<op>()'.
M5019	reinit-in-algorithm	error	'reinit()' is only allowed in 'when' equations, not inside an algorithm section.
M5019	failed-to-deduce-dimension	error	Failed to deduce array dimension size from binding expression for '<name>'.
M5020	reinit-arg1-not-variable	error	First argument to reinit() must be a component reference to a state variable.
M5020	output-not-definitely-assigned	warning	Function output '<outName>' is not definitely assigned on all control-flow paths.
M5020	illegal-end-usage	error	Keyword 'end' may only be used inside an array subscript expression.
M5021	reinit-arg2-type-mismatch	error	Second argument to reinit() must be a scalar Real expression, got <actual>.
M5021	uninitialized-variable-read	warning	Local variable '<varName>' may be read before being initialized in algorithm section.
M5022	unreachable-statement	warning	Unreachable statement detected after 'return' or 'break'.
M5025	implicit-iterator-subscript-missing	error	Implicit iteration variable '<iter>' is not declared and has no explicit range.
M5026	implicit-iteration-dim-mismatch	error	Dimension mismatch in array comprehension / implicit iteration.
M5027	when-condition-not-discrete	error	Condition '<expr>' of 'when' clause must be a discrete-time Boolean expression or vector.
M5028	when-if-branch-lhs-mismatch	error	'if' equation inside a 'when' clause must have an 'else' branch solving for identical variables.
M5029	clocked-when-has-elsewhen	error	A 'when' clause with a Clock condition cannot have 'elsewhen' branches.
M5030	clocked-when-branch-in-when	error	Cannot mix Clock conditions and Boolean event conditions in the same 'when' clause.
M5031	invalid-when-lhs	error	Left-hand side '<lhs>' of equation inside 'when' clause must be a simple component reference.
M5032	when-conditional-expected-boolean	error	'when' condition must be of type Boolean or Clock, got <actualType>.

`M6xxx` — Synchronous Clocks & State Machines (`MLS §16` & `§17`)

Code	Rule Identifier	Severity	Message Template / Condition
M6001	mixedClockDomains	error	Cannot mix variables from different clock partitions without an explicit transition operator ('sample', 'hold', 'subSample', 'superSample', 'shiftSample', 'backSample').
M6002	sampleFactorNotPositive	error	Sub-sampling or super-sampling factor in '<op>()' must be a positive Integer (> 0).
M6003	previousOutsideClocked	error	Synchronous operator 'previous()' may only be used in a clocked discrete-time partition.
M6004	holdInContinuous	error	Operator 'hold()' requires a clocked discrete-time argument and produces a continuous-time signal.
M6010	duplicateInitialState	error	State machine partition contains multiple 'initialState()' declarations.
M6020	impureCalledInPure	error	Impure function '<fnName>' cannot be called from inside a pure function.
M6021	impureInEquationSection	error	Impure function '<fnName>' may only be called inside a 'when' clause or 'initial' section.
M6040	breakConnectionNotFound	error	Target connection for 'break()' operator does not exist in the current scope.

---

2. SysML v2 (`@modelscript/sysml2`)

SysML v2 diagnostics (`languages/sysml2/src/errors.ts` — 34 rules) validate OMG SysML v2 / KerML syntax, naming conventions, port conjugation, SI unit algebra, ontological specialization, requirement satisfaction cycles, and behavioral state-machine/activity soundness.

Code	Rule Identifier	Severity	Description / Diagnostic Message
`S1001`	`parse-error`	`error`	Syntax error reported by the native WASM GLR SysML v2 parser.
`S1002`	`parse-missing`	`error`	Expected closing delimiter or keyword missing in `.sysml` source.
`S1003`	`definition-naming`	`warning`	Definition element (`part def` , `port def` , `action def` , `requirement def`) should use `PascalCase` .
`S1004`	`usage-naming`	`warning`	Usage element (`part` , `port` , `attribute` , `ref`) should use `camelCase` .
`S1005`	`package-naming`	`info`	Package declaration does not follow standard `PascalCase` naming convention.
`S2001`	`duplicate-member`	`error`	Duplicate member name declared within the same namespace or definition body.
`S2002`	`symbol-not-found`	`error`	Referenced feature, type, or requirement symbol could not be resolved in scope.
`S2003`	`unresolved-target` (`allocation-target-unresolved`)	`error`	Target endpoint of `connect` , `allocate` , `satisfy` , or `flow` cannot be resolved.
`S2004`	`unresolved-source` (`allocation-source-unresolved`)	`error`	Source endpoint of `connect` , `allocate` , or `flow` cannot be resolved.
`S2005`	`unresolved-import`	`error`	Imported package or member path (`import Pkg::*`) does not exist in the workspace index.
`S3001`	`type-mismatch`	`error`	Attribute binding or feature redefinition value is incompatible with declared KerML type.
`S3002`	`incompatible-port-connection`	`error`	Connected ports have incompatible definitions or un-conjugated (`~`) directional flow items.
`S3003`	`dimensional-mismatch`	`error`	Incompatible ISQ/SI physical unit dimensions in expression or attribute binding.
`S3004`	`invalid-multiplicity-bounds`	`error`	Multiplicity lower bound exceeds upper bound or is negative (e.g., `[5..2]`).
`S3005`	`target-not-requirement`	`error`	Target of a `satisfy` relationship is not a `requirement def` or `requirement` usage.
`S3006`	`verify-target-not-a-requirement`	`error`	Target of a `verify` relationship in a `verification def` is not a requirement.
`S3007`	`dimensional-connection-mismatch`	`error`	Connected port attributes carry incompatible physical units across an interface connection.
`S4001`	`circular-specialization`	`error`	Circular inheritance chain detected in `:>` (`specializes`) or `:>>` (`redefines`) hierarchy.
`S4002`	`circular-satisfaction`	`error`	Cyclic requirement dependency or circular `satisfy` derivation chain detected.
`S4003`	`empty-definition`	`info`	Structural definition (`part def` , `port def`) has no features, ports, or constraints.
`S4004`	`abstract-requirement-violation`	`warning`	Concrete component directly satisfies an `abstract requirement` instead of a concrete specialization.
`S4005`	`ontological-inconsistency`	`error`	Disjoint OWL2/KerML ontological classifiers applied to the same element.
`S4006`	`missing-subject`	`warning`	`requirement def` or `verification def` does not declare a `subject` parameter.
`S4007`	`empty-package`	`info`	Package declaration contains no child elements or imports.
`S5001`	`constraint-violated`	`error`	Static `require constraint` or `assume constraint` evaluates to `false` under nominal parameters.
`S5002`	`requirement-failed`	`error`	Requirement constraint expression is violated by the satisfying part's attribute bindings.

Code	Rule Identifier	Severity	Description / Diagnostic Message
S5003	broken-interface-contract (interface-contract-broken)	error	interface connection violates port assume/guarantee constraints or item flow directionality.
S5004	activity-cfa-unreachable	warning	Action node in an activity diagram (action def) is unreachable from the first start node.
S5005	activity-soundness-violation	error	Activity control-flow graph contains an unjoined fork , deadlocking join , or improper merge .
S5006	definite-assignment-violation	warning	out parameter of an action def or calc def is not definitely assigned along all execution paths.
S5007	statemachine-nondeterministic	warning	State in state def has overlapping outgoing transition guards capable of simultaneous firing.
S5008	statemachine-deadlock	warning	Non-final state in state def has no outgoing transitions (sink deadlock).

---

3. ModelScript Native .msx (@modelscript/lang)

The native .msx language (languages/modelscript/src/errors.ts — 10 rules) combines declarative equations with imperative scripts and XQuery-style FLWOR (for ... let ... where ... order by ... return) graph queries.

Code	Rule Identifier	Severity	Message Template
MSX1001	parse-error	error	Syntax error near '<token>'.
MSX2001	undefined-variable	error	Undefined identifier '<name>' in current scope.
MSX2002	duplicate-declaration	error	Identifier '<name>' is already declared in this scope.
MSX2003	unknown-type	error	Unknown type or struct '<typeName>'.
MSX3001	type-mismatch	error	Type mismatch: expected '<expected>', got '<actual>'.
MSX3002	invalid-binary-operand	error	Operator '<op>' cannot be applied to types '<leftType>' and '<rightType>'.
MSX4001	non-iterable-collection	error	FLWOR 'in' binding expression must evaluate to an iterable collection, got '<actualType>'.
MSX4002	flwor-unbound-variable	error	FLWOR variable '$<varName>' is not bound in any enclosing 'for' or 'let' clause.
MSX5001	argument-count-mismatch	error	Function '<fnName>' expects <expected> argument(s), got <actual>.
MSX5002	unknown-struct-field	error	Field '<fieldName>' does not exist on type '<typeName>'.

---

4. PSCAD Electromagnetic Transients (@modelscript/pscad)

PSCAD workspace and case linting (languages/pscad/src/lints/index.ts — 11 rules, invoked via msx lint or msx pscad lint) detects electrical topology errors, numerical EMT stability hazards, and power-system equipment misconfigurations in .pscx and .pswx files.

Code	Rule Identifier	Severity	Electrical / EMT Condition Checked
`M6001`	`floating-electrical-node`	`warning`	Electrical wire/bus node is connected to only one terminal without a ground reference or shunt path, causing singular nodal conductance matrices $\mathbf{G}$.
`M6002`	`transformer-winding-mismatch`	`error`	Three-phase transformer primary/secondary nominal voltage ratio or MVA base rating is inconsistent with connected bus voltage levels.
`M6003`	`invalid-parameter-bounds`	`error`	Passive component parameter (R, L, C, or base frequency f_0) is negative or zero where strictly positive values are required.
`M6004`	`weak-grid-warning`	`warning`	Point of Interconnection (POI) Short-Circuit Ratio ($\text{SCR} = S_{\text{sc}}/P_{\text{rated}} < 2.0$) indicates a weak grid prone to PLL synchronization instability.
`M6005`	`csmf-unresolved-input`	`error`	Continuous System Model Functions (CSMF) control block (PI controller, comparator, or transfer function) has an unconnected input signal pin.
`M6006`	`line-travel-time-underflow`	`error`	Bergeron / frequency-dependent transmission line propagation delay $\tau = \ell/v$ is smaller than the EMT simulation time step Δt (`--step`), violating traveling-wave decoupling.
`M6007`	`sync-machine-reactance-order`	`error`	Synchronous machine subtransient, transient, and synchronous reactances violate physical ordering ($X_d'' \le X_d' \le X_d$ or $X_q'' \le X_q' \le X_q$).
`M6008`	`mov-arrester-under-rated`	`warning`	Metal-Oxide Varistor (MOV) surge arrester Maximum Continuous Operating Voltage (V_{MCOV}) is below the bus nominal line-to-ground RMS voltage.
`M6009`	`duplicate-component-id`	`error`	Two component instances in the same `.pscx` canvas share an identical `id` attribute.
`M6010`	`bus-voltage-conflict`	`error`	Direct galvanic connection (wire or closed breaker without a transformer) bridges two buses with different nominal base voltages ($V_{\text{base},1} = V_{\text{base},2}$).
`M6011`	`port-domain-mismatch`	`error`	A scalar control signal wire (CSMF real/integer/boolean) is connected directly to a 3-phase electrical power terminal (or vice versa).

---

5. OpenSCAD Parametric CAD (`@modelscript/scad`)

OpenSCAD static analysis (`languages/scad/src/linter.ts` — 9 rules) validates parametric 3D CSG scripts (`.scad`) before invoking the OpenCASCADE B-Rep kernel, preventing degenerate zero-volume solids and unresolved variables.

Code	Severity	Condition Checked
`SCAD_INVALID_DIMENSION`	`error`	Geometric primitive (`cube` , `sphere` , `cylinder` , `polyhedron` , `linear_extrude`) has a non-positive radius, height, or size (≤ 0), or `cylinder` has both `r` and `d` conflicting.
`SCAD_UNDEFINED_VARIABLE`	`error`	Reference to an undeclared variable in an expression or transformation matrix.
`SCAD_UNDEFINED_FUNCTION`	`error`	Call to an undeclared user function (and not a built-in math/list function).
`SCAD_UNDEFINED_MODULE`	`error`	Instantiation of an undeclared CSG module (and not a built-in primitive or boolean operator).
`SCAD_UNKNOWN_METHOD`	`error`	Invalid member/method call on a vector or geometry value.
`SCAD_DIVISION_BY_ZERO`	`error`	Division or modulo by constant zero (`x / 0` or `x % 0`) in a parameter expression.
`SCAD_ARGUMENT_COUNT`	`error`	Built-in or user-defined module/function called with too few or too many positional arguments.
`SCAD_UNKNOWN_PARAMETER`	`warning`	Named keyword argument in a module/function call does not match any declared parameter of the target signature.
`SCAD_DUPLICATE_DECLARATION`	`warning`	Variable, function, or module is declared multiple times in the same lexical scope (violating OpenSCAD single-assignment semantics).

---

6. SSP 1.0 & SSV Parameter Archives (`@modelscript/ssp`)

The System Structure and Parameterization linter (`languages/ssp/src/linter.ts` — 20 rules across `.ssd` System Structure Descriptions and `.ssv` System Structure Parameter Values) validates multi-FMU co-simulation topologies and parameter bindings against the Modelica Association SSP 1.0 standard.

SSP Architecture Rules (`.ssd` / `.ssp` — 13 Rules)

Code	Severity	Condition Checked
`SSP_EMPTY_CONNECTOR_NAME`	`error`	`<ssd:Connector>` element is missing a non-empty `name` attribute.
`SSP_DUPLICATE_CONNECTOR`	`error`	Multiple connectors with the same `name` are declared on the same system or component.
`SSP_INVALID_CONNECTOR_KIND`	`error`	Connector `kind` attribute is not one of the valid SSP kinds (`input` , `output` , `parameter` , `calculatedParameter` , `local` , `inout`).
`SSP_EMPTY_COMPONENT_NAME`	`error`	`<ssd:Component>` element has an empty or missing `name` attribute.
`SSP_DUPLICATE_COMPONENT`	`error`	Two `<ssd:Component>` elements in the same `<ssd:Elements>` block share the same `name` .
`SSP_MISSING_COMPONENT_SOURCE`	`error`	`<ssd:Component>` of type `application/x-fmu-sharedlibrary` is missing its `source` URI attribute pointing to `resources/*.fmu` .
`SSP_UNRESOLVED_ELEMENT`	`error`	`<ssd:Connection>` `startElement` or `endElement` references a component name that does not exist in `<ssd:Elements>` .
`SSP_UNRESOLVED_CONNECTOR`	`error`	`<ssd:Connection>` `startConnector` or `endConnector` does not exist on the referenced endpoint component or system boundary.
`SSP_SELF_CONNECTION`	`warning`	`<ssd:Connection>` connects a connector directly back to itself (`startElement == endElement` and `startConnector == endConnector`).
`SSP_CONNECTOR_TYPE_MISMATCH`	`error`	Connected source and target connectors have incompatible data types (e.g., `Real` connected to `Boolean` or `String`).
`SSP_CONNECTOR_CAUSALITY_CONFLICT`	`error`	Invalid causal connection direction (e.g., connecting two `output` connectors together or an internal `input` to another internal `input`).
`SSP_UNRESOLVED_BINDING_PREFIX`	`error`	`<ssd:ParameterBinding>` `prefix` attribute references a component hierarchy path that cannot be resolved.
`SSP_INVALID_EXPERIMENT_BOUNDS`	`error`	`<ssd:DefaultExperiment>` specifies `startTime >= stopTime` .

SSV Parameter Set Rules (`.ssv` — 7 Rules)

Code	Severity	Condition Checked
`SSV_EMPTY_PARAMETER_NAME`	`error`	`<ssv:Parameter>` entry has an empty or missing `name` attribute.
`SSV_DUPLICATE_PARAMETER`	`error`	Duplicate `<ssv:Parameter>` entry overriding the same qualified parameter name in a single `.ssv` file.
`SSV_MISSING_PARAMETER_TYPE`	`error`	`<ssv:Parameter>` does not contain a typed value child element (`<ssv:Real>` , `<ssv:Integer>` , `<ssv:Boolean>` , `<ssv:String>` , or `<ssv:Enumeration>`).
`SSV_MISSING_PARAMETER_VALUE`	`error`	Typed parameter element (`<ssv:Real>` , etc.) is missing its required `value` attribute.
`SSV_INVALID_REAL_VALUE`	`error`	`<ssv:Real value="..."/>` cannot be parsed as a finite IEEE 754 floating-point number.
`SSV_INVALID_INTEGER_VALUE`	`error`	`<ssv:Integer value="..."/>` cannot be parsed as a valid base-10 integer.
`SSV_INVALID_BOOLEAN_VALUE`	`error`	`<ssv:Boolean value="..."/>` is neither `"true"` / `"false"` nor `"1"` / `"0"` .

---

7. CSV Trajectory & Lookup Tables (`@modelscript/csv`)

CSV validation (`languages/csv/src/csv-parser.ts` — 6 rules) checks external stimulus tables (`CombiTimeTable`), experimental telemetry datasets, and simulation reference trajectories.

Code	Severity	Condition Checked
`CSV_UNCLOSED_QUOTE`	`error`	Quoted RFC 4180 string field is missing its closing double-quote delimiter before end-of-file.
`CSV_EMPTY_HEADER`	`warning`	Header row contains an empty column name.
`CSV_DUPLICATE_HEADER`	`warning`	Two or more columns in the header row share an identical variable identifier.
`CSV_COLUMN_COUNT_MISMATCH`	`error`	Data row has a different number of fields than the header row.
`CSV_INVALID_NUMERIC_CELL`	`error`	Cell in a numeric signal column contains a non-numeric token (excluding `NaN` / `Inf` when explicitly allowed).
`CSV_NON_MONOTONIC_TIME`	`warning`	The leading `time` column is not monotonically increasing ($t_k < t_{k-1}$), which invalidates time-series interpolation and trajectory diffing.

CHAPTER 44

Export & Exchange Formats Reference

ModelScript serves as an open interoperability hub across systems engineering, multi-physics simulation, CAD/ECAD, formal verification, and regulatory compliance. Through `@modelscript/exchange`, `@modelscript/cad`, `@modelscript/runtime`, and `@modelscript/simulate`, the platform natively imports and exports **over 30 industry-standard formats**.

---

1. Functional Mock-up Interface (FMI 2.0 & FMI 3.0)

ModelScript compiles flattened DAE systems into self-contained `.fmu` archives (and imports external third-party `.fmu` binaries) conforming to the Modelica Association **FMI 2.0** and **FMI 3.0** specifications for both **Model Exchange (ME)** and **Co-Simulation (CS)**.

```
# Export an FMI 3.0 Co-Simulation FMU with embedded C99 source and WebAssembly binary
npx msx fmu DC_Motor motor.mo --fmi-version 3 --type cs --wasm --output DC_Motor.fmu

# Export a universal FMI 2.0 + 3.0 archive supporting both ME and CS
npx msx fmu ThermalPlant plant.mo --fmi-version both --type both --target all -o ThermalPlant.fmu

# Import an external third-party FMU and synthesize a typed Modelica wrapper block
npx msx import-fmu ./vendor/BatteryController.fmu -o BatteryControllerWrapper.mo
```

Archive Layout & Capabilities

Artifact Path	Standard	Description
`modelDescription.xml`	FMI 2.0 / 3.0	Declares `ModelVariables`, causalities (`input`, `output`, `parameter`, `local`), physical units, `ModelStructure` (`Outputs`, `Derivatives`, `InitialUnknowns`), directional derivative sparsity patterns, and FMI 3.0 `Clock` / multidimensional array dimensions.
`sources/*.c`, `sources/*.h`	ANSI C99	Zero-dependency generated C99 implementation of `fmi2*` / `fmi3*` entry points, analytic Jacobians (`fmi2GetDirectionalDerivative` / `fmi3GetDirectionalDerivative`), and embedded RK4/BDF Co-Simulation steppers.
`binaries/<platform>/*.so|.dll|.dylib`	Native ABI	Pre-compiled host shared libraries generated when `--compile` is passed.
`binaries/wasm32-wasi/<model>.wasm`	WebAssembly	Portable, sandboxed WASM FMU binary generated when `--wasm` or `--target wasm` is passed, executable directly in browser workers and edge runtimes.

---

2. System Structure and Parameterization (SSP 1.0 & SSV 1.0)

ModelScript natively parses, lints (`SSP_*` and `SSV_*` rules), simulates, and packages **Modelica Association SSP 1.0** system containers:

Extension	XML Namespace	Purpose
`.ssp`	ZIP Container	Bundles `SystemStructure.ssd`, `resources/*.fmu` components, parameter sets (`*.ssv`), and signal dictionaries (`*.ssm`).
`.ssd`	`http://ssp-standard.org/SSP1/SystemStructureDescription`	Defines hierarchical system architectures, FMU component references, typed connectors, causal signal connections, and `<ssd:DefaultExperiment>` metadata.
`.ssv`	`http://ssp-standard.org/SSP1/SystemStructureParameterValues`	Externalizes parameter sets (`Real`, `Integer`, `Boolean`, `String`, `Enumeration`, and units) for variant management without recompiling FMUs.
`.ssm`	`http://ssp-standard.org/SSP1/SystemStructureParameterMapping`	Maps abstract parameter names in `.ssv` catalogs to concrete hierarchical component paths.

```
# Lint an SSP architecture and its SSV parameter bindings
npx msx lint SystemStructure.ssd NominalParams.ssv

# Export a multi-project PSCAD workspace to an SSP 1.0 container
npx msx pscad export-ssp GridInterconnection.pswx -o GridInterconnection.ssp
```

---

3. Formal Verification & Theorem Proving Formats

The `msx export-formal` command lowers flattened DAE systems, synchronous clock partitions, and hybrid state machines into five formal verification languages for external model checkers and interactive theorem provers:

```
# Export a discrete-time transition encoding to SMT-LIB v2 for Z3 / cvc5
npx msx export-formal FlightControl fcs.mo --format smt2 --dt 0.005 -o fcs.smt2

# Export synchronous dataflow equations to Lustre v6 (Kind 2 / Lesar)
npx msx export-formal FlightControl fcs.mo --format lustre -o fcs.lus

# Export finite-state and quantized hybrid dynamics to NuSMV / nuXmv
npx msx export-formal FlightControl fcs.mo --format nusmv -o fcs.smv

# Export distributed state transitions and temporal invariants to TLA+ (TLC / Apalache)
npx msx export-formal RedundantBus bus.mo --format tla -o RedundantBus.tla

# Export continuous ODEs + discrete guards to KeYmaera X Differential Dynamic Logic (dL)
npx msx export-formal AdaptiveCruise acc.mo --format keymaera -o acc.kyx
```

Format Flag	Extension	Target Toolchains	Encoding Semantics
`smt2`	`.smt2`	Z3, cvc5, MathSAT, Yices 2	QF*NRA / QF_LRA unrolled transition relation $\mathrm{I}(\mathbf{x}_0) \wedge \bigwedge *{k\mathrm{T}}(\mathbf{x}_k, \mathbf{x}_{k+1})$ with negated safety assertions.
`lustre`	`.lus`	Kind 2, JKind, Lustre v6	Synchronous dataflow `node` with `pre` / `->` temporal operators and `--%PROPERTY` observer annotations.
`nusmv`	`.smv`	NuSMV, nuXmv	Symbolic transition system (`MODULE main`, `VAR`, `INIT`, `TRANS`) with `INVARSPEC` and `LTLSPEC` properties.
`tla`	`.tla`	TLC, Apalache	TLA+ `MODULE` with `Init`, `Next`, state variables `VARIABLES`, and temporal `Spec == Init /\ [][Next]_vars`.
`keymaera`	`.kyx`	KeYmaera X	Differential Dynamic Logic (dL) `ArchiveEntry` combining hybrid programs (`{x' = f(x) & H(x)}*`) and postcondition invariants.

---

4. CI/CD Quality Gates & Regulatory Compliance Dossiers

The `msx verify` and `msx dhf` commands serialize static diagnostics, formal proof certificates, and requirement traceability matrices into industry-standard CI/CD and regulatory formats:

```
# SARIF v2.1.0 for GitHub Code Scanning / Azure DevOps Security Tab
npx msx verify system.mo --format sarif --report linter-report.sarif

# CTRF (Common Test Report Format) JSON for unified CI dashboards
npx msx verify system.mo --format ctrf --report ctrf-report.json

# JUnit XML for Jenkins, GitLab CI, and CircleCI test summaries
npx msx verify system.mo --format junit --report junit-report.xml

# FDA 21 CFR Part 820 / ISO 13485 / ISO 26262 Design History File (DHF) Dossier
npx msx verify system.mo --format dhf --report dossier.md
npx msx dhf export ./workspace --format markdown -o DHF_Dossier.md
```

Format	Schema / Standard	Key Payload Contents
SARIF (`sarif`)	OASIS SARIF v2.1.0	Rule descriptors (`M1000` – `M6040` , `S1001` – `S5008`), physical file locations (1-indexed line/column spans), severity levels, and counterexample code flows.
CTRF (`ctrf`)	CTRF v0.0.0 JSON	Standardized `results.summary` (`tests` , `passed` , `failed` , `skipped`) and per-property verification duration and witness traces.
JUnit (`junit`)	JUnit XML	`<testsuites>` and `<testcase>` elements with `<failure>` stack traces for falsified invariants or lint errors.
DHF (`dhf`)	FDA 21 CFR 820.30 / ISO 13485	End-to-end Requirements Traceability Matrix (RTM), formal proof summaries, simulation verification evidence, and SHA-256 artifact integrity digests.

---

5. Systems Engineering & Requirements Interop (SysML v2, ReqIF, OSLC, OWL2)

ModelScript bridges descriptive MBSE specifications and semantic knowledge graphs with executable physics:

Standard	Extensions / Media Types	CLI Command / API	Description
OMG SysML v2 / KerML	`.sysml` , `.kerml` , `application/json`	`msx sysml <export|import|sync>` & REST `/api/v1/sysml2`	Textual SysML v2 notation and full **OMG Systems Modeling API and Services v2.0** REST compliance (`projects` , `commits` , `branches` , `tags` , `elements` , `relationships`).
OMG ReqIF 1.2	`.reqif` , `.reqifz`	`msx reqif <import|export>`	Bidirectional mapping between DOORS/Polarion ReqIF `SPEC-OBJECT` / `SPEC-RELATION` hierarchies and SysML v2 `requirement def` / `satisfy` blocks.
OASIS OSLC 3.0 & TRS 3.0	`.ttl` , `.jsonld` , `.rdf`	`msx oslc <export|serve>`	Exposes requirements (`oslc_rm:Requirement`) and architecture resources (`oslc_am:Resource`) with Tracked Resource Set (TRS 3.0) incremental change logs and delegated UI previews.
W3C OWL 2 / RDF / SPARQL 1.1	`.owl` (Functional Syntax), `.ttl` (Turtle), `.nt` (N-Triples), `.jsonld`	`/api/v1/libraries/rdf` , `/api/v1/libraries/sparql` , MCP `query_ontology_sparql`	Extracts ontological axioms, executes OWL 2 RL forward-chaining reasoning, and answers SPARQL 1.1 queries over workspace models.

---

6. Industrial Digital Twin & Open Hardware Packaging (DDP, AASX, OKH)

For manufacturing handover, Industry 4.0 asset integration, and open-source hardware certification, ModelScript supports three complementary packaging standards:

```
# Pack a workspace into an ISO/PAS 81346 Digital Data Package (.ddp)
npx msx ddp pack ./workspace -o PumpStation.ddp

# Convert a DDP archive into an IDTA Asset Administration Shell v3.0 (.aasx) package
npx msx ddp to-aasx PumpStation.ddp -o PumpStation.aasx
```

Standard	Format / Endpoint	Description
ISO/PAS 81346 DDP	`.ddp` (ZIP + XML/JSON)	Structures multi-domain engineering deliverables by function (`=`), product (`-`), and location (`+`) reference designation aspect trees.
IDTA AAS v3.0	`.aasx` (Open Packaging Conventions) & `?lens=aas`	Packages Asset Administration Shells containing standardized `Nameplate` , `TechnicalData` , `HandoverDocumentation` , and `SimulationModels` (FMU) submodels.
DIN SPEC 3105 / OSHWA OKH	`okh.json` / `okh.toml` & `?lens=okh`	Generates Open Know-How (OKH v1.0) manifests detailing bill of materials (BOM), CAD/ECAD design files, manufacturing tolerances, and SPDX licenses.

---

7. CAD, ECAD & Multi-Physics CAE Formats (STEP, CSG, Gerber, SU2, CalculiX)

`@modelscript/cad` and the polyglot language plugins provide direct geometric and mesh interoperability:

Domain	Format / Standard	Extensions	Capabilities
3D B-Rep CAD	**ISO 10303-21 STEP** (AP203 / AP214 / AP242)	`.stp` , `.step`	Native GLR parser for `DATA` entity graphs (`ADVANCED_FACE` , `B_SPLINE_SURFACE_WITH_KNOTS` , `MANIFOLD_SOLID_BREP` , `PRODUCT_DEFINITION`) and OpenCASCADE B-Rep export.
Parametric CSG & Meshes	**OpenSCAD**, **STL**, **OBJ**, **glTF 2.0 / GLB**	`.scad` , `.stl` , `.obj` , `.gltf` , `.glb`	Evaluates `.scad` scripts and Modelica `Visualizers` (`msx csg`) into watertight boundary-representation solids and triangulated 3D web assets.
ECAD / PCB Manufacturing	**Gerber RS-274X / X2**, **Excellon**, **IPC-2581**, **ODB++**	`.gbr` , `.gtl` , `.gbl` , `.drl`	Parses aperture definitions (`%ADD...*%`), copper traces, solder mask layers, and NC drill coordinates for 3D PCB stackup rendering and thermal-electrical co-simulation.
CFD (Fluid Dynamics)	**SU2**, **OpenFOAM**, **CGNS / VTK**	`.cfg` , `.su2` , `.vtk`	Parses and validates SU2 solver configurations (`SOLVER=RANS` , boundary markers, CFL schedules) and unstructured finite-volume meshes (`NDIME=` , `NELEM=` , `NPOIN=`).
FEA (Structural / Thermal)	**Abaqus / CalculiX**, **Nastran BDF**	`.inp` , `.bdf` , `.frd`	Parses `*NODE` , `*ELEMENT` , `*MATERIAL` , `*ELASTIC` , `*STEP` , and `*BOUNDARY` decks for thermo-mechanical ROM extraction and stress verification.

---

8. Power Systems & Electronics (PSCAD & SPICE)

Format	Extensions	CLI / Tooling	Capabilities
PSCAD EMT	`.pscx` , `.pswx` , `.psmx`	`msx pscad <lint|diagram|sim|verify-ieee2800|export-modelica|export-ssp>`	Parses hierarchical XML schematics, Blackbox/CSMF control blocks, transmission line segments, and executes nodal admittance EMT simulation and IEEE Std 2800-2022 compliance checks.
SPICE Netlists	`.sp` , `.cir` , `.net` , `.sub`	TGG Synchronization (`msx align`) & `@modelscript/modelica`	Bi-directionally maps SPICE netlists (`.subckt` , `R` , `L` , `C` , `M` MOSFET, `Q` BJT, `V` , `I`) to `Modelica.Electrical.Analog` / `Spice3` components.

---

9. Surrogate Modeling & Machine Learning (ONNX, C99, Modelica ROM)

The `msx surrogate` pipeline exports trained Reduced-Order Models (MLP neural networks, multivariate polynomials, and Radial Basis Functions) with embedded domain-validity guardrails:

```
# Train an MLP surrogate and export ONNX, standalone C99, and pure Modelica (.mo) simultaneously
npx msx surrogate Compressor compressor.mo \
  --inputs "rpm,pressureRatio" \
  --input-bounds "rpm:1000:12000,pressureRatio:1.1:4.5" \
  --outputs "massFlow,efficiency" \
  --format all \
  --out-file build/compressor_rom
```

- **ONNX (`.onnx`)**: Serializes weights, biases, input normalization scalers, and activation layers (`Tanh` , `ReLU` , `GELU`) as a standard ONNX protobuf graph for deployment in ONNX Runtime, TensorRT, or PyTorch. Can also be imported via `--from-onnx` .
- **Standalone ANSI C99 (`.c` / `.h`)**: Zero-dependency static C inference function with compile-time weight arrays and bounding-box extrapolation flags for embedded MCU deployment.
- **Modelica Block (`.mo`)**: Self-contained Modelica `block` containing pure matrix/vector equations, `isExtrapolating` boolean output port, and `assert()` guardrails.

---

10. Simulation Trajectories & Scientific Data (CSV, JSON, MAT v4, HDF5, Parquet)

Simulation and Monte Carlo runners (`msx simulate` , `msx mc` , and Cloud HPC workers) produce structured time-series outputs:

```
# Export standard RFC 4180 CSV trajectory
npx msx simulate DC_Motor motor.mo --stop-time 5.0 --format csv > trajectory.csv

# Export structured JSON trajectory with variable metadata and solver statistics
npx msx simulate DC_Motor motor.mo --stop-time 5.0 --format json > trajectory.json
```

Format	Source / Consumer	Description
CSV (`.csv`)	`msx simulate --format csv` , `msx mc --format csv` , `@modelscript/csv`	Leading monotonic `time` column followed by state, derivative (`der(...)`), and algebraic variable columns; directly consumable by `Modelica.Blocks.Tables.CombiTimeTable` and `msl-comparison-runner` .
JSON (`.json`)	`msx simulate --format json` , `msx mc --format json`	Contains `time: number[]` , `signals: Record<string, number[]>` , `units: Record<string, string>` , `events` , and solver convergence telemetry.
MATLAB Level 4 (`.mat`)	OpenModelica / Dymola compatibility	Dual-matrix (`Aclass` , `name` , `description` , `dataInfo` , `data_1` parameters, `data_2` trajectories) binary format for legacy post-processing scripts.
Apache Parquet & HDF5	Cloud HPC (`msx cloud download`) & Digital Twin Archive	Columnar Zstandard-compressed trajectory storage for multi-gigabyte Monte Carlo ensembles and long-horizon operational twin logs.

---

11. Interactive Notebooks & Visual Diagrams (`.monb` , SVG, HTML)

Artifact	Producer	Description
ModelScript Notebook (`.monb`)	VS Code / Web IDE (`modelscript.monbNotebook`)	JSON document format interleaving GitHub-Flavored Markdown + KaTeX cells with executable `.mo` , `.sysml` , and `.msx` cells and inline interactive Plotly/SVG simulation outputs.
Vector Diagrams (`.svg`)	`msx render` , `msx pscad diagram` , `msx pr-diff --format visual-svg`	Standalone, CSS-styled SVG schematics with ELK orthogonal port routing, Modelica `Icon` / `Diagram` coordinate graphics, SysML v2 BDD/IBD/State/Activity compartments, and dark/blueprint theme support.
Interactive Diff & Benchmark Dashboards (`.html`)	`msx pr-diff --format visual-html` , `npm run test:msl -- --report-html`	Self-contained, dark-mode HTML reports with pan/zoom SVG diff overlays and interactive trajectory error charts.

CHAPTER 45

MCP Server & IDE Workbench Reference

ModelScript provides first-class AI agent integration via the **Model Context Protocol (`@modelscript/mcp`)** alongside a comprehensive polyglot **VS Code Extension & Web IDE (`@modelscript/ide-client` , `@modelscript/lsp`)**. Both surfaces share the same zero-allocation WebAssembly GLR parsers, Salsa `QueryEngine` , and linear-memory `DAEBuilder` .

Part I: Model Context Protocol (MCP) Server

The `@modelscript/mcp` package exposes ModelScript's compilation, simulation, formal verification, ontology reasoning, Triple Graph Grammar (TGG) synchronization, and requirements traceability engines to MCP-compatible AI coding assistants (Claude Desktop, Cursor, Windsurf, Antigravity, Zed, and custom LLM orchestrators).

1. Deployment Modes & Transports

ModelScript supports three MCP execution architectures:

A. Local `stdio` Server (`modelscript-mcp`)

For local desktop AI agents, run the `modelscript-mcp` binary over standard input/output (`StdioServerTransport`):

```
{
  "mcpServers": {
    "modelscript": {
      "command": "npx",
      "args": ["-y", "@modelscript/mcp"],
      "env": {
        "MODELICAPATH": "/path/to/ModelicaStandardLibrary"
      }
    }
  }
}
```

B. Hosted Enterprise SSE Gateway (`/api/v1/mcp`)

The ModelScript Enterprise Hub (`apps/api/src/routes/mcp.ts`) exposes a multi-tenant HTTP Server-Sent Events (SSE) MCP gateway with built-in compute credit metering:

- **`GET /api/v1/mcp/sse`** : Establishes a persistent SSE stream (`text/event-stream`), authenticates the caller via `Authorization: Bearer <JWT | ms_bot_*>` , and returns a session-scoped `endpoint` event (`/api/v1/mcp/messages?sessionId=<uuid>`).
- **`POST /api/v1/mcp/messages?sessionId=<uuid>`** : Receives JSON-RPC 2.0 `tools/call` and `resources/read` requests, checks organization compute credit balances via `BillingService` (1 credit per lightweight query, 5 credits for `modelica_simulate` / `hybrid_simulate` / `modelica_doe` / `modelscript_closed_loop_synthesize`), and streams results over the SSE channel.

C. In-WASM AOT Polyglot Dispatch (`PolyglotMcpHost`)

In browser workers and edge sandboxes (`packages/mcp/src/polyglot-server.ts`), `PolyglotMcpHost` dynamically synthesizes MCP tool schemas from every registered `LanguageDefinition` (`defineLanguage({ mcp: { tools, resources } })`). When a tool declares an `aotExport` symbol, `PolyglotMcpHost` writes the arguments directly into WebAssembly linear memory (`__newString`), invokes the compiled AssemblyScript kernel (`mcpDispatchTool(toolId, targetPtr, paramsJsonPtr)`), and reads back the JSON result with zero JS object marshaling overhead.

---

2. Complete Reference of All 31 Core MCP Tools

The canonical MCP server (`packages/mcp/src/tools.ts`) registers **31 specialized engineering tools** grouped across seven functional domains:

Domain 1: Modelica Compilation, Simulation & Sensitivity (9 Tools)

Tool Name	Required Arguments	Optional Arguments	Description
`modelica_load`	`path: string`	—	Load a `.mo` file or package directory from disk into the stateful MCP workspace session and return all discovered class FQNs.
`modelica_parse`	—	`code?: string`, `path?: string`	Parse Modelica source text or a file using the native WASM GLR parser and return CST syntax diagnostics and top-level class names.
`modelica_flatten`	`className: string`	`code?: string`, `path?: string`	Flatten a hierarchical Modelica class into a canonical DAE (`ArenaDAEPrinter` output) and return variable/equation counts and flattened source text.
`modelica_lint`	—	`code?: string`, `path?: string`	Execute all `M1xxx` – `M6xxx` static lint rules and Salsa semantic queries, returning structured diagnostics with line/column spans.
`modelica_simulate`	`className: string`	`code?: string`, `path?: string`, `tStart?: number` (`0`), `tEnd?: number` (`1`), `dt?: number` (`0.01`), `tolerance?: number` (`1e-6`), `method?: "euler" | "rk4" | "dopri5" | "bdf" | "radau5" | "cvode" | "ida"` (`"rk4"`), `variables?: string[]`	Flatten, index-reduce (Pantelides + BLT tearing), and numerically simulate a Modelica model, returning downsampled time-series trajectories and solver statistics.
`hybrid_simulate`	`className: string`	`code?: string`, `path?: string`, `tStart?: number` (`0`), `tEnd?: number` (`10`), `dt?: number` (`0.01`), `maxEvents?: number` (`500`)	Run hybrid continuous/discrete event-driven simulation with zero-crossing root finding (`when` clauses, `reinit()`, state machines, and synchronous clocks).
`modelica_doe`	`className: string`, `parameters: Record<string, number[]>`	`code?: string`, `path?: string`, `tEnd?: number` (`1`), `dt?: number` (`0.01`), `metrics?: string[]`	Execute a Cartesian/factorial Design of Experiments (DoE) sweep across parameter combinations and return terminal/summary metrics for each run.
`modelica_sensitivity`	`className: string`, `parameters: string[]`, `outputs: string[]`	`code?: string`, `path?: string`, `tEnd?: number` (`1`), `dt?: number` (`0.01`), `perturbation?: number` (`0.01`)	Compute normalized local sensitivity Jacobians ($\frac{\partial y_i}{\partial p_j} \cdot \frac{p_j}{y_i}$) via central finite differences or forward AD.
`modelica_query`	`queryType: "list_classes" | "class_info" | "components" | "equations" | "connections"`	`className?: string`	Query the active Salsa `SymbolIndex` for class specializations, typed component declarations, equations, or `connect()` topology edges.

Domain 2: Cross-Domain Consistency & Ontology Reasoning (5 Tools)

Tool Name	Required Arguments	Optional Arguments	Description
`validate_system_consistency`	`sysmlCode: string`, `modelicaCode: string`	`mapping?: Record<string, string>`	Cross-check a SysML v2 architecture definition against a Modelica physics implementation for port mismatches, unit conflicts, and unsatisfied requirement constraints.
`query_ontology_sparql`	`ontologyText: string`, `sparqlQuery: string`	`enableReasoning?: boolean` (`true`)	Parse an OWL 2 Functional Syntax or Turtle ontology, optionally run OWL 2 RL forward-chaining inference, and execute a SPARQL 1.1 `SELECT` / `ASK` query.
`query_ontology_bgp`	`ontologyText: string`, `patterns: Array<{subject?: string, predicate?: string, object?: string}>`	`enableReasoning?: boolean` (`true`)	Match Basic Graph Patterns (BGP) against the inferred RDF triple store and return variable bindings.
`trace_fault_propagation`	`ontologyText: string`, `faultSourceIri: string`	`propagationPropertyIri?: string`, `maxDepth?: number` (`10`)	Traverse transitive causal/containment relationships across the ontological knowledge graph to compute downstream failure blast radius and FMEA impact chains.
`explain_inference`	`ontologyText: string`, `subject: string`, `predicate: string`, `object: string`	—	Construct a formal proof derivation tree explaining which OWL 2 RL rules and asserted axioms entailed a specific inferred triple (s, p, o).

Domain 3: Triple Graph Grammar (TGG) Digital Thread Synchronization (3 Tools)

Tool Name	Required Arguments	Optional Arguments	Description
`tgg_query_thread`	`sysmlCode: string`, `modelicaCode: string`	`spiceCode?: string`	Build the multi-domain Triple Graph Grammar correspondence graph across SysML v2, Modelica, and SPICE, returning synchronized element triples and orphan nodes.
`tgg_diagnose_conflict`	`sysmlCode: string`, `modelicaCode: string`	`baseSysmlCode?: string`, `baseModelicaCode?: string`	Run Critical Pair Analysis (CPA) over concurrent modifications in SysML v2 and Modelica to detect conflicting edits on shared parameters or ports.
`tgg_reconcile_slot`	`sysmlCode: string`, `modelicaCode: string`, `slotId: string`, `authority: "sysml" | "modelica"`	—	Resolve a parameter or structural divergence on `slotId` by propagating the authoritative domain's value across the TGG rule engine and returning updated source code.

Domain 4: Requirements Traceability Matrix (RTM) Intelligence (3 Tools)

Tool Name	Required Arguments	Optional Arguments	Description
`rtm_query_matrix`	`files: Record<string, string>`	—	Extract all requirements, architectural elements, physics models, and verification cases across a polyglot file map and compute coverage percentages and orphan counts.
`rtm_suggest_links`	`files: Record<string, string>`	`threshold?: number` (`0.35`)	Analyze identifier lexical similarity, shared physical quantities/units, and structural proximity to recommend missing `@satisfies` and `@verifies` traceability links.
`rtm_diagnose_suspect`	`baseFiles: Record<string, string>`, `headFiles: Record<string, string>`	—	Compare two workspace snapshots via semantic AST fingerprinting to flag downstream `@satisfies` / `@verifies` links as `SUSPECT` when upstream requirement expressions change.

Domain 5: Autonomous Closed-Loop Synthesis, Calibration & Workspace Editing (5 Tools)

Tool Name	Required Arguments	Optional Arguments	Description
`modelscript_closed_loop_synthesize`	`className: string`, `modelicaCode: string`, `requirements: Array<{variable: string, condition: "lt" | "gt" | "eq" | "between", threshold: number, upperThreshold?: number}>`	`tunableParameters?: string[]`, `maxIterations?: number` (`10`), `tEnd?: number` (`5.0`)	Execute an autonomous compile → lint → simulate → STL robustness evaluation → sensitivity-guided parameter tuning loop until all formal requirements pass.
`modelscript_patch_code`	`filePath: string`, `search: string`, `replace: string`	`validateBeforeWrite?: boolean` (`true`)	Apply a surgical search-and-replace edit to a workspace file, automatically running the WASM GLR parser and linter before committing the write to disk.
`modelscript_read_file`	`filePath: string`	`startLine?: number`, `endLine?: number`	Read source lines from a workspace file within the sandboxed root directory.
`modelscript_list_workspace_files`	—	`directory?: string`, `extensions?: string[]`	Recursively list engineering source files (`.mo`, `.sysml`, `.msx`, `.pscx`, `.scad`, `.stp`, `.owl`, `.ssp`, `.csv`) in the active workspace.
`modelica_diff_calibrate`	`className: string`, `modelicaCode: string`, `parameters: string[]`, `referenceData: {time: number[], signals: Record<string, number[]>}`	`learningRate?: number` (`0.05`), `maxEpochs?: number` (`50`), `parameterBounds?: Record<string, [number, number]>`	Calibrate unknown physical parameters against experimental CSV/reference trajectories using differentiable simulation and gradient descent (Adam / L-BFGS) to minimize trajectory MSE.

Domain 6: PSCAD Power Systems & IEEE 2800 Grid Compliance (4 Tools)

Tool Name	Required Arguments	Optional Arguments	Description
`pscad_lint`	`xmlContent: string`	`fileName?: string`	Run all 11 electrical EMT lint rules (`M6001`–`M6011`) on a `.pscx` or `.pswx` XML payload.
`pscad_simulate`	`xmlContent: string`	`tStop?: number` (`1.0`), `stepUs?: number` (`20.0`)	Execute a fixed-step nodal admittance EMT simulation on a `.pscx` case and return three-phase bus voltage, branch current, and CSMF control signal waveforms.
`pscad_verify_ieee2800`	`xmlContent: string`	`tStop?: number` (`3.0`), `stepUs?: number` (`20.0`)	Run automated **IEEE Std 2800-2022** LVRT/HVRT voltage ride-through, frequency ride-through, and reactive current injection compliance verification.
`pscad_export`	`xmlContent: string`, `targetFormat: "modelica" | "ssp" | "svg"`	—	Convert a PSCAD `.pscx` schematic into an executable Modelica `.mo` model, an SSP 1.0 `.ssd` descriptor, or a rendered Single-Line Diagram SVG.

Domain 7: Social Engineering Network & Bot Collaboration (2 Tools)

Tool Name	Required Arguments	Optional Arguments	Description
`social_read_thread`	`postId: number`	—	Read a social discussion thread with its full conversation tree, parent posts, child replies, and attached engineering artifact views.
`social_post_reply`	`replyToId: number`, `content: string`	`artifactViewId?: number`	Post an engineering response or simulation synthesis back into a discussion thread on behalf of an authorized bot or automated agent.

Declarative Polyglot Language Tools (`PolyglotMcpHost`)

When using `PolyglotMcpHost` (`packages/mcp/src/polyglot-server.ts`), additional language-declared tools from `languages/*/src/language.ts` are automatically mounted:

- `sysml2_extract_topology` : Extracts hierarchical part definitions, ports, and item flow connections from a `.sysml` workspace symbol (`aotExport: "mcp_sysml2_extract_topology"`).
- `sysml2_verify_requirements` : Evaluates SysML v2 `requirement def` constraint satisfaction across the workspace (`aotExport: "mcp_sysml2_verify_requirements"`).

---

3. MCP Resources (`resources/list` & `resources/read`)

The MCP server exposes live workspace state as URI-addressable resources (`packages/mcp/src/resources.ts` and `languages/*/src/language.ts`):

Resource URI / Template	MIME Type	Description
`modelica://libraries`	`application/json`	Lists all loaded Modelica package roots and top-level libraries in the current session.
`modelica://classes`	`application/json`	Lists every indexed class FQN, specialized kind (`model` , `block` , `connector` , `record` , `function` , `package`), and description.
`modelica://classes/{name}`	`application/json`	Returns full class metadata for `{name}` , including source text, components, equations, and `connect()` clauses.
`modelica://workspace/{className}/ast`	`application/json`	(`PolyglotMcpHost`) Serializes the linear CST/AST structure of `{className}` .
`modelica://workspace/{className}/dae`	`application/json`	(`PolyglotMcpHost`) Returns the lowered DAE variables, parameters, and equations for `{className}` .
`sysml2://workspace/{partName}/topology`	`application/json`	(`PolyglotMcpHost`) Returns the resolved port and connection graph for a SysML v2 part.

Part II: VS Code Extension & Web IDE Workbench

The official ModelScript extension (`modelscript.modelscript` , source in `apps/ide/` and `packages/lsp/`) runs both in **VS Code Desktop** (Node.js extension host) and **VS Code for the Web / `ide.modelscript.org`** (100% browser WebWorker host).

1. Polyglot Language Server (`@modelscript/lsp`)

A single unified Language Server Protocol instance provides sub-millisecond incremental intelligence across all 12 registered languages (`.mo` , `.sysml` , `.kerml` , `.msx` , `.pscx` , `.pswx` , `.scad` , `.stp` , `.step` , `.owl` , `.ssp` , `.ssd` , `.ssv` , `.csv` , `.su2` , `.cfg` , `.inp` , `.gbr`):

- **Real-Time Diagnostics**: Incremental GLR parsing + Salsa `lint__*` queries + cross-file unit/type checking.
- **Navigation & Refactoring**: Go-to-Definition, Find All References (across `.sysml` ↔ `.mo` ↔ `.msx` boundaries), Workspace Symbol Search, Rename Symbol, and Document Outline.
- **Code Lenses & Inlay Hints**: Inline equation/variable balance badges (`[Balanced: 14 vars / 14 eqs]`), inferred SI unit inlay hints, and one-click `▶ Simulate` , `⚡ Verify` , and `📐 Render Diagram` CodeLenses above every `model` , `part def` , and `verification def` .

2. Debug Adapter Protocol (DAP) Simulation Debugger

ModelScript registers a full **Debug Adapter Protocol** implementation (`"type": "modelscript"`) that lets engineers step through continuous-time DAE integrations, state-machine transitions, and zero-crossing events exactly like stepping through imperative code:

- **Time & Condition Breakpoints**: Set breakpoints on equation lines, `when` clauses, state transitions, or conditional simulation expressions (e.g., `time >= 2.5` or `der(omega) < 0`).
- **Execution Controls**: `Continue` , `Step Over` (advance one solver step Δt), `Step Into` (step into Newton-Raphson / event-iteration micro-steps), `Step Back` (reverse-time trajectory scrubbing), and `Restart` .

- **Live Inspection Scopes**: Inspect `States (x)`, `Derivatives (der(x))`, `Algebraic Variables (y)`, `Parameters (p)`, `Discrete / Clocked`, and `Solver Telemetry (h, localError, jacobianCond)` in the VS Code Variables and Watch panes.

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "modelscript",
      "request": "launch",
      "name": "Debug Modelica Simulation",
      "model": "Modelica.Mechanics.Rotational.Examples.First",
      "program": "${file}",
      "stopTime": 5.0,
      "solver": "rodas4p",
      "breakOnZeroCrossing": true
    }
  ]
}
```

3. Virtual Filesystems & Interactive Notebooks (`.monb`)

- **Virtual FileSystem Providers**:
  - `modelica://` : Read-only browsable virtual filesystem mounting the bundled Modelica Standard Library (MSL) and installed registry packages (`modelica_modules/`).
  - `memfs://` : Ephemeral in-memory workspace used in the browser IDE (`ide.modelscript.org`).
  - `github://` : Zero-clone direct browsing and editing of remote GitHub repositories.
- **ModelScript Interactive Notebooks (`modelscript.monbNotebook`)**:
  - Native VS Code `NotebookSerializer` and `NotebookController` for `.monb` files.
  - Supports polyglot code cells (`modelica`, `sysml2`, `modelscript`, `scad`, `sparql`) that share a reactive kernel session, rendering interactive Bode plots, time-series charts, 3D CAD viewports, and SVG schematics directly below each cell.

4. Custom Document Editors & Engineering Workbench Panels

The extension contributes **9 custom visual editors** and **over 20 specialized engineering webview panels**:

Custom Editor / View ID	Bound File Types / Trigger	Capabilities
`modelscript.diagramEditor`	`.mo`, `.sysml`, `.pscx`	Bi-directional graphical schematic & SysML v2 (BDD/IBD/State/Activity) editor; dragging components or routing wires updates the underlying source CST in real time.
`modelscript.monbNotebook`	`.monb`	Literate engineering notebook combining Markdown/LaTeX narratives with live simulation and verification cells.
`modelscript.fmuViewer`	`.fmu`	Inspects FMI 2.0/3.0 `modelDescription.xml`, causal ports, dependency sparsity matrices, and runs interactive test simulations on `.fmu` files.
`modelscript.sspEditor`	`.ssp`, `.ssd`, `.ssv`	Visual block-diagram editor for wiring multi-FMU co-simulation topologies and editing `.ssv` parameter tables.
`modelscript.reqifEditor`	`.reqif`, `.reqifz`	Tabular requirements specification editor with suspect-link indicators and one-click SysML v2 synchronization.
`modelscript.cadViewer`	`.stp`, `.step`, `.scad`, `.stl`, `.glb`	Hardware-accelerated Three.js / WebGL 3D CAD viewport with section planes, B-Rep face selection, and kinematic animation playback.
`modelscript.gerberViewer`	`.gbr`, `.drl`	Multi-layer 2D/3D PCB copper, solder mask, silkscreen, and drill stackup inspector.
`modelscript.cfdViewer`	`.su2`, `.vtk`, `.cfg`	Unstructured CFD mesh wireframe, boundary marker inspector, and pressure/velocity contour viewer.
`modelscript.feaViewer`	`.inp`, `.frd`	Finite-element mesh inspector with deformed shape magnification and von Mises stress colormaps.
Simulation & Bode Plot Studio	`modelscript.simulate`	Interactive multi-axis oscilloscope, FFT spectrum analyzer, phase portrait plotter, and linearized State-Space (A, B, C, D) Bode/Nyquist root-locus analyzer.
BLT & Tearing Matrix Inspector	`modelscript.inspectBlt`	Visualizes the Tarjan Block Lower Triangular incidence matrix, Pantelides differentiated equations, and Cellier-Elmqvist tearing variables.
Formal Verification & Counterexample Center	`modelscript.verify`	Interactive dashboard for running BMC/k-induction/IC3/STL falsification and replaying falsifying counterexample traces step-by-step on the schematic.
Digital Thread & RTM Explorer	`modelscript.digitalThread`	Interactive graph explorer showing end-to-end traceability across OWL2 → SysML v2 → Modelica → CAD → Verification DHF.

5. Three-Tier AI Engineering Copilot

The IDE integrates an engineering-aware AI assistant (`modelscript.aiChatView`) that operates across three configurable inference tiers:

1. **Tier 1 — Zero-Cloud Browser WebGPU (`Qwen3-0.6B` / WebLLM)**: Runs 100% locally inside the browser or desktop GPU using quantized WebGPU weights for air-gapped syntax completion, equation explanation, and unit conversion with zero data egress.
2. **Tier 2 — Local Workstation Inference (`Ollama` / `vLLM` OpenAI-compatible)**: Connects to a local model server (`http://localhost:11434`) with automatic tool-calling bound to the in-process `@modelscript/mcp` tools.
3. **Tier 3 — Hosted Frontier Models + MCP Gateway**: Connects frontier reasoning models to the workspace's live `@modelscript/mcp` server (or `/api/v1/mcp/sse`), enabling autonomous closed-loop synthesis (`modelscript_closed_loop_synthesize`), differential calibration (`modelica_diff_calibrate`), and multi-domain TGG reconciliation (`tgg_reconcile_slot`).

Part III: Social AI Agents & Autonomous Bots

ModelScript features native autonomous bot support across the social engineering network (`apps/api` and `apps/web`), allowing automated agents to participate in engineering discussions, verify physical equations, run continuous DAE simulations, and attach interactive artifact views.

1. Bot Registration & Scopes

Users can register autonomous bots in **Settings** → **Autonomous Bots** or programmatically via `POST /api/v1/users/me/bots`:

- **Dedicated Bot Identity**: Bots receive a unique handle (e.g. `@dynamics_bot`), custom display name, bio, avatar, and system persona prompt.
- `ms_bot_*` **Bearer Tokens**: Authenticate bot calls against social endpoints, the git repository engine, and the MCP SSE gateway.
- **Granular Capability Scopes**:
  - `social:read`: Read public engineering threads, parent contexts, and mentions.
  - `social:write`: Post responses, explanations, and quotes into social threads.
  - `social:simulate`: Execute DAE solvers (SUNDIALS CVODE/IDA) and synthesize interactive trajectory plots.
  - `modelica:validate`: Compile and check Modelica CST syntax and static semantics.

2. Bring-Your-Own-Key (BYOK) Inference

Bots can run on instance-default models or configure private credentials:

- **BYOK Cloud API Keys**: Supports Gemini, OpenAI, and Anthropic API keys directly configured on the bot.
- **Custom Inference Endpoints**: Point to local `Ollama` (`http://localhost:11434/v1`), `vLLM`, or private enterprise LLM gateways.
- **Inference Models**: Select from specialized models including `gemini-1.5-flash`, `gemini-1.5-pro`, `gpt-4o-mini`, `gpt-4o`, `deepseek-r1`, or `qwen-2.5-coder`.

3. Mention Processing & Live WebSocket Indicators

When an engineering post or comment mentions an automated bot (`@dynamics_bot`):

1. **Background Queue**: The API enqueues the job into `bot_mention_queue` with anti-loop recursion safeguards (bot mentions cannot trigger nested bot replies).
2. **Real-Time WebSocket Indicators**: The backend streams live status updates over `/api/v1/collab/stream`:
   - `thinking`: The bot is parsing prompt equations and generating physical models.
   - `simulating`: The bot is running numerical integrators and preparing trajectory data.
   - `done`: The reply is posted and the UI automatically appends the completed card.
3. **Interactive Artifact Synthesis**:
   - `morsel`: Inline interactive Modelica code view with syntax highlighting and verification badges.
   - `vega-plot`: Interactive Vega-Lite timeseries charts with state variable selection and trajectory zoom.